

# Core Java (Java 8 and Java 17) - Master Notes

A Complete Reference for Interview Preparation, Production Work, Real-World Projects, and System Understanding.

**Java Full Stack Master Course**

# Table of Contents

## CORE JAVA (Java 8 & Java 17) — MASTER NOTES 39

A Complete Reference Book for Interviews, Production Work & System Understanding . . . . . 39

## TOPIC 1: INTRODUCTION TO JAVA 39

|                                                                                                 |    |
|-------------------------------------------------------------------------------------------------|----|
| 1. Introduction . . . . .                                                                       | 39 |
| 1.1 What is Java? . . . . .                                                                     | 39 |
| 1.2 Why was Java introduced? . . . . .                                                          | 39 |
| 1.3 Real-world problem it solves . . . . .                                                      | 40 |
| 1.4 Historical Background (Full Timeline) . . . . .                                             | 41 |
| 1.5 Interview Definition . . . . .                                                              | 42 |
| 1.6 Simple Definition . . . . .                                                                 | 42 |
| 1.7 Technical Definition . . . . .                                                              | 42 |
| 2. Real Life Analogy . . . . .                                                                  | 43 |
| 3. Internal Working — How Java Code Actually Runs . . . . .                                     | 43 |
| 3.1 The Two-Step Execution Model . . . . .                                                      | 43 |
| 3.2 Detailed Step-by-Step Internal Flow . . . . .                                               | 44 |
| 3.3 Memory Changes During Execution (Preview — detailed in JVM Internals topic later) . . . . . | 44 |
| 4. Diagrams . . . . .                                                                           | 45 |
| 4.1 Java Platform Components — JDK vs JRE vs JVM . . . . .                                      | 45 |
| 4.2 Compilation + Execution Flow (Full Pipeline) . . . . .                                      | 45 |
| 4.3 WORA (Write Once, Run Anywhere) Diagram . . . . .                                           | 46 |
| 5. Syntax — The First Java Program Explained Symbol by Symbol . . . . .                         | 46 |
| Line-by-line / Keyword-by-keyword Explanation . . . . .                                         | 47 |
| Why must main have exactly this signature? . . . . .                                            | 47 |
| 6. "Every Method" for This Topic — Core Entry-Point APIs . . . . .                              | 48 |
| 6.1 System.out.println() family . . . . .                                                       | 48 |
| 7. Code Examples . . . . .                                                                      | 48 |
| 7.1 Basic Example . . . . .                                                                     | 48 |
| 7.2 Intermediate Example — Using Command-Line Arguments . . . . .                               | 49 |
| 7.3 Advanced Example — Multiple Classes + Entry Point . . . . .                                 | 49 |
| 7.4 Real Company-Style Example — Environment Info Printer (Production style) . . . . .          | 50 |
| 7.5 Bad Code vs Good Code Comparison . . . . .                                                  | 50 |

|                                                                                 |    |
|---------------------------------------------------------------------------------|----|
| 8. Dry Run . . . . .                                                            | 51 |
| Program: . . . . .                                                              | 51 |
| Step-by-step Dry Run . . . . .                                                  | 51 |
| 9. Edge Cases . . . . .                                                         | 52 |
| 10. Internal Interview Questions . . . . .                                      | 52 |
| Easy . . . . .                                                                  | 52 |
| Medium . . . . .                                                                | 52 |
| Hard . . . . .                                                                  | 53 |
| Company-Level / Tricky . . . . .                                                | 53 |
| 11. Coding Questions . . . . .                                                  | 53 |
| Easy . . . . .                                                                  | 53 |
| Medium . . . . .                                                                | 54 |
| Hard / Interview Level (Company Tagged: TCS/Infosys screening rounds) . . . . . | 54 |
| 12. Common Mistakes . . . . .                                                   | 55 |
| Beginner Mistakes . . . . .                                                     | 55 |
| Experienced Developer Mistakes . . . . .                                        | 55 |
| Debugging Process for "Main method not found" error: . . . . .                  | 55 |
| 13. Best Practices . . . . .                                                    | 55 |
| 14. Production Usage . . . . .                                                  | 56 |
| 15. Performance . . . . .                                                       | 56 |
| 16. Frequently Asked Interview Questions (50+ with Detailed Answers) . . . . .  | 56 |
| 17. Revision Notes (One-Page Cheat Sheet) . . . . .                             | 61 |
| 18. Homework . . . . .                                                          | 61 |
| 19. Mini Project — "System Info Reporter" CLI Tool . . . . .                    | 62 |
| 20. Final Summary . . . . .                                                     | 62 |

## TOPIC 2: JDK, JRE, JVM 63

|                                                   |    |
|---------------------------------------------------|----|
| 1. Introduction . . . . .                         | 63 |
| 1.1 What is it? . . . . .                         | 63 |
| 1.2 Why was this separation introduced? . . . . . | 63 |
| 1.3 Real-world problem it solves . . . . .        | 64 |
| 1.4 Historical Background . . . . .               | 64 |
| 1.5 Interview Definition . . . . .                | 65 |
| 1.6 Simple Definition . . . . .                   | 65 |
| 1.7 Technical Definition . . . . .                | 65 |

|                                                                                                                             |    |
|-----------------------------------------------------------------------------------------------------------------------------|----|
| 2. Real Life Analogy . . . . .                                                                                              | 66 |
| 3. Internal Working . . . . .                                                                                               | 67 |
| 3.1 What Exactly Is Inside Each Layer . . . . .                                                                             | 67 |
| 3.2 Step-by-Step: What Happens When You Type <code>javac HelloWorld.java</code> Then <code>java HelloWorld</code> . . . . . | 67 |
| 3.3 Memory View — Where Each Component "Lives" . . . . .                                                                    | 68 |
| 4. Diagrams . . . . .                                                                                                       | 68 |
| 4.1 JDK vs JRE vs JVM — Containment Diagram . . . . .                                                                       | 68 |
| 4.2 JVM Architecture (Detailed) . . . . .                                                                                   | 69 |
| 4.3 Class Loader Hierarchy (Delegation Model) . . . . .                                                                     | 69 |
| 5. Syntax — Key Commands Explained . . . . .                                                                                | 69 |
| 5.1 Compilation Command . . . . .                                                                                           | 69 |
| 5.2 Execution Command . . . . .                                                                                             | 70 |
| 5.3 <code>jlink</code> Syntax (Custom Runtime Creation, Java 9+) . . . . .                                                  | 70 |
| 6. Every Method / Tool — Detailed Reference . . . . .                                                                       | 71 |
| 6.1 <code>javac</code> (Compiler) . . . . .                                                                                 | 71 |
| 6.2 <code>java</code> (Launcher) . . . . .                                                                                  | 71 |
| 6.3 <code>javadoc</code> . . . . .                                                                                          | 71 |
| 6.4 <code>jar</code> . . . . .                                                                                              | 72 |
| 6.5 <code>jshell</code> (REPL, Java 9+) . . . . .                                                                           | 72 |
| 6.6 <code>jlink</code> (Java 9+) . . . . .                                                                                  | 72 |
| 7. Code Examples . . . . .                                                                                                  | 72 |
| 7.1 Basic — Checking Your Installed Versions . . . . .                                                                      | 72 |
| 7.2 Intermediate — Inspecting Bytecode with <code>javap</code> . . . . .                                                    | 73 |
| 7.3 Advanced — Building a Custom Minimal Runtime with <code>jlink</code> . . . . .                                          | 73 |
| 7.4 Company-style Example — Programmatic Runtime Diagnostics . . . . .                                                      | 74 |
| 8. Dry Run . . . . .                                                                                                        | 74 |
| Program (same as topic 1, revisited to trace class loading in detail): . . . . .                                            | 74 |
| 9. Edge Cases . . . . .                                                                                                     | 76 |
| 10. Internal Interview Questions . . . . .                                                                                  | 76 |
| Easy . . . . .                                                                                                              | 76 |
| Medium . . . . .                                                                                                            | 76 |
| Hard . . . . .                                                                                                              | 77 |
| Company-Level / Tricky . . . . .                                                                                            | 77 |
| 11. Coding Questions . . . . .                                                                                              | 77 |
| Easy . . . . .                                                                                                              | 77 |
| Medium . . . . .                                                                                                            | 78 |

|                                                                             |           |
|-----------------------------------------------------------------------------|-----------|
| Hard (Company Tagged: Infra/DevOps screening)                               | 78        |
| <b>12. Common Mistakes</b>                                                  | <b>78</b> |
| Beginner Mistakes                                                           | 78        |
| Experienced Developer Mistakes                                              | 79        |
| Debugging Process for UnsupportedClassVersionError                          | 79        |
| <b>13. Best Practices</b>                                                   | <b>79</b> |
| <b>14. Production Usage</b>                                                 | <b>79</b> |
| <b>15. Performance</b>                                                      | <b>80</b> |
| <b>16. Frequently Asked Interview Questions (50+ with Detailed Answers)</b> | <b>80</b> |
| <b>17. Revision Notes (One-Page Cheat Sheet)</b>                            | <b>83</b> |
| <b>18. Homework</b>                                                         | <b>83</b> |
| <b>19. Mini Project — "JVM Health Reporter" CLI Tool</b>                    | <b>84</b> |
| <b>20. Final Summary</b>                                                    | <b>85</b> |

## **TOPIC 3: JAVA COMPILATION PROCESS 85**

|                                                                 |           |
|-----------------------------------------------------------------|-----------|
| <b>1. Introduction</b>                                          | <b>85</b> |
| 1.1 What is it?                                                 | 85        |
| 1.2 Why was it introduced / what problem it solves              | 86        |
| 1.3 Real-world problem it solves                                | 86        |
| 1.4 Historical Background                                       | 86        |
| 1.5 Interview Definition                                        | 86        |
| 1.6 Simple Definition                                           | 86        |
| 1.7 Technical Definition                                        | 87        |
| <b>2. Real Life Analogy</b>                                     | <b>87</b> |
| <b>3. Internal Working</b>                                      | <b>88</b> |
| 3.1 The Four Phases of javac Compilation (Detailed)             | 88        |
| 3.2 Phase 1 — Lexical Analysis (Deep Dive)                      | 88        |
| 3.3 Phase 2 — Syntax Analysis / Parsing (Deep Dive)             | 89        |
| 3.4 Phase 3 — Semantic Analysis (Deep Dive)                     | 89        |
| 3.5 Phase 4 — Bytecode Generation (Deep Dive)                   | 90        |
| <b>4. Diagrams</b>                                              | <b>90</b> |
| 4.1 Full Compilation Pipeline (End-to-End)                      | 90        |
| 4.2 .class File Binary Structure (JVMS §4.1)                    | 90        |
| 4.3 Generics Type Erasure Diagram                               | 91        |
| 4.4 Lambda Desugaring — invokedynamic (Not an Anonymous Class!) | 91        |

|                                                                                                  |     |
|--------------------------------------------------------------------------------------------------|-----|
| 5. Syntax — Compiler Flags and Their Exact Meaning . . . . .                                     | 91  |
| 6. Every Method / Component — Deep Reference . . . . .                                           | 92  |
| 6.1 The Constant Pool . . . . .                                                                  | 92  |
| 6.2 Bytecode Instructions (Opcodes) — Key Families . . . . .                                     | 93  |
| 7. Code Examples . . . . .                                                                       | 93  |
| 7.1 Basic — Viewing the Compiled .class File's Header Bytes . . . . .                            | 93  |
| 7.2 Intermediate — Observing Type Erasure Firsthand . . . . .                                    | 94  |
| 7.3 Advanced — Compiling with --release to Catch API Misuse Early . . . . .                      | 94  |
| 7.4 Production-style Example — Enforcing Strict Compilation in a Build Pipeline . . . . .        | 94  |
| 8. Dry Run . . . . .                                                                             | 95  |
| Source: . . . . .                                                                                | 95  |
| Trace Through All Four Compiler Phases . . . . .                                                 | 95  |
| 9. Edge Cases . . . . .                                                                          | 96  |
| 10. Internal Interview Questions . . . . .                                                       | 96  |
| Easy . . . . .                                                                                   | 96  |
| Medium . . . . .                                                                                 | 96  |
| Hard . . . . .                                                                                   | 97  |
| Company-Level / Tricky . . . . .                                                                 | 97  |
| 11. Coding Questions . . . . .                                                                   | 97  |
| Easy . . . . .                                                                                   | 97  |
| Medium . . . . .                                                                                 | 97  |
| Hard (Company Tagged: Compiler-internals rounds, e.g., Oracle/product-based companies) . . . . . | 98  |
| 12. Common Mistakes . . . . .                                                                    | 98  |
| Beginner Mistakes . . . . .                                                                      | 98  |
| Experienced Developer Mistakes . . . . .                                                         | 98  |
| Debugging Process for Erasure-related ClassCastException . . . . .                               | 99  |
| 13. Best Practices . . . . .                                                                     | 99  |
| 14. Production Usage . . . . .                                                                   | 99  |
| 15. Performance . . . . .                                                                        | 100 |
| 16. Frequently Asked Interview Questions (50+ with Detailed Answers) . . . . .                   | 100 |
| 17. Revision Notes (One-Page Cheat Sheet) . . . . .                                              | 104 |
| 18. Homework . . . . .                                                                           | 104 |
| 19. Mini Project — "Bytecode Inspector" CLI Tool . . . . .                                       | 104 |
| 20. Final Summary . . . . .                                                                      | 105 |

## TOPIC 4: JAVA MEMORY MODEL

106

|                                                                      |     |
|----------------------------------------------------------------------|-----|
| 1. Introduction                                                      | 106 |
| 1.1 What is it?                                                      | 106 |
| 1.2 Why was it introduced                                            | 107 |
| 1.3 Real-world problem it solves                                     | 107 |
| 1.4 Historical Background                                            | 107 |
| 1.5 Interview Definition                                             | 107 |
| 1.6 Simple Definition                                                | 107 |
| 1.7 Technical Definition                                             | 108 |
| 2. Real Life Analogy                                                 | 108 |
| 3. Internal Working                                                  | 109 |
| 3.1 Runtime Data Areas — Where Things Actually Live                  | 109 |
| 3.2 Step-by-Step: Object Creation and Memory Movement                | 109 |
| 3.3 Stack vs Heap — What Exactly Goes Where                          | 109 |
| 4. Diagrams                                                          | 110 |
| 4.1 Full Memory Layout                                               | 110 |
| 4.2 Heap Generational Structure (HotSpot default)                    | 110 |
| 4.3 Stack Frame Structure (Per Method Call)                          | 110 |
| 5. Syntax — JVM Memory Flags                                         | 111 |
| 6. Key APIs for Inspecting Memory                                    | 111 |
| 7. Code Examples                                                     | 111 |
| 7.1 Basic — Observing Heap Memory Stats                              | 111 |
| 7.2 Intermediate — Proving References Live on Stack, Objects on Heap | 112 |
| 7.3 Advanced — Demonstrating StackOverflowError                      | 112 |
| 8. Dry Run                                                           | 112 |
| 9. Edge Cases                                                        | 113 |
| 10. Interview Questions                                              | 113 |
| Easy                                                                 | 113 |
| Medium                                                               | 113 |
| Hard / Company-Level                                                 | 113 |
| 11. Coding Questions                                                 | 114 |
| 12. Common Mistakes                                                  | 114 |
| 13. Best Practices                                                   | 115 |
| 14. Production Usage                                                 | 115 |
| 15. Performance                                                      | 115 |

|                                                        |     |
|--------------------------------------------------------|-----|
| 16. Frequently Asked Interview Questions . . . . .     | 116 |
| 17. Revision Notes . . . . .                           | 117 |
| 18. Homework . . . . .                                 | 117 |
| 19. Mini Project — "Memory Watchdog" Utility . . . . . | 118 |
| 20. Final Summary . . . . .                            | 118 |

## **TOPIC 5: VARIABLES 118**

|                                                    |     |
|----------------------------------------------------|-----|
| 1. Introduction . . . . .                          | 118 |
| 2. Real Life Analogy . . . . .                     | 119 |
| 3. Internal Working . . . . .                      | 119 |
| 4. Types of Variables . . . . .                    | 119 |
| 5. Syntax . . . . .                                | 119 |
| 6. Code Examples . . . . .                         | 120 |
| 7. Edge Cases . . . . .                            | 120 |
| 8. Common Mistakes . . . . .                       | 120 |
| 9. Best Practices . . . . .                        | 120 |
| 10. Frequently Asked Interview Questions . . . . . | 120 |
| 11. Revision Notes . . . . .                       | 121 |
| 12. Homework . . . . .                             | 121 |

## **TOPIC 6: DATA TYPES 121**

|                                                     |     |
|-----------------------------------------------------|-----|
| 1. Introduction . . . . .                           | 121 |
| 2. Two Categories . . . . .                         | 122 |
| 3. The 8 Primitive Types — Full Reference . . . . . | 122 |
| 4. Real Life Analogy . . . . .                      | 122 |
| 5. Internal Working — Autoboxing Preview . . . . .  | 122 |
| 6. Code Examples . . . . .                          | 123 |
| 7. Edge Cases . . . . .                             | 123 |
| 8. Common Mistakes . . . . .                        | 123 |
| 9. Best Practices . . . . .                         | 123 |
| 10. Frequently Asked Interview Questions . . . . .  | 123 |
| 11. Revision Notes . . . . .                        | 124 |
| 12. Homework . . . . .                              | 124 |



## TOPIC 7: OPERATORS 124

|                                                          |     |
|----------------------------------------------------------|-----|
| 1. Introduction . . . . .                                | 125 |
| 2. Categories of Operators . . . . .                     | 125 |
| 3. Real Life Analogy . . . . .                           | 125 |
| 4. Internal Working — Short-Circuit Evaluation . . . . . | 125 |
| 5. Bitwise Operators — Deep Reference . . . . .          | 126 |
| 6. Code Examples . . . . .                               | 126 |
| 7. Edge Cases . . . . .                                  | 126 |
| 8. Common Mistakes . . . . .                             | 126 |
| 9. Best Practices . . . . .                              | 127 |
| 10. Frequently Asked Interview Questions . . . . .       | 127 |
| 11. Revision Notes . . . . .                             | 127 |
| 12. Homework . . . . .                                   | 127 |

## TOPIC 8: TYPE CASTING 128

|                                                                |     |
|----------------------------------------------------------------|-----|
| 1. Introduction . . . . .                                      | 128 |
| 2. Primitive Casting — Widening vs Narrowing . . . . .         | 128 |
| 3. Real Life Analogy . . . . .                                 | 128 |
| 4. Reference Type Casting — Upcasting vs Downcasting . . . . . | 129 |
| 5. Internal Working . . . . .                                  | 129 |
| 6. Code Examples . . . . .                                     | 129 |
| 7. Edge Cases . . . . .                                        | 130 |
| 8. Common Mistakes . . . . .                                   | 130 |
| 9. Best Practices . . . . .                                    | 130 |
| 10. Frequently Asked Interview Questions . . . . .             | 130 |
| 11. Revision Notes . . . . .                                   | 131 |
| 12. Homework . . . . .                                         | 131 |

## TOPIC 9: KEYWORDS 131

|                                                 |     |
|-------------------------------------------------|-----|
| 1. Introduction . . . . .                       | 131 |
| 2. Complete Reference Table (Java 17) . . . . . | 132 |
| Data Type Keywords . . . . .                    | 132 |

|                                                                                     |     |
|-------------------------------------------------------------------------------------|-----|
| Modifier Keywords . . . . .                                                         | 132 |
| Control Flow Keywords . . . . .                                                     | 132 |
| Exception Handling Keywords . . . . .                                               | 133 |
| Class/Object-Oriented Keywords . . . . .                                            | 133 |
| Reserved Literals (NOT technically "keywords" per JLS, but reserved) . . . . .      | 133 |
| Unused / Reserved-but-not-implemented . . . . .                                     | 133 |
| 3. Contextual Keywords (NOT reserved — can still be used as identifiers!) . . . . . | 133 |
| 4. Real Life Analogy . . . . .                                                      | 134 |
| 5. Code Examples . . . . .                                                          | 134 |
| 6. Edge Cases . . . . .                                                             | 134 |
| 7. Common Mistakes . . . . .                                                        | 134 |
| 8. Frequently Asked Interview Questions . . . . .                                   | 134 |
| 9. Revision Notes . . . . .                                                         | 135 |
| 10. Homework . . . . .                                                              | 135 |

## **TOPIC 10: CONTROL STATEMENTS** **135**

|                                                                                      |     |
|--------------------------------------------------------------------------------------|-----|
| 1. Introduction . . . . .                                                            | 135 |
| 2. Conditional Statements . . . . .                                                  | 136 |
| 2.1 if / else if / else . . . . .                                                    | 136 |
| 2.2 Traditional switch (fall-through by default) . . . . .                           | 136 |
| 2.3 Modern Switch Expression (Java 14+) — No Fall-Through, Returns a Value . . . . . | 136 |
| 3. Looping Statements . . . . .                                                      | 136 |
| 4. Jump Statements . . . . .                                                         | 137 |
| Labeled break/continue — controlling OUTER loops from an inner loop . . . . .        | 137 |
| 5. Real Life Analogy . . . . .                                                       | 137 |
| 6. Edge Cases . . . . .                                                              | 138 |
| 7. Common Mistakes . . . . .                                                         | 138 |
| 8. Best Practices . . . . .                                                          | 138 |
| 9. Frequently Asked Interview Questions . . . . .                                    | 138 |
| 10. Revision Notes . . . . .                                                         | 139 |
| 11. Homework . . . . .                                                               | 139 |

## **TOPIC 11: ARRAYS** **139**

|                           |     |
|---------------------------|-----|
| 1. Introduction . . . . . | 139 |
|---------------------------|-----|

|                                                    |     |
|----------------------------------------------------|-----|
| 2. Real Life Analogy . . . . .                     | 140 |
| 3. Internal Working . . . . .                      | 140 |
| 4. Syntax . . . . .                                | 140 |
| 5. Key Methods (via java.util.Arrays) . . . . .    | 141 |
| 6. Code Examples . . . . .                         | 141 |
| 7. Edge Cases . . . . .                            | 142 |
| 8. Common Mistakes . . . . .                       | 142 |
| 9. Best Practices . . . . .                        | 142 |
| 10. Production Usage . . . . .                     | 142 |
| 11. Frequently Asked Interview Questions . . . . . | 142 |
| 12. Revision Notes . . . . .                       | 143 |
| 13. Homework . . . . .                             | 143 |

## **TOPIC 12: METHODS** **144**

|                                                          |     |
|----------------------------------------------------------|-----|
| 1. Introduction . . . . .                                | 144 |
| 2. Syntax — Every Part Explained . . . . .               | 144 |
| 3. Real Life Analogy . . . . .                           | 144 |
| 4. Pass-by-Value — The #1 Interview Trap Topic . . . . . | 145 |
| 5. Varargs (Variable Arguments) . . . . .                | 145 |
| 6. Recursion . . . . .                                   | 145 |
| 7. Static vs Instance Methods . . . . .                  | 146 |
| 8. Edge Cases . . . . .                                  | 146 |
| 9. Common Mistakes . . . . .                             | 146 |
| 10. Best Practices . . . . .                             | 146 |
| 11. Frequently Asked Interview Questions . . . . .       | 146 |
| 12. Revision Notes . . . . .                             | 147 |
| 13. Homework . . . . .                                   | 147 |

## **TOPIC 13: COMMAND LINE ARGUMENTS** **147**

|                                |     |
|--------------------------------|-----|
| 1. Introduction . . . . .      | 148 |
| 2. Real Life Analogy . . . . . | 148 |
| 3. Internal Working . . . . .  | 148 |
| 4. Syntax . . . . .            | 148 |

|                                                                       |     |
|-----------------------------------------------------------------------|-----|
| 5. Code Examples . . . . .                                            | 149 |
| Basic — Safe Access with Defaults . . . . .                           | 149 |
| Intermediate — Parsing Numeric Arguments . . . . .                    | 149 |
| Advanced — Flag-Style Argument Parsing (production pattern) . . . . . | 149 |
| 6. Edge Cases . . . . .                                               | 150 |
| 7. Common Mistakes . . . . .                                          | 150 |
| 8. Best Practices . . . . .                                           | 150 |
| 9. Frequently Asked Interview Questions . . . . .                     | 150 |
| 10. Revision Notes . . . . .                                          | 151 |
| 11. Homework . . . . .                                                | 151 |

## **TOPIC 14: CLASSES & OBJECTS** **151**

|                                                                           |     |
|---------------------------------------------------------------------------|-----|
| 1. Introduction . . . . .                                                 | 151 |
| 1.1 What is it? . . . . .                                                 | 151 |
| 1.2 Why was it introduced . . . . .                                       | 151 |
| 1.3 Real-world problem it solves . . . . .                                | 151 |
| 1.4 Interview Definition . . . . .                                        | 152 |
| 1.5 Simple Definition . . . . .                                           | 152 |
| 2. Real Life Analogy . . . . .                                            | 152 |
| 3. Internal Working — What Happens When You Create an Object . . . . .    | 152 |
| 3.1 The this Keyword . . . . .                                            | 153 |
| 4. Diagrams . . . . .                                                     | 153 |
| 4.1 Class as a Blueprint — Multiple Objects, One Class . . . . .          | 153 |
| 4.2 Object Creation Timeline . . . . .                                    | 154 |
| 5. Syntax . . . . .                                                       | 154 |
| 6. Code Examples . . . . .                                                | 155 |
| 6.1 Basic . . . . .                                                       | 155 |
| 6.2 Intermediate — Encapsulated Class with Constructor . . . . .          | 155 |
| 6.3 Production-style — Multiple Objects Managed in a Collection . . . . . | 156 |
| 7. Dry Run . . . . .                                                      | 156 |
| 8. Edge Cases . . . . .                                                   | 157 |
| 9. Common Mistakes . . . . .                                              | 157 |
| 10. Best Practices . . . . .                                              | 157 |
| 11. Production Usage . . . . .                                            | 157 |
| 12. Frequently Asked Interview Questions . . . . .                        | 158 |

|                                                     |     |
|-----------------------------------------------------|-----|
| 13. Revision Notes . . . . .                        | 158 |
| 14. Homework . . . . .                              | 158 |
| 15. Mini Project — Simple Library Catalog . . . . . | 159 |
| 16. Final Summary . . . . .                         | 159 |

## **TOPIC 15: CONSTRUCTORS** **160**

|                                                                                   |     |
|-----------------------------------------------------------------------------------|-----|
| 1. Introduction . . . . .                                                         | 160 |
| 2. Real Life Analogy . . . . .                                                    | 160 |
| 3. Types of Constructors . . . . .                                                | 161 |
| 3.1 Default Constructor (Compiler-Generated) . . . . .                            | 161 |
| 3.2 No-Argument Constructor (Explicitly Written) . . . . .                        | 161 |
| 3.3 Parameterized Constructor . . . . .                                           | 161 |
| 3.4 Copy Constructor (Java has no built-in one — you write it manually) . . . . . | 161 |
| 3.5 Private Constructor (Singleton / Utility Class Pattern) . . . . .             | 161 |
| 4. Constructor Chaining — <code>this()</code> and <code>super()</code> . . . . .  | 162 |
| 5. Rules of Constructors . . . . .                                                | 162 |
| 6. Code Examples . . . . .                                                        | 163 |
| 7. Edge Cases . . . . .                                                           | 163 |
| 8. Common Mistakes . . . . .                                                      | 163 |
| 9. Best Practices . . . . .                                                       | 164 |
| 10. Production Usage . . . . .                                                    | 164 |
| 11. Frequently Asked Interview Questions . . . . .                                | 164 |
| 12. Revision Notes . . . . .                                                      | 165 |
| 13. Homework . . . . .                                                            | 165 |

## **TOPIC 16: CONSTRUCTOR CHAINING** **165**

|                                                                                  |     |
|----------------------------------------------------------------------------------|-----|
| 1. Introduction . . . . .                                                        | 165 |
| 2. Real Life Analogy . . . . .                                                   | 166 |
| 3. The Two Forms of Chaining . . . . .                                           | 166 |
| 4. Full JVM Object Initialization Order (Critical — Frequently Tested) . . . . . | 166 |
| Diagram — 3-Level Hierarchy Initialization Order . . . . .                       | 167 |
| 5. Rules . . . . .                                                               | 167 |
| 6. Code Examples . . . . .                                                       | 168 |
| 7. Edge Cases . . . . .                                                          | 168 |

|                                                    |     |
|----------------------------------------------------|-----|
| 8. Common Mistakes . . . . .                       | 168 |
| 9. Best Practices . . . . .                        | 169 |
| 10. Frequently Asked Interview Questions . . . . . | 169 |
| 11. Revision Notes . . . . .                       | 170 |
| 12. Homework . . . . .                             | 170 |

## **TOPIC 17: METHOD OVERLOADING 170**

|                                                                                            |     |
|--------------------------------------------------------------------------------------------|-----|
| 1. Introduction . . . . .                                                                  | 170 |
| 2. Real Life Analogy . . . . .                                                             | 170 |
| 3. What Counts as a Valid Overload . . . . .                                               | 171 |
| 4. Internal Working — Overload Resolution Priority (Critical, Frequently Tested) . . . . . | 171 |
| 5. Ambiguous Overload Errors . . . . .                                                     | 171 |
| 6. Overloading vs Overriding — Comparison Table . . . . .                                  | 172 |
| 7. Code Examples . . . . .                                                                 | 172 |
| 8. Edge Cases . . . . .                                                                    | 172 |
| 9. Common Mistakes . . . . .                                                               | 172 |
| 10. Best Practices . . . . .                                                               | 173 |
| 11. Frequently Asked Interview Questions . . . . .                                         | 173 |
| 12. Revision Notes . . . . .                                                               | 174 |
| 13. Homework . . . . .                                                                     | 174 |

## **TOPIC 18: METHOD OVERRIDING 174**

|                                                                                |     |
|--------------------------------------------------------------------------------|-----|
| 1. Introduction . . . . .                                                      | 174 |
| 1.1 What is it? . . . . .                                                      | 174 |
| 1.2 Why was it introduced . . . . .                                            | 174 |
| 1.3 Interview Definition . . . . .                                             | 175 |
| 2. Real Life Analogy . . . . .                                                 | 175 |
| 3. Rules for Valid Overriding . . . . .                                        | 176 |
| 4. Internal Working — Dynamic Method Dispatch (Virtual Method Table) . . . . . | 176 |
| 4.1 Step-by-Step . . . . .                                                     | 176 |
| 4.2 Diagram — Virtual Method Table Concept . . . . .                           | 177 |
| 4.3 super.methodName() — Calling the Parent's Version Explicitly . . . . .     | 177 |
| 5. The @Override Annotation . . . . .                                          | 177 |
| 6. Covariant Return Types (Java 5+) . . . . .                                  | 177 |

|                                                                          |     |
|--------------------------------------------------------------------------|-----|
| 7. Static Method "Hiding" vs Overriding — Critical Distinction . . . . . | 178 |
| 8. Code Examples . . . . .                                               | 178 |
| 9. Dry Run . . . . .                                                     | 178 |
| 10. Edge Cases . . . . .                                                 | 179 |
| 11. Common Mistakes . . . . .                                            | 179 |
| 12. Best Practices . . . . .                                             | 179 |
| 13. Production Usage . . . . .                                           | 180 |
| 14. Frequently Asked Interview Questions . . . . .                       | 180 |
| 15. Revision Notes . . . . .                                             | 181 |
| 16. Homework . . . . .                                                   | 181 |

## **TOPIC 19: INHERITANCE 181**

|                                                                                      |     |
|--------------------------------------------------------------------------------------|-----|
| 1. Introduction . . . . .                                                            | 181 |
| 1.1 What is it? . . . . .                                                            | 181 |
| 1.2 Why was it introduced . . . . .                                                  | 182 |
| 1.3 Real-world problem it solves . . . . .                                           | 182 |
| 1.4 Interview Definition . . . . .                                                   | 182 |
| 2. Real Life Analogy . . . . .                                                       | 182 |
| 3. Types of Inheritance in Java . . . . .                                            | 182 |
| Why Java does NOT support multiple class inheritance — The Diamond Problem . . . . . | 183 |
| 4. Internal Working — What a Subclass Object Actually Contains . . . . .             | 183 |
| 5. Syntax . . . . .                                                                  | 184 |
| 6. super Keyword — Three Uses . . . . .                                              | 184 |
| 7. Code Examples . . . . .                                                           | 185 |
| 7.1 Hierarchical Inheritance + Polymorphism . . . . .                                | 185 |
| 7.2 Multiple Interface "Inheritance" (Diamond Problem Solved Explicitly) . . . . .   | 185 |
| 8. Edge Cases . . . . .                                                              | 186 |
| 9. Common Mistakes . . . . .                                                         | 186 |
| 10. Best Practices . . . . .                                                         | 186 |
| 11. Production Usage . . . . .                                                       | 186 |
| 12. Frequently Asked Interview Questions . . . . .                                   | 187 |
| 13. Revision Notes . . . . .                                                         | 188 |
| 14. Homework . . . . .                                                               | 188 |
| 15. Mini Project — Employee Payroll Hierarchy . . . . .                              | 189 |

|                             |     |
|-----------------------------|-----|
| 16. Final Summary . . . . . | 189 |
|-----------------------------|-----|

## **TOPIC 20: POLYMORPHISM** **190**

|                                                                           |     |
|---------------------------------------------------------------------------|-----|
| 1. Introduction . . . . .                                                 | 190 |
| 2. Real Life Analogy . . . . .                                            | 190 |
| 3. The Two Types — Side-by-Side Comparison . . . . .                      | 191 |
| 4. Polymorphism via Upcasting — The Core Pattern . . . . .                | 191 |
| 5. Polymorphism with Collections — The Most Common Real Pattern . . . . . | 191 |
| 6. Polymorphism and Interfaces (The Preferred Modern Style) . . . . .     | 191 |
| 7. Edge Cases . . . . .                                                   | 192 |
| 8. Common Mistakes . . . . .                                              | 192 |
| 9. Best Practices . . . . .                                               | 192 |
| 10. Production Usage . . . . .                                            | 192 |
| 11. Frequently Asked Interview Questions . . . . .                        | 193 |
| 12. Revision Notes . . . . .                                              | 193 |
| 13. Homework . . . . .                                                    | 193 |

## **TOPIC 21: ABSTRACTION** **194**

|                                                                                                        |     |
|--------------------------------------------------------------------------------------------------------|-----|
| 1. Introduction . . . . .                                                                              | 194 |
| 2. Real Life Analogy . . . . .                                                                         | 194 |
| 3. Abstraction vs Encapsulation — A Critical Distinction (Frequently Confused in Interviews) . . . . . | 194 |
| 4. How Java Achieves Abstraction . . . . .                                                             | 195 |
| 5. Levels of Abstraction . . . . .                                                                     | 195 |
| 6. Code Example — Abstraction in a Real-World-Style Design . . . . .                                   | 196 |
| 7. Edge Cases . . . . .                                                                                | 196 |
| 8. Common Mistakes . . . . .                                                                           | 196 |
| 9. Best Practices . . . . .                                                                            | 196 |
| 10. Production Usage . . . . .                                                                         | 197 |
| 11. Frequently Asked Interview Questions . . . . .                                                     | 197 |
| 12. Revision Notes . . . . .                                                                           | 197 |
| 13. Homework . . . . .                                                                                 | 198 |

## **TOPIC 22: ENCAPSULATION** **198**



|                                                               |     |
|---------------------------------------------------------------|-----|
| 1. Introduction . . . . .                                     | 198 |
| 2. Real Life Analogy . . . . .                                | 198 |
| 3. How to Achieve Encapsulation in Java . . . . .             | 199 |
| 4. Levels of Encapsulation (Getter/Setter Patterns) . . . . . | 199 |
| 5. Code Examples . . . . .                                    | 200 |
| 6. Edge Cases . . . . .                                       | 200 |
| 7. Common Mistakes . . . . .                                  | 200 |
| 8. Best Practices . . . . .                                   | 201 |
| 9. Production Usage . . . . .                                 | 201 |
| 10. Frequently Asked Interview Questions . . . . .            | 201 |
| 11. Revision Notes . . . . .                                  | 202 |
| 12. Homework . . . . .                                        | 202 |

## **TOPIC 23: INTERFACES** **202**

|                                                                                    |     |
|------------------------------------------------------------------------------------|-----|
| 1. Introduction . . . . .                                                          | 202 |
| 2. Real Life Analogy . . . . .                                                     | 203 |
| 3. Syntax and Evolution Across Java Versions . . . . .                             | 203 |
| 4. Why default Methods Were Added (Java 8) — A Real Historical Problem . . . . .   | 204 |
| 5. Multiple Interface Implementation & Diamond Resolution . . . . .                | 204 |
| 6. Functional Interfaces (Preview — Full Depth in Java 8 Features Topic) . . . . . | 204 |
| 7. Code Examples . . . . .                                                         | 204 |
| 8. Edge Cases . . . . .                                                            | 205 |
| 9. Common Mistakes . . . . .                                                       | 205 |
| 10. Best Practices . . . . .                                                       | 205 |
| 11. Production Usage . . . . .                                                     | 205 |
| 12. Frequently Asked Interview Questions . . . . .                                 | 206 |
| 13. Revision Notes . . . . .                                                       | 206 |
| 14. Homework . . . . .                                                             | 206 |

## **TOPIC 24: ABSTRACT CLASSES** **207**

|                                |     |
|--------------------------------|-----|
| 1. Introduction . . . . .      | 207 |
| 2. Real Life Analogy . . . . . | 207 |
| 3. Syntax . . . . .            | 207 |

|                                                                             |     |
|-----------------------------------------------------------------------------|-----|
| 4. Abstract Class vs Interface — Full Comparison (Critical Interview Table) | 208 |
| 5. Code Examples                                                            | 208 |
| 6. Edge Cases                                                               | 209 |
| 7. Common Mistakes                                                          | 209 |
| 8. Best Practices                                                           | 209 |
| 9. Production Usage                                                         | 209 |
| 10. Frequently Asked Interview Questions                                    | 209 |
| 11. Revision Notes                                                          | 210 |
| 12. Homework                                                                | 210 |

## **TOPIC 25: OBJECT CLASS** **210**

|                                                       |     |
|-------------------------------------------------------|-----|
| 1. Introduction                                       | 210 |
| 2. Key Methods of Object                              | 211 |
| 3. equals() and hashCode() — The Critical Contract    | 211 |
| 3.1 Why Override Both Together (Never Just One)       | 211 |
| 3.2 The equals()/hashCode() Contract (Interview Gold) | 211 |
| 4. toString()                                         | 212 |
| 5. getClass()                                         | 212 |
| 6. Edge Cases                                         | 212 |
| 7. Common Mistakes                                    | 212 |
| 8. Best Practices                                     | 213 |
| 9. Production Usage                                   | 213 |
| 10. Frequently Asked Interview Questions              | 213 |
| 11. Revision Notes                                    | 214 |
| 12. Homework                                          | 214 |

## **TOPIC 26: PACKAGES** **214**

|                                  |     |
|----------------------------------|-----|
| 1. Introduction                  | 214 |
| 2. Real Life Analogy             | 214 |
| 3. Syntax and Naming Conventions | 215 |
| 4. Types of Packages             | 215 |
| 5. Fully Qualified Names         | 215 |
| 6. Code Example                  | 216 |

|                                                    |     |
|----------------------------------------------------|-----|
| 7. Edge Cases . . . . .                            | 216 |
| 8. Common Mistakes . . . . .                       | 216 |
| 9. Best Practices . . . . .                        | 216 |
| 10. Frequently Asked Interview Questions . . . . . | 217 |
| 11. Revision Notes . . . . .                       | 217 |
| 12. Homework . . . . .                             | 217 |

## **TOPIC 27: ACCESS MODIFIERS 218**

|                                                                 |     |
|-----------------------------------------------------------------|-----|
| 1. Introduction . . . . .                                       | 218 |
| 2. The Four Levels — Full Visibility Table . . . . .            | 218 |
| 3. Real Life Analogy . . . . .                                  | 218 |
| 4. Applying Access Modifiers — Where Each Can Be Used . . . . . | 218 |
| 5. Code Examples . . . . .                                      | 219 |
| 6. Edge Cases . . . . .                                         | 219 |
| 7. Common Mistakes . . . . .                                    | 219 |
| 8. Best Practices . . . . .                                     | 219 |
| 9. Production Usage . . . . .                                   | 219 |
| 10. Frequently Asked Interview Questions . . . . .              | 220 |
| 11. Revision Notes . . . . .                                    | 220 |
| 12. Homework . . . . .                                          | 220 |

## **ADVANCED CORE JAVA 221**

### **TOPIC 28: WRAPPER CLASSES 221**

|                                                                          |     |
|--------------------------------------------------------------------------|-----|
| 1. Introduction . . . . .                                                | 221 |
| 2. The Complete Mapping . . . . .                                        | 221 |
| 3. Autoboxing and Unboxing (Java 5+) . . . . .                           | 221 |
| 4. Internal Working — Integer Caching (A Major Interview Trap) . . . . . | 222 |
| 5. Key Methods . . . . .                                                 | 222 |
| 6. Code Examples . . . . .                                               | 222 |
| 7. Edge Cases . . . . .                                                  | 223 |
| 8. Common Mistakes . . . . .                                             | 223 |
| 9. Best Practices . . . . .                                              | 223 |

|                                                    |     |
|----------------------------------------------------|-----|
| 10. Frequently Asked Interview Questions . . . . . | 223 |
| 11. Revision Notes . . . . .                       | 224 |
| 12. Homework . . . . .                             | 224 |

## **TOPIC 29: STRING** **224**

|                                                     |     |
|-----------------------------------------------------|-----|
| 1. Introduction . . . . .                           | 224 |
| 2. The String Constant Pool (SCP) . . . . .         | 225 |
| 3. Why String is Immutable — Deep Reasons . . . . . | 225 |
| 4. Key Methods . . . . .                            | 226 |
| 5. Code Examples . . . . .                          | 226 |
| 6. Edge Cases . . . . .                             | 227 |
| 7. Common Mistakes . . . . .                        | 227 |
| 8. Best Practices . . . . .                         | 227 |
| 9. Production Usage . . . . .                       | 227 |
| 10. Frequently Asked Interview Questions . . . . .  | 227 |
| 11. Revision Notes . . . . .                        | 228 |
| 12. Homework . . . . .                              | 228 |

## **TOPIC 30: STRINGBUILDER** **229**

|                                                                         |     |
|-------------------------------------------------------------------------|-----|
| 1. Introduction . . . . .                                               | 229 |
| 2. Internal Working . . . . .                                           | 229 |
| 3. Key Methods . . . . .                                                | 229 |
| 4. Code Examples . . . . .                                              | 230 |
| 5. StringBuilder vs String vs StringBuffer — Comparison Table . . . . . | 230 |
| 6. Edge Cases . . . . .                                                 | 230 |
| 7. Common Mistakes . . . . .                                            | 230 |
| 8. Best Practices . . . . .                                             | 231 |
| 9. Production Usage . . . . .                                           | 231 |
| 10. Frequently Asked Interview Questions . . . . .                      | 231 |
| 11. Revision Notes . . . . .                                            | 232 |
| 12. Homework . . . . .                                                  | 232 |

## **TOPIC 31: STRINGBUFFER** **232**

|                                                                       |     |
|-----------------------------------------------------------------------|-----|
| 1. Introduction . . . . .                                             | 232 |
| 2. StringBuffer vs StringBuilder — The Only Real Difference . . . . . | 232 |
| 3. Code Example . . . . .                                             | 233 |
| 4. When StringBuffer Is Actually Justified . . . . .                  | 233 |
| 5. Edge Cases . . . . .                                               | 233 |
| 6. Common Mistakes . . . . .                                          | 233 |
| 7. Best Practices . . . . .                                           | 233 |
| 8. Frequently Asked Interview Questions . . . . .                     | 233 |
| 9. Revision Notes . . . . .                                           | 234 |
| 10. Homework . . . . .                                                | 234 |

## **TOPIC 32: IMMUTABLE OBJECTS** **234**

|                                                                  |     |
|------------------------------------------------------------------|-----|
| 1. Introduction . . . . .                                        | 234 |
| 2. Rules to Create a Truly Immutable Class . . . . .             | 235 |
| The 5 Rules (Full Checklist) . . . . .                           | 235 |
| Defensive Copying Example (Rule 4 — Frequently Tested) . . . . . | 235 |
| 3. Real Life Analogy . . . . .                                   | 236 |
| 4. Edge Cases . . . . .                                          | 236 |
| 5. Common Mistakes . . . . .                                     | 236 |
| 6. Best Practices . . . . .                                      | 236 |
| 7. Production Usage . . . . .                                    | 237 |
| 8. Frequently Asked Interview Questions . . . . .                | 237 |
| 9. Revision Notes . . . . .                                      | 237 |
| 10. Homework . . . . .                                           | 237 |

## **TOPIC 33: ENUMS** **238**

|                                                                     |     |
|---------------------------------------------------------------------|-----|
| 1. Introduction . . . . .                                           | 238 |
| 2. Real Life Analogy . . . . .                                      | 238 |
| 3. Syntax — From Simple to Advanced . . . . .                       | 238 |
| 3.1 Simple Enum . . . . .                                           | 238 |
| 3.2 Enum with Fields, Constructor, and Methods . . . . .            | 239 |
| 3.3 Enum with Abstract Methods (Constant-Specific Bodies) . . . . . | 239 |
| 4. Internal Working . . . . .                                       | 239 |
| 5. Key Methods (inherited from java.lang.Enum) . . . . .            | 239 |

|                                                    |     |
|----------------------------------------------------|-----|
| 6. Enums in switch Statements . . . . .            | 240 |
| 7. Code Examples . . . . .                         | 240 |
| 8. Edge Cases . . . . .                            | 240 |
| 9. Common Mistakes . . . . .                       | 240 |
| 10. Best Practices . . . . .                       | 241 |
| 11. Production Usage . . . . .                     | 241 |
| 12. Frequently Asked Interview Questions . . . . . | 241 |
| 13. Revision Notes . . . . .                       | 242 |
| 14. Homework . . . . .                             | 242 |

## **TOPIC 34: ANNOTATIONS** **242**

|                                                                             |     |
|-----------------------------------------------------------------------------|-----|
| 1. Introduction . . . . .                                                   | 242 |
| 2. Real Life Analogy . . . . .                                              | 242 |
| 3. Built-in Annotations . . . . .                                           | 243 |
| 4. Meta-Annotations (Annotations That Annotate Other Annotations) . . . . . | 243 |
| 5. Creating a Custom Annotation . . . . .                                   | 243 |
| 6. Reading Annotations via Reflection . . . . .                             | 244 |
| 7. Edge Cases . . . . .                                                     | 244 |
| 8. Common Mistakes . . . . .                                                | 244 |
| 9. Best Practices . . . . .                                                 | 244 |
| 10. Production Usage . . . . .                                              | 245 |
| 11. Frequently Asked Interview Questions . . . . .                          | 245 |
| 12. Revision Notes . . . . .                                                | 245 |
| 13. Homework . . . . .                                                      | 246 |

## **TOPIC 35: GENERICS** **246**

|                                                                                         |     |
|-----------------------------------------------------------------------------------------|-----|
| 1. Introduction . . . . .                                                               | 246 |
| 2. Real Life Analogy . . . . .                                                          | 246 |
| 3. Generic Classes . . . . .                                                            | 247 |
| 4. Generic Methods . . . . .                                                            | 247 |
| 5. Bounded Type Parameters . . . . .                                                    | 247 |
| 6. Wildcards — ?, ? extends, ? super . . . . .                                          | 247 |
| The PECS Rule (Producer Extends, Consumer Super) — Critical Interview Concept . . . . . | 247 |

|                                                                             |     |
|-----------------------------------------------------------------------------|-----|
| 7. Type Erasure Recap (Full Detail in [[Java Compilation Process Topic 3]]) | 248 |
| 8. Code Examples                                                            | 248 |
| 9. Edge Cases                                                               | 249 |
| 10. Common Mistakes                                                         | 249 |
| 11. Best Practices                                                          | 249 |
| 12. Production Usage                                                        | 249 |
| 13. Frequently Asked Interview Questions                                    | 249 |
| 14. Revision Notes                                                          | 250 |
| 15. Homework                                                                | 250 |

## **TOPIC 36: REFLECTION API 250**

|                                         |     |
|-----------------------------------------|-----|
| 1. Introduction                         | 251 |
| 2. Real Life Analogy                    | 251 |
| 3. Getting a Class Object — Three Ways  | 251 |
| 4. Key Reflection Operations            | 252 |
| 5. Edge Cases                           | 252 |
| 6. Common Mistakes                      | 252 |
| 7. Best Practices                       | 253 |
| 8. Production Usage                     | 253 |
| 9. Frequently Asked Interview Questions | 253 |
| 10. Revision Notes                      | 254 |
| 11. Homework                            | 254 |

## **TOPIC 37: INNER CLASSES 254**

|                                         |     |
|-----------------------------------------|-----|
| 1. Introduction                         | 254 |
| 2. Syntax and Internal Working          | 255 |
| 3. Real Life Analogy                    | 255 |
| 4. Code Example                         | 255 |
| 5. Edge Cases                           | 256 |
| 6. Common Mistakes                      | 256 |
| 7. Best Practices                       | 256 |
| 8. Frequently Asked Interview Questions | 256 |
| 9. Revision Notes                       | 257 |

|                        |     |
|------------------------|-----|
| 10. Homework . . . . . | 257 |
|------------------------|-----|

## **TOPIC 38: NESTED CLASSES** **257**

|                                                                    |     |
|--------------------------------------------------------------------|-----|
| 1. Introduction . . . . .                                          | 257 |
| 2. Full Classification of Nested Classes . . . . .                 | 257 |
| 3. Static Nested Class . . . . .                                   | 258 |
| 4. Local Inner Class (Defined Inside a Method) . . . . .           | 258 |
| 5. Anonymous Inner Class . . . . .                                 | 258 |
| 6. Static Nested Class vs Inner Class — Comparison Table . . . . . | 259 |
| 7. Real Life Analogy . . . . .                                     | 259 |
| 8. Edge Cases . . . . .                                            | 259 |
| 9. Common Mistakes . . . . .                                       | 259 |
| 10. Best Practices . . . . .                                       | 259 |
| 11. Production Usage . . . . .                                     | 259 |
| 12. Frequently Asked Interview Questions . . . . .                 | 260 |
| 13. Revision Notes . . . . .                                       | 260 |
| 14. Homework . . . . .                                             | 260 |

## **TOPIC 39: RECORDS (JAVA 17)** **261**

|                                                       |     |
|-------------------------------------------------------|-----|
| 1. Introduction . . . . .                             | 261 |
| 2. Syntax — Before and After . . . . .                | 261 |
| Before Records (Manual, Java 8-style) . . . . .       | 261 |
| With Records (Java 16+) . . . . .                     | 261 |
| 3. What the Compiler Auto-Generates . . . . .         | 262 |
| 4. Compact Constructors — Adding Validation . . . . . | 262 |
| 5. Records Can Have Additional Members . . . . .      | 262 |
| 6. Rules and Restrictions . . . . .                   | 263 |
| 7. Real Life Analogy . . . . .                        | 263 |
| 8. Code Examples . . . . .                            | 263 |
| 9. Edge Cases . . . . .                               | 264 |
| 10. Common Mistakes . . . . .                         | 264 |
| 11. Best Practices . . . . .                          | 264 |
| 12. Production Usage . . . . .                        | 264 |



|                                                    |     |
|----------------------------------------------------|-----|
| 13. Frequently Asked Interview Questions . . . . . | 264 |
| 14. Revision Notes . . . . .                       | 265 |
| 15. Homework . . . . .                             | 265 |

## **TOPIC 40: SEALED CLASSES (JAVA 17) 266**

|                                                                      |     |
|----------------------------------------------------------------------|-----|
| 1. Introduction . . . . .                                            | 266 |
| 2. Syntax . . . . .                                                  | 266 |
| 3. The Three Required Modifiers for Permitted Subclasses . . . . .   | 266 |
| 4. Why Sealed Classes Matter — Exhaustive Pattern Matching . . . . . | 267 |
| 5. Real Life Analogy . . . . .                                       | 267 |
| 6. Sealed Classes vs Enums — When to Use Which . . . . .             | 267 |
| 7. Code Examples . . . . .                                           | 268 |
| 8. Edge Cases . . . . .                                              | 268 |
| 9. Common Mistakes . . . . .                                         | 268 |
| 10. Best Practices . . . . .                                         | 268 |
| 11. Production Usage . . . . .                                       | 269 |
| 12. Frequently Asked Interview Questions . . . . .                   | 269 |
| 13. Revision Notes . . . . .                                         | 269 |
| 14. Homework . . . . .                                               | 269 |

## **EXCEPTION HANDLING 270**

### **TOPIC 41: EXCEPTION HIERARCHY, CHECKED & UNCHECKED EXCEPTIONS<sup>270</sup>**

|                                                          |     |
|----------------------------------------------------------|-----|
| 1. Introduction . . . . .                                | 270 |
| 2. The Full Exception Hierarchy . . . . .                | 270 |
| 3. Checked vs Unchecked — The Core Distinction . . . . . | 271 |
| 4. Real Life Analogy . . . . .                           | 271 |
| 5. Code Examples . . . . .                               | 271 |
| 6. Edge Cases . . . . .                                  | 272 |
| 7. Common Mistakes . . . . .                             | 272 |
| 8. Best Practices . . . . .                              | 272 |
| 9. Frequently Asked Interview Questions . . . . .        | 272 |
| 10. Revision Notes . . . . .                             | 273 |

## **TOPIC 42: TRY-CATCH-FINALLY** **273**

|                                                                                              |     |
|----------------------------------------------------------------------------------------------|-----|
| 1. Introduction . . . . .                                                                    | 273 |
| 2. Syntax and Execution Order . . . . .                                                      | 273 |
| 3. Multi-Catch (Java 7+) . . . . .                                                           | 274 |
| 4. Try-With-Resources (Java 7+) — Automatic Resource Cleanup . . . . .                       | 274 |
| 5. Internal Working — Desugaring (Recap from [[Java Compilation Process Topic 3]]) . . . . . | 274 |
| 6. Edge Cases . . . . .                                                                      | 275 |
| 7. Common Mistakes . . . . .                                                                 | 275 |
| 8. Best Practices . . . . .                                                                  | 275 |
| 9. Frequently Asked Interview Questions . . . . .                                            | 275 |
| 10. Revision Notes . . . . .                                                                 | 276 |

## **TOPIC 43: THROW AND THROWS** **276**

|                                                   |     |
|---------------------------------------------------|-----|
| 1. Introduction . . . . .                         | 276 |
| 2. throw vs throws — Comparison Table . . . . .   | 276 |
| 3. Code Examples . . . . .                        | 277 |
| 4. Rethrowing and Exception Chaining . . . . .    | 277 |
| 5. Edge Cases . . . . .                           | 277 |
| 6. Common Mistakes . . . . .                      | 277 |
| 7. Best Practices . . . . .                       | 278 |
| 8. Frequently Asked Interview Questions . . . . . | 278 |
| 9. Revision Notes . . . . .                       | 278 |

## **TOPIC 44: CUSTOM EXCEPTIONS** **279**

|                                                             |     |
|-------------------------------------------------------------|-----|
| 1. Introduction . . . . .                                   | 279 |
| 2. Syntax — Checked vs Unchecked Custom Exception . . . . . | 279 |
| 3. Real Life Analogy . . . . .                              | 279 |
| 4. Code Examples . . . . .                                  | 280 |
| 5. Best Practices for Custom Exceptions . . . . .           | 280 |
| 6. Common Mistakes . . . . .                                | 280 |
| 7. Production Usage . . . . .                               | 281 |
| 8. Frequently Asked Interview Questions . . . . .           | 281 |

|                                              |     |
|----------------------------------------------|-----|
| 9. Revision Notes . . . . .                  | 281 |
| 10. Homework (Covers Topics 41-44) . . . . . | 281 |

## **COLLECTIONS FRAMEWORK 282**

### **TOPIC 45: THE COLLECTIONS FRAMEWORK & THE LIST INTERFACE 282**

|                                                                   |     |
|-------------------------------------------------------------------|-----|
| 1. Introduction . . . . .                                         | 282 |
| 2. The Core Hierarchy . . . . .                                   | 282 |
| 3. The List Interface . . . . .                                   | 282 |
| 4. List vs Set vs Map — High-Level Comparison (Preview) . . . . . | 283 |
| 5. Real Life Analogy . . . . .                                    | 283 |
| 6. Code Example — Common List Operations . . . . .                | 283 |
| 7. Edge Cases . . . . .                                           | 283 |
| 8. Common Mistakes . . . . .                                      | 284 |
| 9. Frequently Asked Interview Questions . . . . .                 | 284 |

### **TOPIC 46: ARRAYLIST 284**

|                                                   |     |
|---------------------------------------------------|-----|
| 1. Introduction . . . . .                         | 284 |
| 2. Internal Working . . . . .                     | 284 |
| 3. Key Methods with Time Complexity . . . . .     | 285 |
| 4. Code Example . . . . .                         | 285 |
| 5. Edge Cases . . . . .                           | 285 |
| 6. Best Practices . . . . .                       | 285 |
| 7. Frequently Asked Interview Questions . . . . . | 285 |

### **TOPIC 47: LINKEDLIST 286**

|                                                              |     |
|--------------------------------------------------------------|-----|
| 1. Introduction . . . . .                                    | 286 |
| 2. Internal Working . . . . .                                | 286 |
| 3. ArrayList vs LinkedList — Full Comparison Table . . . . . | 286 |
| 4. Code Example . . . . .                                    | 287 |
| 5. Frequently Asked Interview Questions . . . . .            | 287 |

### **TOPIC 48: VECTOR 287**

|                                                     |     |
|-----------------------------------------------------|-----|
| 1. Introduction . . . . .                           | 287 |
| 2. Vector vs ArrayList — Comparison Table . . . . . | 288 |
| 3. Code Example . . . . .                           | 288 |
| 4. Frequently Asked Interview Questions . . . . .   | 288 |

## **TOPIC 49: STACK 288**

|                                                         |     |
|---------------------------------------------------------|-----|
| 1. Introduction . . . . .                               | 288 |
| 2. Key Methods . . . . .                                | 289 |
| 3. Code Example . . . . .                               | 289 |
| 4. Real Life Analogy . . . . .                          | 289 |
| 5. Modern Alternative — ArrayDeque as a Stack . . . . . | 289 |
| 6. Frequently Asked Interview Questions . . . . .       | 289 |

## **TOPIC 50: QUEUE 290**

|                                                           |     |
|-----------------------------------------------------------|-----|
| 1. Introduction . . . . .                                 | 290 |
| 2. Key Methods — The "Safe" vs "Throwing" Pairs . . . . . | 290 |
| 3. Real Life Analogy . . . . .                            | 290 |
| 4. Code Example . . . . .                                 | 290 |
| 5. Frequently Asked Interview Questions . . . . .         | 290 |

## **TOPIC 51: PRIORITYQUEUE 291**

|                                                   |     |
|---------------------------------------------------|-----|
| 1. Introduction . . . . .                         | 291 |
| 2. Internal Working — The Binary Heap . . . . .   | 291 |
| 3. Key Methods with Time Complexity . . . . .     | 291 |
| 4. Custom Priority via Comparator . . . . .       | 291 |
| 5. Real Life Analogy . . . . .                    | 291 |
| 6. Edge Cases . . . . .                           | 292 |
| 7. Frequently Asked Interview Questions . . . . . | 292 |

## **TOPIC 52: DEQUE 292**

|                                                             |     |
|-------------------------------------------------------------|-----|
| 1. Introduction . . . . .                                   | 292 |
| 2. Key Methods . . . . .                                    | 292 |
| 3. ArrayDeque vs LinkedList (as a Deque) vs Stack . . . . . | 293 |

|                                                   |     |
|---------------------------------------------------|-----|
| 4. Code Example . . . . .                         | 293 |
| 5. Real Life Analogy . . . . .                    | 293 |
| 6. Frequently Asked Interview Questions . . . . . | 293 |

## **TOPIC 53: SET** **294**

|                                                       |     |
|-------------------------------------------------------|-----|
| 1. Introduction . . . . .                             | 294 |
| 2. Real Life Analogy . . . . .                        | 294 |
| 3. The Three Main Implementations (Preview) . . . . . | 294 |
| 4. Frequently Asked Interview Questions . . . . .     | 294 |

## **TOPIC 54: HASHSET** **294**

|                                                   |     |
|---------------------------------------------------|-----|
| 1. Introduction . . . . .                         | 294 |
| 2. Internal Working . . . . .                     | 295 |
| 3. Code Example . . . . .                         | 295 |
| 4. Frequently Asked Interview Questions . . . . . | 295 |

## **TOPIC 55: LINKEDHASHSET** **295**

|                                                                     |     |
|---------------------------------------------------------------------|-----|
| 1. Introduction . . . . .                                           | 295 |
| 2. Code Example . . . . .                                           | 296 |
| 3. HashSet vs LinkedHashSet vs TreeSet — Comparison Table . . . . . | 296 |
| 4. Frequently Asked Interview Questions . . . . .                   | 296 |

## **TOPIC 56: TREESET** **296**

|                                                   |     |
|---------------------------------------------------|-----|
| 1. Introduction . . . . .                         | 296 |
| 2. Key Methods (Beyond Standard Set) . . . . .    | 296 |
| 3. Code Example . . . . .                         | 297 |
| 4. Edge Cases . . . . .                           | 297 |
| 5. Frequently Asked Interview Questions . . . . . | 297 |

## **TOPIC 57: MAP** **297**

|                                |     |
|--------------------------------|-----|
| 1. Introduction . . . . .      | 297 |
| 2. Real Life Analogy . . . . . | 298 |
| 3. Core Methods . . . . .      | 298 |

|                                                   |     |
|---------------------------------------------------|-----|
| 4. Frequently Asked Interview Questions . . . . . | 298 |
|---------------------------------------------------|-----|

|                          |            |
|--------------------------|------------|
| <b>TOPIC 58: HASHMAP</b> | <b>298</b> |
|--------------------------|------------|

|                                                                                       |     |
|---------------------------------------------------------------------------------------|-----|
| 1. Introduction . . . . .                                                             | 298 |
| 2. Internal Working — Deep Dive (Critical, Frequently Tested) . . . . .               | 299 |
| 2.1 The Bucket Array . . . . .                                                        | 299 |
| 2.2 Step-by-Step: put("apple", 5) . . . . .                                           | 299 |
| 2.3 Load Factor and Resizing . . . . .                                                | 299 |
| 2.4 Treeification (Java 8+ Optimization) . . . . .                                    | 299 |
| 2.5 Diagram — Full put() Flow . . . . .                                               | 300 |
| 3. Key Methods with Time Complexity . . . . .                                         | 300 |
| 4. Code Examples . . . . .                                                            | 300 |
| 5. HashMap vs Hashtable vs LinkedHashMap vs TreeMap — Full Comparison Table . . . . . | 301 |
| 6. Edge Cases . . . . .                                                               | 301 |
| 7. Common Mistakes . . . . .                                                          | 301 |
| 8. Best Practices . . . . .                                                           | 301 |
| 9. Frequently Asked Interview Questions . . . . .                                     | 302 |
| 10. Revision Notes . . . . .                                                          | 302 |

|                                |            |
|--------------------------------|------------|
| <b>TOPIC 59: LINKEDHASHMAP</b> | <b>302</b> |
|--------------------------------|------------|

|                                                                   |     |
|-------------------------------------------------------------------|-----|
| 1. Introduction . . . . .                                         | 303 |
| 2. Building an LRU Cache — A Classic Production Pattern . . . . . | 303 |
| 3. Frequently Asked Interview Questions . . . . .                 | 303 |

|                          |            |
|--------------------------|------------|
| <b>TOPIC 60: TREEMAP</b> | <b>303</b> |
|--------------------------|------------|

|                                                   |     |
|---------------------------------------------------|-----|
| 1. Introduction . . . . .                         | 304 |
| 2. Key Methods (via NavigableMap) . . . . .       | 304 |
| 3. Code Example . . . . .                         | 304 |
| 4. Frequently Asked Interview Questions . . . . . | 304 |

|                            |            |
|----------------------------|------------|
| <b>TOPIC 61: HASHTABLE</b> | <b>304</b> |
|----------------------------|------------|

|                                                     |     |
|-----------------------------------------------------|-----|
| 1. Introduction . . . . .                           | 304 |
| 2. Hashtable vs HashMap — Key Differences . . . . . | 305 |
| 3. Frequently Asked Interview Questions . . . . .   | 305 |

|                                                               |            |
|---------------------------------------------------------------|------------|
| <b>TOPIC 62: COMPARABLE</b>                                   | <b>305</b> |
| 1. Introduction . . . . .                                     | 305        |
| 2. Syntax . . . . .                                           | 305        |
| 3. The compareTo() Contract . . . . .                         | 306        |
| 4. Code Example . . . . .                                     | 306        |
| 5. Frequently Asked Interview Questions . . . . .             | 306        |
| <b>TOPIC 63: COMPARATOR</b>                                   | <b>306</b> |
| 1. Introduction . . . . .                                     | 306        |
| 2. Comparable vs Comparator — Full Comparison Table . . . . . | 307        |
| 3. Syntax — Old Style vs Modern (Java 8+) Style . . . . .     | 307        |
| 4. Chaining Comparators (Java 8+) . . . . .                   | 307        |
| 5. Code Example . . . . .                                     | 308        |
| 6. Edge Cases . . . . .                                       | 308        |
| 7. Frequently Asked Interview Questions . . . . .             | 308        |
| <b>TOPIC 64: ITERATOR</b>                                     | <b>308</b> |
| 1. Introduction . . . . .                                     | 309        |
| 2. Key Methods . . . . .                                      | 309        |
| 3. ConcurrentModificationException — Why It Happens . . . . . | 309        |
| The Correct, Safe Way . . . . .                               | 309        |
| 4. Real Life Analogy . . . . .                                | 309        |
| 5. Frequently Asked Interview Questions . . . . .             | 309        |
| <b>TOPIC 65: LISTITERATOR</b>                                 | <b>310</b> |
| 1. Introduction . . . . .                                     | 310        |
| 2. Key Additional Methods (Beyond Iterator) . . . . .         | 310        |
| 3. Code Example . . . . .                                     | 311        |
| 4. Iterator vs ListIterator — Comparison Table . . . . .      | 311        |
| 5. Frequently Asked Interview Questions . . . . .             | 311        |
| <b>JAVA 8 FEATURES</b>                                        | <b>311</b> |

## **TOPIC 66: LAMBDA EXPRESSIONS 311**

|                                                     |     |
|-----------------------------------------------------|-----|
| 1. Introduction . . . . .                           | 312 |
| 2. Syntax Evolution . . . . .                       | 312 |
| 3. Real Life Analogy . . . . .                      | 312 |
| 4. Variable Capture — "Effectively Final" . . . . . | 312 |
| 5. Edge Cases . . . . .                             | 313 |
| 6. Common Mistakes . . . . .                        | 313 |
| 7. Frequently Asked Interview Questions . . . . .   | 313 |

## **TOPIC 67: FUNCTIONAL INTERFACES 313**

|                                                                                |     |
|--------------------------------------------------------------------------------|-----|
| 1. Introduction . . . . .                                                      | 313 |
| 2. The Four Core Built-in Functional Interfaces (java.util.function) . . . . . | 314 |
| 3. Other Notable Built-in Functional Interfaces . . . . .                      | 314 |
| 4. Code Example . . . . .                                                      | 314 |
| 5. Frequently Asked Interview Questions . . . . .                              | 314 |

## **TOPIC 68: METHOD REFERENCES 315**

|                                                   |     |
|---------------------------------------------------|-----|
| 1. Introduction . . . . .                         | 315 |
| 2. The Four Kinds of Method References . . . . .  | 315 |
| 3. Code Examples . . . . .                        | 315 |
| 4. Frequently Asked Interview Questions . . . . . | 315 |

## **TOPIC 69: STREAMS API 316**

|                                                      |     |
|------------------------------------------------------|-----|
| 1. Introduction . . . . .                            | 316 |
| 2. Real Life Analogy . . . . .                       | 316 |
| 3. Stream Pipeline Structure . . . . .               | 316 |
| 4. Key Intermediate Operations . . . . .             | 317 |
| 5. Key Terminal Operations . . . . .                 | 317 |
| 6. Code Examples . . . . .                           | 317 |
| 6.1 Basic Pipeline . . . . .                         | 317 |
| 6.2 flatMap — Flattening Nested Structures . . . . . | 317 |
| 6.3 reduce — Aggregation . . . . .                   | 318 |
| 6.4 Collectors — The Aggregation Toolkit . . . . .   | 318 |



|                                                                     |     |
|---------------------------------------------------------------------|-----|
| 6.5 Parallel Streams . . . . .                                      | 318 |
| 7. Internal Working — Laziness (Critical Interview Point) . . . . . | 318 |
| 8. Edge Cases . . . . .                                             | 318 |
| 9. Common Mistakes . . . . .                                        | 319 |
| 10. Best Practices . . . . .                                        | 319 |
| 11. Production Usage . . . . .                                      | 319 |
| 12. Frequently Asked Interview Questions . . . . .                  | 319 |
| 13. Revision Notes . . . . .                                        | 320 |

## **TOPIC 70: OPTIONAL 320**

|                                                   |     |
|---------------------------------------------------|-----|
| 1. Introduction . . . . .                         | 320 |
| 2. Creating Optionals . . . . .                   | 320 |
| 3. Key Methods . . . . .                          | 321 |
| 4. Code Examples . . . . .                        | 321 |
| 5. Edge Cases . . . . .                           | 322 |
| 6. Common Mistakes . . . . .                      | 322 |
| 7. Best Practices . . . . .                       | 322 |
| 8. Frequently Asked Interview Questions . . . . . | 322 |

## **TOPIC 71: DATE & TIME API 323**

|                                                                       |     |
|-----------------------------------------------------------------------|-----|
| 1. Introduction . . . . .                                             | 323 |
| 2. Key Classes . . . . .                                              | 323 |
| 3. Code Examples . . . . .                                            | 323 |
| 4. java.util.Date vs java.time.LocalDate — Comparison Table . . . . . | 324 |
| 5. Frequently Asked Interview Questions . . . . .                     | 324 |

## **TOPIC 72: PREDICATE, FUNCTION, CONSUMER, SUPPLIER 324**

|                                                     |     |
|-----------------------------------------------------|-----|
| 1. Introduction . . . . .                           | 324 |
| 2. Predicate — A Condition . . . . .                | 324 |
| 3. Function — A Transformation . . . . .            | 325 |
| 4. Consumer — A Side-Effect Action . . . . .        | 325 |
| 5. Supplier — A Value Producer (No Input) . . . . . | 325 |
| 6. Comparison Table . . . . .                       | 325 |

|                                                   |     |
|---------------------------------------------------|-----|
| 7. Frequently Asked Interview Questions . . . . . | 325 |
|---------------------------------------------------|-----|

|                                                                 |            |
|-----------------------------------------------------------------|------------|
| <b>TOPIC 73: DEFAULT METHODS &amp; STATIC INTERFACE METHODS</b> | <b>326</b> |
|-----------------------------------------------------------------|------------|

|                                                              |     |
|--------------------------------------------------------------|-----|
| 1. Recap . . . . .                                           | 326 |
| 2. Why They Were Essential for Java 8 Specifically . . . . . | 326 |
| 3. Static Interface Methods — Utility/Factory Role . . . . . | 326 |
| 4. Frequently Asked Interview Questions . . . . .            | 326 |

|                       |            |
|-----------------------|------------|
| <b>MULTITHREADING</b> | <b>327</b> |
|-----------------------|------------|

|                                              |            |
|----------------------------------------------|------------|
| <b>TOPIC 74: THREAD CLASS &amp; RUNNABLE</b> | <b>327</b> |
|----------------------------------------------|------------|

|                                                      |     |
|------------------------------------------------------|-----|
| 1. Introduction . . . . .                            | 327 |
| 2. Two Ways to Create a Thread . . . . .             | 327 |
| 3. start() vs run() — Critical Distinction . . . . . | 327 |
| 4. Real Life Analogy . . . . .                       | 328 |
| 5. Key Thread Methods . . . . .                      | 328 |
| 6. Code Example . . . . .                            | 328 |
| 7. Edge Cases . . . . .                              | 329 |
| 8. Common Mistakes . . . . .                         | 329 |
| 9. Best Practices . . . . .                          | 329 |
| 10. Frequently Asked Interview Questions . . . . .   | 329 |

|                                   |            |
|-----------------------------------|------------|
| <b>TOPIC 75: THREAD LIFECYCLE</b> | <b>329</b> |
|-----------------------------------|------------|

|                                                         |     |
|---------------------------------------------------------|-----|
| 1. Introduction . . . . .                               | 330 |
| 2. The Six States . . . . .                             | 330 |
| 3. Real Life Analogy . . . . .                          | 330 |
| 4. Code Example — Observing State Transitions . . . . . | 331 |
| 5. Frequently Asked Interview Questions . . . . .       | 331 |

|                                  |            |
|----------------------------------|------------|
| <b>TOPIC 76: SYNCHRONIZATION</b> | <b>331</b> |
|----------------------------------|------------|

|                                                    |     |
|----------------------------------------------------|-----|
| 1. Introduction . . . . .                          | 331 |
| 2. The Race Condition Problem . . . . .            | 331 |
| 3. synchronized — Method and Block Forms . . . . . | 332 |

|                                                             |     |
|-------------------------------------------------------------|-----|
| 4. Real Life Analogy . . . . .                              | 332 |
| 5. Code Example — Fixing the Race Condition . . . . .       | 332 |
| 6. volatile — Visibility Without Mutual Exclusion . . . . . | 333 |
| 7. Edge Cases . . . . .                                     | 333 |
| 8. Common Mistakes . . . . .                                | 333 |
| 9. Best Practices . . . . .                                 | 333 |
| 10. Frequently Asked Interview Questions . . . . .          | 333 |

## **TOPIC 77: LOCKS (`java.util.concurrent.locks`) 334**

|                                                               |     |
|---------------------------------------------------------------|-----|
| 1. Introduction . . . . .                                     | 334 |
| 2. synchronized vs ReentrantLock — Comparison Table . . . . . | 334 |
| 3. Code Example . . . . .                                     | 335 |
| 4. Deadlock — What It Is and How to Avoid It . . . . .        | 335 |
| 5. Frequently Asked Interview Questions . . . . .             | 335 |

## **TOPIC 78: EXECUTOR FRAMEWORK 335**

|                                                   |     |
|---------------------------------------------------|-----|
| 1. Introduction . . . . .                         | 335 |
| 2. Creating an Executor . . . . .                 | 336 |
| 3. Submitting Tasks . . . . .                     | 336 |
| 4. Real Life Analogy . . . . .                    | 336 |
| 5. Key Methods . . . . .                          | 336 |
| 6. Edge Cases . . . . .                           | 337 |
| 7. Common Mistakes . . . . .                      | 337 |
| 8. Best Practices . . . . .                       | 337 |
| 9. Frequently Asked Interview Questions . . . . . | 337 |

## **TOPIC 79: CALLABLE & FUTURE 338**

|                                                      |     |
|------------------------------------------------------|-----|
| 1. Introduction . . . . .                            | 338 |
| 2. Runnable vs Callable — Comparison Table . . . . . | 338 |
| 3. Code Example . . . . .                            | 338 |
| 4. Key Future Methods . . . . .                      | 338 |
| 5. Edge Cases . . . . .                              | 339 |
| 6. Frequently Asked Interview Questions . . . . .    | 339 |

## **TOPIC 80: COMPLETABLEFUTURE 339**

|                                                              |     |
|--------------------------------------------------------------|-----|
| 1. Introduction . . . . .                                    | 339 |
| 2. Creating and Chaining . . . . .                           | 340 |
| 3. Key Methods . . . . .                                     | 340 |
| 4. Real Life Analogy . . . . .                               | 340 |
| 5. Code Example — Combining and Exception Handling . . . . . | 341 |
| 6. Future vs CompletableFuture — Comparison Table . . . . .  | 341 |
| 7. Frequently Asked Interview Questions . . . . .            | 341 |

## **TOPIC 81: CONCURRENT COLLECTIONS 342**

|                                                               |     |
|---------------------------------------------------------------|-----|
| 1. Introduction . . . . .                                     | 342 |
| 2. Key Concurrent Collections . . . . .                       | 342 |
| 3. ConcurrentHashMap vs Hashtable/synchronizedMap() . . . . . | 342 |
| 4. Producer-Consumer with BlockingQueue . . . . .             | 343 |
| 5. Real Life Analogy . . . . .                                | 343 |
| 6. Frequently Asked Interview Questions . . . . .             | 343 |
| Homework (Covers Topics 74-81: Multithreading) . . . . .      | 343 |

## **FILE HANDLING 344**

### **TOPIC 82: FILE API (java.io.File) 344**

|                                                                         |     |
|-------------------------------------------------------------------------|-----|
| 1. Introduction . . . . .                                               | 344 |
| 2. Key Methods . . . . .                                                | 344 |
| 3. Code Example . . . . .                                               | 345 |
| 4. Reading/Writing Content (Streams — the OLD way, Java 1.0+) . . . . . | 345 |
| 5. Frequently Asked Interview Questions . . . . .                       | 345 |

### **TOPIC 83: NIO (NEW I/O) 345**

|                                                                          |     |
|--------------------------------------------------------------------------|-----|
| 1. Introduction . . . . .                                                | 346 |
| 2. Path and Files — The Modern Way (Java 7+ NIO.2) . . . . .             | 346 |
| 3. java.io.File vs java.nio.file.Path/Files — Comparison Table . . . . . | 346 |
| 4. Streams-Based Directory Walking (Java 8+) . . . . .                   | 346 |

|                                                   |     |
|---------------------------------------------------|-----|
| 5. Frequently Asked Interview Questions . . . . . | 346 |
|---------------------------------------------------|-----|

|                                                      |            |
|------------------------------------------------------|------------|
| <b>TOPIC 84: SERIALIZATION &amp; DESERIALIZATION</b> | <b>347</b> |
|------------------------------------------------------|------------|

|                                                   |     |
|---------------------------------------------------|-----|
| 1. Introduction . . . . .                         | 347 |
| 2. Serializable — A Marker Interface . . . . .    | 347 |
| 3. Serializing and Deserializing . . . . .        | 347 |
| 4. serialVersionUID — Why It Matters . . . . .    | 347 |
| 5. Edge Cases . . . . .                           | 348 |
| 6. Common Mistakes . . . . .                      | 348 |
| 7. Best Practices . . . . .                       | 348 |
| 8. Production Usage . . . . .                     | 348 |
| 9. Frequently Asked Interview Questions . . . . . | 349 |

|                      |            |
|----------------------|------------|
| <b>JVM INTERNALS</b> | <b>349</b> |
|----------------------|------------|

|                                                      |            |
|------------------------------------------------------|------------|
| <b>TOPIC 85: HEAP, STACK &amp; METASPACE (RECAP)</b> | <b>349</b> |
|------------------------------------------------------|------------|

|                                                   |     |
|---------------------------------------------------|-----|
| 1. Recap . . . . .                                | 349 |
| 2. Quick Reference Diagram . . . . .              | 350 |
| 3. Frequently Asked Interview Questions . . . . . | 350 |

|                                     |            |
|-------------------------------------|------------|
| <b>TOPIC 86: GARBAGE COLLECTION</b> | <b>350</b> |
|-------------------------------------|------------|

|                                                                  |     |
|------------------------------------------------------------------|-----|
| 1. Introduction . . . . .                                        | 350 |
| 2. The Generational Hypothesis — Why Generations Exist . . . . . | 350 |
| 3. Minor GC vs Major/Full GC . . . . .                           | 351 |
| Minor GC Step-by-Step (Simplified) . . . . .                     | 351 |
| 4. Modern Garbage Collectors — Comparison Table . . . . .        | 351 |
| 5. G1 (Garbage First) — The Modern Default . . . . .             | 351 |
| 6. Real Life Analogy . . . . .                                   | 351 |
| 7. System.gc() Revisited . . . . .                               | 352 |
| 8. Edge Cases . . . . .                                          | 352 |
| 9. Common Mistakes . . . . .                                     | 352 |
| 10. Best Practices . . . . .                                     | 352 |
| 11. Production Usage . . . . .                                   | 352 |

12. Frequently Asked Interview Questions . . . . . 353

**TOPIC 87: JVM ARCHITECTURE (RECAP & FULL SUMMARY) 353**

1. Recap . . . . . 353

2. The Complete JVM Architecture — Final Consolidated Diagram . . . . . 354

3. The Full Journey: Source Code to Output (End-to-End Capstone Summary) . . . . . 354

4. Frequently Asked Interview Questions . . . . . 354

FINAL SUMMARY — CORE JAVA COMPLETE . . . . . 354

# CORE JAVA (Java 8 & Java 17) — MASTER NOTES

*A Complete Reference Book for Interviews, Production Work & System Understanding*

---

## TOPIC 1: INTRODUCTION TO JAVA

---

### 1. Introduction

#### 1.1 What is Java?

Java is a **high-level, class-based, object-oriented, platform-independent, general-purpose programming language** that was designed to have as few implementation dependencies as possible.

It follows the principle of **WORA — Write Once, Run Anywhere**. This means a Java program compiled on one machine (say Windows) can run on any other machine (Linux, Mac, Solaris) **without recompilation**, as long as a Java Virtual Machine (JVM) is installed on that machine.

Java is:

- A **programming language** (syntax and rules to write code)
- A **platform** (an environment that runs compiled Java code — the JVM)

#### 1.2 Why was Java introduced?

**Historical problem (Before Java, early 1990s):**

| Problem                         | Description                                                                                  |
|---------------------------------|----------------------------------------------------------------------------------------------|
| Platform dependency             | C/C++ programs compiled for Windows would not run on Solaris or Mac without recompilation.   |
| Manual memory management        | C/C++ required manual <code>malloc/free</code> , causing memory leaks and dangling pointers. |
| Pointer arithmetic bugs         | Direct pointer manipulation in C/C++ caused security vulnerabilities and crashes.            |
| No built-in network support     | C/C++ had no standard, easy networking libraries built into the language.                    |
| Complex multiple inheritance    | C++ multiple inheritance created the "Diamond Problem."                                      |
| No automatic garbage collection | Developers had to track and free every allocated object manually.                            |

**The real trigger — Consumer Electronics (Green Project, 1991):**

Sun Microsystems started a project called "**Green Project**", led by **James Gosling, Mike Sheridan, and Patrick Naughton**, to build software for embedded consumer electronics (set-top boxes, toasters, TVs). These devices used **different processor chips** from different manufacturers. Writing separate C/C++ code for each chip was expensive and error-prone.

They needed a language that:

1. Could run on **any hardware/chip** without modification.
2. Was **secure** (embeddable in consumer devices).
3. Was **small and efficient**.
4. Had **automatic memory management**.

This led to the creation of a new language, first called "**Oak**" (named after an oak tree outside Gosling's office), later renamed "**Java**" (named after Java coffee, inspired by the coffee the team drank) in **1995**.

### ***1.3 Real-world problem it solves***

- **Cross-platform deployment problem:** Banks, e-commerce companies, and enterprises run software across thousands of heterogeneous servers (Linux, Windows, Cloud VMs). Java lets them build once and deploy everywhere the JVM exists.
- **Memory-safety problem:** Automatic Garbage Collection removes an entire class of bugs (memory leaks, dangling pointers, double-free errors).
- **Security problem:** No direct pointer access; the JVM sandbox (Bytecode Verifier, Security Manager historically) protects against malicious code — critical for applets and enterprise systems.
- **Scalability problem:** Multi-threading is built into the language core (not a third-party library), making Java ideal for large concurrent enterprise systems (banking engines, trading systems).



## 1.4 Historical Background (Full Timeline)

| Year  | Event                                                                                                                                                                                      |
|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1991  | Green Project starts at Sun Microsystems. Language named "Oak".                                                                                                                            |
| 1994  | Oak is used to build a web browser called "WebRunner" (later HotJava) demonstrating dynamic, embeddable content.                                                                           |
| 1995  | Renamed to <b>Java</b> . Officially announced at SunWorld. Java 1.0 released with the slogan "Write Once, Run Anywhere".                                                                   |
| 1996  | JDK 1.0 released.                                                                                                                                                                          |
| 1997  | JDK 1.1 — added Inner classes, JDBC, RMI, JavaBeans.                                                                                                                                       |
| 1998  | <b>J2SE 1.2</b> — Major release, called "Java 2". Added Swing, Collections Framework.                                                                                                      |
| 2000  | J2SE 1.3                                                                                                                                                                                   |
| 2002  | J2SE 1.4 — added <code>assert</code> , exception chaining, NIO, regex.                                                                                                                     |
| 2004  | <b>J2SE 5.0 (1.5)</b> — HUGE release: Generics, Enhanced for-loop, Autoboxing, Varargs, Enums, Annotations, Static Import.                                                                 |
| 2006  | Java SE 6 — Sun releases Java as <b>open source</b> (OpenJDK).                                                                                                                             |
| 2009  | <b>Oracle acquires Sun Microsystems</b> (and hence Java).                                                                                                                                  |
| 2011  | Java SE 7 — try-with-resources, diamond operator <code>&lt;&gt;</code> , switch on Strings.                                                                                                |
| 2014  | <b>Java SE 8</b> — Landmark release: Lambda Expressions, Stream API, Default methods, Optional, new Date/Time API.                                                                         |
| 2017  | Java SE 9 — Module System (Project Jigsaw), JShell (REPL).                                                                                                                                 |
| 2018  | Java SE 10 — <code>var</code> (local variable type inference). Oracle shifts to a <b>6-month release cycle</b> .                                                                           |
| 2018  | Java SE 11 — <b>LTS (Long Term Support)</b> release. <code>var</code> in lambda params, new HTTP Client, String methods ( <code>isBlank</code> , <code>strip</code> ).                     |
| 2021  | <b>Java SE 17 — LTS release</b> . Sealed Classes, Pattern Matching for <code>instanceof</code> , Records (finalized), new macOS rendering pipeline, strong encapsulation of JDK internals. |
| 2023+ | Java 21 (LTS) — Virtual Threads (Project Loom), Pattern Matching for switch, Record Patterns.                                                                                              |

### Why Java 8 and Java 17 specifically (as in this course)?

- **Java 8** is the most widely used version in the industry for **10+ years** — Streams, Lambdas, and Optional fundamentally changed how Java code is written. Most legacy enterprise systems (banks, insurance) still run on Java 8.
- **Java 17** is the current **LTS (Long Term Support)** standard adopted by new Spring Boot projects, modern microservices, and cloud-native applications (Records, Sealed Classes, pattern matching). Spring Boot 3.x **requires** Java 17+.

### **1.5 Interview Definition**

"Java is a statically-typed, object-oriented, platform-independent programming language that compiles to an intermediate bytecode executed by the Java Virtual Machine (JVM), providing automatic memory management, built-in multithreading, and strong security through its managed runtime."

### **1.6 Simple Definition**

Java is a programming language used to write code once and run it on any computer or device, without worrying about what operating system that device uses.

### **1.7 Technical Definition**

Java is a general-purpose, concurrent, strongly-typed, class-based, object-oriented programming language and computing platform, whose specification is maintained via the **Java Community Process (JCP)**, and whose reference implementation compiles source code (`.java`) into architecture-neutral **bytecode** (`.class`), interpreted or JIT-compiled at runtime by a **Java Virtual Machine (JVM)** conforming to the **Java Virtual Machine Specification (JVMS)**.

---

## 2. Real Life Analogy

| Domain     | Analogy                                                                                                                                                                                                                                                                                                                                          |
|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Bank       | A bank issues ATM cards ( <code>.class</code> bytecode) that work in <b>any</b> ATM machine of <b>any</b> bank, in <b>any</b> country (any JVM on any OS) — as long as the machine understands the card format (the JVM understands bytecode). You don't need a special card for each ATM brand.                                                 |
| Hospital   | A doctor's <b>prescription written in a universal drug-code format</b> can be read and dispensed by <b>any pharmacy in any city</b> , regardless of the pharmacy's local language — similar to bytecode being understood by any JVM regardless of the underlying OS.                                                                             |
| WhatsApp   | WhatsApp is written once by engineers, but the <b>same core logic</b> runs on Android, iOS, Windows, and Web — much like Java's WORA promise (though WhatsApp uses different native compiled code per platform, the <i>analogy</i> is: single source of truth, deployed everywhere).                                                             |
| Amazon     | Amazon's backend order-processing systems (many written in Java) run on thousands of heterogeneous servers across the world (different hardware/data centers) using the <b>same deployed .jar/.war file</b> — no need to recompile per data center.                                                                                              |
| Uber       | Uber's surge-pricing/matching engines need to run reliably 24/7, handling thousands of concurrent ride requests — Java's built-in <b>multithreading &amp; concurrency</b> model is why many such systems choose it.                                                                                                                              |
| Library    | The JVM is like a <b>universal translator/librarian</b> standing at every library (computer) in the world. No matter which language (OS) the library speaks, you hand the librarian the same <b>standard request slip</b> (bytecode), and the librarian (JVM) fetches the book (executes instructions) in a way native to that specific library. |
| Restaurant | Java's automatic Garbage Collector is like a restaurant's automatic busser who <b>clears unused plates (unused objects) from tables (memory/heap)</b> automatically, so waiters (developers) don't have to manually clean up after every customer — reducing human error (memory leaks).                                                         |
| College    | The <b>JDK</b> is like a <b>complete college</b> (has classrooms, library, hostel — i.e., compiler + tools + JRE). The <b>JRE</b> is like <b>just the hostel + canteen</b> (environment to "live" and run, no teaching tools). The <b>JVM</b> is like <b>just your dorm room</b> (the actual place where you, the program, execute).             |

## 3. Internal Working — How Java Code Actually Runs

### 3.1 The Two-Step Execution Model

Unlike pure **compiled languages** (C, C++ — compile directly to native machine code) and pure **interpreted languages** (Python, JavaScript — line-by-line interpretation), Java uses a **hybrid model**:

```
STEP 1 (Compile-Time):  
MyProgram.java ---[javac compiler]--> MyProgram.class (bytecode)
```

STEP 2 (Run-Time):

MyProgram.class ---[JVM: Class Loader + Interpreter/JIT]---> Machine Code ---> Output

## 3.2 Detailed Step-by-Step Internal Flow

### Step 1: Writing source code

You write human-readable code in a file `HelloWorld.java`.

### Step 2: Compilation (javac)

The `javac` (Java Compiler) tool:

- Checks syntax (semicolons, brackets, keywords).
- Checks semantics (type mismatches, undeclared variables).
- Converts `.java` → `.class` file containing **bytecode** — NOT native machine code, but an intermediate, platform-independent instruction set.

### Step 3: Class Loading

When you run `java HelloWorld`, the **Class Loader Subsystem** of the JVM:

1. **Loads** the `.class` file into memory (Method Area/Metaspace).

2. **Links** it:

- *Verification*: Bytecode Verifier checks the bytecode is valid and safe (no stack underflow, no illegal type casts) — this is Java's core **security** feature.
- *Preparation*: Allocates memory for static variables, sets default values.
- *Resolution*: Converts symbolic references to direct references.

3. **Initializes**: Executes static initializers and static blocks, assigns actual values to static variables.

### Step 4: Execution

The **Execution Engine** takes over:

- **Interpreter**: Reads bytecode line-by-line and executes it (slow, but starts immediately).
- **JIT (Just-In-Time) Compiler**: Detects "hot" code (methods called repeatedly) and compiles them directly into **native machine code**, caching it for reuse — dramatically speeding up execution on repeated calls.
- **Garbage Collector**: Runs in the background, reclaiming heap memory of objects no longer referenced.

## 3.3 Memory Changes During Execution (Preview — detailed in JVM Internals topic later)

```
STACK (per thread) HEAP (shared)
+-----+ +-----+
| main() frame | | |
| local vars | | Objects created via |
| return address | ---ref----> | 'new' keyword live here |
+-----+ | |
+-----+
METHOD AREA / METASPACE
+-----+
| Class metadata, static variables, constant pool, |
| method bytecode |
+-----+
```

- **Stack**: Stores method call frames, local (primitive) variables, and **references** to objects.
- **Heap**: Stores actual **objects** (instances created with `new`).
- **Metaspace** (Java 8+, replaced PermGen): Stores class metadata (structure of classes, not instances).

## 4. Diagrams

### 4.1 Java Platform Components — JDK vs JRE vs JVM

```
+-----+
| JDK |
| (Java Development Kit – for DEVELOPERS) |
| |
| +-----+ |
| | JRE | |
| | (Java Runtime Environment – to RUN programs) | |
| | |
| | +-----+ | |
| | | JVM | | |
| | | (Java Virtual Machine – EXECUTES bytecode)| | |
| | | - Class Loader | | |
| | | - Runtime Data Areas (Heap, Stack, etc.) | | |
| | | - Execution Engine (Interpreter + JIT) | | |
| | +-----+ | |
| | |
| | + Core Class Libraries (java.lang, java.util..) | |
| +-----+ |
| |
| + javac (compiler), javadoc, jar, jdb (debugger), jshell |
+-----+
```

### 4.2 Compilation + Execution Flow (Full Pipeline)

```
[HelloWorld.java]
|
| javac HelloWorld.java
v
[HelloWorld.class] <-- Bytecode (platform-independent)
|
| java HelloWorld
v
+-----+
| JVM |
| |
| 1. Class Loader --> loads HelloWorld.class |
| | |
| v |
| 2. Bytecode Verifier --> checks safety |
| | |
| v |
| 3. Execution Engine |
| - Interpreter (line by line) |
| - JIT Compiler (hot code -> native code) |
| | |
| v |
| 4. Native Machine Code |
+-----+
|
v
OS / Hardware executes native instructions
|
v
OUTPUT
```

### 4.3 WORA (Write Once, Run Anywhere) Diagram

```
+-----+
| HelloWorld.java |
+-----+
|
| javac (compile ONCE)
|
v
+-----+
| HelloWorld.class | <-- SAME bytecode file
+-----+
/ | \
/ | \
v v v
+-----+ +-----+ +-----+
| JVM Windows | | JVM Linux | | JVM MacOS |
+-----+ +-----+ +-----+
| | |
v v v
Runs correctly Runs correctly Runs correctly
```

## 5. Syntax — The First Java Program Explained Symbol by Symbol

```
// File name MUST match public class name: HelloWorld.java
public class HelloWorld {

    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

## Line-by-line / Keyword-by-keyword Explanation

| Element                                 | Meaning                                                                                                                                                                                                                                |
|-----------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>public</code> (on class)          | Access modifier — this class is visible/accessible from <b>any other class</b> , in any package.                                                                                                                                       |
| <code>class</code>                      | Keyword that declares a new <b>class</b> (a blueprint for objects).                                                                                                                                                                    |
| <code>HelloWorld</code>                 | Name of the class. <b>Must exactly match the file name</b> ( <code>HelloWorld.java</code> ) if the class is <code>public</code> .                                                                                                      |
| <code>{ }</code> (class body)           | Curly braces define the <b>scope/boundary</b> of the class — everything between belongs to <code>HelloWorld</code> .                                                                                                                   |
| <code>public</code> (on main)           | Access modifier — the <code>main</code> method must be <code>public</code> so the <b>JVM</b> (which is "outside" the class, technically launching it) can call it.                                                                     |
| <code>static</code>                     | Means this method belongs to the <b>class itself</b> , not to an instance/object. The JVM calls <code>main()</code> <b>without creating an object</b> of <code>HelloWorld</code> first — hence it <b>MUST</b> be <code>static</code> . |
| <code>void</code>                       | Return type — <code>main</code> returns <b>nothing</b> to the JVM.                                                                                                                                                                     |
| <code>main</code>                       | The special method name the JVM looks for as the <b>entry point</b> of any standalone Java application.                                                                                                                                |
| <code>(String[] args)</code>            | Parameter — an <b>array of Strings</b> representing command-line arguments passed when running the program (e.g., <code>java HelloWorld arg1 arg2</code> ).                                                                            |
| <code>String[] args</code> — why array? | Because you can pass <b>multiple</b> command-line arguments; the JVM stores each as a <code>String</code> inside this array.                                                                                                           |
| <code>{ }</code> (method body)          | Scope of the <code>main</code> method — all executable statements go here.                                                                                                                                                             |
| <code>System</code>                     | A <b>final class</b> in <code>java.lang</code> package, provides access to system resources (standard input/output/error streams, environment).                                                                                        |
| <code>.out</code>                       | A <b>static field</b> of <code>System</code> class, of type <code>PrintStream</code> , representing the <b>standard output stream</b> (usually the console).                                                                           |
| <code>.println(...)</code>              | A <b>method</b> of <code>PrintStream</code> that prints the given argument <b>and moves to a new line</b> afterward.                                                                                                                   |
| <code>"Hello, World!"</code>            | A <b>String literal</b> — the actual text to print.                                                                                                                                                                                    |
| <code>;</code>                          | Statement terminator — every executable statement in Java must end with a semicolon.                                                                                                                                                   |

### Why must `main` have exactly this signature?

The JVM specification hardcodes a search for:

```
public static void main(String[] args)
```

If you change **any** part (e.g., remove `static`, or use `int` return type instead of `void`, or lowercase `Main`), the JVM will throw:

```
Error: Main method not found in class HelloWorld, please define the main method as:
public static void main(String[] args)
```

Valid variations (all equivalent, accepted by JVM):

```
public static void main(String[] args)
public static void main(String args[])
public static void main(String... args) // varargs – Java 5+
static public void main(String[] args) // order of modifiers doesn't matter
final public static void main(String[] args) // 'final' is allowed too
```

## 6. "Every Method" for This Topic — Core Entry-Point APIs

Since "Introduction to Java" doesn't have a class of its own, we cover the **foundational APIs** every beginner touches immediately.

### 6.1 `System.out.println()` family

| Method                                                 | Purpose                           | Parameter               | Return Type                                     | Notes                                                                      |
|--------------------------------------------------------|-----------------------------------|-------------------------|-------------------------------------------------|----------------------------------------------------------------------------|
| <code>println()</code>                                 | Print empty line                  | none                    | <code>void</code>                               | Just moves to next line                                                    |
| <code>println(String x)</code>                         | Print + newline                   | <code>String</code>     | <code>void</code>                               | Most common                                                                |
| <code>println(int x) / println(double x) / etc.</code> | Print + newline for primitives    | primitive               | <code>void</code>                               | Overloaded for every primitive type                                        |
| <code>print(String x)</code>                           | Print, <b>no</b> newline          | <code>String</code>     | <code>void</code>                               | Cursor stays on same line                                                  |
| <code>printf(String format, Object... args)</code>     | Formatted print (like C's printf) | format string + varargs | <code>PrintStream</code> (itself, for chaining) | Uses format specifiers <code>%d</code> , <code>%s</code> , <code>%f</code> |

**Time/Space Complexity:**  $O(n)$  where  $n$  = length of string being printed; printing is I/O bound, not CPU bound — actual performance depends on console/stream buffering, not Big-O.

#### Common mistakes:

- Using `+` for many concatenations inside a loop before printing (creates many intermediate `String` objects) — better to build with `StringBuilder` first.
- Forgetting `System.out.flush()` is usually **not** needed for `println` (auto-flushes on newline for console), but matters for custom streams.

## 7. Code Examples

### 7.1 Basic Example

```
public class BasicExample {
public static void main(String[] args) {
System.out.println("Hello, World!"); // simplest possible Java program
}
}
```



## 7.2 Intermediate Example — Using Command-Line Arguments

```
public class GreetUser {
    public static void main(String[] args) {
        if (args.length == 0) {
            System.out.println("No name provided. Usage: java GreetUser <name>");
            return; // exits main early
        }
        String name = args[0]; // first command-line argument
        System.out.println("Hello, " + name + "! Welcome to Java.");
    }
}
```

**Run it:**

```
javac GreetUser.java
java GreetUser Soumya
```

**Output:** Hello, Soumya! Welcome to Java.

## 7.3 Advanced Example — Multiple Classes + Entry Point

```
// File: Application.java
public class Application {

    public static void main(String[] args) {
        System.out.println("Application starting...");
        Greeter greeter = new Greeter(); // object creation happens on heap
        greeter.greet("Production User");
        System.out.println("Application finished.");
    }
}

class Greeter { // NON-public class, same file allowed
    void greet(String name) {
        System.out.println("Hello, " + name + " – from a helper class!");
    }
}
```

**Rule:** Only ONE public class allowed per .java file, and the file name must match that public class's name. Additional **non-public** (package-private) classes can exist in the same file.

## 7.4 Real Company-Style Example — Environment Info Printer (Production style)

```
package com.company.diagnostics; // package declaration – real projects always use packages

/**
 * Utility class that prints JVM/runtime diagnostic information.
 * Used commonly in production startup logs to confirm environment.
 */
public final class RuntimeDiagnostics { // 'final' - prevents subclassing, utility class pattern

    private RuntimeDiagnostics() { // private constructor - prevents instantiation
        throw new AssertionError("Utility class - do not instantiate");
    }

    public static void printEnvironmentInfo() {
        System.out.println("=== Runtime Environment ===");
        System.out.println("Java Version : " + System.getProperty("java.version"));
        System.out.println("Java Vendor : " + System.getProperty("java.vendor"));
        System.out.println("OS Name : " + System.getProperty("os.name"));
        System.out.println("Available Processors : " + Runtime.getRuntime().availableProcessors());
        System.out.println("Max Memory (bytes) : " + Runtime.getRuntime().maxMemory());
    }

    public static void main(String[] args) {
        printEnvironmentInfo();
    }
}
```

### Why this is "production quality":

- Uses a **package** (real enterprise code is never in the default package).
- Utility class pattern (**final** class + **private** constructor) — prevents misuse.
- Uses **Javadoc comments** (**/\*\* ... \*/**) for documentation generation.
- Reads real JVM system properties instead of hardcoding values.

## 7.5 Bad Code vs Good Code Comparison

### ■ Bad (beginner mistakes):

```
public class prog1{
    public static void main(String a[]){
        int x=5;String s="value is "+x;System.out.println(s);}}
```

Problems: No package, poor naming (`prog1`, not descriptive), no formatting/indentation, everything on fewer lines, unclear variable names.

### ■ Good (industry standard):

```
package com.company.demo;

public class NumberPrinter {

    public static void main(String[] args) {
        int value = 5;
        String message = "Value is " + value;
        System.out.println(message);
    }
}
```

Improvements: Proper package, PascalCase class name describing intent, camelCase variables, one statement per line, consistent indentation (4 spaces), space around operators.

## 8. Dry Run

### Program:

```
public class DryRunExample {  
    public static void main(String[] args) {  
        int a = 10;  
        int b = 20;  
        int sum = a + b;  
        System.out.println("Sum is: " + sum);  
    }  
}
```

### Step-by-step Dry Run

| Step | Code Line                             | Stack Frame (main)                                                     | Heap                                                                                                                                                                                                             | Output                        |
|------|---------------------------------------|------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------|
| 1    | JVM starts, loads DryRunExample.class | main() frame created, pushed onto Thread Stack                         | (empty, no objects yet)                                                                                                                                                                                          | —                             |
| 2    | int a = 10;                           | a = 10                                                                 | —                                                                                                                                                                                                                | —                             |
| 3    | int b = 20;                           | a = 10, b = 20                                                         | —                                                                                                                                                                                                                | —                             |
| 4    | int sum = a + b;                      | a = 10, b = 20, sum = 30                                               | —                                                                                                                                                                                                                | —                             |
| 5    | System.out.println("Sum is: " + sum); | (String concatenation creates a temporary String object: "Sum is: 30") | Temporary String object "Sum is: 30" created on Heap (String Pool, since compile-time constant folding does NOT apply here as sum is a variable — this is a runtime-created String via StringBuilder internally) | Sum is: 30 printed to console |
| 6    | main() returns                        | main() frame popped off stack                                          | Heap objects become eligible for GC                                                                                                                                                                              | Program ends                  |

### What really happens internally at Step 5 (Java 8+ compiler optimization):

The compiler transforms "Sum is: " + sum into (conceptually):

```
new StringBuilder().append("Sum is: ").append(sum).toString();
```

This is because + on Strings is **not a real operator** — Java has no operator overloading; the compiler secretly uses `StringBuilder` for concatenation involving variables.

## 9. Edge Cases

| Edge Case                                                     | What Happens                                                                                               | Why                                                                 |
|---------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| Class name doesn't match file name (for public class)         | <b>Compile-time error:</b> class X is public, should be declared in a file named X.java                    | Java enforces 1 public class = 1 matching file name rule            |
| No main method at all                                         | <b>Runtime error:</b> Error: Main method not found in class X                                              | JVM specifically searches for exact main signature at launch        |
| main method is not static                                     | <b>Runtime error:</b> Same "Main method not found" error                                                   | JVM must call main without creating an instance                     |
| Multiple classes with main in the same project                | No error — you specify which class to run: java ClassNameWithMain                                          | Each class can independently have its own main                      |
| Running java HelloWorld.class (with .class extension)         | <b>Error:</b> Could not find or load main class HelloWorld.class                                           | The java launcher expects the <b>class name</b> , not the file name |
| Command-line arguments not passed but args[0] accessed        | ArrayIndexOutOfBoundsException at runtime                                                                  | args array is empty (length 0), not null                            |
| Two public classes in one file                                | <b>Compile-time error:</b> class Y is public, should be declared in a file named Y.java                    | Java allows only ONE public top-level class per file                |
| File saved with wrong extension (e.g., .txt)                  | javac won't recognize/compile it properly unless extension is exactly .java                                | Compiler expects .java extension                                    |
| Very large/huge print statements or infinite loop with prints | Can cause extremely slow I/O / console flooding, potential OutOfMemoryError if building huge strings first | I/O bottleneck and/or heap exhaustion                               |

## 10. Internal Interview Questions

### Easy

1 **Q: What is Java and why is it called platform-independent?**

A: Java is a high-level, object-oriented language that compiles to bytecode, which is executed by a JVM present on every platform, making the same .class file runnable everywhere without recompilation.

2 **Q: What does WORA stand for?**

A: Write Once, Run Anywhere.

3 **Q: Who created Java and when?**

A: James Gosling and his team at Sun Microsystems, in 1995 (project started 1991 as "Green Project", originally named "Oak").

### Medium

1 **Q: Is Java 100% platform-independent? Explain.**

A: The **language/bytecode** is platform-independent, but the **JVM itself** is platform-dependent (a different native JVM binary is needed per OS/architecture). Also, native code called via JNI, or OS-specific file path separators, can break pure platform independence in practice.

2 **Q: Why is Java called both a compiled AND an interpreted language?**

A: Source code (.java) is first **compiled** into bytecode (.class) by javac. Then, the JVM **interprets** that bytecode (or JIT-compiles hot paths into native code) at runtime. It uses both techniques — hence a "hybrid" model.

3 **Q: What is the difference between JDK, JRE, and JVM?**

A: JVM executes bytecode; JRE = JVM + core libraries (to *run* Java programs); JDK = JRE + development tools like javac, javadoc, debugger (to *develop* Java programs).

## Hard

1 **Q: If Java has a JVM overhead (interpretation/JIT warm-up), why is it still used for high-performance systems (e.g., trading platforms)?**

A: Because after **JIT warm-up**, hot code paths run as native machine code — comparable to C/C++ speed for long-running processes. Additionally, JVMs like HotSpot use advanced optimizations (inlining, escape analysis) that can sometimes outperform naively written C++ for long-running server applications. Short-lived CLI tools suffer more from JVM startup cost; long-running servers benefit from JIT.

2 **Q: Explain what happens internally between typing `java HelloWorld` in the terminal and seeing output on the screen.**

A: (Expected full answer covering: OS launches java launcher → JVM instance created → Class Loader loads HelloWorld.class from classpath → Bytecode Verifier validates → Execution Engine (interpreter/JIT) executes main() → System calls write to stdout buffer → OS displays to terminal.)

## Company-Level / Tricky

1 **(Amazon-style) Q: Can you have a Java program with NO `main` method that still runs successfully via `java` command?**

A: Yes — using **static initializer blocks** was historically possible for some early JVMs to trigger execution but modern JVMs still require `main` for the class you explicitly launch — HOWEVER, if the launched class has a **static block** but no `main`, the JVM will still throw "Main method not found" **after** running the static block, because it needs `main` to actually be invoked (static blocks run during class loading regardless, but `main` invocation is what the JVM specifically waits for). This is a good trick question testing understanding of class loading vs execution phases.

2 **(Cross-question) Q: You said Java is object-oriented. Then why does `main` need to be static?**

A: Because when the JVM starts, **no object of the class exists yet**. Making `main` static allows the JVM to invoke it directly on the class itself, without first needing an instance — solving the "chicken-and-egg" problem of needing an object to call a method, when no object exists yet at program start.

---

## 11. Coding Questions

### Easy

**Q1: Write a Java program to print your name and the current Java version.**

```
public class NameAndVersion {
    public static void main(String[] args) {
        System.out.println("Name: Soumya");
    }
}
```

```
System.out.println("Java Version: " + System.getProperty("java.version"));
}
}
```

**Q2: Write a program that prints all command-line arguments passed to it, one per line.**

```
public class PrintArgs {
    public static void main(String[] args) {
        for (String arg : args) {
            System.out.println(arg);
        }
    }
}
```

- Brute force / Only approach: Iterate with enhanced for-loop. **Time Complexity:**  $O(n)$  where  $n$  = number of arguments.

## Medium

**Q3: Write a program that prints "Even number of arguments" or "Odd number of arguments" based on args.length, and if zero arguments, print "No arguments provided".**

```
public class ArgCountChecker {
    public static void main(String[] args) {
        if (args.length == 0) {
            System.out.println("No arguments provided");
        } else if (args.length % 2 == 0) {
            System.out.println("Even number of arguments");
        } else {
            System.out.println("Odd number of arguments");
        }
    }
}
```

## Hard / Interview Level (Company Tagged: TCS/Infosys screening rounds)

**Q4: Without using any library method, write a program to reverse each command-line argument string and print it.**

```
public class ReverseArgs {
    public static void main(String[] args) {
        for (String arg : args) {
            char[] chars = arg.toCharArray();
            StringBuilder reversed = new StringBuilder();
            for (int i = chars.length - 1; i >= 0; i--) { // Brute force: O(n) per string
                reversed.append(chars[i]);
            }
            System.out.println(reversed.toString());
        }
    }
}
```

- **Brute Force:** Manual loop from end to start —  $O(n)$  time,  $O(n)$  space per string (as shown).
- **Better/Optimal:** `new StringBuilder(arg).reverse().toString()` — uses built-in optimized reverse (still  $O(n)$ , but far less code, uses internal array swap in-place).

```
System.out.println(new StringBuilder(arg).reverse());
```

## 12. Common Mistakes

### *Beginner Mistakes*

- 1 Naming the file differently from the public class (`Test.java` containing `public class Main`) → compile error.
- 2 Forgetting `static` on `main` → runtime error, program compiles but won't launch.
- 3 Using `String args[]` and being confused why it's different from `String[] args` (it's **not** different — both are valid, just different declaration styles).
- 4 Assuming `args[0]` always exists → `ArrayIndexOutOfBoundsException`.
- 5 Confusing `print` vs `println` — forgetting `print` doesn't add a newline, causing jumbled console output.
- 6 Writing everything in the default package (no `package` statement) for real projects — this breaks JAR packaging conventions and causes classpath conflicts later.

### *Experienced Developer Mistakes*

- 1 Ignoring JVM startup/warm-up costs when benchmarking Java code with **too few iterations** (JIT hasn't kicked in yet — leads to wrong performance conclusions).
- 2 Forgetting that `System.out` is a `PrintStream`, which is **not exception-safe silent** — I/O errors on `println` are swallowed unless you check `System.out.checkError()`.
- 3 Hardcoding OS-specific paths (`C:\...\`) instead of using `File.separator` / `Path` APIs, breaking WORA benefits in real deployments.
- 4 Using `System.exit()` improperly inside libraries (should only be used in `main`-level application shutdown logic, never inside reusable library code, as it kills the entire JVM abruptly).

### *Debugging Process for "Main method not found" error:*

- 1 Check method signature exactly matches `public static void main(String[] args)`.
  - 2 Check for typos: `Main` vs `main` (case-sensitive).
  - 3 Check you're running the correct class name matching your compiled `.class` file.
  - 4 Check the classpath includes the directory containing your `.class` file.
- 

## 13. Best Practices

- 1 **Always use packages** — never leave classes in the default (unnamed) package for real projects.
- 2 **One public class per file**, file name = class name exactly (case-sensitive).
- 3 Follow **naming conventions**: `PascalCase` for classes, `camelCase` for methods/variables, `UPPER_SNAKE_CASE` for constants.
- 4 Use **meaningful names** — `calculateInterest()` not `calc()`.

- 5 Add **Javadoc comments** (`/** ... */`) on public classes/methods for maintainability.
  - 6 Keep `main` methods **thin** — delegate real logic to other methods/classes (Single Responsibility Principle).
  - 7 Avoid `System.out.println` for real application logging — use a **logging framework** (SLF4J/Logback) in production instead (covered later).
  - 8 Consistent indentation (typically 4 spaces), and consistent brace style (Java community standard: opening brace on same line).
- 

## 14. Production Usage

- **Backend servers:** Almost every large bank (JPMorgan, Goldman Sachs), airline booking system, and e-commerce backend (Amazon, Flipkart) has core services written in Java due to reliability, mature tooling, and long-term support.
  - **Android development:** Historically, Android apps were written in Java (now largely Kotlin, but Java remains fully supported and huge legacy codebases exist).
  - **Big Data:** Hadoop, Spark's core engines are written in/for the JVM (Scala/Java).
  - **Enterprise middleware:** JBoss, WebLogic, WebSphere application servers run Java EE / Jakarta EE applications.
  - **Every "Hello World"** shown here is conceptually how **every microservice's entry point** (e.g., a Spring Boot `@SpringBootApplication` class) still boils down to a `main` method under the hood.
- 

## 15. Performance

- **Startup time:** JVM has a warm-up cost (class loading + JIT warm-up) — historically a weakness for CLI tools/serverless "cold starts", improved significantly by **AppCDS (Application Class Data Sharing)** and **Project CRaC** in modern JVMs, and by **GraalVM Native Image** for near-instant startup.
  - **Throughput after warm-up:** Comparable to natively compiled languages for long-running server processes, due to adaptive JIT optimization.
  - **Memory usage:** Higher baseline memory footprint compared to C/C++ (JVM overhead, object headers, GC bookkeeping), but this trade-off buys memory safety and developer productivity.
  - **Trade-off summary:** Java trades a small amount of raw peak performance and memory efficiency for massive gains in **developer productivity, safety, portability, and maintainability** — which is why it dominates enterprise backend systems.
- 

## 16. Frequently Asked Interview Questions (50+ with Detailed Answers)



**1 What is Java?**

High-level, object-oriented, platform-independent programming language that compiles to bytecode executed by the JVM.

**2 Why is Java platform-independent?**

Because compiled bytecode (`.class`) is not tied to any specific OS/hardware — any machine with a compatible JVM can execute it.

**3 What is the JVM?**

Java Virtual Machine — an abstract computing machine that provides a runtime environment to execute Java bytecode.

**4 What is JIT compilation?**

Just-In-Time compilation — the JVM compiles frequently executed ("hot") bytecode into native machine code at runtime for faster execution.

**5 Difference between JDK, JRE, JVM?**

JDK = development kit (JRE + compiler + tools). JRE = runtime (JVM + libraries). JVM = the engine that executes bytecode.

**6 What does WORA mean?**

Write Once, Run Anywhere — compile once, run on any platform with a JVM.

**7 Is Java purely object-oriented?**

No — Java has primitive types (`int`, `char`, etc.) which are not objects, so it's not 100% pure OOP (unlike languages like Smalltalk).

**8 What is bytecode?**

Platform-independent intermediate instructions generated by `javac`, stored in `.class` files, understood by the JVM.

**9 Why was Java originally called "Oak"?**

Named after an oak tree outside James Gosling's office; renamed to "Java" due to a trademark conflict, and Java was chosen inspired by coffee.

**10 What is the Green Project?**

Sun Microsystems' 1991 initiative to build software for embedded consumer electronics, which led to the creation of Java.

**11 What year was Java officially released?**

1995.

**12 Who is the father of Java?**

James Gosling.

**13 Which company currently owns Java?**

Oracle Corporation (after acquiring Sun Microsystems in 2009/2010).

**14 What is OpenJDK?**

The free, open-source reference implementation of the Java Platform, Standard Edition.

**15 Why must the `main` method be `static`?**

Because the JVM invokes it without creating an instance of the class first.

**16 Why must `main` be `public`?**

So the JVM (external to the class) can access and invoke it.

**17 Why does `main` return `void`?**

Because the JVM doesn't expect a direct return value from `main`; program exit status is instead controlled via `System.exit(int status)`.

**18 What happens if the class name and file name don't match (for a public class)?**

Compile-time error.

**19 Can a Java file have multiple classes?**

Yes, but only one can be `public`, and it must match the file name.

**20 What is the entry point of a Java application?**

The `main` method: `public static void main(String[] args)`.

**21 Can we overload the `main` method?**

Yes — you can have multiple `main` methods with different signatures, but the JVM only calls the exact `public static void main(String[] args)` version as the entry point; others are just regular overloaded methods you'd call manually.

**22 What is `args` in `main(String[] args)`?**

An array holding command-line arguments passed when running the program.

**23 What happens if you access `args[0]` when no arguments were passed?**

`ArrayIndexOutOfBoundsException` is thrown at runtime.

**24 Difference between `System.out.print` and `System.out.println`?**

`print` doesn't add a newline; `println` does.

**25 What is `System.out`?**

A static field of type `PrintStream` in the `System` class, representing standard output.

**26 Is Java case-sensitive?**

Yes — `Main` and `main` are different identifiers.

**27 What is the difference between a compiler and an interpreter, and how does Java use both?**

A compiler translates the whole program before execution (fast execution, upfront translation); an interpreter translates and executes line by line (slower, no upfront compile step). Java's `javac` compiles source to bytecode; the JVM interprets/JIT-compiles that bytecode at runtime — a hybrid of both models.

**28 Why does Java use an intermediate bytecode instead of compiling directly to native machine code?**

To achieve platform independence — the same bytecode file can run on any OS/architecture with a compatible JVM, unlike native machine code which is CPU/OS specific.

**29 What is the Bytecode Verifier?**

A JVM component that checks loaded bytecode for illegal operations (e.g., invalid type casts, stack overflows) before execution, ensuring security and stability.

**30 Name the core JDK tools.**

`javac` (compiler), `java` (launcher), `javadoc` (documentation generator), `jar` (archiver), `jdb` (debugger), `jshell` (REPL, since Java 9).

**31 What is JShell?**

An interactive REPL (Read-Eval-Print-Loop) tool introduced in Java 9 for quickly testing Java code snippets without creating full classes/files.

**32 What's new in Java 8 relevant to "introduction" level understanding?**

Lambda expressions, Stream API, method references, default/static interface methods, new Date/Time API, Optional (covered in depth in the Java 8 Features topic).

**33 What's new in Java 17 relevant to "introduction" level understanding?**

Sealed classes, Records finalized, Pattern matching for `instanceof`, strong encapsulation of internal JDK APIs.

**34 Why is Java 17 an LTS (Long Term Support) release?**

Oracle designates certain versions (8, 11, 17, 21) for extended support (patches/security updates for years), making them the standard choice for production enterprise systems, unlike non-LTS versions supported for only 6 months.

**35 What is the difference between Java SE, Java EE (Jakarta EE), and Java ME?**

Java SE = Standard Edition (core language/APIs); Java EE/Jakarta EE = Enterprise Edition (web/enterprise APIs like Servlets, EJB); Java ME = Micro Edition (embedded/mobile devices, largely legacy now).

**36 Can Java run without installing a JDK, only a JRE?**

Yes, to just *run* compiled `.class/.jar` files you only need a JRE; you need the JDK only to *compile* (develop) Java source code. (Note: since Java 11, Oracle no longer ships a separate JRE download — you install a full JDK even just to run programs.)

**37 What replaced the separate JRE download starting Java 11?**

Oracle stopped shipping standalone JRE packages; developers now use the full JDK (or create custom minimal runtimes using `jlink`) even to just run applications.

**38 What is `jlink`?**

A tool (Java 9+) that creates a custom, minimal runtime image containing only the modules your application needs, reducing deployment footprint.

**39 Is Java free to use?**

OpenJDK builds are free and open-source (GPL). Oracle's JDK has had shifting commercial licensing terms for production use in certain versions — always check the specific version's license.

**40 What does "class-based" mean in Java's definition?**

Objects are created from class blueprints (as opposed to prototype-based languages like JavaScript, where objects can be created and modified without an underlying class definition).

**41 Explain why Java's memory management is called "automatic".**

The Garbage Collector automatically identifies and reclaims memory occupied by objects no longer reachable/referenced, without explicit `free()/delete` calls from the developer.

**42 What are the four pillars of OOP Java supports? (preview, detailed in OOP topic)**

Encapsulation, Inheritance, Polymorphism, Abstraction.

**43 Give one real-world reason a bank would choose Java over Python for its core transaction engine.**

Java's static typing catches many errors at compile-time (safer for financial correctness), and the JVM's mature JIT and multithreading model handle very high-throughput, low-latency transaction processing better than typical Python (GIL-limited) implementations.

**44 What is the difference between `javac` and `java` commands?**

`javac` compiles `.java` source into `.class` bytecode; `java` launches the JVM to execute a compiled class (or JAR).

**45 Can you run a `.java` file directly (without a separate compile step) since a certain Java version?**

Yes — since **Java 11**, you can run `java HelloWorld.java` directly for single-file source-code programs; the JVM compiles it in-memory and runs it immediately (useful for quick scripts).

**46 What is the classpath?**

The list of locations (directories/JARs) the JVM searches to find compiled classes needed at runtime.

**47 What does "strongly typed" mean for Java?**

Every variable and expression has a type known (checked) at compile-time; you cannot assign incompatible types without explicit casting, reducing a class of runtime errors.

**48 Why can't `main`'s return type be `int` in standard Java (unlike C's `int main()`)?**

By JVM specification, the entry point signature is fixed as `void`; to return an exit status to the OS, you must explicitly call `System.exit(int status)`.

**49 What is the significance of the year 2004 (J2SE 5.0) in Java's history?**

Introduced Generics, enhanced for-loop, autoboxing/unboxing, varargs, enums, and annotations — arguably the most feature-rich release before Java 8.

**50 Why do modern frameworks like Spring Boot still require Java 17+ for their latest major versions?**

They leverage newer language features (Records, sealed classes, pattern matching) and performance/security improvements only available from Java 17 onward, and Java 17 (being LTS) offers long-term production stability guarantees.

**51 What is meant by "Java is secure by design"?**

Features like the Bytecode Verifier, absence of explicit pointers, automatic memory management, and (historically) the Security Manager/sandboxing model collectively reduce common vulnerability classes (buffer overflows, memory corruption) compared to unmanaged languages.

**52 What is the significance of "Java is robust"?**

Strong compile-time type checking, automatic garbage collection, and comprehensive exception handling collectively reduce runtime crashes compared to languages requiring manual memory/error management.

---

## 17. Revision Notes (One-Page Cheat Sheet)

### JAVA – QUICK REVISION SHEET

=====

WHAT: High-level, OOP, platform-independent language -> compiles to bytecode -> run by JVM

WHY CREATED: Green Project (1991) for embedded consumer electronics needing hardware independence

ORIGINAL NAME: Oak -> renamed Java (1995)

CREATOR: James Gosling (Sun Microsystems)

CURRENT OWNER: Oracle (since 2009/2010 acquisition)

KEY PRINCIPLE: WORA - Write Once, Run Anywhere

JDK = JRE + Development Tools (javac, javadoc, jar, jshell)

JRE = JVM + Core Libraries (to RUN programs)

JVM = Executes bytecode (Class Loader + Runtime Data Areas + Execution Engine)

#### EXECUTION FLOW:

.java --(javac)--> .class (bytecode) --(JVM: load/verify/execute)--> native machine code --> output

#### MAIN METHOD (must match EXACTLY):

```
public static void main(String[] args)
```

- public: JVM (outside class) can call it

- static: called without object creation

- void: no return value to JVM

- String[] args: command-line arguments

#### KEY TIMELINE:

1995 - Java 1.0 2004 - J2SE 5 (Generics) 2014 - Java 8 (Lambdas/Streams)

2006 - Open sourced 2011 - Java 7 (try-w-res) 2021 - Java 17 LTS (Records/Sealed)

#### WHY JAVA 8 & 17 MATTER MOST TODAY:

Java 8 -> most widely deployed legacy/enterprise version (Streams/Lambdas)

Java 17 -> current LTS standard; required by Spring Boot 3+

## 18. Homework

- 1 Install JDK 17 on your machine and verify with `java -version` and `javac -version`.
- 2 Write and run a `HelloWorld.java` program that prints your full name, today's date (hardcoded as text for now), and your favorite programming language.
- 3 Modify the program to accept your name as a command-line argument instead of hardcoding it, and print a greeting.
- 4 Deliberately introduce each of the 5 edge-case errors listed in Section 9 above, one at a time, and observe/write down the exact compiler/runtime error messages.
- 5 Research and write 5 sentences (in your own words) on why Java's automatic garbage collection is considered safer than C++'s manual memory management.
- 6 Find and note the exact Java version and vendor (e.g., Oracle, Eclipse Temurin, Amazon Corretto) installed on your machine using `System.getProperty("java.vendor")`.

## 19. Mini Project — "System Info Reporter" CLI Tool

**Goal:** Apply everything learned in this topic (class structure, `main` method, command-line arguments, `System` class, packages) to build a small, real, runnable CLI tool.

### Requirements:

- Package: `com.jobsradar.diagnostics`
- Accepts an optional command-line argument: `--verbose`
- Without `--verbose`: prints Java version and OS name only.
- With `--verbose`: also prints vendor, number of available processors, and max JVM memory.

```
package com.jobsradar.diagnostics;

/**
 * Mini Project: System Info Reporter
 * Demonstrates: packages, main method, command-line args, System class usage.
 */
public class SystemInfoReporter {

    public static void main(String[] args) {
        boolean verbose = containsVerboseFlag(args);

        System.out.println("Java Version : " + System.getProperty("java.version"));
        System.out.println("OS Name : " + System.getProperty("os.name"));

        if (verbose) {
            System.out.println("Java Vendor : " + System.getProperty("java.vendor"));
            System.out.println("Processors : " + Runtime.getRuntime().availableProcessors());
            System.out.println("Max Memory : " + Runtime.getRuntime().maxMemory() + " bytes");
        }

        private static boolean containsVerboseFlag(String[] args) {
            for (String arg : args) {
                if (arg.equals("--verbose")) {
                    return true;
                }
            }
            return false;
        }
    }
}
```

**Why this project matters:** It's the smallest possible "real" program — it has a package, a clear responsibility, handles optional CLI input safely (no `ArrayIndexOutOfBoundsException`), and separates logic into a helper method rather than crowding `main`.

## 20. Final Summary

- Java is a **class-based, object-oriented, platform-independent** language created in 1995 by James Gosling's team at Sun Microsystems, born out of the need for hardware-independent code for embedded consumer devices (the "Green Project").
- It achieves **platform independence (WORA)** by compiling source code into intermediate **bytecode**, executed by a **JVM** — a hybrid of compilation and interpretation, further optimized at runtime by the **JIT compiler**.

- **JDK**  $\supset$  **JRE**  $\supset$  **JVM** — JDK is for developers (has compiler), JRE is for running programs, JVM is the actual execution engine.
- The **entry point** of every standalone Java program is the exact signature `public static void main(String[] args)` — each keyword exists for a specific, JVM-mandated reason.
- Java's design goals — **simplicity, robustness, security, portability, multithreading, and automatic memory management** — are why it remains a dominant language for enterprise, backend, and Android systems even 30 years later.
- **Java 8** (Streams/Lambdas) and **Java 17** (Records/Sealed classes, current LTS) are the two most important versions to master for modern industry work.
- Things to remember forever: the compile $\rightarrow$ bytecode $\rightarrow$ JVM execution pipeline, why `main` must be `public static void`, and the JDK/JRE/JVM containment relationship — these fundamentals recur in every advanced Java/Spring Boot topic ahead.

---

(End of Topic 1: Introduction to Java.)

---

## TOPIC 2: JDK, JRE, JVM

---

### 1. Introduction

#### 1.1 What is it?

**JDK (Java Development Kit)**, **JRE (Java Runtime Environment)**, and **JVM (Java Virtual Machine)** are the three foundational layers of the Java platform. Each serves a distinct purpose, and they are **nested inside each other**:

JDK  $\supset$  JRE  $\supset$  JVM

- **JVM** — the engine that actually **executes** Java bytecode.
- **JRE** — the JVM **plus** the core class libraries (`java.lang`, `java.util`, etc.) needed to **run** a compiled Java program.
- **JDK** — the JRE **plus** development tools (`javac`, `javadoc`, `jar`, `jshell`, debugger) needed to **write and compile** Java programs.

#### 1.2 Why was this separation introduced?

Sun Microsystems designed Java with a clear separation of concerns:

| Need                                                                   | Who needs it | Component |
|------------------------------------------------------------------------|--------------|-----------|
| Just running a Java app (e.g., an end-user opening a Java desktop app) | End users    | JRE only  |

| Need                                                                     | Who needs it             | Component |
|--------------------------------------------------------------------------|--------------------------|-----------|
| Writing, compiling, and packaging Java programs                          | Developers               | Full JDK  |
| Actually interpreting/executing bytecode, regardless of who's running it | Everyone (transparently) | JVM       |

This separation meant:

- **End users** didn't need to download a heavy toolkit (compiler, debugger) — just enough to *run* apps (JRE).
- **Developers** needed the complete toolchain (JDK).
- The **JVM specification** could be implemented independently by different vendors (Oracle HotSpot, Eclipse OpenJ9, Azul Zing, GraalVM) while still guaranteeing bytecode compatibility — enabling competition and innovation in JVM implementations without breaking the "write once, run anywhere" promise.

**Important update (Java 11+):** Oracle **stopped shipping a separate standalone JRE download** starting with Java 11. Today, you install the **JDK** even if you only intend to run Java programs. The conceptual JRE (runtime libraries + JVM) still exists **inside** the JDK — it's just no longer distributed as a separate lightweight package. You can still build a *custom* minimal runtime using `jlink` (Java 9+), which packages only the modules your app actually needs.

### 1.3 Real-world problem it solves

- **Problem:** Without a clear boundary between "build tools" and "runtime," every deployment machine would need the full heavyweight compiler/toolchain just to run software — wasteful for production servers, mobile devices, and embedded systems.
- **Solution:** JDK/JRE/JVM separation lets **production servers** ship only what's needed to *run* code (historically just JRE; today a `jlink`-trimmed custom runtime), while **developer workstations and CI/CD build servers** carry the full JDK.

### 1.4 Historical Background

| Version         | Milestone                                                                                                                                                                     |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| JDK 1.0 (1996)  | First public release — bundled compiler + basic runtime, no formal JDK/JRE split marketing yet.                                                                               |
| JDK 1.2 (1998)  | Sun officially began using "JRE" terminology to describe the separately distributable runtime-only package.                                                                   |
| J2SE 5.0 (2004) | JVM specification matured significantly (JSR 924).                                                                                                                            |
| Java 6 (2006)   | Sun open-sourced the JDK as <b>OpenJDK</b> under GPLv2.                                                                                                                       |
| Java 9 (2017)   | <b>Project Jigsaw</b> (Module System) introduced <code>jlink</code> , enabling custom minimal runtime images.                                                                 |
| Java 11 (2018)  | Oracle <b>stopped providing a separate JRE installer</b> ; only full JDK builds are distributed from Oracle/OpenJDK vendors from this point forward.                          |
| Java 17 (2021)  | Current LTS; JDK, JRE (conceptually embedded), and JVM (HotSpot by default) all significantly hardened for security and performance (improved G1 GC, better startup via CDS). |



## 1.5 Interview Definition

"The JVM is the abstract runtime engine that executes Java bytecode; the JRE bundles the JVM with the standard class libraries required to run compiled applications; the JDK is the complete development environment, bundling the JRE together with the compiler (`javac`) and other tools needed to write, build, and debug Java source code."

## 1.6 Simple Definition

JVM runs your program. JRE is what you need installed to run a Java program. JDK is what you need installed to write and build a Java program.

## 1.7 Technical Definition

The **JVM** is a concrete implementation of the abstract computing machine defined by the **Java Virtual Machine Specification (JVMS)**, providing class loading, bytecode verification, a runtime data area model (Heap, Stack, Method Area/Metaspace, PC Register, Native Method Stack), and an execution engine (interpreter + JIT compiler + garbage collector). The **JRE** is the JVM implementation packaged together with the **Java Class Library (JCL)** — the standard API (`java.lang.*`, `java.util.*`, `java.io.*`, etc.). The **JDK** is a superset distribution bundling the JRE with the **Software Development Kit (SDK)** tools: `javac`, `javadoc`, `javap`, `jar`, `jdb`, `jshell`, `jlink`, `jmod`, and others, all governed by the **Java Community Process (JCP)**.

---

## 2. Real Life Analogy

| Domain     | Analogy                                                                                                                                                                                                                                                                                                                                                                                                                           |
|------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Bank       | JVM = the bank's core vault-processing machine that actually moves money (executes the transaction). JRE = the whole bank branch (vault machine + tellers + standard forms/services) that a <b>customer</b> interacts with to get things done. JDK = the bank's <b>head office + engineering team</b> — includes everything a branch has, PLUS the tools to design new financial products, audit systems, and build new services. |
| Hospital   | JVM = the operating theatre where the actual surgery (execution) happens. JRE = the entire ward (theatre + nurses + medicines) a <b>patient</b> needs to receive treatment (run). JDK = the <b>medical college</b> — has the ward/theatre PLUS training tools, research labs, and equipment to train new doctors (develop new "treatments"/programs).                                                                             |
| Restaurant | JVM = the kitchen where food is actually cooked (executed). JRE = the kitchen + dining area — enough for a <b>customer</b> to be served a meal (run the app). JDK = the entire restaurant business including the <b>recipe R&amp;D kitchen + chef training academy</b> (develop new recipes/programs).                                                                                                                            |
| Library    | JVM = the librarian who fetches and reads a specific book's content aloud (executes instructions). JRE = librarian + entire reading room + catalog system, enough for a <b>reader</b> (end user) to consume books (run programs). JDK = the librarian + reading room + <b>the entire publishing house</b> (tools to write, edit, and print new books — i.e., write and compile new programs).                                     |
| College    | JDK = the complete college campus (classrooms + hostel + library + labs). JRE = just the hostel + canteen (a place to "live" and consume, no teaching tools). JVM = your specific dorm room where you actually stay and do your work (execution happens).                                                                                                                                                                         |
| Amazon     | Amazon's production servers running a compiled Java microservice only need the <b>runtime</b> (conceptually the JRE, i.e., JVM + libraries) to serve customer traffic. Amazon's internal <b>build/CI servers</b> and developer laptops need the <b>full JDK</b> to compile and package new service versions before they're deployed.                                                                                              |
| Uber       | The JVM instance running inside each Uber backend server container is like the <b>engine</b> of a car — it's what actually makes the car (application) move (execute). The JRE is the whole car (engine + dashboard + controls) a <b>driver</b> (production system) needs to operate. The JDK is the <b>car factory</b> — everything needed to design and manufacture new cars (build new application versions).                  |

## 3. Internal Working

### 3.1 What Exactly Is Inside Each Layer

```
JDK (Java Development Kit)
■
■■■ Development Tools
■ ■■■ javac (compiler: .java -> .class)
■ ■■■ javadoc (generates HTML documentation from comments)
■ ■■■ jar (packages .class files into .jar archives)
■ ■■■ javap (disassembler - inspects bytecode)
■ ■■■ jdb (command-line debugger)
■ ■■■ jshell (REPL, Java 9+)
■ ■■■ jlink (creates custom minimal runtime images, Java 9+)
■ ■■■ jmod (creates/inspects .jmod module files, Java 9+)
■ ■■■ jconsole/jvisualvm (monitoring/profiling tools)
■
■■■ JRE (Java Runtime Environment)
■
■■■ Java Class Library (JCL)
■ ■■■ java.lang.* (core: Object, String, Math, Thread, etc.)
■ ■■■ java.util.* (Collections, Date/Time, etc.)
■ ■■■ java.io.* / java.nio.* (I/O)
■ ■■■ java.net.* (networking)
■ ■■■ ... hundreds of packages
■
■■■ JVM (Java Virtual Machine)
■
■■■ Class Loader Subsystem
■ ■■■ Bootstrap Class Loader
■ ■■■ Platform/Extension Class Loader
■ ■■■ Application/System Class Loader
■
■■■ Runtime Data Areas
■ ■■■ Method Area / Metaspace (shared across all threads)
■ ■■■ Heap (shared across all threads)
■ ■■■ Stack (per-thread)
■ ■■■ PC Register (per-thread)
■ ■■■ Native Method Stack (per-thread)
■
■■■ Execution Engine
■ ■■■ Interpreter
■ ■■■ JIT Compiler (C1 - Client, C2 - Server tiers)
■ ■■■ Garbage Collector (Serial, Parallel, G1, ZGC, Shenandoah)
```

### 3.2 Step-by-Step: What Happens When You Type `javac HelloWorld.java` Then `java HelloWorld`

#### Phase A — Compilation (uses JDK's `javac`, part of the SDK layer, NOT the JVM):

1. `javac` (a standalone program, itself written in Java and run by a JVM internally) reads `HelloWorld.java`.
2. Performs **lexical analysis** (tokenizing), **syntax analysis** (parsing into an Abstract Syntax Tree), and **semantic analysis** (type checking).
3. Emits `HelloWorld.class` — a binary file containing **bytecode** (a sequence of one-byte opcodes + operands, e.g., `iconst_1`, `invokevirtual`, `areturn`).

#### Phase B — Execution (uses the JRE's JVM):

1. The `java` launcher starts a **new JVM process**.
2. The **Bootstrap Class Loader** loads core JDK classes (`java.lang.Object`, `java.lang.String`) from `rt.jar` (pre-Java 9) or the module system (`java.base` module, Java 9+).
3. The **Application Class Loader** loads your `HelloWorld.class` from the classpath/module path.

4. **Bytecode Verifier** checks the loaded class for illegal instructions.
5. **Execution Engine** begins interpreting bytecode instruction by instruction inside `main()`.
6. If a method is called repeatedly (becomes "hot"), the **JIT compiler** compiles it to native machine code for faster subsequent execution.
7. **Garbage Collector** runs periodically in the background to reclaim unused heap memory.
8. When `main()` returns, the JVM process terminates (unless non-daemon threads are still running).

### 3.3 Memory View — Where Each Component "Lives"

```
DISK RUNNING JVM PROCESS (RAM)
+-----+ +-----+
| HelloWorld.java | | Method Area / Metaspace |
| HelloWorld.class | --(loaded by)--> | - HelloWorld class metadata |
| rt.jar / modules | | - java.lang.Object metadata |
+-----+ +-----+
| Heap |
| - all objects (new HelloWorld()) |
| |
| Thread Stack (per thread) |
| - main() frame, local vars |
+-----+ +-----+
```

## 4. Diagrams

### 4.1 JDK vs JRE vs JVM — Containment Diagram

```
+-----+
| JDK |
| +-----+ |
| | JRE | |
| | +-----+ | |
| | | JVM | | |
| | | Class Loader | Runtime Data Areas | Engine | | |
| | +-----+ | |
| | + Java Class Library (java.lang, java.util...) | |
| +-----+ |
| + javac, javadoc, jar, jdb, jshell, jlink, jmod |
+-----+
```

## 4.2 JVM Architecture (Detailed)

```
JVM ARCHITECTURE
+-----+
| 1. CLASS LOADER SUBSYSTEM |
| Loading -> Linking (Verify, Prepare, Resolve) -> Initializing |
+-----+
| 2. RUNTIME DATA AREAS |
| +-----+ +-----+ +-----+ +-----+ +-----+ |
| | Method Area| | Heap | | Stack | | PC Reg | |Native| |
| |(Metaspace) | | | |(per- | |(per-thread)| |Method| |
| | | | | thread)| | | |Stack | |
| +-----+ +-----+ +-----+ +-----+ +-----+ |
+-----+
| 3. EXECUTION ENGINE |
| +-----+ +-----+ +-----+ +-----+ |
| | Interpreter | | JIT Compiler | | Garbage Collector | |
| | | | (C1/C2 tiers) | | (G1 / ZGC / etc.) | |
| +-----+ +-----+ +-----+ +-----+ |
+-----+
| 4. NATIVE METHOD INTERFACE (JNI) --> Native Method Libraries |
+-----+
```

## 4.3 Class Loader Hierarchy (Delegation Model)

```
+-----+
| Bootstrap Class Loader | loads java.base module (core JDK classes)
| (written in native C/C++) | e.g. java.lang.Object, java.lang.String
+-----+
| parent of
v
+-----+
| Platform Class Loader | loads other JDK platform modules
| (formerly "Extension" pre-J9) | e.g. java.sql, java.xml
+-----+
| parent of
v
+-----+
| Application (System) | loads YOUR classes
| Class Loader | from classpath/-cp or module path
+-----+
```

DELEGATION MODEL: A class loader ALWAYS asks its PARENT to try loading a class FIRST before attempting to load it itself. This prevents core classes (like java.lang.String) from being maliciously overridden by user code.

## 5. Syntax — Key Commands Explained

### 5.1 Compilation Command

```
javac HelloWorld.java
```

| Part              | Meaning                                                |
|-------------------|--------------------------------------------------------|
| javac             | Invokes the JDK's Java compiler                        |
| HelloWorld.java   | Source file to compile                                 |
| (implicit output) | Produces HelloWorld.class in same directory by default |

### Useful javac flags:

```
javac -d out/ HelloWorld.java # -d : output directory for compiled .class files
javac -cp lib/mylib.jar App.java # -cp : classpath, where to find dependency classes
javac --release 17 App.java # --release : compile targeting a specific Java version
javac -Xlint:all App.java # -Xlint : enable all recommended compiler warnings
```

## 5.2 Execution Command

```
java HelloWorld
```

| Part       | Meaning                                                    |
|------------|------------------------------------------------------------|
| java       | Launches a new JVM instance                                |
| HelloWorld | Fully-qualified class name (NOT file name) containing main |

### Useful java flags:

```
java -cp out/ com.company.HelloWorld # -cp : classpath to locate compiled classes
java -Xmx512m -Xms256m HelloWorld # -Xmx : max heap size, -Xms : initial heap size
java -jar app.jar # runs an executable JAR (manifest specifies main class)
java --version # prints JVM/Java version info
java HelloWorld.java # (Java 11+) compiles & runs single-file source directly, no separate javac step
```

## 5.3 [jlink](#) Syntax (Custom Runtime Creation, Java 9+)

```
jlink --module-path $JAVA_HOME/jmods --add-modules java.base \
--output custom-runtime --strip-debug --no-header-files --no-man-pages
```

| Flag          | Meaning                                                |
|---------------|--------------------------------------------------------|
| --module-path | Where to find .jmod module files                       |
| --add-modules | Which modules to include (only what your app needs)    |
| --output      | Directory to write the custom minimal JRE-like runtime |
| --strip-debug | Removes debug symbols to reduce size                   |

## 6. Every Method / Tool — Detailed Reference

### 6.1 `javac` (Compiler)

| Aspect                   | Detail                                                                                                                                               |
|--------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| Purpose                  | Translates <code>.java</code> source files into <code>.class</code> bytecode files                                                                   |
| Common Parameters        | <code>-d</code> (output dir), <code>-cp/-classpath</code> (dependencies),<br><code>--release</code> (target version), <code>-Xlint</code> (warnings) |
| "Return"                 | Exit code 0 = success, non-zero = compilation errors                                                                                                 |
| Time Complexity          | Roughly linear in source code size for parsing; can be superlinear for complex type inference/generics resolution in large files                     |
| When to use              | Every time before running any hand-written <code>.java</code> file (unless using Java 11+ single-file execution)                                     |
| When NOT to use directly | In real projects — build tools (Maven/Gradle) invoke <code>javac</code> for you with proper dependency resolution                                    |
| Common mistakes          | Forgetting <code>-cp</code> when the class depends on external JARs, causing <code>ClassNotFoundException</code>                                     |

### 6.2 `java` (Launcher)

| Aspect            | Detail                                                                                                                                                                           |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Purpose           | Starts a JVM instance and executes the specified class's <code>main</code> method (or a JAR's entry point)                                                                       |
| Common Parameters | <code>-cp</code> , <code>-jar</code> , <code>-Xmx/-Xms</code> (heap sizing),<br><code>-D&lt;property&gt;=&lt;value&gt;</code> (system properties),<br><code>--module-path</code> |
| Time Complexity   | JVM startup itself has fixed overhead (class loading, JIT warm-up); irrelevant to Big-O of your program logic                                                                    |
| When to use       | To run any compiled Java application                                                                                                                                             |
| Common mistakes   | Passing the <code>.class</code> extension or file path instead of the fully-qualified class name                                                                                 |

### 6.3 `javadoc`

| Aspect         | Detail                                                                                                                                        |
|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| Purpose        | Generates HTML API documentation from specially formatted <code>/** ... */</code> comments in source code                                     |
| When to use    | Before releasing a library/API for other developers to consume                                                                                |
| Common mistake | Writing regular <code>//</code> comments instead of <code>/** */</code> Javadoc comments — <code>javadoc</code> tool ignores regular comments |

## 6.4 jar

| Aspect       | Detail                                                                                                                                    |
|--------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Purpose      | Packages multiple <code>.class</code> files (and resources) into a single compressed <code>.jar</code> archive (a specialized ZIP format) |
| Common flags | <code>-c</code> (create), <code>-f</code> (specify file name), <code>-m</code> (include manifest), <code>-v</code> (verbose)              |
| Example      | <code>jar -cvfm app.jar manifest.txt -C out/ .</code>                                                                                     |
| When to use  | To distribute a compiled application/library as a single deployable file                                                                  |

## 6.5 jshell (REPL, Java 9+)

| Aspect          | Detail                                                                                            |
|-----------------|---------------------------------------------------------------------------------------------------|
| Purpose         | Interactive Read-Eval-Print-Loop — test Java snippets instantly without writing a full class/file |
| When to use     | Quick experiments, learning, prototyping small logic                                              |
| When NOT to use | Building real applications (not a replacement for actual project structure)                       |

## 6.6 jlink (Java 9+)

| Aspect         | Detail                                                                                                                                                |
|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| Purpose        | Builds a custom, minimal runtime image containing only required modules — reduces deployment size significantly (useful for containers/microservices) |
| When to use    | Production Docker images where you want the smallest possible JVM footprint                                                                           |
| Common mistake | Forgetting to include a module your app actually needs at runtime, causing <code>NoClassDefFoundError</code> in the trimmed runtime                   |

# 7. Code Examples

## 7.1 Basic — Checking Your Installed Versions

```
public class VersionCheck {
    public static void main(String[] args) {
        System.out.println("java.version : " + System.getProperty("java.version"));
        System.out.println("java.vendor : " + System.getProperty("java.vendor"));
        System.out.println("java.home : " + System.getProperty("java.home"));
    }
}
```

Command line equivalent:

```
java -version
javac -version
```



## 7.2 Intermediate — Inspecting Bytecode with `javap`

```
public class SimpleMath {  
    public int add(int a, int b) {  
        return a + b;  
    }  
}
```

```
javac SimpleMath.java  
javap -c SimpleMath
```

### Output (disassembled bytecode):

```
public int add(int, int);  
Code:  
0: iload_1  
1: iload_2  
2: iadd  
3: ireturn
```

This proves that `a + b` becomes actual bytecode instructions (`iload`, `iadd`, `ireturn`) that the JVM's execution engine processes — the JVM never sees your original Java source at all.

## 7.3 Advanced — Building a Custom Minimal Runtime with `jlink`

```
# Step 1: Identify required modules for your app  
jdeps --print-module-deps --ignore-missing-deps app.jar  
  
# Step 2: Build a custom runtime containing ONLY those modules  
jlink --module-path $JAVA_HOME/jmods \  
--add-modules java.base,java.logging \  
--output my-custom-jre \  
--strip-debug --no-header-files --compress=2  
  
# Step 3: Run using the custom runtime instead of the full JDK  
./my-custom-jre/bin/java -cp app.jar com.company.App
```

**Why this is "production-quality":** Real companies (especially in microservices/Docker deployments) use `jlink` to slim down container images from ~300MB (full JDK) to ~40-60MB (custom runtime), reducing deployment time and attack surface.

## 7.4 Company-style Example — Programmatic Runtime Diagnostics

```
package com.jobsradar.diagnostics;

import java.lang.management.ManagementFactory;
import java.lang.management.RuntimeMXBean;

/**
 * Prints detailed JVM runtime information – commonly used in production
 * health-check/diagnostic endpoints (e.g., a Spring Boot actuator-style report).
 */
public final class JvmDiagnostics {

    private JvmDiagnostics() {
        throw new AssertionError("Utility class - do not instantiate");
    }

    public static void printFullReport() {
        RuntimeMXBean runtimeBean = ManagementFactory.getRuntimeMXBean();

        System.out.println("JVM Name : " + runtimeBean.getVmName());
        System.out.println("JVM Vendor : " + runtimeBean.getVmVendor());
        System.out.println("JVM Version : " + runtimeBean.getVmVersion());
        System.out.println("Uptime (ms) : " + runtimeBean.getUptime());
        System.out.println("Input Arguments: " + runtimeBean.getInputArguments());
    }

    public static void main(String[] args) {
        printFullReport();
    }
}
```

## 8. Dry Run

**Program (same as topic 1, revisited to trace class loading in detail):**

```
public class DryRunJvm {
    public static void main(String[] args) {
        Helper h = new Helper();
        h.sayHello();
    }
}

class Helper {
    void sayHello() {
        System.out.println("Hello from Helper!");
    }
}
```

| Step | Action                       | Class Loader Subsystem                                                                | Method Area                       | Heap  | Stack |
|------|------------------------------|---------------------------------------------------------------------------------------|-----------------------------------|-------|-------|
| 1    | java<br>DryRunJvm<br>invoked | Bootstrap loader<br>loads java.lang.<br>Object, java.<br>lang.String,<br>System, etc. | Core JDK class<br>metadata loaded | empty | empty |

| Step | Action                                                                     | Class Loader Subsystem                                                                        | Method Area                              | Heap                                                         | Stack                                                                          |
|------|----------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|------------------------------------------|--------------------------------------------------------------|--------------------------------------------------------------------------------|
| 2    | Application loader loads <code>DryRunJvm.class</code>                      | Loading → Linking (verify/prepare/resolve) → Initializing                                     | <code>DryRunJvm</code> metadata added    | empty                                                        | <code>main()</code> frame pushed                                               |
| 3    | <code>Helper h = new Helper();</code> executes                             | Application loader <b>lazily</b> loads <code>Helper.class</code> (only when first referenced) | <code>Helper</code> class metadata added | <code>Helper</code> object allocated on Heap                 | <code>main()</code> frame: local var <code>h</code> → reference to Heap object |
| 4    | <code>h.sayHello();</code> called                                          | —                                                                                             | —                                        | —                                                            | <code>sayHello()</code> frame pushed on top of <code>main()</code> frame       |
| 5    | <code>System.out.println(...)</code> executes inside <code>sayHello</code> | —                                                                                             | —                                        | —                                                            | Output printed; <code>sayHello()</code> frame popped after return              |
| 6    | <code>main()</code> returns                                                | —                                                                                             | —                                        | <code>Helper</code> object now unreachable (eligible for GC) | <code>main()</code> frame popped; stack empty                                  |

**Key insight:** `Helper.class` is loaded **lazily** — only at the exact point it's first referenced (`new Helper()`), NOT upfront when `DryRunJvm` is loaded. This is called "**lazy/on-demand class loading**", a core JVM behavior.

## 9. Edge Cases

| Edge Case                                                                                                          | What Happens                                                                                                                                                                                                                                                                | Why                                                                                                          |
|--------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| Running <code>java</code> without any JDK/JRE installed                                                            | OS reports "command not found"                                                                                                                                                                                                                                              | No Java runtime is available on the PATH                                                                     |
| Compiling with a newer <code>javac</code> but running with an older <code>java</code>                              | <code>UnsupportedClassVersionError: class file version X, this version of the Java Runtime only recognizes up to Y</code>                                                                                                                                                   | Bytecode has a version number tied to compiler version; older JVMs reject newer bytecode versions            |
| Two different JDKs installed, wrong one on PATH                                                                    | Compiles/runs using unexpected Java version, causing subtle feature unavailability (e.g., <code>var</code> not recognized)                                                                                                                                                  | <code>JAVA_HOME</code> / <code>PATH</code> environment variable points to the wrong installation             |
| Using <code>--release 8</code> flag with newer JDK but referencing Java 17-only APIs (e.g., <code>Records</code> ) | Compile-time error: symbol not found                                                                                                                                                                                                                                        | <code>--release</code> restricts both API and bytecode target version, catching accidental use of newer APIs |
| Missing a module in a <code>jlink</code> -trimmed custom runtime                                                   | <code>NoClassDefFoundError</code> or <code>ClassNotFoundException</code> at runtime                                                                                                                                                                                         | The custom runtime doesn't include the module containing that class                                          |
| Circular class dependency during loading                                                                           | JVM handles it correctly via its class loading/linking phases (this is NOT invalid, unlike some naive assumptions) — but circular <b>static initializer</b> logic referencing not-yet-initialized static fields can yield unexpected default values ( <code>0/null</code> ) | Java's class initialization order guarantees don't fully resolve circular static dependencies intuitively    |
| Corrupted/tampered <code>.class</code> file                                                                        | <code>ClassFormatError</code> or bytecode verification failure                                                                                                                                                                                                              | Bytecode Verifier rejects malformed/invalid instructions before execution                                    |

## 10. Internal Interview Questions

### Easy

**1 Q: What is the relationship between JDK, JRE, and JVM?**

A: JDK contains JRE, and JRE contains JVM. JDK = JRE + dev tools. JRE = JVM + core libraries.

**2 Q: Can you run a compiled `.class` file with only a JVM, no JRE?**

A: No — practically, you always need at least the JRE (JVM + standard class library), since even the simplest program depends on core classes like `java.lang.Object`, `System`, etc., which live in the JRE's class library, not the bare JVM alone.

### Medium

**1 Q: Since Java 11, Oracle stopped shipping a separate JRE. What do you install now to just run Java apps?**

A: You install the full JDK (or use `jlink` to build a custom minimal runtime image containing exactly the modules you need).

**2 Q: What is the Class Loader delegation model, and why does it matter for security?**

A: Each class loader asks its parent to attempt loading a class first before trying itself. This prevents

malicious user code from spoofing/overriding core classes like `java.lang.String`, since the Bootstrap loader (trusted, loads from the JDK itself) always gets first chance to load core class names.

- 3 **Q: Why can different JVM vendors (HotSpot, OpenJ9, Zing, GraalVM) all claim "Java compatibility"?**  
A: Because the **JVM Specification (JVMS)** defines the required behavior (bytecode instruction semantics, class file format, runtime data area guarantees) independent of implementation — any vendor that conforms to the spec can produce a valid, interchangeable JVM.

## Hard

- 1 **Q: Explain what `UnsupportedClassVersionError` means and how you'd debug it in a CI/CD pipeline.**  
A: It means the bytecode's major/minor version (embedded in the `.class` file header) is newer than what the running JVM supports. Debugging: check `javac -version` vs `java -version` used in the pipeline, ensure both stages use matching (or compatible `--release` targeted) JDK versions, check `JAVA_HOME` in build agents.
- 2 **Q: Your production Docker image is 350MB just from the JDK. How would you reduce it using knowledge of JDK/JRE/JVM internals?**  
A: Use `jdeps` to discover actual required modules, then use `jlink` to build a custom minimal runtime image containing only those modules, cutting the base image down dramatically (often to 50-80MB), and use a multi-stage Docker build so the final image doesn't carry the full JDK.

## Company-Level / Tricky

- 1 **(Cross-question) Q: If the JVM is what actually executes code, why do we even need the JRE as a separate concept?**  
A: The bare JVM specification only defines execution mechanics (bytecode interpretation, memory model) — it says nothing about the actual **standard library classes** (`String`, `ArrayList`, `Thread`) that virtually every real program depends on. The JRE bundles these together so the JVM has something meaningful to execute against.
- 2 **(Tricky) Q: Is `javac` itself a Java program?**  
A: Yes — `javac` is written in Java and is itself run by a bootstrapping JVM instance under the hood, demonstrating Java's own self-hosting compiler design.

---

## 11. Coding Questions

### Easy

**Q1: Write a program that prints whether the running JVM is 64-bit, using system properties.**

```
public class BitnessCheck {
    public static void main(String[] args) {
        String arch = System.getProperty("os.arch");
        System.out.println("Architecture: " + arch);
        System.out.println("Likely 64-bit: " + arch.contains("64"));
    }
}
```

## Medium

**Q2: Write a program that prints all JVM system properties sorted alphabetically.**

```
import java.util.Properties;
import java.util.TreeMap;
import java.util.Map;

public class AllSystemProperties {
    public static void main(String[] args) {
        Properties props = System.getProperties();
        Map<String, String> sorted = new TreeMap<>(); // TreeMap auto-sorts by key
        for (String name : props.stringPropertyNames()) {
            sorted.put(name, props.getProperty(name));
        }
        for (Map.Entry<String, String> entry : sorted.entrySet()) {
            System.out.println(entry.getKey() + " = " + entry.getValue());
        }
    }
}
```

- **Time Complexity:**  $O(n \log n)$  for TreeMap insertion of  $n$  properties.

## Hard (Company Tagged: Infra/DevOps screening)

**Q3: Write a program that programmatically detects and prints the currently active Garbage Collector being used by the JVM.**

```
import java.lang.management.GarbageCollectorMXBean;
import java.lang.management.ManagementFactory;
import java.util.List;

public class ActiveGCDetector {
    public static void main(String[] args) {
        List<GarbageCollectorMXBean> gcBeans = ManagementFactory.getGarbageCollectorMXBeans();
        for (GarbageCollectorMXBean bean : gcBeans) {
            System.out.println("GC Name: " + bean.getName()
                + " | Collection Count: " + bean.getCollectionCount()
                + " | Collection Time (ms): " + bean.getCollectionTime());
        }
    }
}
```

---

## 12. Common Mistakes

### Beginner Mistakes

- 1 Confusing "installing Java" with "installing the JDK" vs "installing just a JRE" — leading to `javac` command not found when only a JRE (pre-Java 11 concept) was installed.
- 2 Assuming `java -version` and `javac -version` will always match if multiple JDKs are installed but `PATH` isn't configured consistently.
- 3 Thinking `.jar` files are directly executable machine code — they still require a JVM to interpret their contained bytecode.

## Experienced Developer Mistakes

- 1 Not pinning the exact JDK version in CI/CD pipelines, leading to "works on my machine" bytecode version mismatches (`UnsupportedClassVersionError`) between build and deploy environments.
- 2 Shipping full 300+MB JDK-based Docker images to production instead of using `jlink`/distroless-style minimal runtimes, wasting storage, bandwidth, and increasing attack surface.
- 3 Forgetting that different JVM vendors (HotSpot vs OpenJ9 vs GraalVM) can have meaningfully different default GC behavior, startup time, and memory footprint — assuming "a JVM is a JVM" without benchmarking for the target environment (e.g., serverless/cold-start-sensitive workloads benefit hugely from GraalVM Native Image or CRaC).

## Debugging Process for `UnsupportedClassVersionError`

- 1 Run `java -version` and `javac -version` — compare major version numbers.
  - 2 Check the class file's version header using `javap -verbose YourClass` (major version field).
  - 3 Ensure both build and runtime environments use the same (or `--release-compatible`) JDK.
- 

## 13. Best Practices

- 1 Always pin an **exact JDK version** in your build tool (Maven `<release>`/Gradle `sourceCompatibility`) and CI/CD YAML — never rely on "whatever's on the machine."
  - 2 Use **LTS versions** (8, 11, 17, 21) for production systems — avoid non-LTS versions for long-lived services.
  - 3 Use `jlink`/multi-stage Docker builds for production deployment images to minimize size and attack surface.
  - 4 Keep development, CI, and production environments on **matching major JDK versions** to avoid `UnsupportedClassVersionError` surprises.
  - 5 Regularly update to the latest patch release within your chosen LTS line for security fixes (JVMs receive CVE patches regularly).
  - 6 Document the required JDK version clearly in your project's README/build files (`.java-version`, `pom.xml`, etc.).
- 

## 14. Production Usage

- **CI/CD pipelines:** Build agents install a specific pinned JDK (via tools like `sdkman`, Docker base images `eclipse-temurin:17-jdk`) to guarantee reproducible builds.
- **Containerized microservices:** Production Docker images commonly use **JRE-only** or `jlink-trimmed` **custom runtime** base images (e.g., `eclipse-temurin:17-jre`) — NOT the full JDK — to minimize image size and reduce the attack surface (no compiler needed in production).
- **Enterprise app servers:** WebLogic/WebSphere/JBoss deployments specify exact supported JDK vendor+version combinations for certified production support.

- **Cloud-native/serverless:** AWS Lambda, GraalVM Native Image, and Project CRaC address JVM cold-start weaknesses for latency-sensitive production workloads.
- 

## 15. Performance

- **JDK vs JRE size:** Full JDK (~300MB+) vs `jlink`-trimmed custom runtime (as small as 40-70MB) — directly impacts container image pull time and cold-start latency in cloud deployments.
  - **JVM vendor choice affects performance:** HotSpot (default, balanced), OpenJ9 (lower memory footprint, good for microservices), GraalVM (fast startup via Native Image, ahead-of-time compilation), Azul Zing/Prime (low-pause GC for latency-critical trading systems).
  - **Trade-off:** A trimmed custom runtime reduces deployment footprint and attack surface, but requires careful dependency analysis (via `jdeps`) — missing a module causes runtime failures, not compile-time failures, so this is a genuine engineering trade-off between size and safety-net completeness.
- 

## 16. Frequently Asked Interview Questions (50+ with Detailed Answers)

- 1 **What is JDK?** Java Development Kit — full toolkit (JRE + compiler + dev tools) to develop Java applications.
- 2 **What is JRE?** Java Runtime Environment — JVM + standard class libraries, needed to run compiled Java applications.
- 3 **What is JVM?** Java Virtual Machine — the engine that executes Java bytecode, providing platform independence.
- 4 **What is the relationship between the three?**  $\text{JDK} \supset \text{JRE} \supset \text{JVM}$  (each containing the next).
- 5 **Can JRE run without JVM?** No — the JVM is the execution core inside the JRE; JRE without JVM has nothing to execute bytecode.
- 6 **Is JVM platform-independent?** No — the JVM itself is platform-**dependent** (a different native binary per OS/architecture); it's the **bytecode** that's platform-independent, because any platform's JVM can run it.
- 7 **What changed about JRE distribution starting Java 11?** Oracle stopped shipping a standalone JRE installer; only full JDK builds are distributed, though the JRE concept (JVM + libraries) still exists inside it.
- 8 **What tool lets you build a custom minimal runtime?** `jlink` (introduced in Java 9 with Project Jigsaw).
- 9 **Name the class loaders in the JVM's delegation hierarchy.** Bootstrap, Platform (formerly Extension), and Application (System) class loaders.
- 10 **What is the Class Loader delegation model?** Each loader asks its parent to try loading a class first, before attempting itself — protects core classes from being overridden maliciously.
- 11 **What does `javac` do?** Compiles `.java` source files into `.class` bytecode files.
- 12 **What does `java` do?** Launches a JVM instance to execute a compiled class's `main` method or a JAR's entry point.



- 13 **What is `javap` used for?** Disassembling/inspecting compiled bytecode of a class.
- 14 **What is `javadoc` used for?** Generating HTML API documentation from specially formatted source comments.
- 15 **What is `jsHELL`?** An interactive REPL (Java 9+) for quickly testing Java code snippets.
- 16 **What is `jar`?** A tool to package multiple `.class` files and resources into a single compressed archive.
- 17 **What does the Bootstrap Class Loader load?** Core JDK classes (e.g., `java.lang.Object`, `java.lang.String`), historically from `rt.jar`, now from the `java.base` module.
- 18 **Why is the Bootstrap Class Loader written in native code (C/C++), not Java?** Because it must load the very first Java classes (like `Object`) before any Java code can run — there's a bootstrapping "chicken and egg" problem it must solve natively.
- 19 **What is `UnsupportedClassVersionError`?** An error thrown when running bytecode compiled by a newer `javac` on an older, incompatible JVM.
- 20 **How do you fix `UnsupportedClassVersionError`?** Ensure the runtime JVM version is equal to or newer than the version used to compile the bytecode (or recompile targeting an older `--release`).
- 21 **What are the major runtime data areas inside the JVM?** Method Area/Metaspace, Heap, Stack, PC Register, Native Method Stack.
- 22 **Which runtime data areas are shared across threads, and which are per-thread?** Method Area and Heap are shared; Stack, PC Register, and Native Method Stack are per-thread.
- 23 **What replaced PermGen in Java 8?** Metaspace — allocated from native (off-heap) memory instead of the JVM heap, removing the fixed-size `PermGen` limitation and its common `OutOfMemoryError: PermGen space` issue.
- 24 **What is the Execution Engine composed of?** Interpreter, JIT Compiler, and Garbage Collector.
- 25 **What is JIT compilation and why does it matter?** Just-In-Time compilation converts frequently executed ("hot") bytecode into native machine code at runtime, improving performance for long-running processes.
- 26 **Name some real JVM implementations besides Oracle HotSpot.** Eclipse OpenJ9, Azul Zing/Prime, GraalVM, Amazon Corretto (HotSpot-based distribution).
- 27 **What is GraalVM known for?** Fast startup and low memory footprint via Ahead-of-Time (AOT) compilation into Native Images, useful for serverless/CLI tools.
- 28 **What is `jdeps`?** A tool that analyzes class-level dependencies of your application, useful for determining exactly which modules `jlink` needs to include.
- 29 **Why would a company use `jlink` in production?** To ship a minimal, custom runtime image instead of the full JDK, reducing container size, attack surface, and startup overhead.
- 30 **What is the difference between the Method Area and the Heap?** Method Area stores class-level metadata (structure, static variables, bytecode); Heap stores actual object instances created at runtime.
- 31 **Can two different JVM vendors run the exact same `.class` file identically?** Yes, in principle — any JVM-compliant implementation must correctly execute standard bytecode, though performance characteristics (GC pause times, JIT aggressiveness) can differ.
- 32 **What is the purpose of the Bytecode Verifier?** To ensure loaded bytecode doesn't violate JVM safety rules (illegal type casts, stack overflows, illegal jumps) before execution — a core security mechanism.

- 33 Does the JDK include a JVM?** Yes — the JDK includes the JRE, which includes the JVM.
- 34 What's the smallest thing you need installed to just run a `.jar` file today (Java 11+)?** A full JDK installation (since standalone JRE downloads were discontinued) or a custom `jlink`-built runtime.
- 35 What is a "module" in the context of `jlink`/Project Jigsaw?** A named, self-describing collection of packages and resources (declared via `module-info.java`) introduced in Java 9 to enable strong encapsulation and minimal runtime composition.
- 36 Why did Java 9 introduce the Module System?** To solve the "JAR Hell" problem (no way to express strong dependencies/versions between JARs) and to allow building smaller, more secure custom runtimes.
- 37 What is `rt.jar` and does it still exist?** `rt.jar` was the pre-Java 9 archive containing all core runtime classes; it was replaced by the modular `java.base` and other JDK modules starting Java 9.
- 38 What does `-Xmx` control?** The maximum heap size the JVM is allowed to use.
- 39 What does `-Xms` control?** The initial heap size allocated when the JVM starts.
- 40 What is the difference between `-cp` and `--module-path`?** `-cp` (classpath) is used for traditional unnamed-module/classpath-based class loading; `--module-path` is used for the Java Platform Module System (JPMS) introduced in Java 9.
- 41 How would you check which exact JDK vendor/build is installed?** `java -version` (shows vendor string, e.g., "OpenJDK", "Eclipse Adoptium/Temurin", "Oracle") and `System.getProperty("java.vendor")` programmatically.
- 42 What is Amazon Corretto?** Amazon's free, production-ready, multi-platform distribution of OpenJDK, commonly used in AWS-hosted Java workloads.
- 43 What happens internally when you run `java -jar app.jar`?** The JVM reads the JAR's `META-INF/MANIFEST.MF` file to find the `Main-Class` attribute, then loads and executes that class's `main` method.
- 44 What must a JAR file contain to be directly executable via `java -jar`?** A manifest file (`META-INF/MANIFEST.MF`) specifying the `Main-Class` attribute.
- 45 Why can't you just copy `.class` files between machines with completely different JVM major versions and expect them to always run?** Because bytecode carries a version number tied to the compiler that produced it; older JVMs reject bytecode versions newer than they support.
- 46 What's the benefit of Java being self-hosting (compiler written in Java itself)?** It proves language maturity/robustness and allows the compiler to benefit from ongoing JVM performance improvements just like any other Java application.
- 47 Does the JVM require an internet connection to run?** No — the JVM is a local execution engine; internet access is only needed if the application logic itself performs network operations.
- 48 What is the significance of `--release` versus `-source/-target` flags in `javac`?** `--release N` ensures BOTH the language features AND the API set are restricted to what was available in Java version N, preventing accidental use of newer APIs even if only bytecode target version was intended to be restricted (a common historical pitfall with separate `-source/-target` flags).
- 49 What is the typical production strategy for choosing between HotSpot, OpenJ9, and GraalVM?** HotSpot for general-purpose balanced workloads (default, most mature); OpenJ9 for memory-constrained environments (many container instances); GraalVM for ultra-fast startup/low memory serverless or CLI-tool use cases.

**50 Why is understanding JDK/JRE/JVM important for an interview even though "everyone just installs Java"?** Because production incidents (version mismatches, container bloat, class loading errors, `OutOfMemoryError` types) trace directly back to misunderstanding these layers — interviewers use this topic to gauge depth of real operational Java knowledge, not just syntax familiarity.

**51 What's the difference between the Application Class Loader and a custom class loader you might write yourself?** The Application Class Loader is the JVM's default loader for user classpath/module-path classes; a custom class loader (extending `ClassLoader`) lets you implement specialized loading logic (e.g., loading classes from a database, network, or with hot-reload support), commonly used in plugin architectures and application servers.

---

## 17. Revision Notes (One-Page Cheat Sheet)

```
JDK / JRE / JVM – QUICK REVISION SHEET
=====
JVM = Executes bytecode (Class Loader + Runtime Data Areas + Execution Engine)
JRE = JVM + Java Class Library (java.lang, java.util, ...) -> "enough to RUN"
JDK = JRE + Dev Tools (javac, javadoc, jar, jshell, jlink) -> "enough to BUILD"

CONTAINMENT: JDK ⊃ JRE ⊃ JVM

SINCE JAVA 11: No separate JRE download – install full JDK, or build custom
runtime with `jlink` (Java 9+, Project Jigsaw / Module System).

JVM INTERNAL STRUCTURE:
1. Class Loader Subsystem -> Loading, Linking (Verify/Prepare/Resolve), Init
2. Runtime Data Areas -> Method Area/Metaspace, Heap (shared)
   Stack, PC Register, Native Method Stack (per-thread)
3. Execution Engine -> Interpreter + JIT Compiler + Garbage Collector

CLASS LOADER HIERARCHY (delegation - parent asked first):
Bootstrap -> Platform (Extension) -> Application (System)

KEY TOOLS:
javac - compile .java -> .class
java - run compiled class/JAR
javap - disassemble bytecode
javadoc - generate API docs
jar - package classes into archive
jshell - REPL (Java 9+)
jlink - build custom minimal runtime (Java 9+)
jdeps - analyze class dependencies

COMMON ERROR: UnsupportedClassVersionError
-> Bytecode compiled with NEWER javac than the JVM running it supports.
-> Fix: match JDK versions between build & run, or use --release flag.
```

---

## 18. Homework

1 Run `java -version` and `javac -version` on your machine and note the exact output, including vendor string.

- 2 Compile a simple class, then use `javap -c YourClass` to view its disassembled bytecode and identify at least 5 different bytecode instructions.
  - 3 Use `jdeps` on a small compiled program to list its module dependencies.
  - 4 Attempt to build a custom minimal runtime using `jlink` for a simple "Hello World" program, and compare its folder size to a full JDK installation.
  - 5 Deliberately compile a class using a newer JDK with `--release 17`, then try running it with an older Java 8 JVM (if available) and record the exact `UnsupportedClassVersionError` message.
  - 6 Write 5 sentences explaining, in your own words, why production Docker images should avoid bundling the full JDK.
- 

## 19. Mini Project — "JVM Health Reporter" CLI Tool

**Goal:** Build a small diagnostic tool using `java.lang.management` APIs that reports JVM internals learned in this topic — useful as a foundation for real production health-check endpoints.

```
package com.jobsradar.diagnostics;

import java.lang.management.*;
import java.util.List;

/**
 * Mini Project: JVM Health Reporter
 * Reports class loading stats, memory usage, and active garbage collectors.
 */
public class JvmHealthReporter {

    public static void main(String[] args) {
        printClassLoadingStats();
        printMemoryStats();
        printGarbageCollectors();
    }

    private static void printClassLoadingStats() {
        ClassLoadingMXBean clBean = ManagementFactory.getClassLoadingMXBean();
        System.out.println("=== Class Loading ===");
        System.out.println("Loaded Classes : " + clBean.getLoadedClassCount());
        System.out.println("Total Loaded (ever) : " + clBean.getTotalLoadedClassCount());
        System.out.println("Unloaded Classes : " + clBean.getUnloadedClassCount());
    }

    private static void printMemoryStats() {
        MemoryMXBean memBean = ManagementFactory.getMemoryMXBean();
        System.out.println("\n=== Memory ===");
        System.out.println("Heap Memory Usage : " + memBean.getHeapMemoryUsage());
        System.out.println("Non-Heap Memory Usage: " + memBean.getNonHeapMemoryUsage());
    }

    private static void printGarbageCollectors() {
        List<GarbageCollectorMXBean> gcBeans = ManagementFactory.getGarbageCollectorMXBeans();
        System.out.println("\n=== Garbage Collectors ===");
        for (GarbageCollectorMXBean bean : gcBeans) {
            System.out.println(bean.getName() + " -> collections: " + bean.getCollectionCount()
                + ", time(ms): " + bean.getCollectionTime());
        }
    }
}
```

```
}
```

**Why this matters:** This is conceptually a simplified version of what production monitoring tools (Spring Boot Actuator's `/actuator/metrics`, JMX consoles, Prometheus JVM exporters) do under the hood — reading the exact same `java.lang.management` MXBeans.

---

## 20. Final Summary

- **JDK**  $\supset$  **JRE**  $\supset$  **JVM** — JDK is for developers (adds compiler + tools), JRE is for running programs (JVM + class libraries), JVM is the actual bytecode execution engine.
  - Since **Java 11**, there's no separate JRE download — you install the full JDK, or build a custom minimal runtime with `jlink` (Java 9+, part of Project Jigsaw's Module System).
  - The JVM internally consists of the **Class Loader Subsystem** (Loading  $\rightarrow$  Linking  $\rightarrow$  Initializing, using a **parent-delegation model** across Bootstrap  $\rightarrow$  Platform  $\rightarrow$  Application loaders), **Runtime Data Areas** (Method Area/Metaspace + Heap shared; Stack + PC Register + Native Method Stack per-thread), and the **Execution Engine** (Interpreter + JIT Compiler + Garbage Collector).
  - Classes are loaded **lazily** — only when first referenced, not upfront.
  - Different **JVM vendors** (HotSpot, OpenJ9, GraalVM, Azul Zing) all conform to the same JVM Specification but differ meaningfully in startup time, memory footprint, and GC behavior — an important production/architecture decision, not just a checkbox.
  - Key production tools to remember forever: `javac` (compile), `java` (run), `javap` (disassemble), `jar` (package), `jlink` (minimal custom runtime), `jdeps` (dependency analysis) — and the common real-world error `UnsupportedClassVersionError`, which always traces back to a JDK version mismatch between build and run environments.
- 

*(End of Topic 2: JDK, JRE, JVM.)*

---

# TOPIC 3: JAVA COMPILATION PROCESS

---

## 1. Introduction

### 1.1 What is it?

The **Java Compilation Process** is the complete pipeline that transforms human-readable Java source code (`.java`) into a `.class` file containing **bytecode** — a well-defined binary format that the JVM can load, verify, and execute. This is distinct from Topic 2 (which covered *what* JDK/JRE/JVM are); this topic goes deep into **exactly what** `javac` **does internally**, phase by phase, and the **exact byte-level structure** of the resulting `.class` file.

## 1.2 Why was it introduced / what problem it solves

Before understanding "why," recall the goal of Java: **platform independence**. A direct source-to-native-machine-code compiler (like C's GCC) would tie the output to one specific CPU/OS. Instead, Sun Microsystems designed `javac` to compile to an **intermediate, well-specified, platform-neutral bytecode format**, deferring the final "real" machine-code translation to the JVM at runtime (via interpretation/JIT). This single design decision is *the* mechanical reason WORA (Write Once, Run Anywhere) works at all.

Additionally, a formal, multi-phase compilation process (lexical → syntax → semantic analysis → bytecode generation) solves:

- **Catching errors early** (at compile-time) rather than at runtime — type mismatches, undeclared variables, unreachable code.
- **Producing a compact, standardized binary format** (`.class`) that any conforming JVM can load and verify for safety before execution.

## 1.3 Real-world problem it solves

- Prevents shipping broken code to production — a huge percentage of bugs (typos, type errors, missing imports) are caught **before** the program ever runs.
- Enables **tooling** (IDEs, static analyzers, `javap`) to reason about code structure via a standardized intermediate format.
- Enables **security verification** — the JVM's Bytecode Verifier can statically check the compiled output for illegal operations before ever executing it, because the format is well-specified and predictable.

## 1.4 Historical Background

- The original Sun `javac` compiler (1995) was a relatively simple single-pass-ish translator.
- Over decades, `javac` evolved to support new language features (generics erasure in Java 5, lambda desugaring in Java 8, pattern matching in Java 17+) while keeping the `.class` **file format** largely backward compatible (with incrementing version numbers).
- The **Java Language Specification (JLS)** and **Java Virtual Machine Specification (JVMS)** — both maintained by Oracle/JCP — formally define exactly how source code must be compiled and how the resulting bytecode must behave, ensuring every compliant compiler (not just `javac` — e.g., the Eclipse Compiler for Java, ECJ) produces JVM-compatible output.

## 1.5 Interview Definition

"The Java compilation process is the sequence of phases — lexical analysis, syntax analysis, semantic analysis, and bytecode generation — performed by `javac` (or any JLS-compliant compiler) to transform `.java` source files into standardized `.class` bytecode files, which are then loaded, verified, and executed by a JVM."

## 1.6 Simple Definition

The compilation process is what happens step by step inside `javac` when it turns your written code into the `.class` file the computer can actually run.

## 1.7 Technical Definition

Compilation in Java is a multi-phase, static, ahead-of-time (AOT) source-to-bytecode transformation governed by the **Java Language Specification (JLS)**, consisting of: (1) **lexical analysis** (tokenization), (2) **syntax analysis** (parsing into an Abstract Syntax Tree per JLS grammar), (3) **semantic analysis** (type checking, name resolution, generics erasure, desugaring of syntactic sugar like enhanced-for/lambda/try-with-resources), and (4) **bytecode generation**, emitting a binary `.class` file conforming to the **class file format** defined in the JVM Specification §4, including the constant pool, method/field tables, and attribute structures.

---

## 2. Real Life Analogy

| Domain     | Analogy                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Bank       | Submitting a loan application (source code) goes through: (1) a form-format check (lexical — are all fields filled with valid characters?), (2) a structural check (syntax — is the form filled in the right order/sections?), (3) an eligibility/credit check (semantic — does your income type match the loan type requested; are all cross-references valid?), (4) finally, the bank issues a <b>standardized approval certificate</b> (bytecode) that any of the bank's branches (JVMs) can process identically. |
| Hospital   | A patient's symptoms (source code) are: (1) transcribed into standard medical terms (lexical), (2) organized into a structured case history following medical documentation format (syntax), (3) cross-checked against the patient's known allergies/history for contradictions (semantic — type checking), (4) converted into a standardized referral form (bytecode) any specialist hospital (JVM) can act on immediately without re-interviewing the patient from scratch.                                        |
| Restaurant | A handwritten customer order (source code) is: (1) checked that the handwriting/words are valid menu items (lexical), (2) structured into a proper "starter → main → dessert" ticket format (syntax), (3) cross-checked against kitchen inventory and dietary restrictions (semantic), (4) printed as a <b>standardized kitchen order ticket</b> (bytecode) that any chef (JVM) in any branch can read and cook from identically.                                                                                    |
| College    | A hand-written exam answer sheet (source code) goes through: (1) legibility check (lexical), (2) checking answers are in the correct question-numbered sections (syntax), (3) verifying each answer actually addresses the question asked and doesn't contradict itself (semantic), (4) producing a <b>standardized grade report</b> (bytecode) any examination board office (JVM) anywhere can process the same way.                                                                                                |
| Amazon     | An engineer's pull request (source code) goes through automated linting (lexical/syntax checks), code review + type-checking in CI (semantic analysis), and finally produces a <b>build artifact (JAR)</b> that can be deployed identically across every one of Amazon's global data centers (JVMs) — the artifact behaves the same everywhere it runs.                                                                                                                                                              |

---

## 3. Internal Working

### 3.1 The Four Phases of `javac` Compilation (Detailed)

```
.java SOURCE FILE
|
v
+-----+
| PHASE 1: LEXICAL ANALYSIS | (Scanning / Tokenizing)
| - Reads raw characters |
| - Groups into TOKENS |
| - Removes whitespace/comments|
+-----+
|
v
+-----+
| PHASE 2: SYNTAX ANALYSIS | (Parsing)
| - Applies JLS grammar rules |
| - Builds Abstract Syntax Tree|
| (AST) |
| - Detects syntax errors |
| (missing semicolons, etc.) |
+-----+
|
v
+-----+
| PHASE 3: SEMANTIC ANALYSIS | (Type Checking + Resolution)
| - Symbol table construction |
| - Type checking |
| - Name/scope resolution |
| - Generics type erasure |
| - Desugaring (lambdas, enhanced|
| for-loop, try-with-resources,|
| autoboxing, varargs) |
+-----+
|
v
+-----+
| PHASE 4: BYTECODE GENERATION |
| - Emits .class binary file |
| - Constant Pool populated |
| - Method bytecode instructions|
| generated |
+-----+
|
v
.class BYTECODE FILE
```

### 3.2 Phase 1 — Lexical Analysis (Deep Dive)

The **Lexer (Scanner)** reads the raw source file character-by-character and groups characters into meaningful **tokens** — the smallest units of meaning in the language.

**Example source:**

```
int sum = a + 10;
```

**Tokens produced:**

```
KEYWORD("int") IDENTIFIER("sum") OPERATOR("=") IDENTIFIER("a")
OPERATOR("+") LITERAL(10) SEMICOLON(";")
```



- Whitespace and comments are **discarded** at this stage (they have no semantic meaning to the compiler, only to humans).
- Invalid characters (e.g., `@#$` outside of annotations/valid contexts) cause a **lexical error**.

### 3.3 Phase 2 — Syntax Analysis / Parsing (Deep Dive)

The **Parser** takes the token stream and checks it against the **formal grammar rules** defined in the JLS, building a tree structure called the **Abstract Syntax Tree (AST)**.

**AST for** `int sum = a + 10;`:

```
VariableDeclaration
/ | \
Type Identifier Initializer
"int" "sum" |
BinaryExpression("+")
/ \
Identifier("a") Literal(10)
```

- If tokens don't match any valid grammar rule (e.g., a missing semicolon, unbalanced braces, `int = ;`), a **syntax error** is reported — this is the classic "cannot find symbol" /  `';' expected` compiler error category.

### 3.4 Phase 3 — Semantic Analysis (Deep Dive)

This is the most complex phase. The compiler walks the AST and performs:

- 1 **Symbol Table Construction**: Records every declared variable/method/class name, its type, and its scope.
- 2 **Type Checking**: Verifies that operations are valid for their operand types (e.g., you cannot do `String + boolean` in most contexts without triggering specific concatenation rules, or assign an `int` to a `boolean` variable).
- 3 **Name/Scope Resolution**: Ensures every identifier used actually refers to something declared and visible in that scope (catches "cannot find symbol" errors).
- 4 **Generics Type Erasure**: `List<String>` is compiled down to raw `List` bytecode with compiler-inserted casts, since **generics are a compile-time-only feature** — no generic type information exists in the actual bytecode (this is why you can't do `list instanceof List<String>` at runtime).
- 5 **Desugaring** — converting "syntactic sugar" into more primitive equivalent code:
  - Enhanced for-loop `for (String s : list)` → desugars to an explicit `Iterator`-based loop.
  - Try-with-resources → desugars to an explicit try-finally block calling `close()`.
  - Autoboxing `Integer i = 5;` → desugars to `Integer i = Integer.valueOf(5);`.
  - Lambda expressions (Java 8+) → desugar to `invokedynamic` bytecode instructions bound to synthetic methods (NOT simply anonymous inner classes, contrary to popular belief — this is a common interview trick question).
  - Varargs `void foo(String... args)` → compiled as `String[] args` under the hood.

### 3.5 Phase 4 — Bytecode Generation (Deep Dive)

The compiler emits the final `.class` binary file. This includes populating:

- The **Constant Pool** — a table of all literals, class/method/field references used in the class.
  - **Method bytecode** — a sequence of stack-based instructions (opcodes) per method.
  - **Attributes** — metadata like line number tables (for stack traces), local variable tables (for debuggers), and annotations.
- 

## 4. Diagrams

### 4.1 Full Compilation Pipeline (End-to-End)

```
HelloWorld.java
|
v
+-----+
| javac |
| |
| [Lexer] -> tokens -> [Parser] -> AST -> [Semantic Analyzer] |
| -> annotated/resolved AST -> [Bytecode Generator] |
+-----+
|
v
HelloWorld.class (binary bytecode file)
|
v
[Loaded by JVM's Class Loader at runtime – see Topic 2]
```

### 4.2 `.class` File Binary Structure (JVMS §4.1)

```
+-----+
| Magic Number 0xCAFEBABE (4 bytes) |
+-----+
| Minor Version (2 bytes) |
| Major Version (2 bytes) e.g. 61 = Java 17 |
+-----+
| Constant Pool Count (2 bytes) |
| Constant Pool [table of literals/refs] |
+-----+
| Access Flags (2 bytes) e.g. public, final |
+-----+
| This Class (2 bytes, index into pool) |
| Super Class (2 bytes, index into pool) |
+-----+
| Interfaces Count + Interfaces[] |
+-----+
| Fields Count + Fields[] |
+-----+
| Methods Count + Methods[] (contains actual bytecode) |
+-----+
| Attributes Count + Attributes[] |
| (e.g. SourceFile, LineNumberTable, LocalVariableTable) |
+-----+
```

### 4.3 Generics Type Erasure Diagram

SOURCE CODE (what you write):

```
List<String> names = new ArrayList<String>();
names.add("Java");
String first = names.get(0);
```

AFTER TYPE ERASURE (what's actually compiled to bytecode - conceptually):

```
List names = new ArrayList(); // generic info REMOVED
names.add("Java");
String first = (String) names.get(0); // compiler auto-inserts CAST
```

=> At runtime, the JVM has NO IDEA the list was ever "of Strings" – this is why `list.getClass() == otherList.getClass()` is TRUE even if one is `List<String>` and other is `List<Integer>`.

### 4.4 Lambda Desugaring — `invokedynamic` (Not an Anonymous Class!)

SOURCE CODE:

```
Runnable r = () -> System.out.println("Hello");
```

COMMON MISCONCEPTION: "This compiles to an anonymous inner class"

REALITY (verified via `javap -c`):

The lambda body is compiled into a private synthetic method (e.g., `lambda$main$0`), and the call site uses the `invokedynamic` bytecode instruction with a bootstrap method (`LambdaMetafactory`) that generates the actual functional interface implementation CLASS AT RUNTIME (on first invocation), not at compile time.

This is more efficient than generating a full `.class` file per lambda at compile time (which is what anonymous classes do).

## 5. Syntax — Compiler Flags and Their Exact Meaning

```
javac [options] SourceFile.java
```

| Flag                             | Meaning                                                                                                                                                | Example                                         |
|----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------|
| <code>-d &lt;dir&gt;</code>      | Output directory for generated <code>.class</code> files                                                                                               | <code>javac -d out/ App.java</code>             |
| <code>-cp / -classpath</code>    | Where to find dependency classes during compilation                                                                                                    | <code>javac -cp libs/*.jar App.java</code>      |
| <code>--release &lt;N&gt;</code> | Compile targeting exactly Java version N's language features AND API set                                                                               | <code>javac --release 8 App.java</code>         |
| <code>-source &lt;N&gt;</code>   | (Legacy) Restrict language features to version N (does NOT restrict API — can cause <code>--release</code> vs <code>-source/-target</code> mismatches) | <code>javac -source 8 -target 8 App.java</code> |
| <code>-target &lt;N&gt;</code>   | (Legacy) Set the output bytecode's target JVM version                                                                                                  | (used with <code>-source</code> )               |
| <code>-Xlint:all</code>          | Enable all recommended compiler warnings (unchecked casts, deprecation, etc.)                                                                          | <code>javac -Xlint:all App.java</code>          |
| <code>-g</code>                  | Include ALL debugging info (line numbers, local variable names, source file) in the <code>.class</code> file                                           | <code>javac -g App.java</code>                  |

| Flag                       | Meaning                                                                                       | Example                                            |
|----------------------------|-----------------------------------------------------------------------------------------------|----------------------------------------------------|
| <code>-g:none</code>       | Strip ALL debugging info (smaller <code>.class</code> , but no useful stack traces/debugging) | <code>javac -g:none App.java</code>                |
| <code>-verbose</code>      | Print detailed info about each compilation step (files loaded, class files written)           | <code>javac -verbose App.java</code>               |
| <code>-Werror</code>       | Treat all warnings as compile errors (strict CI/CD pipelines often enable this)               | <code>javac -Xlint:all -Werror App.java</code>     |
| <code>--add-modules</code> | Include specific modules during compilation (Java 9+ module system)                           | <code>javac --add-modules java.sql App.java</code> |

**Why `--release` is preferred over `-source/-target` together (interview favorite):**

Using `-source 8 -target 8` only restricts the **bytecode version and language syntax** — it does **NOT** prevent you from accidentally calling a Java 11-only API method while still targeting Java 8 bytecode, because the compiler still compiles against the **current** JDK's full class library by default. `--release 8` fixes this by compiling against a **snapshot of the actual Java 8 API surface**, catching such mistakes at compile time instead of causing a runtime `NoSuchMethodError` in production.

## 6. Every Method / Component — Deep Reference

### 6.1 The Constant Pool

| Aspect         | Detail                                                                                                                                                                                                                                                            |
|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Purpose        | A table (per-class) storing all constants: string literals, class/method/field symbolic references, numeric literals                                                                                                                                              |
| Why it exists  | Bytecode instructions are extremely compact (often just 1-3 bytes) — instead of embedding full class/method names directly in every instruction, they reference an <b>index into the Constant Pool</b> , keeping bytecode small and reusable                      |
| Inspect it via | <code>javap -v YourClass</code> (verbose mode shows the full constant pool table)                                                                                                                                                                                 |
| Common mistake | Assuming string literals are always deduplicated across the ENTIRE application — they're only guaranteed pooled within the JVM's <b>String Pool</b> at runtime (a heap-adjacent intern pool), which draws from each class's constant pool entries as classes load |

## 6.2 Bytecode Instructions (Opcodes) — Key Families

| Instruction Family | Example Opcodes                                                                                                                                 | Purpose                                                                                             |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| Load/Store         | <code>iload</code> , <code>istore</code> , <code>aload</code> , <code>astore</code>                                                             | Move values between local variable slots and the operand stack                                      |
| Arithmetic         | <code>iadd</code> , <code>isub</code> , <code>imul</code> , <code>idiv</code>                                                                   | Perform arithmetic on values on the operand stack                                                   |
| Type Conversion    | <code>i2l</code> , <code>i2d</code> , <code>d2i</code>                                                                                          | Convert between primitive types                                                                     |
| Object Creation    | <code>new</code> , <code>newarray</code> , <code>anewarray</code>                                                                               | Allocate new objects/arrays on the heap                                                             |
| Method Invocation  | <code>invokevirtual</code> , <code>invokestatic</code> , <code>invokespecial</code> , <code>invokeinterface</code> , <code>invokedynamic</code> | Call methods (each variant has different dispatch rules — covered deeply in OOP/Polymorphism topic) |
| Control Flow       | <code>ifeq</code> , <code>goto</code> , <code>tableswitch</code> , <code>lookupswitch</code>                                                    | Conditional/unconditional jumps, switch statements                                                  |
| Return             | <code>ireturn</code> , <code>areturn</code> , <code>return</code>                                                                               | Return from a method (typed by return value kind)                                                   |

**Time/Space note:** Each bytecode instruction executes in the interpreter as effectively  $O(1)$  per instruction (ignoring what the instruction itself does, like a method call which recurses into more instructions) — the overall program's complexity is determined by your algorithm, not by compilation.

## 7. Code Examples

### 7.1 Basic — Viewing the Compiled `.class` File's Header Bytes

```
import java.io.FileInputStream;
import java.io.IOException;

public class ClassFileHeaderReader {
    public static void main(String[] args) throws IOException {
        try (FileInputStream fis = new FileInputStream("ClassFileHeaderReader.class")) {
            byte[] header = new byte[8];
            fis.read(header);
            System.out.printf("Magic Number: 0x%02X%02X%02X%02X%n",
                header[0], header[1], header[2], header[3]);
            int minor = ((header[4] & 0xFF) << 8) | (header[5] & 0xFF);
            int major = ((header[6] & 0xFF) << 8) | (header[7] & 0xFF);
            System.out.println("Minor Version: " + minor);
            System.out.println("Major Version: " + major + " (Java " + (major - 44) + ")");
        }
    }
}
```

**Note:** Major version 61 = Java 17, 52 = Java 8 (formula: `major - 44` gives the Java version for versions 49+).

## 7.2 Intermediate — Observing Type Erasure Firsthand

```
import java.util.ArrayList;
import java.util.List;

public class TypeErasureDemo {
    public static void main(String[] args) {
        List<String> stringList = new ArrayList<>();
        List<Integer> intList = new ArrayList<>();

        // Proves generics info is erased at runtime:
        System.out.println(stringList.getClass() == intList.getClass()); // true!

        // This would NOT compile (generics only exist at compile-time):
        // if (stringList instanceof List<String>) { } // COMPILE ERROR
    }
}
```

## 7.3 Advanced — Compiling with `--release` to Catch API Misuse Early

```
// File: NewApiUsage.java
import java.util.List;

public class NewApiUsage {
    public static void main(String[] args) {
        // List.of() was introduced in Java 9 – NOT available in Java 8
        List<String> names = List.of("Alice", "Bob");
        System.out.println(names);
    }
}

# This will FAIL to compile with a clear error, catching the mistake early:
javac --release 8 NewApiUsage.java
# error: cannot find symbol
# symbol: method of(String,String)
# location: interface List
```

Without `--release`, using just `-source 8 -target 8`, this mistake would compile successfully against your current (newer) JDK's `List` class, but then throw `NoSuchMethodError` when actually run on a real Java 8 JVM in production — a dangerous, hard-to-catch bug that `--release` prevents entirely.

## 7.4 Production-style Example — Enforcing Strict Compilation in a Build Pipeline

```
javac --release 17 \
-Xlint:all \
-Werror \
-d target/classes \
-cp "libs/*" \
$(find src/main/java -name "*.java")
```

**Why this is production-quality:** Real CI/CD pipelines enforce `-Werror` (no warnings allowed) and pin `--release` explicitly, ensuring the exact same compilation strictness across every developer's machine and every build server, preventing "it compiled fine on my machine" class-version mismatches.

## 8. Dry Run

### Source:

```
public class CompileTrace {  
    public static void main(String[] args) {  
        int x = 5;  
        int y = x * 2;  
        System.out.println(y);  
    }  
}
```

### Trace Through All Four Compiler Phases

| Phase                  | What Happens to This Code                                                                                                                                                                                                                                                 |
|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. Lexical Analysis    | Tokens produced: KEYWORD(public), KEYWORD(class), IDENTIFIER(CompileTrace), LBRACE, ... KEYWORD(int), IDENTIFIER(x), OP(=), LITERAL(5), SEMI, ... IDENTIFIER(System), DOT, IDENTIFIER(out), DOT, IDENTIFIER(println), LPAREN, IDENTIFIER(y), RPAREN, SEMI, RBRACE, RBRACE |
| 2. Syntax Analysis     | Builds an AST: ClassDeclaration(CompileTrace) → MethodDeclaration(main) → [VarDecl(x,5), VarDecl(y, BinaryExpr(x,*,2)), MethodCall(System.out.println, [y])]                                                                                                              |
| 3. Semantic Analysis   | Symbol table: x:int, y:int registered in main's scope. Type check: $x * 2 \rightarrow \text{int} * \text{int} = \text{int}$ ✓ valid. println(int) overload resolved (matches PrintStream.println(int)). No errors.                                                        |
| 4. Bytecode Generation | Emits CompileTrace.class with a main method containing bytecode like: iconst_5 istore_1 iload_1 iconst_2 imul istore_2 getstatic System.out iload_2 invokevirtual println(I)V return                                                                                      |

### Verify it yourself:

```
javac CompileTrace.java  
javap -c CompileTrace
```

## 9. Edge Cases

| Edge Case                                                                                                      | What Happens                                                                                               | Why                                                                                                                                 |
|----------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| Unicode escape sequences in source (A) this looks like one line but isn't!')                                   | Processed <b>before</b> lexical analysis even begins (a distinct, earlier "translation" step per JLS)      | Java allows Unicode escapes anywhere in source, even inside what looks like comments — a classic obscure gotcha ( <code>//</code> ) |
| Two classes in different files with identical fully-qualified names on the classpath                           | Compile-time or class-loading ambiguity/error depending on classpath order                                 | The compiler/loader can only resolve one definitive symbol per fully-qualified name                                                 |
| Compiling code using <code>var</code> (Java 10+) with <code>--release 8</code>                                 | Compile-time error — <code>var</code> is a contextual keyword unavailable in Java 8                        | <code>--release</code> restricts language features to the target version                                                            |
| Extremely deep expression nesting (thousands of nested parentheses)                                            | Can cause <code>StackOverflowError</code> <b>inside the compiler itself</b> (the recursive-descent parser) | Parsing many languages' grammars (including Java's) is implemented recursively                                                      |
| Diamond operator <code>&lt;&gt;</code> with ambiguous type inference                                           | Compile-time error requiring explicit type arguments                                                       | Semantic analysis cannot always infer the intended generic type unambiguously                                                       |
| Compiling a file with no package statement, then another with a package statement, both referencing each other | Can cause "cannot find symbol" unless proper classpath/package structure is used                           | Java requires directory structure to (usually) mirror package declarations                                                          |
| Source file encoding mismatch (e.g., UTF-8 source read as a different encoding)                                | Garbled string literals or lexical errors on special characters                                            | <code>javac -encoding UTF-8</code> should be specified explicitly in production builds to avoid platform-default encoding surprises |

## 10. Internal Interview Questions

### Easy

1 **Q: What are the four main phases of Java compilation?**

A: Lexical analysis, syntax analysis, semantic analysis, bytecode generation.

2 **Q: What is a token?**

A: The smallest meaningful unit of source code identified during lexical analysis (keywords, identifiers, literals, operators, punctuation).

### Medium

1 **Q: What is type erasure, and why does Java use it for generics?**

A: Type erasure removes generic type parameter information during compilation, replacing it with raw types and inserted casts, for **backward compatibility** with pre-Java-5 bytecode/libraries that had no concept of generics — allowing generic and non-generic code to interoperate seamlessly on the same JVM.

2 **Q: Do lambda expressions compile into anonymous inner classes?**

A: No — a common misconception. Lambdas compile into a private synthetic method plus an `invokedynamic` call site bound to `LambdaMetafactory`, which generates the actual implementing class **at runtime on first invocation**, not at compile time like anonymous classes.



### 3 Q: What's the difference between `--release` and using `-source/-target` together?

A: `--release N` restricts BOTH language syntax AND the API surface to exactly what existed in version N. `-source/-target` only restricts language syntax/bytecode version but still compiles against the CURRENT JDK's full API, risking accidental use of newer methods that don't exist at runtime on an older JVM.

## Hard

### 1 Q: Why can bytecode instructions be so compact (often 1-3 bytes), given how verbose class/method names can be?

A: Because instructions reference an **index into the Constant Pool** rather than embedding full names/literals directly — the actual string data lives once in the pool, and instructions just point to it.

### 2 Q: Explain what happens if you use `instanceof List<String>` — why is it a compile error?

A: Because of type erasure, generic type parameters don't exist in bytecode at runtime — the JVM literally cannot check "is this specifically a List of Strings" since that information was erased during compilation; only `raw instanceof List` is legal.

## Company-Level / Tricky

### 1 (Tricky) Q: If Unicode escapes are processed before lexical analysis, can (newline) inside what looks like a single-line comment actually break the comment early?

A: Yes — this is a genuinely obscure but real JLS behavior: `// some text this becomes code, not comment!` because Unicode translation happens in an earlier phase than comment/token recognition, so the escape becomes an actual newline character before the lexer even decides where the comment ends.

### 2 (Cross-question) Q: Since generics are erased, how does the JVM prevent you from accidentally adding an `Integer` to a `List<String>` obtained via unchecked reflection tricks?

A: It doesn't, at the bytecode level — this is exactly why `ClassCastException` can occur later when retrieving an element and the compiler-inserted cast fails; type safety for generics is a **compile-time-only guarantee**, not enforced by the runtime type system.

---

## 11. Coding Questions

### Easy

**Q1: Write a program and use `javap -v` to find the exact major/minor version number of its compiled `.class` file, then map it to the Java version name.**

```
javac HelloWorld.java
javap -v HelloWorld | findstr "major minor"
```

(Major version 61 = Java 17, 55 = Java 11, 52 = Java 8.)

### Medium

**Q2: Write two overloaded methods that would look identical after generic type erasure, and observe the compile-time error Java gives you.**

```
import java.util.List;

public class ErasureClash {
```

```
// These two methods erase to the SAME signature: process(List) -> void
// Uncommenting the second causes: "name clash" compile-time error
static void process(List<String> list) { }
// static void process(List<Integer> list) { } // COMPILE ERROR if uncommented
}
```

**Explanation:** After type erasure, both methods would have the identical bytecode signature `process(Ljava/util/List;)V` — the compiler catches this ambiguity at compile time and refuses to allow it.

## Hard (Company Tagged: Compiler-internals rounds, e.g., Oracle/product-based companies)

**Q3: Demonstrate, using `javap -c`, that a `for-each` loop over a `List` desugars into explicit `Iterator` calls.**

```
import java.util.List;
import java.util.ArrayList;

public class ForEachDesugar {
    public static void main(String[] args) {
        List<String> items = new ArrayList<>();
        items.add("A");
        for (String item : items) {
            System.out.println(item);
        }
    }
}

javac ForEachDesugar.java
javap -c ForEachDesugar
```

**Expected bytecode observation:** You'll see explicit calls to `invokeinterface java/util/List.iterator`, `invokeinterface java/util/Iterator.hasNext`, and `invokeinterface java/util/Iterator.next` — proving the enhanced for-loop is pure syntactic sugar with zero special bytecode instructions of its own.

## 12. Common Mistakes

### Beginner Mistakes

- 1 Assuming `-source/-target` alone is enough to guarantee cross-version compatibility — missing that it doesn't restrict the API surface (should use `--release` instead).
- 2 Confusing compile-time generic type safety with runtime enforcement — being surprised by `ClassCastException` in code that "should have been type-safe" due to erasure + unchecked warnings being ignored.
- 3 Not realizing enhanced for-loops, try-with-resources, and autoboxing are all "sugar" that desugars into more verbose equivalent code — leading to confusion when reading decompiled/disassembled bytecode that "doesn't match" the source.

### Experienced Developer Mistakes

- 1 Ignoring `-Xlint` compiler warnings (like unchecked cast warnings from generics) in day-to-day development, only to have them manifest as production `ClassExceptions` months later.

- 2 Not pinning `--release` in build tools, leading to accidental use of newer JDK APIs that silently pass CI (built and tested on a newer JDK) but fail with `NoSuchMethodError` in an older production runtime environment.
- 3 Assuming lambda expressions have identical performance characteristics to anonymous inner classes without verifying via bytecode inspection or benchmarking — the `invokedynamic`-based lazy class generation has different startup/warm-up characteristics.

### *Debugging Process for Erasure-related* `ClassCastException`

- 1 Identify the exact line throwing the exception via the stack trace.
  - 2 Check if unchecked/raw-type warnings were present at compile time (`-Xlint:unchecked`) around that code path.
  - 3 Trace back to where a raw type or unchecked cast bypassed the compiler's normal generic type safety.
- 

## 13. Best Practices

- 1 Always compile with `--release <target-version>` explicitly in build tools/CI, never rely on implicit "whatever JDK happens to be installed."
  - 2 Enable `-Xlint:all` and treat unchecked/generic warnings seriously — they signal potential runtime `ClassCastException` risk.
  - 3 Never suppress `@SuppressWarnings("unchecked")` without a clear, documented justification comment explaining why it's actually safe.
  - 4 Use `javap -c/-v` periodically during learning to build accurate mental models of what your code "actually becomes" — don't rely purely on source-level intuition for performance-sensitive code.
  - 5 Keep source file encoding explicit (`-encoding UTF-8`) in build configuration to avoid platform-dependent compilation surprises.
- 

## 14. Production Usage

- **Build tools** (Maven/Gradle) wrap `javac` internally, exposing `--release/source/target` as simple configuration properties (`<maven.compiler.release>17</maven.compiler.release>`).
  - **Static analysis tools** (SonarQube, Checkstyle, SpotBugs) operate on the AST or bytecode produced by this pipeline to detect code smells and bugs before deployment.
  - **Bytecode manipulation libraries** (ASM, Byte Buddy, used internally by Spring/Hibernate for proxies and AOP) operate directly on the `.class` file structure described in Section 4.2 — understanding this structure is foundational to understanding how frameworks generate dynamic proxies at runtime.
  - **Decompilers** (CFR, Fernflower — used inside IntelliJ IDEA to show "decompiled" library source) reverse this pipeline approximately, turning bytecode back into readable (though re-sugared, not identical) Java-like source.
-

## 15. Performance

- **Compilation itself** is a one-time, build-time cost — it doesn't affect runtime performance of the final program, but slow compilation impacts developer iteration speed and CI pipeline duration.
  - **Type erasure has a runtime performance benefit:** since no generic type metadata needs to be checked/stored per-object at runtime, there's zero runtime overhead for using generics compared to raw types — a deliberate design trade-off (compile-time-only safety, in exchange for zero runtime cost).
  - **Desugaring choices matter for performance:** lambdas' `invokedynamic` approach has a small one-time class-generation cost on first invocation per lambda call site, but avoids the class-loading overhead of many separate anonymous-class `.class` files that would otherwise bloat JAR size and class-loading time at startup.
- 

## 16. Frequently Asked Interview Questions (50+ with Detailed Answers)

- 1 **What are the four phases of Java compilation?** Lexical analysis, syntax analysis, semantic analysis, bytecode generation.
- 2 **What does the lexer/scanner do?** Converts raw source characters into a stream of tokens (keywords, identifiers, literals, operators, punctuation), discarding whitespace/comments.
- 3 **What does the parser do?** Builds an Abstract Syntax Tree (AST) from the token stream, validating it against the JLS grammar.
- 4 **What is an AST?** Abstract Syntax Tree — a tree representation of the program's grammatical structure used internally by the compiler.
- 5 **What happens during semantic analysis?** Symbol table construction, type checking, scope/name resolution, generics erasure, and desugaring of syntactic sugar.
- 6 **What is type erasure?** The process of removing generic type parameter information at compile time, replacing it with raw types and compiler-inserted casts, for backward compatibility.
- 7 **Why does Java use type erasure instead of reifying generics (like C#)?** To maintain backward compatibility with the vast amount of pre-Java-5 bytecode and libraries, allowing generic and non-generic code to interoperate on the same JVM without a bytecode format change.
- 8 **Does `instanceof` work with parameterized generic types (e.g., `list instanceof List<String>`)?** No — this is a compile-time error, because generic type info doesn't exist at runtime due to erasure.
- 9 **What is desugaring?** The compiler's process of translating higher-level, convenient syntax (sugar) like enhanced for-loops, try-with-resources, and autoboxing into more primitive, explicit equivalent code.
- 10 **Do lambda expressions compile to anonymous inner classes?** No — they compile to a private synthetic method plus an `invokedynamic` instruction using `LambdaMetafactory`, generating the implementing class lazily at runtime.
- 11 **What is the Constant Pool?** A per-class table storing all literal values and symbolic references (classes, methods, fields), allowing bytecode instructions to stay compact by referencing pool indices instead of embedding data directly.

- 12 **What is the magic number in a `.class` file?** `0xCAFEBABE` — the first 4 bytes identifying the file as valid Java bytecode.
- 13 **How do you find a `.class` file's target Java version?** Inspect its major version field (via `javap -v` or manual byte reading); e.g., major version 61 = Java 17, 52 = Java 8.
- 14 **What's the difference between `--release` and `-source/-target`?** `--release` restricts both language features AND the API surface to the target version; `-source/-target` alone only restricts syntax/bytecode version, still compiling against the current JDK's full (possibly newer) API.
- 15 **What's a real production risk of NOT using `--release`?** Accidentally calling a newer JDK API method that compiles fine (against the build machine's JDK) but throws `NoSuchMethodError` when deployed to an older production JVM.
- 16 **What does `javap` do?** Disassembles compiled `.class` files, showing method signatures and (with `-c`) actual bytecode instructions.
- 17 **What is `-Xlint` used for?** Enabling additional recommended compiler warnings (unchecked casts, deprecation usage, etc.) not shown by default.
- 18 **What does `-Werror` do?** Treats all compiler warnings as hard errors, failing the build — commonly used in strict CI/CD pipelines.
- 19 **Why can extremely deeply nested expressions crash the compiler itself?** Because `javac`'s parser is typically implemented as a recursive-descent parser, and extreme nesting can exhaust the compiler's own JVM call stack, causing a `StackOverflowError` during compilation.
- 20 **What does `invokevirtual` do?** A bytecode instruction that invokes an instance method using dynamic (runtime) dispatch based on the actual object's class.
- 21 **What does `invokestatic` do?** Invokes a static method — resolved at compile time, no dynamic dispatch needed since static methods aren't polymorphic.
- 22 **What does `invokespecial` do?** Invokes constructors, private methods, and superclass methods — cases where dynamic dispatch is NOT wanted (exact method must be called).
- 23 **What does `invokeinterface` do?** Invokes a method declared in an interface, requiring extra runtime lookup since the actual implementing class isn't known until runtime.
- 24 **What does `invokedynamic` do?** A more flexible invocation mechanism (added in Java 7, heavily used by lambdas in Java 8+) that defers the exact method-linking logic to a runtime-generated "call site," enabling behaviors like lazy lambda class generation.
- 25 **What is a Bootstrap Method in the context of `invokedynamic`?** A special method (e.g., in `LambdaMetafactory`) invoked once per call site to determine how to actually link/dispatch that dynamic call, typically generating and caching an implementation class.
- 26 **What is the significance of the Java Language Specification (JLS)?** It's the authoritative document defining exact Java language grammar, semantics, and compilation rules — any compliant compiler (`javac`, `ECJ`) must produce behavior matching it.
- 27 **What is the significance of the Java Virtual Machine Specification (JVMS)?** It defines the exact `.class` file format and bytecode instruction semantics that any compliant JVM must support, independent of which language/compiler produced the bytecode.

- 28 Can languages other than Java produce valid JVM bytecode?** Yes — Kotlin, Scala, Groovy, Clojure all compile to standard JVM bytecode conforming to the JVM specification, proving the bytecode format is language-agnostic.
- 29 What is autoboxing, and when does it happen during compilation?** The automatic conversion of a primitive to its wrapper type (e.g., `int` to `Integer`); it's inserted during semantic analysis/desugaring as an explicit `Integer.valueOf()` call.
- 30 What is a symbol table?** An internal compiler data structure mapping declared identifiers (variables, methods, classes) to their type and scope information, used throughout semantic analysis.
- 31 What is name clash due to erasure?** When two overloaded generic methods would produce identical bytecode signatures after type erasure, causing a compile-time error since the JVM cannot distinguish them.
- 32 What tool would you use to see the raw bytes of a `.class` file's header?** A hex editor, or a small custom Java program reading the file as raw bytes (as shown in Section 7.1), or `javap -v`.
- 33 Are comments and whitespace preserved in the AST?** No — they are discarded during lexical analysis since they carry no semantic meaning for compilation (though some tools preserve them separately for formatting/documentation purposes).
- 34 What is Unicode translation, and when does it happen relative to lexical analysis?** A JLS-mandated step that converts `\uxxxx` escape sequences into actual Unicode characters BEFORE lexical analysis even begins — meaning such escapes can appear (and have effect) even inside what visually looks like a comment.
- 35 What does the `-g` compiler flag control?** How much debugging information (line numbers, local variable names, source file name) is embedded in the output `.class` file.
- 36 Why might a production build use `-g:none`?** To slightly reduce `.class` file size and obscure internal variable names, though this comes at the cost of much less useful stack traces during production debugging — a real trade-off.
- 37 What is a class file's "major version" used for?** It tells any JVM loading the file the exact bytecode format/feature version it must support to run it correctly; mismatches cause `UnsupportedClassVersionError`.
- 38 What is the relationship between the JLS and `javac`?** `javac` is Oracle's reference implementation that must conform to the rules defined in the JLS; other compilers (like Eclipse's ECJ) independently implement the same specification.
- 39 Why does try-with-resources desugar into a try-finally block?** Because prior to Java 7, resource cleanup required manually written finally blocks calling `close()`; try-with-resources is pure syntactic sugar generating that same boilerplate automatically, including proper suppressed-exception handling.
- 40 What does varargs (`String... args`) compile down to?** A regular array parameter (`String[] args`); the calling syntax sugar (passing individual arguments) is converted into implicit array creation at each call site.
- 41 Is bytecode instruction execution time related to Big-O notation of your algorithm?** No directly — each bytecode instruction is roughly constant-time at the instruction-dispatch level; your algorithm's overall complexity (loops, recursion) is what determines Big-O, not the compilation process itself.
- 42 What is the benefit of compiling to an intermediate bytecode instead of directly to native code?** Platform independence (WORA) — the same bytecode runs on any conforming JVM, decoupling the compiler from any specific CPU architecture.

- 43 Can bytecode be decompiled back to readable Java-like source?** Yes, approximately, using decompilers (CFR, Fernflower, JD-GUI) — though desugared constructs (like lambdas' `invokedynamic`) may not perfectly "re-sugar" back to the exact original source form.
- 44 What is ASM (as used by frameworks like Spring/Hibernate)?** A low-level bytecode manipulation/generation library that works directly with the `.class` file format, used internally to generate dynamic proxy classes at runtime for AOP, ORM lazy-loading, etc.
- 45 Why do frameworks like Hibernate generate proxy classes at runtime instead of requiring you to write them?** To implement lazy-loading and interception transparently — bytecode generation libraries let them synthesize new `.class` bytes on the fly conforming to the same format `javac` produces, without requiring the developer to hand-write proxy boilerplate.
- 46 What's a real interview red flag if a candidate says "Java compiles directly to machine code"?** It shows a fundamental misunderstanding of Java's architecture — Java compiles to an intermediate bytecode format, NOT directly to native machine code (that final native translation is the JVM's/JIT's job at runtime, not `javac`'s).
- 47 Does the compiler perform any optimization, or is that entirely the JVM's job?** `javac` performs minimal optimization (mostly constant folding of compile-time-constant expressions like `"a" + "b" → "ab"`); the vast majority of runtime performance optimization (inlining, dead code elimination, escape analysis) is the JIT compiler's responsibility at runtime, not `javac`'s.
- 48 What is constant folding?** A compile-time optimization where expressions involving only compile-time constants (e.g., `final int x = 2 + 3;`) are pre-computed by the compiler and embedded directly as a single literal in the bytecode/constant pool.
- 49 Why is understanding the compilation process valuable for framework developers specifically?** Because frameworks that generate/manipulate bytecode (Spring AOP, Hibernate, Mockito) directly interact with the exact `.class` file structures and instruction semantics this topic covers — deep framework debugging often requires this knowledge.
- 50 What would happen if you tried to run `.class` files produced by a non-`javac` JVM language compiler (e.g., Kotlin) using the standard `java` command?** It would work perfectly fine, PROVING the JVM only cares about valid, spec-conformant bytecode — it has no awareness of, or dependency on, which source language or compiler originally produced that bytecode.
-

## 17. Revision Notes (One-Page Cheat Sheet)

### JAVA COMPILATION PROCESS – QUICK REVISION SHEET

=====

#### FOUR PHASES:

1. Lexical Analysis -> raw chars -> TOKENS (discards whitespace/comments)
2. Syntax Analysis -> tokens -> AST (per JLS grammar rules)
3. Semantic Analysis -> symbol table, type check, name resolution, GENERICS ERASURE, DESUGARING (lambdas, for-each, try-w-resources, autoboxing, varargs)
4. Bytecode Generation -> emits .class file (Constant Pool + method bytecode)

#### KEY FACTS:

- Generics: COMPILE-TIME ONLY (erased) -> zero runtime overhead, but ``instanceof List<String>`` is a compile ERROR
- Lambdas: compile to invokedynamic + LambdaMetafactory (NOT anon classes!)
- Enhanced for-loop: desugars to explicit Iterator calls
- try-with-resources: desugars to try-finally calling close()

#### .class FILE STRUCTURE (JVM §4.1):

Magic (0xCAFEFABE) -> Version -> Constant Pool -> Access Flags -> This/Super Class -> Interfaces -> Fields -> Methods -> Attributes

#### COMPILER FLAGS TO REMEMBER:

- release N : restricts BOTH syntax AND API to version N (PREFERRED)
- source/-target : restricts syntax/bytecode only, NOT the API (risky!)
- Xlint:all : show all recommended warnings
- Werror : treat warnings as errors
- g / -g:none : include / strip debug info

#### MAJOR VERSION QUICK MAP:

52 = Java 8 55 = Java 11 61 = Java 17 65 = Java 21

## 18. Homework

- 1 Write a small program using generics, compile it, and use `javap -v` to confirm the Constant Pool and method signatures show ERASED (raw) types.
- 2 Write a lambda expression, compile it, and use `javap -c` to find the `invokedynamic` instruction and the synthetic `lambda$...method` it generates.
- 3 Compile the same file once with `--release 8` and once with `--release 17` (using an API method only available in 17, e.g., a `String` method added later) and document the exact difference in outcome.
- 4 Manually read the first 8 bytes of any `.class` file (using a hex editor or the code from Section 7.1) and confirm the magic number and major version.
- 5 Write 5 sentences in your own words explaining why type erasure means generics provide "compile-time-only" safety.

## 19. Mini Project — "Bytecode Inspector" CLI Tool

**Goal:** Build a small CLI tool that compiles a given `.java` file and reports its class file's version, constant pool size, and method count — reinforcing everything learned about the `.class` file format.



```

package com.jobsradar.compiler;

import javax.tools.JavaCompiler;
import javax.tools.ToolProvider;
import java.io.DataInputStream;
import java.io.FileInputStream;
import java.io.IOException;
import java.io.File;

/**
 * Mini Project: Bytecode Inspector
 * Compiles a .java file programmatically (via the Compiler API) then reads
 * and reports basic structural facts about the resulting .class file.
 */
public class BytecodeInspector {

    public static void main(String[] args) throws IOException {
        if (args.length == 0) {
            System.out.println("Usage: java BytecodeInspector <SourceFile.java>");
            return;
        }

        String sourceFile = args[0];
        compile(sourceFile);

        String className = new File(sourceFile).getName().replace(".java", "");
        inspectClassFile(className + ".class");
    }

    private static void compile(String sourceFile) {
        JavaCompiler compiler = ToolProvider.getSystemJavaCompiler();
        int result = compiler.run(null, null, null, sourceFile);
        if (result != 0) {
            throw new IllegalStateException("Compilation failed for: " + sourceFile);
        }
        System.out.println("Compiled successfully: " + sourceFile);
    }

    private static void inspectClassFile(String classFile) throws IOException {
        try (DataInputStream in = new DataInputStream(new FileInputStream(classFile))) {
            int magic = in.readInt();
            int minor = in.readUnsignedShort();
            int major = in.readUnsignedShort();
            int constantPoolCount = in.readUnsignedShort();

            System.out.printf("Magic Number : 0x%08X%n", magic);
            System.out.println("Minor Version : " + minor);
            System.out.println("Major Version : " + major + " (Java " + (major - 44) + ")");
            System.out.println("Constant Pool Sz : " + constantPoolCount + " entries (approx, raw count)");
        }
    }
}

```

**Why this matters:** This mini project uses the **Java Compiler API** (`javax.tools.JavaCompiler`) — the same mechanism build tools and IDEs use to invoke `javac` programmatically — and directly parses the binary structure covered in Section 4.2, connecting theory to a real runnable tool.

## 20. Final Summary

- Java compilation is a **four-phase pipeline**: Lexical Analysis (tokenizing) → Syntax Analysis (AST building) → Semantic Analysis (type checking, name resolution, generics erasure, desugaring) → Bytecode Generation (emitting the `.class` binary).
- **Generics are erased at compile time** — this provides compile-time-only type safety with zero runtime overhead, but also means `instanceof List<String>` is illegal, and raw `getClass()` comparisons across different generic instantiations return `true`.
- **Lambdas do NOT compile into anonymous inner classes** — they use `invokedynamic` + `LambdaMetafactory` to lazily generate implementing classes at runtime, a key interview differentiator from naive assumptions.
- **Desugaring** converts convenient syntax (enhanced for-loops, try-with-resources, autoboxing, varargs) into more primitive, explicit equivalent bytecode-generating constructs.
- The `.class` file format is a precisely specified binary structure (JVMS §4.1) starting with the magic number `0xCAFEFABE`, followed by version info, the Constant Pool, and method/field/attribute tables.
- Always prefer `--release <version>` over the legacy `-source/-target` pair in build configurations — it's the only flag that restricts BOTH syntax and the actual API surface, preventing dangerous `NoSuchMethodErrors` in production.
- Understanding this pipeline is foundational for debugging framework "magic" (Spring AOP proxies, Hibernate lazy-loading proxies) since they operate directly on this same `.class` file format using bytecode manipulation libraries like ASM.

---

*(End of Topic 3: Java Compilation Process.)*

---

## TOPIC 4: JAVA MEMORY MODEL

---

### 1. Introduction

#### 1.1 What is it?

The **Java Memory Model (JMM)** describes two related things that are often taught together:

1. **Runtime memory layout** — how the JVM divides memory into regions (Heap, Stack, Metaspace, PC Register, Native Method Stack) and where different kinds of data (objects, local variables, class metadata) actually live.
2. **The formal JMM (JLS Chapter 17)** — the rules governing how threads interact through memory: visibility of writes across threads, reordering guarantees, and the `happens-before` relationship. This second meaning becomes critical in the Multithreading topic later, but its foundation is introduced here.

## 1.2 Why was it introduced

Without a well-defined memory model, different JVM implementations (or even the same JVM with different CPU/compiler optimizations) could behave inconsistently — a variable updated by one thread might never become visible to another thread due to CPU caching or instruction reordering. The JMM was formalized (notably reworked in **JSR-133**, Java 5, 2004) precisely to give developers **portable, well-defined guarantees** about memory visibility and ordering, regardless of the underlying hardware.

## 1.3 Real-world problem it solves

- Prevents `OutOfMemoryError` surprises by giving developers a clear mental model of where memory is consumed (Heap vs Stack vs Metaspace) so they can size and tune the JVM correctly for production workloads.
- Solves the **multithreaded visibility problem** — without the JMM's guarantees (and tools like `volatile`, `synchronization`), one thread's update to a shared variable might never be seen by another thread, causing subtle, hard-to-reproduce bugs.

## 1.4 Historical Background

| Milestone                     | Detail                                                                                                                                         |
|-------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| Java 1.0–1.4                  | Original, informally-specified memory model — had known ambiguities/bugs around reordering and visibility.                                     |
| <b>JSR-133 (Java 5, 2004)</b> | Formal rewrite of the JMM, introducing the precise <code>happens-before</code> relationship and fixing <code>volatile/final</code> semantics.  |
| Java 8                        | <b>PermGen removed</b> , replaced by <b>Metaspace</b> (allocated from native memory, not the JVM heap) — a major runtime memory layout change. |
| Java 9+                       | G1 GC became the default collector, changing typical heap region behavior (region-based, not strictly generational-contiguous).                |

## 1.5 Interview Definition

"The Java Memory Model defines both the runtime areas the JVM divides memory into (Heap, Stack, Metaspace, PC Register, Native Method Stack) and, formally, the rules (via `happens-before`) governing when a write made by one thread becomes visible to another — ensuring predictable behavior across all conforming JVM implementations and hardware."

## 1.6 Simple Definition

It's the set of rules for where Java stores different kinds of data while your program runs, and how threads see each other's changes to that data.

## 1.7 Technical Definition

The JVM Runtime Data Areas (JVMS §2.5) partition memory into **shared areas** (Heap, Method Area/Metaspace) accessible by all threads, and **per-thread areas** (JVM Stack, PC Register, Native Method Stack). Separately, the formal Java Memory Model (JLS §17.4, defined via JSR-133) specifies the **happens-before** partial ordering relationship that determines whether a memory write by one thread is guaranteed to be visible to a read by another thread, without which compilers/CPU's would be free to reorder or cache operations in ways that break intuitive program behavior.

---

## 2. Real Life Analogy

| Domain     | Analogy                                                                                                                                                                                                                                                                                                                                                                                  |
|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Restaurant | The <b>Heap</b> is the restaurant's shared central kitchen storage (all chefs/threads can access the same ingredients/objects). The <b>Stack</b> is each waiter's personal notepad (private per-waiter/thread — one waiter's order notes are invisible to another waiter). The <b>Method Area</b> is the restaurant's master recipe book (shared class "blueprints" everyone refers to). |
| Bank       | The <b>Heap</b> is the bank's central vault (shared, all tellers/threads can deposit/access the same account objects). The <b>Stack</b> is each teller's personal desk drawer holding their current transaction's temporary notes (private per teller/thread).                                                                                                                           |
| Library    | The <b>Heap</b> is the library's shared bookshelves (all readers/threads access the same physical books/objects). The <b>Stack</b> is each reader's personal desk where they keep track of which chapter they're currently on (private call-frame state per reader/thread).                                                                                                              |
| WhatsApp   | Imagine two people (threads) editing a shared group's "pinned message" (a heap object) at the same time from different phones — without a memory model's visibility guarantees, one person's edit might not show up on the other's screen for a while, exactly like a stale/cached read across threads without proper synchronization.                                                   |

---

## 3. Internal Working

### 3.1 Runtime Data Areas — Where Things Actually Live

| Area                          | Shared or Per-Thread | Stores                                                                                              | Created/Destroyed                                                             |
|-------------------------------|----------------------|-----------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| Heap                          | Shared (all threads) | All objects (new instances), arrays                                                                 | Created at JVM startup, objects removed by GC                                 |
| Method Area / Metaspace       | Shared (all threads) | Class metadata, static variables, constant pool, method bytecode                                    | Created at JVM startup; per-class data unloaded when its class loader is GC'd |
| JVM Stack                     | Per-thread           | Method call frames: local variables (primitives + object references), operand stack, return address | Created when thread starts, destroyed when thread ends                        |
| PC (Program Counter) Register | Per-thread           | Address of the currently executing bytecode instruction                                             | Created/destroyed with the thread                                             |
| Native Method Stack           | Per-thread           | State for native (non-Java, e.g., JNI/C) method calls                                               | Created/destroyed with the thread                                             |

### 3.2 Step-by-Step: Object Creation and Memory Movement

```
Person p = new Person("Alice", 30);
```

- 1 `new Person(...)` → JVM allocates memory for a `Person` object **on the Heap**.
- 2 The constructor runs, initializing the object's fields **inside that Heap block**.
- 3 A **reference** (memory address / pointer, conceptually) to that Heap object is returned.
- 4 That reference is stored in the **local variable slot** `p`, **on the current thread's Stack** — NOT the object itself.
- 5 When the method containing `p` returns, the **Stack frame is popped** — the reference `p` disappears, but the actual `Person` object remains on the Heap until the **Garbage Collector** determines it's unreachable (no more references point to it).

### 3.3 Stack vs Heap — What Exactly Goes Where

```
int x = 10; -> primitive VALUE stored directly on Stack
Person p = new Person(); -> reference stored on Stack, actual OBJECT stored on Heap
```

- **Primitives** (`int`, `double`, `boolean`, etc.) declared as **local variables** live entirely on the Stack.
- **Object references** (the "pointer") live on the Stack; the **actual object data** always lives on the Heap.
- **Instance fields** (even primitive ones) inside an object live on the Heap, as part of that object's memory block — NOT the Stack — because they belong to the object, not to a specific method call.

## 4. Diagrams

### 4.1 Full Memory Layout

```
+-----+
| JVM MEMORY |
| |
| SHARED (across all threads) |
| +-----+ +-----+ |
| | HEAP | | METHOD AREA / METASPACE | |
| | - All objects (new X()) | | - Class metadata | |
| | - Arrays | | - Static variables | |
| | - Young Gen (Eden,S0,S1) | | - Constant pool | |
| | - Old Gen (Tenured) | | - Method bytecode | |
| +-----+ +-----+ |
| |
| PER-THREAD (one set per running thread) |
| +-----+ +-----+ +-----+ |
| | Thread-1 | | Thread-2 | | ...more threads... | |
| | Stack | | Stack | | | |
| | PC Reg | | PC Reg | | | |
| | Native Mth | | Native Mth | | | |
| | Stack | | Stack | | | |
| +-----+ +-----+ +-----+ |
+-----+
```

### 4.2 Heap Generational Structure (HotSpot default)

```
HEAP
+-----+
| YOUNG GENERATION |
| +-----+ +-----+ +-----+ |
| | Eden | | Survivor 0 | | Survivor 1 | |
| | (new objects | | (S0) | | (S1) | |
| | created here) | | | | |
| +-----+ +-----+ +-----+ |
+-----+
| OLD GENERATION (Tenured) |
| Objects that survived several Young GC cycles move here |
+-----+
(Metaspace lives OUTSIDE the Heap, in native memory, since Java 8)
```

### 4.3 Stack Frame Structure (Per Method Call)

```
main() calls calculate() calls square():

Thread Stack (grows downward as calls happen):
+-----+
| square() frame | <- currently executing (top of stack)
| local var: n |
| operand stack |
+-----+
| calculate() frame |
| local var: x, result |
+-----+
| main() frame |
| local var: args |
+-----+
```

Each frame is popped when its method returns — this is exactly why deep/infinite recursion causes `StackOverflowError`.

## 5. Syntax — JVM Memory Flags

```
java -Xms256m -Xmx1024m -XX:MetaspaceSize=64m -XX:MaxMetaspaceSize=256m -Xss512k MyApp
```

| Flag                        | Meaning                                                                 |
|-----------------------------|-------------------------------------------------------------------------|
| -Xms<size>                  | Initial Heap size                                                       |
| -Xmx<size>                  | Maximum Heap size                                                       |
| -Xss<size>                  | Stack size <b>per thread</b>                                            |
| -XX:MetaspaceSize=<size>    | Initial Metaspace size (triggers GC-driven expansion sizing after this) |
| -XX:MaxMetaspaceSize=<size> | Maximum Metaspace size (unbounded by default, unlike old PermGen!)      |
| -XX:+PrintGCDetails         | Print detailed GC activity logs (older flag; Java 9+ prefers -Xlog:gc*) |
| -Xlog:gc*                   | (Java 9+) Unified logging for GC events                                 |

## 6. Key APIs for Inspecting Memory

| API                                             | Purpose                                                                               |
|-------------------------------------------------|---------------------------------------------------------------------------------------|
| <code>Runtime.getRuntime().totalMemory()</code> | Total Heap memory currently allocated to the JVM                                      |
| <code>Runtime.getRuntime().freeMemory()</code>  | Free memory within the currently allocated Heap                                       |
| <code>Runtime.getRuntime().maxMemory()</code>   | Maximum Heap the JVM is allowed to grow to (-Xmx)                                     |
| <code>System.gc()</code>                        | <b>Suggests</b> (does NOT guarantee) the JVM run garbage collection                   |
| <code>java.lang.management.MemoryMXBean</code>  | Programmatic access to detailed Heap/non-Heap memory usage (used by monitoring tools) |

**Common mistake:** Calling `System.gc()` expecting immediate, guaranteed collection — it's only a *hint*; the JVM is free to ignore it entirely.

## 7. Code Examples

### 7.1 Basic — Observing Heap Memory Stats

```
public class MemoryStats {
    public static void main(String[] args) {
        Runtime rt = Runtime.getRuntime();
        System.out.println("Max Memory : " + rt.maxMemory() / (1024 * 1024) + " MB");
        System.out.println("Total Memory: " + rt.totalMemory() / (1024 * 1024) + " MB");
        System.out.println("Free Memory : " + rt.freeMemory() / (1024 * 1024) + " MB");
    }
}
```

## 7.2 Intermediate — Proving References Live on Stack, Objects on Heap

```
public class ReferenceDemo {
    public static void main(String[] args) {
        StringBuilder a = new StringBuilder("Hello");
        StringBuilder b = a; // 'b' is a NEW stack reference to the SAME heap object
        b.append(" World");
        System.out.println(a); // prints "Hello World" - proves a and b point to ONE object
    }
}
```

**Explanation:** `a` and `b` are two separate reference variables (on the Stack), but both point to the **same single object on the Heap** — modifying through `b` is visible through `a`.

## 7.3 Advanced — Demonstrating `StackOverflowError`

```
public class StackOverflowDemo {
    static int depth = 0;

    static void recurse() {
        depth++;
        recurse(); // no base case - infinite recursion
    }

    public static void main(String[] args) {
        try {
            recurse();
        } catch (StackOverflowError e) {
            System.out.println("Crashed at depth: " + depth);
        }
    }
}
```

**Why this happens:** Every call to `recurse()` pushes a new frame onto the thread's Stack; since the Stack has a fixed size (`-Xss`), unbounded recursion eventually exhausts it.

## 8. Dry Run

```
public class MemoryDryRun {
    public static void main(String[] args) {
        int a = 5;
        Person p1 = new Person("Bob");
        Person p2 = p1;
        p1 = null;
    }
}

class Person {
    String name;
    Person(String name) { this.name = name; }
}
```

| Step | Stack (main frame)                                    | Heap                                               |
|------|-------------------------------------------------------|----------------------------------------------------|
| 1    | <code>a = 5</code>                                    | —                                                  |
| 2    | <code>p1 = &lt;ref to Obj#1&gt;</code>                | Obj#1 {name: "Bob"} created                        |
| 3    | <code>p2 = &lt;ref to Obj#1&gt;</code> (same object!) | Obj#1 now has 2 references pointing to it (p1, p2) |



| Step | Stack (main frame)                        | Heap                                                              |
|------|-------------------------------------------|-------------------------------------------------------------------|
| 4    | <code>p1 = null</code>                    | Obj#1 still has 1 reference (p2) — <b>NOT eligible for GC yet</b> |
| 5    | <code>main()</code> returns, frame popped | Obj#1 now has 0 references — <b>eligible for GC</b>               |

## 9. Edge Cases

| Edge Case                                                                | What Happens                                   | Why                                                                                                                                                                      |
|--------------------------------------------------------------------------|------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Infinite/very deep recursion                                             | <code>StackOverflowError</code>                | Thread Stack has a fixed, finite size                                                                                                                                    |
| Creating too many large objects, never releasing references              | <code>OutOfMemoryError: Java heap space</code> | Heap has a maximum size ( <code>-Xmx</code> ); GC can't reclaim still-referenced objects                                                                                 |
| Loading thousands of classes dynamically (e.g., many dynamic proxies)    | <code>OutOfMemoryError: Metaspace</code>       | Metaspace, though not bound by Heap size, still has its own limit ( <code>-XX:MaxMetaspaceSize</code> , unbounded by default but constrained by available native memory) |
| Circular references between two objects (a references b, b references a) | Both still get garbage collected correctly     | Java's GC uses <b>reachability from GC Roots</b> , not simple reference counting — a circular pair unreachable from any root is still collected                          |
| Calling <code>System.gc()</code>                                         | May or may not trigger actual collection       | It's only an advisory hint to the JVM, never a guarantee                                                                                                                 |

## 10. Interview Questions

### Easy

- Q: Where are objects stored — Stack or Heap?** A: Heap. Local variables/references live on the Stack; actual object data always lives on the Heap.
- Q: What causes `StackOverflowError`?** A: Exceeding the thread's fixed Stack size, typically via unbounded/infinite recursion.

### Medium

- Q: What replaced PermGen in Java 8, and why?** A: Metaspace — allocated from native (off-heap) memory instead of a fixed-size Heap region, removing the common `OutOfMemoryError: PermGen space` issue caused by loading too many classes.
- Q: Are instance fields stored on the Stack or Heap?** A: Heap — even primitive instance fields live inside the object's memory block on the Heap, because they belong to the object, not to any single method call.

### Hard / Company-Level

- Q: Two objects reference each other (circular reference) and nothing else in the program references either one. Are they garbage collected?** A: Yes — Java's GC determines eligibility via **reachability from**

**GC Roots** (active threads, static references, JNI references), not simple reference counting, so a mutually-referencing but otherwise unreachable pair is still correctly collected.

- 2 **Q: Why is Metaspace's default max size effectively "unbounded" compared to the old PermGen — is that always safe?** A: Not necessarily — while it avoids the old artificial PermGen ceiling, an application with a classloader leak (e.g., repeatedly reloading classes without ever releasing old class loaders) can still exhaust **native machine memory**, causing an `OutOfMemoryError: Metaspace` or even crashing the OS process — so it should still be monitored/bounded explicitly in production via `-XX:MaxMetaspaceSize`.

---

## 11. Coding Questions

**Q1 (Easy): Write a program that prints current heap usage before and after creating one million small objects, showing memory growth.**

```
import java.util.ArrayList;
import java.util.List;

public class HeapGrowthDemo {
    public static void main(String[] args) {
        Runtime rt = Runtime.getRuntime();
        System.out.println("Before: " + usedMemoryMB(rt) + " MB");

        List<int[]> holder = new ArrayList<>();
        for (int i = 0; i < 1_000_000; i++) {
            holder.add(new int[]{i}); // keep references so GC can't reclaim them
        }

        System.out.println("After : " + usedMemoryMB(rt) + " MB");
    }

    private static long usedMemoryMB(Runtime rt) {
        return (rt.totalMemory() - rt.freeMemory()) / (1024 * 1024);
    }
}
```

**Q2 (Medium): Demonstrate that setting a reference to `null` makes an object eligible for GC, using a custom `finalize-free` approach (a `PhantomReference` or simply relying on `-verbose:gc` observation).**

```
public class EligibilityDemo {
    public static void main(String[] args) {
        Object obj = new Object();
        obj = null; // object now unreachable, eligible for GC
        System.gc(); // advisory hint only
        System.out.println("Requested GC after nulling the only reference.");
    }
}
```

---

## 12. Common Mistakes

- 1 **Beginner:** Assuming `System.gc()` immediately and guaranteedly frees memory — it's only a hint.
- 2 **Beginner:** Confusing "setting a reference to null" with "immediately destroying the object" — it only makes it *eligible*; actual collection timing is up to the GC.

- 3 **Experienced:** Holding onto object references longer than necessary (e.g., static collections that keep growing, listener registrations never removed) — the most common real-world cause of memory leaks in a garbage-collected language.
  - 4 **Experienced:** Not setting `-Xmx` explicitly in production, relying on JVM defaults that may not suit the container's actual available memory, causing OOM-kills in Kubernetes/Docker environments.
- 

## 13. Best Practices

- 1 Always explicitly set `-Xms/-Xmx` in production deployments matched to actual available container/host memory.
  - 2 Avoid unnecessary long-lived references (static collections, caches without eviction) — the #1 real-world Java memory leak pattern.
  - 3 Use profiling tools (VisualVM, JFR/Java Flight Recorder, async-profiler) to diagnose memory issues empirically rather than guessing.
  - 4 Prefer bounded caches (e.g., `Caffeine`, `LinkedHashMap` with eviction) over unbounded `HashMap`-based caches.
- 

## 14. Production Usage

- **Container memory sizing:** Kubernetes/Docker deployments must carefully align `-Xmx` with container memory limits — JVMs unaware of cgroup limits (pre-Java 10) famously caused OOM-kills; Java 10+ is cgroup-aware by default.
  - **Memory leak diagnosis:** Production incidents (gradually increasing memory, eventual `OutOfMemoryError`) are diagnosed via heap dumps (`jmap`, `jcmd`) analyzed in tools like Eclipse MAT.
  - **Monitoring:** `MemoryMXBean`-based metrics are exposed by Spring Boot Actuator/Micrometer to Prometheus/Grafana dashboards in real production systems.
- 

## 15. Performance

- Heap sizing directly trades off GC pause frequency (smaller heap = more frequent but shorter GCs) vs pause duration (larger heap = less frequent but potentially longer GCs) — modern collectors like G1/ZGC aim to minimize this trade-off.
  - Stack size (`-Xss`) is rarely a performance lever — it's mainly a safety/recursion-depth setting; too small risks `StackOverflowError`, too large wastes memory per thread (multiplied across many threads).
  - Metaspace being off-heap means it doesn't compete with object allocation space, but its growth still costs native memory and can indirectly cause GC pauses tied to class metadata management.
-

## 16. Frequently Asked Interview Questions

- 1 What are the main JVM runtime data areas? Heap, Method Area/Metaspace, Stack, PC Register, Native Method Stack.
- 2 Which areas are shared across threads vs per-thread? Heap and Method Area are shared; Stack, PC Register, Native Method Stack are per-thread.
- 3 Where do objects live? Always on the Heap.
- 4 Where do local primitive variables live? On the Stack.
- 5 Where do object references live? On the Stack (pointing to Heap objects).
- 6 Where do instance fields live? On the Heap, as part of the owning object.
- 7 What replaced PermGen and when? Metaspace, in Java 8.
- 8 Why was PermGen replaced? It had a fixed maximum size prone to `OutOfMemoryError: PermGen space`; Metaspace uses native memory and grows more flexibly.
- 9 What causes `StackOverflowError`? Exceeding the thread's Stack size limit, typically via deep/infinite recursion.
- 10 What causes `OutOfMemoryError: Java heap space`? Too many objects remaining reachable (referenced), exceeding `-Xmx`.
- 11 What is a GC Root? An object provably reachable from outside the heap (active thread stacks, static fields, JNI references) — the starting points for reachability analysis.
- 12 Does Java use reference counting for garbage collection? No — it uses reachability analysis from GC Roots, which correctly handles circular references.
- 13 Does `System.gc()` guarantee garbage collection? No — it's only an advisory request; the JVM may ignore it.
- 14 What is the Young Generation split into? Eden space and two Survivor spaces (S0, S1).
- 15 Where do newly created objects usually get allocated? In the Eden space of the Young Generation.
- 16 What happens to objects that survive multiple Young GC cycles? They get promoted ("tenured") into the Old Generation.
- 17 What does `-Xms` control? The JVM's initial Heap size.
- 18 What does `-Xmx` control? The JVM's maximum Heap size.
- 19 What does `-Xss` control? The Stack size allocated per thread.
- 20 Is Metaspace part of the Heap? No — it's allocated from native (off-heap) memory.
- 21 Can Metaspace still cause an `OutOfMemoryError`? Yes, if a classloader leak or excessive dynamic class generation exhausts available native memory or the configured `-XX:MaxMetaspaceSize`.
- 22 What's a common real-world cause of Java memory leaks despite having a garbage collector? Long-lived, unintentionally retained references (static caches, unregistered listeners) that keep objects "reachable" forever.
- 23 Since Java 10, is the JVM aware of container (cgroup) memory limits? Yes — earlier JVM versions were not, famously causing OOM-kills in Docker/Kubernetes when the JVM assumed it had the full host's memory available.

24 What tool would you use to take a heap dump for leak analysis? `jmap` or `jcmd`, analyzed later with Eclipse MAT or VisualVM.

25 Why does setting a reference to `null` not immediately free memory? It only makes the object *eligible* for garbage collection; actual reclamation timing is controlled by the GC, not the developer.

---

## 17. Revision Notes

### JAVA MEMORY MODEL — CHEAT SHEET

=====

SHARED (all threads): Heap, Method Area / Metaspace

PER-THREAD: Stack, PC Register, Native Method Stack

HEAP: all objects/arrays live here. Split: Young Gen (Eden+S0+S1) -> Old Gen

STACK: method frames, local primitives, object REFERENCES (not objects!)

METASPACE: class metadata, off-heap since Java 8 (replaced PermGen)

GC basis: REACHABILITY from GC Roots (NOT reference counting)

-> handles circular references correctly

#### KEY ERRORS:

StackOverflowError -> Stack exhausted (deep/infinite recursion)

OutOfMemoryError: heap space -> too many reachable objects, Heap full

OutOfMemoryError: Metaspace -> too many loaded classes / classloader leak

KEY FLAGS: `-Xms` `-Xmx` `-Xss` `-XX:MetaspaceSize` `-XX:MaxMetaspaceSize`

`System.gc()` = ADVISORY HINT ONLY, never guaranteed.

---

## 18. Homework

- 1 Run the `HeapGrowthDemo` program with different `-Xmx` values (e.g., `-Xmx64m`) until you trigger a real `OutOfMemoryError`, and record the exact message.
  - 2 Modify `StackOverflowDemo` to run with `-Xss256k` vs `-Xss8m` and compare the recursion depth reached before crashing.
  - 3 Take a heap dump of a running Java program using `jmap` and open it in VisualVM or Eclipse MAT.
-

## 19. Mini Project — "Memory Watchdog" Utility

```
package com.jobsradar.diagnostics;

/** Periodically logs heap usage percentage; warns if usage exceeds a threshold. */
public class MemoryWatchdog {
    public static void main(String[] args) throws InterruptedException {
        Runtime rt = Runtime.getRuntime();
        for (int i = 0; i < 5; i++) {
            long used = rt.totalMemory() - rt.freeMemory();
            long max = rt.maxMemory();
            double percent = (used * 100.0) / max;
            System.out.printf("Heap usage: %.2f%% (%d MB / %d MB)%n",
                percent, used / (1024 * 1024), max / (1024 * 1024));
            if (percent > 80.0) {
                System.out.println("WARNING: Heap usage above 80%!");
            }
            Thread.sleep(1000);
        }
    }
}
```

## 20. Final Summary

- The JVM splits memory into **shared areas** (Heap, Method Area/Metaspace) and **per-thread areas** (Stack, PC Register, Native Method Stack).
- **Objects always live on the Heap**; the Stack only ever holds primitives and object **references**.
- Garbage collection uses **reachability from GC Roots**, correctly handling circular references — not simple reference counting.
- **Metaspace** (Java 8+) replaced the old fixed-size **PermGen**, moving class metadata to native memory.
- `System.gc()` is an advisory hint only — never a guarantee.
- Real-world memory leaks in Java almost always come from **unintentionally retained references**, not from the language "failing" to garbage collect.

---

*(End of Topic 4: Java Memory Model.)*

---

# TOPIC 5: VARIABLES

---

## 1. Introduction

**What is it?** A variable is a named piece of memory that holds a value which can change during program execution.

**Why introduced / problem solved:** Programs need a way to store, label, and reuse data (a user's age, an order total) instead of hardcoding literal values everywhere. Variables give that data a readable name and a type-checked storage slot.

**Interview definition:** "A variable is an identifier bound to a memory location, with a declared type, that holds a value which may be modified during the program's execution."

**Simple definition:** A labeled box that holds a value you can change later.

## 2. Real Life Analogy

- **Bank:** A variable is like a labeled locker (`accountBalance`) — the label stays the same, but the amount inside can change over time.
- **Restaurant:** A variable is like a table number card — it labels "table 5" consistently, even though different customers (values) sit there at different times.

## 3. Internal Working

```
int age = 25;
```

- The compiler reserves a slot in the current stack frame (for a local variable) sized for an `int` (4 bytes).
- `25` is stored directly in that slot (primitives are stored by value).
- If it were an object (`String name = "Bob";`), the slot on the Stack would store a **reference**, while the actual `String` object lives on the Heap (see [\[\[Java Memory Model|Topic 4\]\]](#)).

## 4. Types of Variables

| Type              | Scope                                  | Lives In                 | Default Value                                    |
|-------------------|----------------------------------------|--------------------------|--------------------------------------------------|
| Local variable    | Inside a method/block                  | Stack (per call)         | None — must be explicitly initialized before use |
| Instance variable | Per-object, non-static field           | Heap (inside the object) | Yes (0, false, null etc.)                        |
| Static variable   | Shared across all instances of a class | Method Area/Metaspace    | Yes                                              |

```
class Counter {  
    static int totalCount = 0; // static variable - ONE copy shared by all objects  
    int id; // instance variable - one copy PER object  
    void increment() {  
        int temp = 1; // local variable - exists only during this method call  
        totalCount += temp;  
    }  
}
```

## 5. Syntax

```
[modifiers] type variableName [= initialValue];
```

- **modifiers** — optional (`static`, `final`, access modifiers for fields).
- **type** — primitive (`int`, `double`) or reference type (`String`, `List<Integer>`).

- `variableName` — must start with a letter, `_`, or `$`; cannot be a reserved keyword.
- `= initialValue` — optional for fields (get defaults), **mandatory before use** for local variables.

## 6. Code Examples

```
public class VariableDemo {
    static int staticCounter = 0; // static: shared across all instances
    int instanceId; // instance: one per object, defaults to 0

    public static void main(String[] args) {
        int localVar = 10; // local: must be initialized before use
        final double PI_APPROX = 3.14; // 'final' -> value cannot be reassigned
        System.out.println(localVar + " " + PI_APPROX);
    }
}
```

## 7. Edge Cases

| Case                                                              | Result                                                                                                |
|-------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| Using a local variable before initializing it                     | Compile-time error: "variable might not have been initialized"                                        |
| Reassigning a <code>final</code> variable                         | Compile-time error                                                                                    |
| Variable name same as a keyword (e.g. <code>int class</code> ;) ) | Compile-time error — reserved word                                                                    |
| Same variable name in nested block scopes shadowing outer one     | Compiles, but the inner variable "shadows" (hides) the outer for that block — a common source of bugs |

## 8. Common Mistakes

- Forgetting local variables have **no default value** (unlike fields) — using them uninitialized is always a compile error, not a runtime `0/null`.
- Overusing mutable static variables — causes hidden shared-state bugs across threads/instances.

## 9. Best Practices

- Use `camelCase` naming, meaningful names (`accountBalance`, not `ab`).
- Prefer `final` for variables that shouldn't change (immutability reduces bugs).
- Keep variable scope as narrow as possible (declare close to first use).

## 10. Frequently Asked Interview Questions

- 1 **What are the three kinds of variables in Java?** Local, instance, and static.
- 2 **Do local variables get default values?** No — they must be explicitly initialized before use, unlike instance/static fields.



- 3 **What's the difference between an instance variable and a static variable?** Instance variables have one copy per object (Heap); static variables have exactly one copy shared across all instances (Method Area/Metaspace).
- 4 **What does `final` do on a variable?** Prevents reassignment after initial assignment — the reference/value becomes constant (for objects, the reference is fixed, but the object's internal state can still change unless the object itself is immutable).
- 5 **What is variable shadowing?** When a variable in an inner scope has the same name as one in an outer scope, temporarily hiding the outer one within that inner block.

## 11. Revision Notes

### VARIABLES – CHEAT SHEET

=====

LOCAL -> in method/block, Stack, NO default value, must init before use

INSTANCE -> per object, Heap, HAS default value

STATIC -> shared per class, Method Area/Metaspace, HAS default value

final -> value/reference cannot be reassigned after first assignment

## 12. Homework

- 1 Write a class with one static, one instance, and one local variable; print all three and explain their default values (or lack thereof).
- 2 Deliberately trigger the "variable might not have been initialized" compiler error and read the exact message.

---

*(End of Topic 5: Variables.)*

---

# TOPIC 6: DATA TYPES

---

## 1. Introduction

**What is it?** A data type defines what **kind of value** a variable can hold (a whole number, a decimal, a single character, true/false, or an object) and how much memory it occupies.

**Why introduced:** The compiler and JVM need to know, ahead of time, exactly how many bytes to reserve and what operations are valid, to catch type errors at compile time and to store data efficiently.

**Interview definition:** "A data type is a classification specifying which values a variable may hold, the operations permitted on it, and its memory representation — in Java, split into primitive types (stored by value) and reference types (stored as a reference to a Heap object)."

## 2. Two Categories

### JAVA DATA TYPES

■■■ PRIMITIVE (8 types) – stored directly, by value, always has a default

■ byte, short, int, long, float, double, char, boolean

■■■ REFERENCE (non-primitive) – stores a reference to an object on the Heap  
String, arrays, classes, interfaces, enums, all wrapper classes

## 3. The 8 Primitive Types — Full Reference

| Type    | Size                                    | Default | Range                                         | Example              |
|---------|-----------------------------------------|---------|-----------------------------------------------|----------------------|
| byte    | 1 byte                                  | 0       | -128 to 127                                   | byte b = 100;        |
| short   | 2 bytes                                 | 0       | -32,768 to 32,767                             | short s = 30000;     |
| int     | 4 bytes                                 | 0       | -2,147,483,648 to 2,147,483,647               | int i = 100000;      |
| long    | 8 bytes                                 | 0L      | $-9.2 \times 10^{18}$ to $9.2 \times 10^{18}$ | long l = 100000L;    |
| float   | 4 bytes                                 | 0.0f    | ~7 significant decimal digits                 | float f = 3.14f;     |
| double  | 8 bytes                                 | 0.0d    | ~15 significant decimal digits                | double d = 3.14159;  |
| char    | 2 bytes                                 | '■'     | 0 to 65,535 (single UTF-16 code unit)         | char c = 'A';        |
| boolean | JVM-dependent (not precisely specified) | false   | true / false only                             | boolean flag = true; |

**Why char is 2 bytes, not 1:** Java uses UTF-16 internally for char, unlike C's 1-byte ASCII char, so it can represent a much wider range of characters natively.

**Why float/long literals need a suffix (f, L):** Numeric literals default to int (whole numbers) or double (decimals) — you must explicitly mark 100L or 3.14f or the compiler either truncates/errors or picks the wrong type.

## 4. Real Life Analogy

- **Bank:** int is like a standard account balance field (whole rupees); double is like a balance that supports paise/decimals; boolean is like an "is account frozen?" flag — only two possible states.
- **Restaurant:** byte/short/int/long are like different-sized order-quantity fields — a byte might suffice for "number of chairs at a table" (max 127), but you'd need long for "total orders served by the chain this year."

## 5. Internal Working — Autoboxing Preview

```
int primitive = 10;
Integer wrapper = primitive; // AUTOBOXING: compiler inserts Integer.valueOf(primitive)
int back = wrapper; // AUTO-UNBOXING: compiler inserts wrapper.intValue()
```

Primitives live directly on the Stack (for locals); their wrapper class equivalents (Integer, Double, etc.) are full objects living on the Heap — covered in depth in the **Wrapper Classes** topic.

## 6. Code Examples

```
public class DataTypesDemo {
    public static void main(String[] args) {
        byte age = 25;
        int population = 1_400_000_000; // underscore for readability (Java 7+)
        long worldPopulation = 8_000_000_000L; // 'L' suffix required - exceeds int range
        float price = 99.99f; // 'f' suffix required for float literal
        double preciseValue = 3.14159265358979;
        char grade = 'A';
        boolean isPassed = true;

        System.out.println(age + " " + population + " " + worldPopulation);
        System.out.println(price + " " + preciseValue + " " + grade + " " + isPassed);
    }
}
```

## 7. Edge Cases

| Case                                                                                     | Result                                                                                                                       |
|------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| <code>int x = 2147483647; x = x + 1;</code>                                              | <b>Silently overflows</b> to -2147483648 — NO exception thrown!                                                              |
| <code>float f = 3.14;</code> (no suffix)                                                 | Compile-time error — 3.14 defaults to <code>double</code> , which doesn't fit in <code>float</code> without an explicit cast |
| <code>byte b = 128;</code>                                                               | Compile-time error — exceeds <code>byte</code> 's max range of 127                                                           |
| <code>char c = 65;</code>                                                                | Valid — 65 is auto-converted to the character 'A' ( <code>char</code> is really an unsigned 16-bit integer under the hood)   |
| Comparing <code>float/double</code> for exact equality ( <code>0.1 + 0.2 == 0.3</code> ) | Returns <code>false</code> — due to binary floating-point representation, not a bug                                          |

## 8. Common Mistakes

- Using `==` to compare `float/double` values expecting exact equality — always use a tolerance/epsilon comparison instead.
- Forgetting integer overflow is **silent** in Java (no exception) — unlike some languages, it just wraps around.
- Using `float/double` for financial/currency calculations instead of `BigDecimal` — leads to precision bugs in money math.

## 9. Best Practices

- Use `BigDecimal` (not `double/float`) for any money/currency calculation.
- Use underscores in large numeric literals for readability (`1_000_000`).
- Choose the smallest type that safely fits your data's realistic range, but don't over-optimize prematurely — `int` and `double` are the sensible defaults for most whole numbers and decimals respectively.

## 10. Frequently Asked Interview Questions

- 1 **How many primitive data types does Java have?** 8 — `byte`, `short`, `int`, `long`, `float`, `double`, `char`, `boolean`.
- 2 **What's the default value of an uninitialized `int` instance field?** `0`.
- 3 **Why does Java need an `L` suffix for long literals?** Because numeric literals default to `int`; without `L`, a value exceeding `int`'s range causes a compile error.
- 4 **Does integer overflow throw an exception in Java?** No — it silently wraps around (e.g., `Integer.MAX_VALUE + 1` becomes `Integer.MIN_VALUE`).
- 5 **Why shouldn't you use `double` for currency?** Binary floating-point cannot exactly represent many decimal fractions, causing rounding errors in financial calculations — use `BigDecimal` instead.
- 6 **Is `char` signed or unsigned in Java?** Unsigned (0 to 65,535) — unlike `byte/short/int/long`, which are all signed.
- 7 **What's the difference between primitive types and reference types in terms of memory?** Primitives store their actual value directly (typically on the Stack for locals); reference types store a reference/pointer to an object on the Heap.

## 11. Revision Notes

### DATA TYPES — CHEAT SHEET

=====

PRIMITIVES (8): `byte(1B)` `short(2B)` `int(4B)` `long(8B)` `float(4B)` `double(8B)` `char(2B)` `boolean`  
- stored BY VALUE, always have a default, never null

REFERENCE TYPES: `String`, arrays, classes, interfaces, wrapper classes  
- store a REFERENCE to a Heap object, default is null

### GOTCHAS:

`int` overflow -> SILENT wraparound, no exception  
`float/double` -> imprecise, NEVER use for currency (use `BigDecimal`)  
`long` literal -> needs '`L`' suffix; `float` literal needs '`f`' suffix

## 12. Homework

- 1 Write a program that deliberately overflows an `int` and a `long`, and print the wraparound results.
- 2 Prove `0.1 + 0.2 != 0.3` in Java with a `System.out.println`, then fix the comparison using `Math.abs(a - b) < epsilon`.

---

(End of Topic 6: Data Types.)

---

## TOPIC 7: OPERATORS

---

## 1. Introduction

**What is it?** Operators are special symbols that perform operations on one, two, or three operands (values/variables) — arithmetic, comparison, logical decisions, bit manipulation, and assignment.

**Interview definition:** "An operator is a language construct that takes one or more operands and produces a result, categorized by arity (unary/binary/ternary) and function (arithmetic, relational, logical, bitwise, assignment)."

## 2. Categories of Operators

| Category                | Operators                                                                     | Purpose                                        |
|-------------------------|-------------------------------------------------------------------------------|------------------------------------------------|
| Arithmetic              | <code>+ - * / %</code>                                                        | Basic math                                     |
| Unary                   | <code>+ - ++ -- !</code>                                                      | Single-operand operations                      |
| Relational              | <code>== != &gt; &lt; &gt;= &lt;=</code>                                      | Comparison, produces <code>boolean</code>      |
| Logical                 | <code>&amp;&amp; \ \  !</code>                                                | Boolean combination, with short-circuiting     |
| Bitwise                 | <code>&amp; \ ^ ~ &lt;&lt; &gt;&gt; &gt;&gt;&gt;</code>                       | Bit-level manipulation                         |
| Assignment              | <code>= += -= *= /= %= &amp;= \ = ^= &lt;&lt;= &gt;&gt;= &gt;&gt;&gt;=</code> | Assign (optionally combined with an operation) |
| Ternary                 | <code>? :</code>                                                              | Inline conditional expression                  |
| <code>instanceof</code> | <code>instanceof</code>                                                       | Runtime type check                             |

## 3. Real Life Analogy

- **Bank:** `balance -= withdrawAmount` is like a teller deducting a withdrawal in one combined step, instead of writing `balance = balance - withdrawAmount` longhand.
- **Restaurant:** `isVIP && hasReservation` — a table is only auto-confirmed if BOTH conditions hold (logical AND); if `isVIP` is `false`, the system doesn't even bother checking the reservation (short-circuiting) — saving a database lookup.

## 4. Internal Working — Short-Circuit Evaluation

```
if (user != null && user.isActive()) { ... }
```

- `&&` and `\|\|` **short-circuit:** if the left operand already determines the result (`false` for `&&`, `true` for `\|\|`), the right operand is **never evaluated**.
- This is not just a performance optimization — it's a **null-safety pattern**: `user.isActive()` would throw `NullPointerException` if evaluated when `user` is `null`, but short-circuiting prevents that call from ever happening.
- `&` and `|` (single) are the **non-short-circuiting** bitwise/logical equivalents — using them on booleans always evaluates both sides (a classic interview trick question).

## 5. Bitwise Operators — Deep Reference

| Operator | Meaning                                           | Example                          |
|----------|---------------------------------------------------|----------------------------------|
| &        | AND each bit                                      | 5 & 3 → 1 (0101 & 0011 = 0001)   |
|          | OR each bit                                       | 5   3 → 7 (0101   0011 = 0111)   |
| ^        | XOR each bit                                      | 5 ^ 3 → 6 (0101 ^ 0011 = 0110)   |
| ~        | Bitwise NOT (invert all bits)                     | ~5 → -6                          |
| <<       | Left shift (multiply by 2^n)                      | 5 << 1 → 10                      |
| >>       | Signed right shift (preserves sign bit)           | -8 >> 1 → -4                     |
| >>>      | Unsigned right shift (fills with 0, ignores sign) | -8 >>> 1 → large positive number |

## 6. Code Examples

```
public class OperatorsDemo {
    public static void main(String[] args) {
        int a = 10, b = 3;
        System.out.println(a / b); // 3 (integer division truncates)
        System.out.println(a % b); // 1 (remainder)
        System.out.println((double) a / b); // 3.333... (cast forces floating-point division)

        int x = 5;
        System.out.println(x++); // prints 5, THEN increments (post-increment)
        System.out.println(++x); // increments THEN prints 7 (pre-increment)

        String name = null;
        System.out.println(name != null && name.length() > 0); // false, no NPE (short-circuit)

        int max = (a > b) ? a : b; // ternary operator
        System.out.println(max); // 10
    }
}
```

## 7. Edge Cases

| Case                                                        | Result                                                                                                    |
|-------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| 5 / 0 (int)                                                 | Throws <code>ArithmeticException: / by zero</code>                                                        |
| 5.0 / 0 (double)                                            | Returns <code>Infinity</code> — NO exception for floating-point division by zero                          |
| 0.0 / 0.0                                                   | Returns <code>NaN</code> (Not a Number)                                                                   |
| x++ vs ++x                                                  | Post-increment returns the OLD value then increments; pre-increment increments THEN returns the new value |
| & /   on booleans                                           | Valid in Java (unlike some languages) — but evaluates BOTH sides always, no short-circuit                 |
| Mixing & with && expecting identical short-circuit behavior | A real bug source — & always evaluates both operands even for booleans                                    |

## 8. Common Mistakes

- Using `&|` instead of `&&||` for boolean logic where a `NullPointerException`-prone right-hand check exists — loses the short-circuit safety net.
- Forgetting integer division truncates (`5 / 2` is 2, not 2.5) — must cast to `double` explicitly.
- Confusing `>>` (signed) with `>>>` (unsigned) right-shift on negative numbers.

## 9. Best Practices

- Always use `&&||` for boolean logic (not `&|`) unless you specifically need both sides always evaluated (rare, e.g., side-effect-dependent code — and even then, prefer clarity over cleverness).
- Cast explicitly when floating-point division is intended: `(double) a / b`.
- Avoid deeply nested ternary operators — they hurt readability; prefer if-else for anything beyond a single simple condition.

## 10. Frequently Asked Interview Questions

- 1 Difference between `&&` and `&` for booleans?** `&&` short-circuits (skips evaluating the right side if the result is already determined); `&` always evaluates both sides.
- 2 What does integer division `7 / 2` return?** 3 — truncates toward zero, discards the remainder.
- 3 What happens dividing an `int` by zero vs a `double` by zero?** `int / 0` throws `ArithmeticException`; `double / 0.0` returns `Infinity` (or `NaN` for `0.0/0.0`) — no exception.
- 4 Difference between `x++` and `++x`?** `x++` (post) returns the current value, then increments; `++x` (pre) increments first, then returns the new value.
- 5 What does `>>>` do differently from `>>`?** `>>>` is an unsigned right shift — it always fills vacated bits with 0, ignoring the sign bit; `>>` preserves the sign bit (arithmetic shift).
- 6 Why is short-circuit evaluation important beyond performance?** It prevents unnecessary/unsafe evaluation of the right operand — commonly used to guard against `NullPointerException` (`obj != null && obj.method()`).

## 11. Revision Notes

```
OPERATORS — CHEAT SHEET
=====
ARITHMETIC: + - * / % (int / truncates; int/0 throws; double/0.0 = Infinity/NaN)
INCREMENT: x++ (post) vs ++x (pre)
RELATIONAL: == != > < >= <= -> always returns boolean
LOGICAL: && || ! -> SHORT-CIRCUIT (safe for null-checks)
& | -> NO short-circuit, evaluates both sides always
BITWISE: & | ^ ~ << >> >>>
TERNARY: condition ? valIfTrue : valIfFalse
```

## 12. Homework

- 1** Write a program proving `5 / 2` (int) differs from `5.0 / 2` (double), and explain why.

- 2 Write a program with a null check using `&&` and prove it avoids `NullPointerException`, then show what happens if you replace `&&` with `&`.

---

(End of Topic 7: Operators.)

---

## TOPIC 8: TYPE CASTING

---

### 1. Introduction

**What is it?** Type casting is converting a value from one data type to another — either between primitive types, or between related reference types (classes/interfaces in an inheritance hierarchy).

**Why introduced:** Real programs constantly mix types (an `int` counter needs to become a `double` average; a generic `Object` needs to be treated as its specific subtype). Java, being strongly-typed, requires this conversion to be explicit in cases where data could be lost or the conversion isn't guaranteed safe — this is a deliberate safety feature, not a limitation.

**Interview definition:** "Type casting is the explicit or implicit conversion of a value from one type to another, categorized as widening (implicit, safe) or narrowing (explicit, potentially lossy) for primitives, and upcasting/downcasting for reference types."

### 2. Primitive Casting — Widening vs Narrowing

```
WIDENING (implicit, automatic, always safe - no data loss):  
byte -> short -> int -> long -> float -> double  
(also: char -> int)
```

```
NARROWING (explicit, requires cast syntax, can LOSE data):  
double -> float -> long -> int -> short -> byte
```

```
int i = 100;  
long l = i; // WIDENING - automatic, no cast needed  
double d = l; // WIDENING - automatic
```

```
double bigValue = 3.99;  
int truncated = (int) bigValue; // NARROWING - explicit cast REQUIRED, decimal part is LOST (becomes 3)
```

### 3. Real Life Analogy

- **Bank:** Converting a smaller currency denomination into a larger one (coins into a note) is like widening — always safe, nothing is lost. Converting a large note into coins might lose a fraction (rounding) — like narrowing, which needs deliberate, explicit handling.
- **College:** Treating a `Student` object as a generic `Person` (upcasting) is always safe — every `Student` IS a `Person`. Treating a generic `Person` reference back as a specific `Student` (downcasting) is risky — it only works if that `Person` object actually WAS a `Student` underneath; otherwise it fails at runtime.



## 4. Reference Type Casting — Upcasting vs Downcasting

```
class Person {}
class Student extends Person {}

Student s = new Student();
Person p = s; // UPCASTING - implicit, always safe (Student IS-A Person)

Person p2 = new Student();
Student s2 = (Student) p2; // DOWNCASTING - explicit cast required, safe HERE because
// p2 actually references a real Student object underneath

Person p3 = new Person();
Student s3 = (Student) p3; // COMPILES, but throws ClassCastException at RUNTIME -
// p3 is a plain Person, NOT actually a Student
```

## 5. Internal Working

- **Widening primitive conversion:** The JVM has dedicated bytecode instructions (`i2l`, `i2d`, etc.) that reinterpret the bit pattern into the larger type's representation — always lossless.
- **Narrowing primitive conversion:** Uses instructions like `d2i`, `l2i` — truncates or loses precision; the JVM does NOT throw an error, it just silently produces a truncated/wrapped result.
- **Reference downcasting:** The JVM inserts a `checkcast` bytecode instruction, which performs a runtime type check against the object's actual class; if it fails, `ClassCastException` is thrown immediately.

## 6. Code Examples

```
public class CastingDemo {
    public static void main(String[] args) {
        // Primitive widening
        int intVal = 42;
        double doubleVal = intVal; // implicit widening
        System.out.println(doubleVal); // 42.0

        // Primitive narrowing
        double pi = 3.14159;
        int truncatedPi = (int) pi; // explicit narrowing
        System.out.println(truncatedPi); // 3 (decimal part lost, NOT rounded)

        // Narrowing overflow example
        int big = 300;
        byte b = (byte) big; // 300 doesn't fit in byte's -128..127 range
        System.out.println(b); // 44 (wraps around, NOT an error)

        // Safe use of instanceof before downcasting (best practice)
        Object obj = "Hello";
        if (obj instanceof String str) { // Java 16+ pattern matching for instanceof
            System.out.println(str.toUpperCase());
        }
    }
}
```

## 7. Edge Cases

| Case                                                                   | Result                                                                                                                                  |
|------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| <code>(int) 3.99</code>                                                | 3 — truncates toward zero, does NOT round                                                                                               |
| <code>(byte) 300</code>                                                | 44 — narrowing silently overflows/wraps, no exception                                                                                   |
| Downcasting to an unrelated/incompatible type                          | <code>ClassCastException</code> at runtime — compiles fine, fails only when executed                                                    |
| Casting null to any reference type                                     | Always succeeds — null has no runtime type to conflict with                                                                             |
| Casting between unrelated classes (no inheritance relationship at all) | <b>Compile-time error</b> — the compiler can statically prove it's impossible, unlike the "compiles but fails at runtime" downcast case |

## 8. Common Mistakes

- Assuming `(int) doubleValue` rounds — it actually **truncates** (use `Math.round()` for rounding).
- Downcasting without an `instanceof` check first, risking `ClassCastException` in production.
- Forgetting narrowing conversions between numeric types can silently overflow/wrap with no warning or exception.

## 9. Best Practices

- Always guard a downcast with `instanceof` (ideally using Java 16+ pattern matching: `if (obj instanceof String str) { ... }`) rather than a bare cast.
- Use `Math.round()`, `Math.floor()`, or `Math.ceil()` explicitly when you need specific rounding behavior instead of relying on truncation.
- Avoid narrowing casts on values whose range you haven't verified — validate bounds first if data loss would be a bug.

## 10. Frequently Asked Interview Questions

- What's the difference between widening and narrowing primitive conversion?** Widening is implicit and always safe (smaller type to larger, e.g., `int` → `double`); narrowing is explicit and can lose data (larger type to smaller, e.g., `double` → `int`), requiring a cast.
- Does `(int) 3.99` round or truncate?** Truncates — returns 3, discarding the decimal part entirely (never rounds).
- What is upcasting?** Implicitly treating a subclass reference as its superclass type — always safe since a subclass IS-A superclass.
- What is downcasting, and why can it fail at runtime?** Explicitly treating a superclass reference as a specific subclass type; it fails with `ClassCastException` at runtime if the object's actual runtime type isn't really that subclass (or a further subclass of it).
- Why does casting between two completely unrelated classes fail at compile time, but casting a Person to a Student (its actual subclass hierarchy) only fails at runtime?** The compiler can only reject

casts it can statically PROVE are impossible (no inheritance relationship at all); when a relationship exists (Student extends Person), the compiler allows the cast syntactically and defers the actual safety check to runtime via the `checkcast` instruction.

- 6 **What happens if you narrow-cast an `int` value that's too large for a `byte`?** It silently overflows/wraps around using modular arithmetic on the bit pattern — no exception is thrown.

## 11. Revision Notes

```
TYPE CASTING – CHEAT SHEET
=====
PRIMITIVE WIDENING (implicit, safe):
byte -> short -> int -> long -> float -> double (also char -> int)

PRIMITIVE NARROWING (explicit cast required, may lose data, NO exception):
double -> float -> long -> int -> short -> byte

REFERENCE UPCASTING : Subclass -> Superclass (implicit, always safe)
REFERENCE DOWNCASTING : Superclass -> Subclass (explicit cast, can throw
ClassCastException at RUNTIME if types don't actually match)

RULE OF THUMB: guard every downcast with `instanceof` first.
(int) 3.99 => 3 (TRUNCATES, does not round)
```

## 12. Homework

- 1 Write a program that narrows an `int` value of 500 to a `byte` and explain the wrapped output.
- 2 Create a small class hierarchy (Animal → Dog, Cat) and demonstrate a downcast that succeeds and one that throws `ClassCastException`, guarding the risky one with `instanceof`.

---

(End of Topic 8: Type Casting.)

---

# TOPIC 9: KEYWORDS

---

## 1. Introduction

**What is it?** Keywords are reserved words with special, fixed meaning to the Java compiler — they cannot be used as identifiers (variable/class/method names).

**Interview definition:** "Keywords are predefined, reserved tokens in the Java language grammar that carry fixed syntactic/semantic meaning and cannot be redeclared as user identifiers."

## 2. Complete Reference Table (Java 17)

### Data Type Keywords

| Keyword                | Meaning                        |
|------------------------|--------------------------------|
| byte, short, int, long | Integer primitive types        |
| float, double          | Floating-point primitive types |
| char                   | Single UTF-16 character        |
| boolean                | true/false                     |
| void                   | No return value                |

### Modifier Keywords

| Keyword                    | Meaning                                                                                                       |
|----------------------------|---------------------------------------------------------------------------------------------------------------|
| public, private, protected | Access modifiers                                                                                              |
| static                     | Belongs to the class, not an instance                                                                         |
| final                      | Prevents reassignment/overriding/subclassing (context-dependent)                                              |
| abstract                   | Declares a class/method with no full implementation, must be extended/implemented                             |
| synchronized               | Restricts concurrent thread access to a method/block                                                          |
| volatile                   | Guarantees visibility of a variable's latest value across threads                                             |
| transient                  | Excludes a field from default serialization                                                                   |
| native                     | Method implemented in platform-native code (e.g., C, via JNI)                                                 |
| strictfp                   | Forces strict IEEE 754 floating-point calculations (largely obsolete since Java 17 makes it default behavior) |

### Control Flow Keywords

| Keyword               | Meaning                                            |
|-----------------------|----------------------------------------------------|
| if, else              | Conditional branching                              |
| switch, case, default | Multi-way branching                                |
| for, while, do        | Loops                                              |
| break, continue       | Loop/switch control                                |
| return                | Exit a method, optionally with a value             |
| yield                 | (Java 13+) Return a value from a switch expression |

## Exception Handling Keywords

| Keyword             | Meaning                                                         |
|---------------------|-----------------------------------------------------------------|
| try, catch, finally | Exception handling blocks                                       |
| throw               | Explicitly raise an exception                                   |
| throws              | Declare a method may propagate a checked exception              |
| assert              | Runtime assertion check (disabled by default, enabled with -ea) |

## Class/Object-Oriented Keywords

| Keyword                                   | Meaning                                  |
|-------------------------------------------|------------------------------------------|
| class, interface, enum, record (Java 16+) | Type declarations                        |
| extends, implements                       | Inheritance/interface implementation     |
| new                                       | Object instantiation                     |
| this                                      | Reference to the current object instance |
| super                                     | Reference to the parent class            |
| instanceof                                | Runtime type check                       |
| package, import                           | Namespace organization                   |
| sealed, non-sealed, permits (Java 17)     | Restrict which classes may extend a type |

## Reserved Literals (NOT technically "keywords" per JLS, but reserved)

| Literal     | Meaning                           |
|-------------|-----------------------------------|
| true, false | Boolean literals                  |
| null        | The absence of a reference/object |

## Unused / Reserved-but-not-implemented

| Keyword     | Note                                                                                                                                                                                   |
|-------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| goto, const | Reserved as keywords (cannot be used as identifiers) but have <b>no actual function</b> in Java — kept reserved to avoid confusing C/C++ developers and to allow potential future use. |

## 3. Contextual Keywords (NOT reserved — can still be used as identifiers!)

| Word                        | Where it's special                       |
|-----------------------------|------------------------------------------|
| var                         | Local variable type inference (Java 10+) |
| record                      | Declaring a record type (Java 16+)       |
| yield                       | Inside switch expressions                |
| sealed, permits, non-sealed | Sealed class declarations (Java 17)      |

**Interview trap:** `var` is NOT a reserved keyword — you can still legally write `int var = 5;` — it's only special when used in the position of a type declaration for local variable inference.

## 4. Real Life Analogy

- **College:** Keywords are like reserved terms in an official exam form (e.g., "Roll No.", "Signature") — you can't name your own custom form field "Roll No." because the form processor (compiler) has a fixed, special meaning already assigned to it.

## 5. Code Examples

```
public class KeywordDemo {
    public static void main(String[] args) {
        final int MAX = 100; // 'final' - cannot be reassigned
        var count = 10; // 'var' - type inferred as int (contextual keyword)
        int var = 5; // LEGAL - 'var' is not reserved, just contextual
        assert count > 0 : "count must be positive"; // 'assert' - disabled unless -ea flag used
        System.out.println(MAX + " " + count + " " + var);
    }
}
```

## 6. Edge Cases

| Case                                                                         | Result                                                                             |
|------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| Using <code>goto</code> or <code>const</code> as an identifier               | Compile-time error — reserved even though unused                                   |
| Using <code>var</code> as a variable name                                    | Legal — it's contextual, not reserved                                              |
| <code>assert</code> statements without <code>-ea</code> flag                 | Silently skipped at runtime — assertions are <b>disabled by default</b> in the JVM |
| Using <code>enum</code> as an identifier (e.g., <code>int enum = 5;</code> ) | Compile-time error — <code>enum</code> became a true reserved keyword since Java 5 |

## 7. Common Mistakes

- Forgetting `assert` does nothing unless the JVM is run with `-ea` (enable assertions) — relying on it for production validation logic is a mistake; use proper exceptions instead.
- Assuming all lowercase words with special meaning (`var`, `record`, `yield`) are reserved keywords — several are only **contextual**.

## 8. Frequently Asked Interview Questions

- 1 **How many true reserved keywords does Java have?** Around 53 in modern Java (Java 17), including context-sensitive additions like `sealed/permits/non-sealed`, plus 2 reserved literals (`true`, `false`) and `null`.
- 2 **Is `var` a reserved keyword?** No — it's a contextual keyword; you can still use `var` as a variable/method name.
- 3 **What do `goto` and `const` do in Java?** Nothing — they're reserved (cannot be used as identifiers) but have no actual implemented behavior.

- 4 **Are assertions (`assert`) enabled by default?** No — they must be explicitly enabled with the `-ea` JVM flag; without it, `assert` statements are silently skipped entirely.
- 5 **What keywords were added in Java 17 specifically?** `sealed`, `non-sealed`, and `permits` (contextual, for sealed class hierarchies).

## 9. Revision Notes

### KEYWORDS — CHEAT SHEET

=====

TRUE RESERVED (cannot be identifiers): ~53 words incl. `class`, `if`, `for`, `static`, `final`, `public`, `private`, `try`, `catch`, `throws`, `extends`, `implements`, `enum`, ...

RESERVED LITERALS: `true`, `false`, `null`

UNUSED BUT RESERVED: `goto`, `const` (no function, kept reserved)

CONTEXTUAL (NOT reserved, can be identifiers!): `var`, `record`, `yield`, `sealed`, `permits`, `non-sealed`

`assert` -> disabled by default; needs `-ea` JVM flag to activate`

## 10. Homework

- 1 Try compiling `int enum = 5;` and `int var = 5;` — note which one fails and which succeeds, and explain why.
- 2 Run a program containing an `assert` statement first normally, then with `java -ea YourClass`, and compare behavior.

---

*(End of Topic 9: Keywords.)*

---

# TOPIC 10: CONTROL STATEMENTS

---

## 1. Introduction

**What is it?** Control statements alter the default top-to-bottom, one-statement-at-a-time execution flow of a program — via conditional branching (`if/switch`), looping (`for/while/do-while`), and jump statements (`break/continue/return`).

**Interview definition:** "Control statements are language constructs that direct the order in which individual statements, instructions, or function calls are executed, enabling conditional branching and repetition."

## 2. Conditional Statements

### 2.1 if / else if / else

```
int score = 85;
if (score >= 90) {
    System.out.println("Grade A");
} else if (score >= 75) {
    System.out.println("Grade B");
} else {
    System.out.println("Grade C or below");
}
```

- Only ONE branch executes — evaluation stops at the first `true` condition.

### 2.2 Traditional switch (fall-through by default)

```
int day = 3;
switch (day) {
    case 1:
        System.out.println("Monday");
        break; // WITHOUT break, execution "falls through" into case 2!
    case 2:
        System.out.println("Tuesday");
        break;
    default:
        System.out.println("Unknown day");
}
```

### 2.3 Modern Switch Expression (Java 14+) — No Fall-Through, Returns a Value

```
String dayName = switch (day) {
    case 1, 2, 3, 4, 5 -> "Weekday"; // arrow syntax - no fall-through, no break needed
    case 6, 7 -> "Weekend";
    default -> {
        yield "Invalid day"; // 'yield' returns a value from a block branch
    }
};
System.out.println(dayName);
```

**Why the arrow (->) form matters (interview favorite):** The classic `case x:` form's biggest historical bug source was **forgetting** `break`, causing unintended fall-through into the next case. The `case x ->` arrow form (Java 14+) executes **ONLY** the matched branch, with no fall-through possible, and can directly be used as an **expression** returning a value.

## 3. Looping Statements

| Loop                                 | Use When                                                                        | Structure                                      |
|--------------------------------------|---------------------------------------------------------------------------------|------------------------------------------------|
| <code>for</code>                     | Number of iterations is known/countable                                         | <code>for (init; condition; update) { }</code> |
| Enhanced <code>for</code> (for-each) | Iterating over a collection/array without needing an index                      | <code>for (Type item : collection) { }</code>  |
| <code>while</code>                   | Condition checked <b>BEFORE</b> each iteration; may run zero times              | <code>while (condition) { }</code>             |
| <code>do-while</code>                | Condition checked <b>AFTER</b> each iteration; always runs <b>AT LEAST</b> once | <code>do { } while (condition);</code>         |



```

for (int i = 0; i < 5; i++) { // classic for
System.out.println(i);
}

int[] nums = {10, 20, 30};
for (int n : nums) { // enhanced for-each (desugars to Iterator calls)
System.out.println(n);
}

int i = 0;
while (i < 5) { // may execute ZERO times if condition starts false
System.out.println(i);
i++;
}

int j = 0;
do { // guaranteed to execute AT LEAST once
System.out.println(j);
j++;
} while (j < 5);

```

## 4. Jump Statements

```

for (int i = 0; i < 10; i++) {
if (i == 5) break; // exits the loop ENTIRELY
if (i % 2 == 0) continue; // skips to the NEXT iteration, doesn't exit
System.out.println(i);
}

```

### *Labeled break/continue — controlling OUTER loops from an inner loop*

```

outer:
for (int i = 0; i < 3; i++) {
for (int j = 0; j < 3; j++) {
if (j == 1) continue outer; // skips to next ITERATION OF THE OUTER loop
if (i == 2) break outer; // breaks OUT OF the outer loop entirely
System.out.println(i + "," + j);
}
}

```

## 5. Real Life Analogy

- **Restaurant:** `switch` on order type is like a kitchen routing station — an order ticket (value) is routed to exactly ONE station (branch) based on its type; the modern arrow syntax is like a station that automatically stops routing further once matched (no accidental fall-through into the wrong station).
- **Bank:** A `do-while` loop is like an ATM's PIN entry — it ALWAYS asks at least once, then repeats only if the condition (wrong PIN) holds; a `while` loop is like checking account balance before allowing ANY withdrawal attempt at all — it may never even start the loop body if the balance is already zero.

## 6. Edge Cases

| Case                                                                                   | Result                                                                                                                                              |
|----------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>switch</code> with no <code>default</code> and no case matches                   | Simply does nothing (traditional) / <b>compile error</b> if used as a <code>switch</code> EXPRESSION that must produce a value and isn't exhaustive |
| Missing <code>break</code> in traditional <code>switch</code>                          | Falls through into subsequent case(s) until a <code>break</code> or the end is reached — classic bug source                                         |
| <code>while (false) { ... }</code>                                                     | Body never executes, not even once                                                                                                                  |
| <code>do { ... } while (false);</code>                                                 | Body executes <b>exactly once</b> , regardless of the condition                                                                                     |
| <code>break/continue</code> without a label, inside nested loops                       | Only affects the <b>innermost</b> enclosing loop                                                                                                    |
| Switch expression (arrow form) on an enum missing a case, with no <code>default</code> | Compile-time error if not exhaustive — a genuine safety improvement over traditional <code>switch</code>                                            |

## 7. Common Mistakes

- Forgetting `break` in traditional `switch` statements, causing silent fall-through bugs.
- Using `while` when `do-while` was intended (or vice versa) — off-by-one logic errors about whether the body should run at least once.
- Assuming unlabeled `break/continue` affects an outer loop from inside nested loops — it only ever affects the innermost one without an explicit label.

## 8. Best Practices

- Prefer the modern arrow-based switch expression (Java 14+) over traditional `switch` wherever possible — eliminates fall-through bugs and can be used directly as an expression.
- Use enhanced for-each loops when you don't need the index — more readable and avoids off-by-one index bugs.
- Avoid deeply nested loops with multiple labeled breaks where possible — consider extracting logic into a separate method with an early `return` instead, for readability.

## 9. Frequently Asked Interview Questions

- What's the difference between `while` and `do-while`?** `while` checks the condition before each iteration (may run zero times); `do-while` checks after (always runs at least once).
- What happens if you forget `break` in a traditional `switch` case?** Execution "falls through" into the next case's code, continuing until a `break` or the switch's end is reached.
- How does the Java 14+ arrow switch expression avoid fall-through?** Each `case X ->` branch executes in isolation with no fall-through by design — no `break` needed or even permitted in that form.
- What does `yield` do inside a switch expression?** Returns a value from a multi-statement block branch (`case X -> { ... yield value; }`) of a switch expression.

- 5 **What does `labeled continue outer`; do differently from a `plain continue`; in nested loops?** Plain `continue` skips to the next iteration of the *innermost* loop; `continue outer` (with a label) skips to the next iteration of the specifically labeled *outer* loop instead.
- 6 **Can a `switch` expression require exhaustiveness?** Yes — when switching over an enum or sealed type without a `default`, the compiler requires all possible cases to be covered, otherwise it's a compile-time error.

## 10. Revision Notes

### CONTROL STATEMENTS — CHEAT SHEET

=====

`if / else if / else` -> exactly one branch runs

`switch (case X: ... break;)` -> TRADITIONAL, falls through without `break`!

`switch (case X -> ...)` -> MODERN (Java 14+), no fall-through, can return a value (`yield`)

`for (init; cond; update)` -> known iteration count

`for (Type t : collection)` -> for-each, no index needed

`while (cond) { }` -> checks BEFORE, may run 0 times

`do { } while (cond);` -> checks AFTER, runs  $\geq 1$  time always

`break` -> exits the loop/switch entirely

`continue` -> skips to next iteration

`label:` -> lets `break/continue` target an OUTER loop specifically

## 11. Homework

- 1 Write a traditional `switch` deliberately missing a `break`, observe the fall-through bug, then fix it two ways: adding `break`, and rewriting it as a Java 14+ arrow `switch` expression.
- 2 Write a nested loop with a labeled `break outer`; and prove, with print statements, exactly which iterations get skipped versus executed.

---

(End of Topic 10: Control Statements.)

---

# TOPIC 11: ARRAYS

---

## 1. Introduction

**What is it?** An array is a fixed-size, ordered, index-based container holding multiple values of the **same type** in one contiguous block of memory.

**Why introduced:** Without arrays, storing "10 test scores" would require 10 separate variables (`score1, score2, ... score10`) — unmanageable for real data. Arrays give a single name and index-based access to a whole collection of same-typed values.

**Interview definition:** "An array in Java is a fixed-length, reference-type object that stores a sequence of elements of a single declared type (primitive or reference), providing constant-time  $O(1)$  indexed access."

## 2. Real Life Analogy

- **College:** An array is like a row of numbered lockers in a hallway — each locker (index) holds exactly one item, all lockers are the same size/type, and the row has a fixed total count decided when installed (array size is fixed at creation).
- **Library:** A bookshelf with numbered slots — you can instantly go to "slot 7" (O(1) access), but you can't add a slot 21 to a shelf built for 20 without building an entirely new, bigger shelf (arrays cannot be resized).

## 3. Internal Working

- Arrays in Java are **objects** — even an array of primitives (`int[]`) is allocated on the **Heap**, and an array variable on the Stack holds a **reference** to it (same reference/object relationship as `[[Java Memory Model|Topic 4]]`).
- The JVM allocates one **contiguous block of memory** sized (`element size × length`) plus a small header (which stores the array's length and type info).
- Because storage is contiguous, accessing `arr[i]` is a direct memory offset calculation — **O(1)**, regardless of array size or which index is accessed.
- Array length is **fixed at creation** — there is no "resize" operation; "growing" an array (as `ArrayList` does internally) actually means allocating a brand-new, larger array and copying all elements over.

```
int[] arr = new int[5];

STACK HEAP
+-----+ +-----+
| arr ---+----->| [length=5][0][0][0][0] |
+-----+ +-----+
```

## 4. Syntax

```
int[] scores = new int[5]; // declaration + allocation, all elements default to 0
int[] values = {10, 20, 30}; // declaration + array literal initializer
int[][] matrix = new int[3][4]; // 2D array: 3 rows, 4 columns
int[][] jagged = new int[3][]; // jagged array: rows can have DIFFERENT lengths
jagged[0] = new int[2];
jagged[1] = new int[5];
```

- `int[] arr` and `int arr[]` are both legal (C-style trailing brackets supported for compatibility, but `Type[] name` is the preferred style).
- Array length is accessed via the `.length` **field** (not a method — no parentheses, unlike `String.length()`).

## 5. Key Methods (via `java.util.Arrays`)

| Method                                     | Purpose                                                                             | Time Complexity           |
|--------------------------------------------|-------------------------------------------------------------------------------------|---------------------------|
| <code>Arrays.sort(arr)</code>              | Sorts in ascending order (dual-pivot quicksort for primitives, TimSort for objects) | $O(n \log n)$             |
| <code>Arrays.binarySearch(arr, key)</code> | Finds an element's index (array MUST be sorted first)                               | $O(\log n)$               |
| <code>Arrays.toString(arr)</code>          | Human-readable string representation                                                | $O(n)$                    |
| <code>Arrays.equals(a, b)</code>           | Element-by-element equality check                                                   | $O(n)$                    |
| <code>Arrays.fill(arr, val)</code>         | Fills every element with the given value                                            | $O(n)$                    |
| <code>Arrays.copyOf(arr, newLength)</code> | Creates a new, resized copy (this is how "growable" arrays are simulated)           | $O(n)$                    |
| <code>Arrays.asList(arr)</code>            | Wraps an array as a <b>fixed-size List</b> view (no add/remove!)                    | $O(1)$ (view, not a copy) |

## 6. Code Examples

```
import java.util.Arrays;

public class ArrayDemo {
    public static void main(String[] args) {
        int[] scores = {85, 92, 78, 95, 60};

        // Basic iteration
        for (int i = 0; i < scores.length; i++) {
            System.out.println("Index " + i + ": " + scores[i]);
        }

        // Enhanced for-each (no index access)
        for (int score : scores) {
            System.out.println(score);
        }

        Arrays.sort(scores);
        System.out.println(Arrays.toString(scores)); // [60, 78, 85, 92, 95]

        int index = Arrays.binarySearch(scores, 92);
        System.out.println("Found 92 at index: " + index);

        // 2D array
        int[][] grid = {{1, 2}, {3, 4}, {5, 6}};
        for (int[] row : grid) {
            System.out.println(Arrays.toString(row));
        }
    }
}
```

## 7. Edge Cases

| Case                                                      | Result                                                                                                                                                                          |
|-----------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Accessing <code>arr[arr.length]</code> (one past the end) | <code>ArrayIndexOutOfBoundsException</code>                                                                                                                                     |
| Accessing <code>arr[-1]</code>                            | <code>ArrayIndexOutOfBoundsException</code>                                                                                                                                     |
| Declaring <code>int[] arr;</code> without initializing    | <code>arr</code> is <code>null</code> — accessing <code>arr.length</code> throws <code>NullPointerException</code>                                                              |
| <code>Arrays.binarySearch</code> on an unsorted array     | Returns an <b>undefined/incorrect</b> result — silently wrong, no exception                                                                                                     |
| Array of objects, uninitialized elements                  | Each element defaults to <code>null</code> (NOT an empty object)                                                                                                                |
| Comparing two arrays with <code>==</code>                 | Compares <b>references</b> , not contents — always <code>false</code> for two separately-created arrays even with identical elements (use <code>Arrays.equals()</code> instead) |
| Multi-dimensional "jagged" arrays                         | Fully supported — rows can have different lengths, since a 2D array in Java is really "an array of array references," not a true rectangular block                              |

## 8. Common Mistakes

- Using `==` to compare array contents instead of `Arrays.equals()` — a very common beginner bug.
- Off-by-one errors (`<=` instead of `<` in loop conditions), causing `ArrayIndexOutOfBoundsException`.
- Forgetting arrays have a **fixed size** and trying to "add" elements directly — must create a new, larger array (or better, use `ArrayList` if resizing is genuinely needed).
- Calling `Arrays.binarySearch` without sorting first.

## 9. Best Practices

- Use arrays when the size is fixed and known, and raw performance/low memory overhead matters; use `ArrayList`/`Collections` when the size needs to change dynamically.
- Always use `Arrays.equals()`/`Arrays.deepEquals()` (for nested arrays) for content comparison, never `==`.
- Prefer `Arrays.toString()`/`Arrays.deepToString()` for debugging output instead of manual loops.

## 10. Production Usage

- Arrays underpin nearly every `Collection` implementation internally — `ArrayList` is backed by a resizable `Object[]`, `HashMap`'s buckets are backed by an array of nodes.
- Performance-critical code (numerical computing, image/audio buffers, low-level parsing) prefers raw primitive arrays over boxed `Collections` to avoid autoboxing overhead and to keep data contiguous in memory (better CPU cache locality).

## 11. Frequently Asked Interview Questions

- 1 **Are arrays objects in Java?** Yes — even primitive arrays (`int[]`) are Heap-allocated objects with a reference on the Stack, and they have a `.length` field and inherit from `Object`.
- 2 **What is the time complexity of accessing an array element by index?**  $O(1)$  — direct memory offset calculation, since storage is contiguous.
- 3 **Can array size change after creation?** No — arrays are fixed-size; "growing" means allocating a new, larger array and copying elements (which is exactly what `ArrayList` does internally).
- 4 **Why does `arr1 == arr2` return false even for two arrays with identical elements?** Because `==` on reference types compares object references (memory addresses), not contents — use `Arrays.equals()` for content comparison.
- 5 **What's a jagged array, and does Java support it?** An array of arrays where each inner array can have a different length; Java fully supports it because a 2D array is really "an array of array references," not one contiguous rectangular block.
- 6 **What happens if you call `Arrays.binarySearch` on an unsorted array?** The result is undefined/incorrect — it may return a wrong index silently, with no exception, because binary search's correctness assumes sorted input.
- 7 **What is `Arrays.asList()`'s major limitation?** It returns a fixed-size list backed directly by the array — calling `add()`/`remove()` on it throws `UnsupportedOperationException`.

## 12. Revision Notes

### ARRAYS — CHEAT SHEET

=====

- Fixed size, same-type elements, contiguous memory,  $O(1)$  indexed access
- Arrays are OBJECTS (Heap-allocated), reference lives on Stack
- `.length` is a FIELD (no parens), unlike `String.length()`
- Default values: `0/0.0/false` for primitives, `null` for object arrays

`Arrays.sort()` ->  $O(n \log n)$

`Arrays.binarySearch()` ->  $O(\log n)$ , REQUIRES sorted array first

`Arrays.equals()` -> content comparison (use this, NOT `==`)

`Arrays.copyOf()` -> resize by copying to a new array

COMMON BUG: `arr1 == arr2` compares REFERENCES not contents

COMMON BUG: off-by-one -> `ArrayIndexOutOfBoundsException`

## 13. Homework

- 1 Write a program that deliberately triggers `ArrayIndexOutOfBoundsException` and `NullPointerException` (on an uninitialized array), and read the exact messages.
- 2 Write a program comparing two arrays with identical contents using both `==` and `Arrays.equals()`, and explain the differing results.
- 3 Implement a simple "growable array" manually using `Arrays.copyOf()` each time capacity is exceeded — mimicking `ArrayList`'s internal resizing strategy.

---

(End of Topic 11: Arrays.)

---

# TOPIC 12: METHODS

## 1. Introduction

**What is it?** A method is a named, reusable block of code that performs a specific task, optionally accepting input (parameters) and optionally returning a value.

**Why introduced:** Without methods, every piece of logic would need to be duplicated everywhere it's needed. Methods enable **code reuse**, **modularity**, and **abstraction** (calling `calculateInterest()` hides the implementation details from the caller).

**Interview definition:** "A method is a named block of code associated with a class or object that defines a set of behaviors, characterized by a signature (name + parameter types), a return type, and an optional body, invoked by reference to that signature."

## 2. Syntax — Every Part Explained

```
public static int add(int a, int b) {  
    return a + b;  
}
```

| Part                           | Meaning                                                                           |
|--------------------------------|-----------------------------------------------------------------------------------|
| <code>public</code>            | Access modifier — visibility of the method                                        |
| <code>static</code>            | Belongs to the class itself, callable without an object instance                  |
| <code>int</code> (return type) | Type of value returned to the caller; <code>void</code> means nothing is returned |
| <code>add</code>               | Method name (verb-based naming convention, camelCase)                             |
| <code>(int a, int b)</code>    | Parameter list — the method's declared inputs                                     |
| <code>return a + b;</code>     | Returns the computed value and immediately exits the method                       |

**Method signature** = method name + parameter types (NOT the return type, and NOT parameter names) — this is exactly what the compiler uses to distinguish overloaded methods.

## 3. Real Life Analogy

- **Bank:** A method is like a specific bank service window (`processWithdrawal(accountId, amount)`) — you don't need to know HOW the vault mechanism works internally; you just provide the required inputs and get a result back.
- **Restaurant:** A method is like a named recipe (`prepareDish(ingredients)`) — the chef (caller) doesn't re-derive the recipe each time; they just invoke it with fresh ingredients (arguments) whenever needed.



## 4. Pass-by-Value — The #1 Interview Trap Topic

Java is ALWAYS pass-by-value — there is no pass-by-reference, ever, for anything, including objects.

What's actually passed depends on the parameter type:

- **Primitive parameter:** a **copy of the value** is passed — changes inside the method never affect the caller's variable.
- **Object/array parameter:** a **copy of the reference (the pointer/address)** is passed — the method CAN mutate the object's internal state (since both copies point to the same Heap object), but **reassigning the parameter itself** to a new object has zero effect on the caller's original reference.

```
public class PassByValueDemo {
    public static void main(String[] args) {
        int x = 10;
        modifyPrimitive(x);
        System.out.println(x); // still 10 - primitive copy was modified, not x itself

        StringBuilder sb = new StringBuilder("Hello");
        mutateObject(sb);
        System.out.println(sb); // "Hello World" - mutation IS visible (same object)

        reassignReference(sb);
        System.out.println(sb); // still "Hello World" - reassignment inside method has NO effect
    }

    static void modifyPrimitive(int val) {
        val = 999; // only changes the LOCAL copy
    }

    static void mutateObject(StringBuilder s) {
        s.append(" World"); // mutates the SAME object both references point to
    }

    static void reassignReference(StringBuilder s) {
        s = new StringBuilder("Different Object"); // only reassigns the LOCAL reference copy
    }
}
```

## 5. Varargs (Variable Arguments)

```
static int sum(int... numbers) { // compiles to int[] numbers internally
    int total = 0;
    for (int n : numbers) total += n;
    return total;
}
// Callable as: sum(), sum(1), sum(1,2,3) - any number of int arguments
```

- Must be the **last** parameter in the list.
- A method can have at most one varargs parameter.

## 6. Recursion

```
static int factorial(int n) {
    if (n <= 1) return 1; // BASE CASE - required to stop recursion
    return n * factorial(n - 1); // RECURSIVE CASE
}
```

- Each recursive call pushes a **new Stack frame** (see [[Java Memory Model|Topic 4]]) — missing/wrong base case causes `StackOverflowError`.

## 7. Static vs Instance Methods

| Aspect                               | Static Method                         | Instance Method                        |
|--------------------------------------|---------------------------------------|----------------------------------------|
| Belongs to                           | The class                             | A specific object                      |
| Called via                           | <code>ClassName.method()</code>       | <code>object.method()</code>           |
| Can access instance fields directly? | No                                    | Yes                                    |
| Can access static fields directly?   | Yes                                   | Yes                                    |
| Typical use                          | Utility/helper logic, factory methods | Behavior tied to an object's own state |

## 8. Edge Cases

| Case                                                                         | Result                                                                                              |
|------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| Calling a static method via an instance ( <code>obj.staticMethod()</code> )  | Legal, but discouraged — compiler resolves it statically anyway (a common IDE warning)              |
| Method with no <code>return</code> but declared <code>non-void</code>        | Compile-time error: "missing return statement"                                                      |
| Recursive method with no base case                                           | <code>StackOverflowError</code> at runtime                                                          |
| Passing <code>null</code> as an object argument, method calls a method on it | <code>NullPointerException</code> inside the method                                                 |
| Two methods differing ONLY by return type (same name + same parameters)      | <b>Compile-time error</b> — return type is NOT part of the method signature for overload resolution |

## 9. Common Mistakes

- Believing Java "passes objects by reference" — it always passes by value; only the *reference's value* is copied for objects (a critical, frequently-misunderstood distinction).
- Forgetting `varargs` must be the last parameter.
- Infinite recursion from a missing or incorrect base case.

## 10. Best Practices

- Keep methods short and focused on a single responsibility (Single Responsibility Principle).
- Use descriptive, verb-based names (`calculateTotal()`, not `calc()` or `doStuff()`).
- Prefer returning new values over mutating input parameters, where practical, for clearer/more predictable code.

## 11. Frequently Asked Interview Questions

- 1 **Is Java pass-by-value or pass-by-reference?** Always pass-by-value — for objects, it's the **reference's value** (the address) that's copied, not the object itself, and not a true reference to the caller's variable.
- 2 **If Java is pass-by-value, why can a method mutate an object passed into it?** Because the copied reference still points to the SAME Heap object — mutating through it (e.g., `list.add(x)`) affects that shared object; but reassigning the parameter to a brand-new object has no effect on the caller's original reference.
- 3 **What is a method signature?** The method's name plus its parameter types (NOT the return type, and NOT parameter names) — used by the compiler to resolve overloaded methods.
- 4 **Why can't two methods differ only by return type?** Because the compiler uses the method signature (name + parameter types) for overload resolution, and a call site with matching arguments would be ambiguous if only return type differed.
- 5 **What must every varargs parameter satisfy?** It must be the last parameter in the method's parameter list, and a method can only have one.
- 6 **What causes `StackOverflowError` in a recursive method?** A missing, unreachable, or incorrect base case, causing unbounded recursive calls that exhaust the thread's Stack.

## 12. Revision Notes

### METHODS — CHEAT SHEET

=====

SIGNATURE = method name + parameter TYPES (NOT return type, NOT param names)

JAVA IS ALWAYS PASS-BY-VALUE:

primitive param -> copy of VALUE passed, caller's variable never changes

object param -> copy of REFERENCE passed:

- mutating through it AFFECTS the shared object

- reassigning the param does NOT affect caller's reference

varargs (Type... name) -> must be LAST param, compiles to an array internally

recursion -> needs a BASE CASE or -> `StackOverflowError`

static method -> belongs to class, no 'this', callable without an instance

instance method -> belongs to an object, has access to 'this'

## 13. Homework

- 1 Write the `PassByValueDemo` example yourself from scratch (without looking) and predict each output BEFORE running it.
- 2 Write a recursive method to compute Fibonacci numbers, then deliberately break its base case to observe `StackOverflowError`.

---

(End of Topic 12: Methods.)

---

## TOPIC 13: COMMAND LINE ARGUMENTS

---

## 1. Introduction

**What is it?** Command-line arguments are values passed to a Java program at the moment it's launched from the terminal/shell, made available inside `main(String[] args)`.

**Why introduced:** Programs often need external, launch-time configuration (a file path, a mode flag, a username) without hardcoding it or requiring interactive input — command-line arguments give a simple, scriptable way to parameterize a program's behavior.

**Interview definition:** "Command-line arguments are String tokens passed after the class/JAR name on the `java` invocation, delivered to the program as the `String[] args` parameter of its `main` method, parsed and split by the shell/OS before the JVM ever sees them."

## 2. Real Life Analogy

- **Restaurant:** Command-line arguments are like special instructions handed to the kitchen at order time ("no onions", "extra spicy") — provided once, at the start, rather than the chef needing to ask interactively partway through cooking.
- **Bank:** Similar to filling out a deposit slip BEFORE approaching the counter — you hand over pre-decided parameters (account number, amount) rather than the teller (program) needing to pause and ask you mid-transaction.

## 3. Internal Working

```
java ReportGenerator sales.csv --verbose 2024
```

- 1 The **shell/OS** splits the command line into whitespace-separated tokens.
- 2 Everything after the class name (`ReportGenerator`) is collected into a `String[]` array: `["sales.csv", "--verbose", "2024"]`.
- 3 The JVM passes this array as the argument to `main(String[] args)`.
- 4 `args[0]` is the FIRST argument AFTER the class name — NOT the program name itself (unlike C's `argv[0]`, which includes the program name; Java's `args` excludes it entirely).

## 4. Syntax

```
public class ReportGenerator {  
    public static void main(String[] args) {  
        for (int i = 0; i < args.length; i++) {  
            System.out.println("arg[" + i + "] = " + args[i]);  
        }  
    }  
}
```

```
java ReportGenerator sales.csv --verbose 2024  
# Output:  
# arg[0] = sales.csv  
# arg[1] = --verbose  
# arg[2] = 2024
```

## 5. Code Examples

### *Basic — Safe Access with Defaults*

```
public class GreetingApp {
    public static void main(String[] args) {
        String name = args.length > 0 ? args[0] : "Guest"; // default if missing
        System.out.println("Hello, " + name + "!");
    }
}
```

### *Intermediate — Parsing Numeric Arguments*

```
public class AreaCalculator {
    public static void main(String[] args) {
        if (args.length != 2) {
            System.out.println("Usage: java AreaCalculator <width> <height>");
            return;
        }
        double width = Double.parseDouble(args[0]); // String -> double conversion
        double height = Double.parseDouble(args[1]);
        System.out.println("Area: " + (width * height));
    }
}
```

### *Advanced — Flag-Style Argument Parsing (production pattern)*

```
import java.util.Arrays;
import java.util.List;

public class ConfigurableApp {
    public static void main(String[] args) {
        List<String> argList = Arrays.asList(args);
        boolean verbose = argList.contains("--verbose");
        int threadCount = 4; // default

        for (int i = 0; i < args.length - 1; i++) {
            if (args[i].equals("--threads")) {
                threadCount = Integer.parseInt(args[i + 1]);
            }
        }
        System.out.println("Verbose: " + verbose + ", Threads: " + threadCount);
    }
}
```

**Production note:** Real CLI tools rarely hand-parse args like this — they use libraries like **Picocli** or **Apache Commons CLI** for robust flag parsing, help-text generation, and validation.

## 6. Edge Cases

| Case                                                                                             | Result                                                                                         |
|--------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| No arguments passed                                                                              | <code>args</code> is an <b>empty array</b> ( <code>length == 0</code> ), NOT <code>null</code> |
| Accessing <code>args[0]</code> with zero arguments                                               | <code>ArrayIndexOutOfBoundsException</code>                                                    |
| Passing a non-numeric string where a number is expected ( <code>Integer.parseInt("abc")</code> ) | <code>NumberFormatException</code>                                                             |
| An argument containing spaces (e.g., "John Doe")                                                 | Must be quoted on the command line, otherwise the shell splits it into TWO separate arguments  |
| Very large number of arguments                                                                   | No language-level limit; practically bounded by OS command-line length limits                  |

## 7. Common Mistakes

- Assuming `args` is `null` when nothing is passed — it's actually an **empty array**.
- Forgetting to quote multi-word arguments on the shell, causing them to be split unexpectedly.
- Not validating `args.length` before indexing, causing `ArrayIndexOutOfBoundsException` in production scripts.
- Using `Integer.parseInt()/Double.parseDouble()` without a `try-catch`, crashing on malformed input instead of showing a helpful usage message.

## 8. Best Practices

- Always validate `args.length` and provide a clear "Usage: ..." message before attempting to parse.
- Wrap numeric parsing in `try-catch` for `NumberFormatException`, giving a helpful error instead of a raw stack trace.
- For any non-trivial CLI tool, use a dedicated argument-parsing library (Picocli, JCommander, Apache Commons CLI) instead of hand-rolled parsing logic.

## 9. Frequently Asked Interview Questions

- What type is `args` in `main(String[] args)`?** An array of `String`, each element being one whitespace-separated token passed after the class/JAR name on the command line.
- Is `args` ever `null`?** No — if no arguments are passed, `args` is a valid but **empty array** (`length == 0`).
- Does `args[0]` include the program/class name (like C's `argv[0]`)?** No — Java's `args[0]` is the FIRST actual argument; the class name is not included at all.
- How do you pass a multi-word single argument?** Wrap it in quotes on the command line, e.g., `java App "John Doe"` — otherwise the shell splits it into two separate arguments.
- How would you safely convert a command-line argument to an `int`?** Use `Integer.parseInt(args[i])` wrapped in a `try-catch` for `NumberFormatException` to handle invalid input gracefully.

## 10. Revision Notes

### COMMAND LINE ARGUMENTS — CHEAT SHEET

=====

```
public static void main(String[] args)
```

- args = String[] of tokens AFTER the class name on the command line
- NO arguments passed -> args.length == 0 (empty array, NEVER null)
- args[0] = FIRST actual argument (unlike C's argv[0], which is the program name)

#### COMMON ERRORS:

ArrayIndexOutOfBoundsException -> accessed args[i] beyond args.length

NumberFormatException -> Integer.parseInt() on non-numeric text

MULTI-WORD ARG: must be quoted on the shell ("John Doe"), else split into 2 args

PRODUCTION TIP: use Picocli / Apache Commons CLI for real CLI tools, not manual parsing

## 11. Homework

- 1 Write a program that prints a helpful usage message if fewer than 2 arguments are passed, otherwise processes them.
- 2 Deliberately pass a non-numeric string to a program expecting a number, observe the raw `NumberFormatException` stack trace, then wrap the parsing in `try-catch` to show a friendly error instead.

---

(End of Topic 13: Command Line Arguments — this concludes the "Java Fundamentals" sub-section. Next: Object-Oriented Programming.)

---

# TOPIC 14: CLASSES & OBJECTS

---

## 1. Introduction

### 1.1 What is it?

A **class** is a blueprint/template that defines the structure (fields) and behavior (methods) that its objects will have. An **object** is a concrete instance of a class, created at runtime, occupying its own memory on the Heap, with its own copy of instance field values.

### 1.2 Why was it introduced

Procedural programming (C-style: separate data structures + free-floating functions operating on them) becomes unmanageable as programs grow — data and the logic that operates on it live far apart, making it easy to misuse data or duplicate logic. **Object-Oriented Programming (OOP)** bundles data and behavior together into a single unit (a class), modeling real-world entities directly in code — improving organization, reuse, and maintainability for large systems.

### 1.3 Real-world problem it solves

- Models real business entities directly (`Customer`, `Order`, `Account`) instead of scattered arrays/structs and disconnected functions.
- Encapsulation (see [[Encapsulation]], covered later) protects an object's internal state from being corrupted by external code.
- Enables **code reuse** through inheritance and **polymorphic** behavior — foundational to virtually all enterprise Java frameworks (Spring beans, JPA entities, DTOs are all just classes/objects).

## 1.4 Interview Definition

"A class is a user-defined blueprint or prototype that defines fields (state) and methods (behavior); an object is a runtime instance of a class, allocated on the Heap, with its own distinct copy of instance field values but sharing the class's method bytecode and static members."

## 1.5 Simple Definition

A class is a recipe. An object is the actual dish made by following that recipe — you can make many dishes (objects) from one recipe (class), and each dish can have its own specific ingredients (field values).

## 2. Real Life Analogy

| Domain     | Analogy                                                                                                                                                                                                                                                                                                                             |
|------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| College    | <code>Student</code> is a class (blueprint: name, rollNumber, marks, methods like <code>attendClass()</code> ). Each actual enrolled student ("Alice, roll 101", "Bob, roll 102") is a separate <b>object</b> — same blueprint, different data.                                                                                     |
| Bank       | <code>Account</code> is a class (fields: <code>accountNumber</code> , <code>balance</code> ; methods: <code>deposit()</code> , <code>withdraw()</code> ). Every customer's actual account is a separate object with its own balance, but they all share the same <code>deposit()/withdraw()</code> logic defined once in the class. |
| Restaurant | <code>MenuItem</code> is a class (name, price, category). "Margherita Pizza – ₹299" and "Cold Coffee – ₹120" are two different objects created from the same <code>MenuItem</code> class.                                                                                                                                           |
| Amazon     | <code>Product</code> class defines what every listed product looks like structurally (title, price, seller, rating); each individual listing on the site is an object instance of that class, stored as a row/document in a database, materialized into memory as needed.                                                           |

## 3. Internal Working — What Happens When You Create an Object

```
Person p = new Person("Alice", 30);
```

### Step-by-step:

1. **Class loading** (if not already loaded): the JVM's Class Loader loads `Person.class` into the Method Area/Metaspace (see [[JDK, JRE, JVM|Topic 2]]) — this only happens ONCE per class, the first time it's referenced.



2. **new keyword:** the JVM allocates a block of memory on the **Heap** exactly large enough for one **Person** object (all its instance fields, plus an object header used internally for things like the hash code and lock state).
3. **Default initialization:** all instance fields are first set to their type's default (**null** for **String** **name**, **0** for **int** **age**) — this happens **BEFORE** the constructor body runs.
4. **Constructor execution:** `Person("Alice", 30)` runs, assigning the actual values (`this.name = "Alice"; this.age = 30;`) into that Heap block.
5. **Reference returned:** the memory address of the newly created object is returned and stored in the **Stack** variable **p** (a reference, not the object itself).

```

STACK HEAP
+-----+ +-----+
| p -----+----->| Person object |
+-----+ | header (hashcode, lock info) |
| name = "Alice" |
| age = 30 |
+-----+

```

### 3.1 The `this` Keyword

`this` refers to the **current object instance** inside an instance method or constructor — most commonly used to disambiguate a field from a same-named parameter:

```

class Person {
    String name;
    Person(String name) {
        this.name = name; // this.name = the FIELD, name (right side) = the PARAMETER
    }
}

```

## 4. Diagrams

### 4.1 Class as a Blueprint — Multiple Objects, One Class

```

CLASS: Person (blueprint, ONE copy in Method Area)
fields: name, age
methods: greet(), haveBirthday()
|
-----
| | |
v v v
Object 1 (Heap) Object 2 (Heap) Object 3 (Heap)
name="Alice" name="Bob" name="Carol"
age=30 age=25 age=40

```

Each object has its **OWN** field values, but they all share the **SAME** method bytecode (`greet()`, `haveBirthday()`) defined once in the class metadata.

## 4.2 Object Creation Timeline

```
new Person("Alice", 30)
|
v
[1] Class loaded? --No--> Load Person.class into Method Area (ONE TIME)
|
v
[2] Allocate memory block on HEAP for one Person instance
|
v
[3] Set ALL fields to default values (null/0/false) FIRST
|
v
[4] Run constructor body -> assign actual field values
|
v
[5] Return reference -> stored in a Stack variable (e.g., 'p')
```

## 5. Syntax

```
public class Person { // class declaration

// Fields (instance variables) - object STATE
private String name;
private int age;

// Constructor - initializes a new object
public Person(String name, int age) {
    this.name = name;
    this.age = age;
}

// Method - object BEHAVIOR
public void greet() {
    System.out.println("Hi, I'm " + name + ", age " + age);
}

// Getter (encapsulation-friendly access)
public String getName() {
    return name;
}
}
```

| Element                           | Meaning                                                                                                        |
|-----------------------------------|----------------------------------------------------------------------------------------------------------------|
| <code>class Person</code>         | Declares the blueprint named <code>Person</code>                                                               |
| <code>private String name;</code> | An instance field — <code>private</code> restricts direct external access (encapsulation, covered fully later) |
| <code>public Person(...)</code>   | A constructor — same name as the class, no return type, runs on <code>new</code>                               |
| <code>public void greet()</code>  | An instance method — operates on <code>this</code> object's own field values                                   |

## 6. Code Examples

### 6.1 Basic

```
public class Car {
    String model;
    int year;

    public static void main(String[] args) {
        Car myCar = new Car(); // no-arg constructor (default, since none defined explicitly)
        myCar.model = "Tesla Model 3"; // direct field access (works since fields aren't private here)
        myCar.year = 2024;
        System.out.println(myCar.model + " (" + myCar.year + ")");
    }
}
```

### 6.2 Intermediate — Encapsulated Class with Constructor

```
public class BankAccount {
    private String accountHolder;
    private double balance;

    public BankAccount(String accountHolder, double initialBalance) {
        this.accountHolder = accountHolder;
        this.balance = initialBalance;
    }

    public void deposit(double amount) {
        if (amount <= 0) {
            throw new IllegalArgumentException("Deposit must be positive");
        }
        balance += amount;
    }

    public double getBalance() {
        return balance;
    }

    public static void main(String[] args) {
        BankAccount acc = new BankAccount("Alice", 1000.0);
        acc.deposit(500.0);
        System.out.println(acc.getBalance()); // 1500.0
    }
}
```

## 6.3 Production-style — Multiple Objects Managed in a Collection

```
package com.jobsradar.banking;

import java.util.ArrayList;
import java.util.List;

/** Demonstrates managing multiple independent object instances. */
public class BankSystem {

    public static void main(String[] args) {
        List<BankAccount> accounts = new ArrayList<>();
        accounts.add(new BankAccount("Alice", 1000.0));
        accounts.add(new BankAccount("Bob", 2500.0));

        for (BankAccount acc : accounts) {
            acc.deposit(100.0);
            System.out.println(acc.getBalance());
        }
    }
}
```

## 7. Dry Run

```
public class DryRunClasses {
    public static void main(String[] args) {
        Person p1 = new Person("Alice");
        Person p2 = new Person("Bob");
        p1.greet();
        p2.greet();
    }
}

class Person {
    String name;
    Person(String name) { this.name = name; }
    void greet() { System.out.println("Hi, I'm " + name); }
}
```

| Step | Stack                                                     | Heap                                           |
|------|-----------------------------------------------------------|------------------------------------------------|
| 1    | main() frame created                                      | —                                              |
| 2    | p1 = <ref>                                                | Object#1 {name:"Alice"} created                |
| 3    | p2 = <ref>                                                | Object#2 {name:"Bob"} created                  |
| 4    | p1.greet() — greet() frame pushed, this bound to Object#1 | reads Object#1.name → prints "Hi, I'm Alice"   |
| 5    | p2.greet() — greet() frame pushed, this bound to Object#2 | reads Object#2.name → prints "Hi, I'm Bob"     |
| 6    | main() returns                                            | both objects now unreachable — eligible for GC |

**Key insight:** Both calls execute the exact SAME `greet()` bytecode (one copy in the Method Area), but `this` is bound to a **different object** each time — this is how one method definition serves unlimited objects correctly.

## 8. Edge Cases

| Case                                                          | Result                                                                                                                             |
|---------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| No constructor defined at all                                 | Compiler auto-generates a public <b>no-argument default constructor</b>                                                            |
| A constructor IS defined (with args), no no-arg one added     | The default no-arg constructor is <b>NOT</b> auto-generated anymore — <code>new Person()</code> becomes a compile error            |
| Accessing a <code>null</code> object reference's field/method | <code>NullPointerException</code>                                                                                                  |
| Comparing two objects with <code>==</code>                    | Compares references (memory addresses), NOT field content — <code>new Person("A") == new Person("A")</code> is false               |
| Two objects created with identical field values               | Are still two DISTINCT objects (different Heap addresses) unless <code>equals()</code> is explicitly overridden to compare content |

## 9. Common Mistakes

- Believing `==` compares object *content* — it compares **references**; use `.equals()` (properly overridden) for content comparison.
- Forgetting that once you define ANY constructor, the compiler stops auto-generating the no-arg default one.
- Making fields `public` directly instead of `private` with getters/setters — breaks encapsulation, a very common beginner anti-pattern.
- Confusing the class (one blueprint, in Method Area) with objects (many instances, on Heap) — a common conceptual gap for beginners moving from procedural languages.

## 10. Best Practices

- Keep fields `private`, expose controlled access via public getters/(validated) setters — the foundation of encapsulation.
- Always provide meaningful constructors that establish valid initial object state (avoid leaving objects in a "half-initialized" state).
- Follow naming conventions: `PascalCase` for class names, `camelCase` for fields/methods.
- One class = one clear responsibility (Single Responsibility Principle).

## 11. Production Usage

- Every JPA/Hibernate **Entity** (`@Entity class User {...}`), every Spring `@Component/@Service/@Repository` bean, and every REST **DTO** is fundamentally just a class being instantiated into managed objects by a framework.

- Frameworks like Spring create and manage object **lifecycles** (construction, dependency injection, destruction) on your behalf — but the underlying mechanics are exactly the class/object/constructor model covered here.
- 

## 12. Frequently Asked Interview Questions

- 1 **What is the difference between a class and an object?** A class is a compile-time blueprint (structure + behavior definition); an object is a runtime instance of that blueprint, occupying its own memory on the Heap.
  - 2 **Where does a class's metadata live, and where do its objects live?** Class metadata lives in the Method Area/Metaspace (one copy); objects live on the Heap (many independent instances).
  - 3 **What is `this` used for?** A reference to the current object instance, commonly used to disambiguate a field from a same-named constructor/method parameter.
  - 4 **Does every class automatically get a default constructor?** Only if you define NO constructors at all — as soon as you define even one constructor (with or without arguments), the auto-generated no-arg default is no longer provided.
  - 5 **Why does `new Person("A").equals(new Person("A"))` return `false` unless `equals()` is overridden?** Because the default `Object.equals()` implementation compares references (same as `==`) — two separately created objects always have different Heap addresses unless equality is explicitly redefined based on field content.
  - 6 **Do all objects of the same class share their method code?** Yes — method bytecode is stored once per class in the Method Area; each object only carries its own instance field values, and `this` is bound per-call to determine which object's data the shared method code operates on.
  - 7 **What happens to fields before a constructor runs?** They're first set to their type's default value (`0`, `false`, `null`) automatically, before any constructor body executes.
- 

## 13. Revision Notes

### CLASSES & OBJECTS – CHEAT SHEET

=====

CLASS = blueprint, ONE copy in Method Area/Metaspace (fields + method bytecode)

OBJECT = runtime instance, on HEAP, own copy of field VALUES, shares method bytecode

#### OBJECT CREATION ORDER:

1. Class loaded (once)
2. Heap memory allocated
3. Fields default-initialized
4. Constructor runs
5. Reference returned to Stack variable

`this` -> refers to the CURRENT object instance

`==` -> compares REFERENCES (memory address), not content

`.equals()` -> compares CONTENT (only if properly overridden)

NO constructor defined -> compiler auto-generates a public no-arg one

ANY constructor defined -> default no-arg one is NO LONGER auto-generated

## 14. Homework

- 1 Create a `Book` class (title, author, price) with a constructor and a `display()` method; instantiate 3 different `Book` objects and print each.
  - 2 Prove `new Book(...) == new Book(...)` is `false` even with identical field values, using `System.out.println`.
  - 3 Remove all constructors from a class, confirm the implicit no-arg constructor works, then add ONE parameterized constructor and confirm `new ClassName()` now fails to compile.
- 

## 15. Mini Project — Simple Library Catalog

```
package com.jobsradar.library;

import java.util.ArrayList;
import java.util.List;

class Book {
    private final String title;
    private final String author;
    private boolean checkedOut;

    public Book(String title, String author) {
        this.title = title;
        this.author = author;
        this.checkedOut = false;
    }

    public void checkOut() {
        if (checkedOut) {
            System.out.println(title + " is already checked out.");
            return;
        }
        checkedOut = true;
        System.out.println(title + " checked out successfully.");
    }

    public String getTitle() { return title; }
    public boolean isCheckedOut() { return checkedOut; }
}

public class LibraryCatalog {
    public static void main(String[] args) {
        List<Book> catalog = new ArrayList<>();
        catalog.add(new Book("Effective Java", "Joshua Bloch"));
        catalog.add(new Book("Clean Code", "Robert Martin"));

        catalog.get(0).checkOut();
        catalog.get(0).checkOut(); // demonstrates state check works per-object

        for (Book b : catalog) {
            System.out.println(b.getTitle() + " -> checked out: " + b.isCheckedOut());
        }
    }
}
```

---

## 16. Final Summary

- A **class** is a compile-time blueprint stored once in the Method Area; an **object** is a runtime instance living on the Heap, with its own copy of field values.
- Object creation follows a strict order: class loading → Heap allocation → default field initialization → constructor execution → reference returned to the caller.
- `this` always refers to the current object instance — essential for disambiguating fields from parameters.
- `==` compares references, never content — content equality requires a properly overridden `equals()` (covered in the `[[Object Class]]` topic).
- Defining any constructor removes the compiler's auto-generated no-arg default — a frequent real-world compile error source.
- Every enterprise Java framework (Spring, Hibernate) is ultimately just orchestrating the same class/object/constructor lifecycle taught here, at scale.

---

*(End of Topic 14: Classes & Objects.)*

---

## TOPIC 15: CONSTRUCTORS

---

### 1. Introduction

**What is it?** A constructor is a special block of code, with the same name as its class and no return type, that runs automatically when an object is created with `new`, responsible for initializing that object's initial state.

**Why introduced:** Without constructors, every object would start with only default field values (`0`, `null`, `false`), forcing callers to manually set every field after creation via separate calls — error-prone and allows objects to exist temporarily in invalid/incomplete states. Constructors guarantee an object is properly initialized in one atomic step, at the moment of creation.

**Interview definition:** "A constructor is a special member of a class, sharing the class's name and having no return type (not even `void`), invoked automatically by the `new` operator to initialize a newly allocated object's state before any reference to it is returned to the caller."

### 2. Real Life Analogy

- **Hospital:** A constructor is like the mandatory intake/admission process for a new patient — before a patient record can exist in the system at all, certain required fields (name, date of birth, emergency contact) **MUST** be captured; you cannot have a "half-created" patient record floating around.
- **College:** A constructor is like the college admission form processing — a `Student` object doesn't exist in the system until admission (construction) is complete with the required minimum data (name, roll number assigned).



## 3. Types of Constructors

### 3.1 Default Constructor (Compiler-Generated)

```
class Person { }  
// Compiler silently generates: public Person() { } -- ONLY if you define NO constructor yourself
```

### 3.2 No-Argument Constructor (Explicitly Written)

```
class Person {  
    String name;  
    Person() { // explicitly written, same effect as default but you control the body  
        this.name = "Unknown";  
    }  
}
```

### 3.3 Parameterized Constructor

```
class Person {  
    String name;  
    int age;  
    Person(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
}
```

### 3.4 Copy Constructor (Java has no built-in one — you write it manually)

```
class Person {  
    String name;  
    int age;  
    Person(String name, int age) { this.name = name; this.age = age; }  
  
    // Copy constructor - manually defined, unlike C++'s automatic one  
    Person(Person other) {  
        this.name = other.name;  
        this.age = other.age;  
    }  
}
```

**Interview note:** Unlike C++, Java does NOT auto-generate a copy constructor — you must write one explicitly if you want that pattern.

### 3.5 Private Constructor (Singleton / Utility Class Pattern)

```
class DatabaseConnection {  
    private static final DatabaseConnection INSTANCE = new DatabaseConnection();  
    private DatabaseConnection() { } // private -> cannot be called with 'new' externally  
    public static DatabaseConnection getInstance() { return INSTANCE; }  
}
```

## 4. Constructor Chaining — `this()` and `super()`

```
class Person {
    String name;
    int age;

    Person() {
        this("Unknown", 0); // this() -> calls ANOTHER constructor in the SAME class
        System.out.println("No-arg constructor finished");
    }

    Person(String name, int age) {
        this.name = name;
        this.age = age;
    }
}
```

- `this(...)` **must be the first statement** in the constructor if used.
- `super(...)` (calling the parent class's constructor) similarly must be the first statement — if you write neither, the compiler **implicitly inserts** `super()` (calling the parent's no-arg constructor) as the very first line. (Full inheritance-related detail covered in the **Constructor Chaining** and **Inheritance** topics.)

## 5. Rules of Constructors

| Rule                                                                          | Detail                                                                                                                                    |
|-------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Same name as the class                                                        | Mandatory — the compiler identifies constructors by exact name match                                                                      |
| No return type                                                                | Not even <code>void</code> — writing a return type turns it into a regular method that happens to share the class name, NOT a constructor |
| Cannot be <code>static</code> , <code>final</code> , or <code>abstract</code> | Constructors are tied to instance creation, incompatible with these modifiers                                                             |
| Can be overloaded                                                             | Multiple constructors with different parameter lists are fully allowed                                                                    |
| Can be <code>private</code>                                                   | Restricts external instantiation — used for Singleton and utility-class patterns                                                          |
| Runs exactly once per object                                                  | Automatically triggered by <code>new</code> , never called again for that same object afterward                                           |

## 6. Code Examples

```
public class ConstructorDemo {
    public static void main(String[] args) {
        Person p1 = new Person(); // no-arg -> chains to this("Unknown", 0)
        Person p2 = new Person("Alice", 30); // parameterized
        Person p3 = new Person(p2); // copy constructor

        System.out.println(p1.name + " " + p1.age);
        System.out.println(p3.name + " " + p3.age); // "Alice 30" - copied from p2
    }
}

class Person {
    String name;
    int age;

    Person() {
        this("Unknown", 0);
    }

    Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    Person(Person other) {
        this.name = other.name;
        this.age = other.age;
    }
}
```

## 7. Edge Cases

| Case                                                                 | Result                                                                                                  |
|----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| Defining any constructor at all                                      | Compiler stops auto-generating the no-arg default constructor                                           |
| Constructor with a return type accidentally added (void Person() {}) | Treated as a REGULAR METHOD, not a constructor — a classic subtle bug                                   |
| Calling <code>this(...)</code> NOT as the first statement            | Compile-time error                                                                                      |
| Recursive constructor chaining (Person() { this(); } calling itself) | Compile-time error: "recursive constructor invocation"                                                  |
| Private constructor, no static factory method provided               | Class can never be instantiated from outside itself (sometimes intentional, e.g., pure utility classes) |

## 8. Common Mistakes

- Accidentally giving a constructor a return type (even `void`), silently turning it into a dead-code method that's never automatically called.
- Forgetting that defining a parameterized constructor removes the free no-arg default — `new ClassName()` then fails to compile unless you add one explicitly.
- Assuming Java auto-generates copy constructors like C++ — it does not; you must write them yourself.
- Placing other statements before `this(...)/super(...)` — always must be the very first line.

## 9. Best Practices

- Use constructor chaining (`this(...)`) to avoid duplicating initialization logic across multiple overloaded constructors.
- Validate arguments inside constructors (throw `IllegalArgumentException` for invalid input) to guarantee objects are never created in an invalid state.
- Prefer `private` constructors + static factory methods (`Person.of(...)`) when you want to control/name object creation more expressively than `new` alone allows.

## 10. Production Usage

- **Singleton pattern** (private constructor + static instance) is used for shared resources like connection pools, configuration managers.
- **Builder pattern** (covered later, often paired with private constructors) is heavily used in frameworks and DTOs for objects with many optional fields (e.g., `HttpClient.newBuilder()...build()`).
- Spring/Hibernate rely on having accessible (often no-arg) constructors to instantiate managed beans/entities via reflection.

## 11. Frequently Asked Interview Questions

- 1 **What is a constructor?** A special class member with the same name as the class and no return type, automatically invoked by `new` to initialize a new object.
- 2 **Can a constructor be `static`?** No — constructors are inherently tied to instance creation and cannot be `static`, `final`, or `abstract`.
- 3 **What happens if you give a constructor a return type?** It's no longer treated as a constructor at all — it becomes a regular method that happens to share the class's name, and won't run automatically on `new`.
- 4 **What is constructor chaining?** Using `this(...)` to call another constructor in the same class (or `super(...)` to call the parent class's constructor) to reuse initialization logic.
- 5 **Does Java provide an automatic copy constructor like C++?** No — Java has no built-in copy constructor; you must write one manually if that pattern is needed.
- 6 **Why must `this(...)/super(...)` be the first statement in a constructor?** To guarantee any delegated/parent initialization completes fully before the current constructor's own body can safely rely on that state.
- 7 **What's a real use case for a `private` constructor?** Singleton pattern (exactly one shared instance) or utility classes (preventing instantiation entirely, exposing only static methods).

## 12. Revision Notes

### CONSTRUCTORS — CHEAT SHEET

=====

- SAME NAME as class, NO return type (not even void)
- Runs automatically via 'new', exactly once per object
- Can be overloaded, can be private (Singleton/utility pattern)
- Cannot be static / final / abstract

TYPES: default (compiler-generated), no-arg (hand-written), parameterized, copy constructor (MANUAL - not auto-generated, unlike C++)

#### CHAINING:

this(...) -> calls another constructor in the SAME class (must be 1st statement)  
super(...) -> calls the PARENT class's constructor (must be 1st statement;  
inserted IMPLICITLY by compiler if you write neither)

GOTCHA: return type on a "constructor" -> becomes a regular METHOD, not a constructor!

GOTCHA: defining ANY constructor removes the free auto-generated no-arg one

## 13. Homework

- 1 Write a class with a no-arg, a parameterized, and a copy constructor, using `this(...)` chaining to avoid duplicated logic.
- 2 Deliberately add a return type to a "constructor" and prove (via a failed `new ClassName()` call, or by observing it's callable like a normal method) that it's no longer a real constructor.
- 3 Implement a Singleton class using a private constructor and a static `getInstance()` method.

---

(End of Topic 15: Constructors.)

---

## TOPIC 16: CONSTRUCTOR CHAINING

---

### 1. Introduction

**What is it?** Constructor chaining is the mechanism by which one constructor invokes another — either another constructor in the **same class** (via `this(...)`) or the constructor of its **superclass** (via `super(...)`) — forming a defined execution order before the calling constructor's own body runs.

**Why introduced:** In a class hierarchy, a subclass object needs its parent's state initialized FIRST (a `Dog` object must have its `Animal` part properly set up before `Dog`-specific fields are set). Chaining guarantees this order automatically and predictably, rather than leaving initialization order to chance.

**Interview definition:** "Constructor chaining is the invocation of one constructor from another, either within the same class (`this(...)`) or from a subclass to its superclass (`super(...)`), always occurring as the first statement of the calling constructor, ensuring a well-defined, top-down initialization order across a class hierarchy."

## 2. Real Life Analogy

- **College:** Admitting a `PostGradStudent` requires first completing the general `Student` admission process (parent constructor via `super()`) — university-wide roll number, ID card — BEFORE any postgraduate-specific setup (thesis advisor assignment) can happen.
- **Hospital:** A `SurgeryPatient` record must first go through the general `Patient` intake process (parent constructor) before surgery-specific fields (consent forms, anesthesiologist assignment) are recorded.

## 3. The Two Forms of Chaining

```
class A {
    A() { System.out.println("A()"); }
    A(int x) { System.out.println("A(int)"); }
}

class B extends A {
    B() {
        this(5); // (1) SAME-CLASS chaining -> calls B(int)
        System.out.println("B()");
    }
    B(int x) {
        super(x); // (2) PARENT-CLASS chaining -> calls A(int)
        System.out.println("B(int)");
    }
}

new B();
// Output:
// A(int)
// B(int)
// B()
```

## 4. Full JVM Object Initialization Order (Critical — Frequently Tested)

When ANY object is created, the JVM follows this **exact** order, top of the hierarchy first:

1. Parent class's static initializers/static blocks (ONCE, first time class is loaded)
2. This class's static initializers/static blocks (ONCE, first time class is loaded)
- 
- (the above only run the very FIRST time the class is ever used – steps below run EVERY time a new object is created)
- 
3. `super(...)` call executes FULLY (recursively following this same order for the parent)
4. This class's INSTANCE variable initializers, in the order they're WRITTEN
5. This class's instance initializer blocks { }, in the order they're WRITTEN
6. This class's constructor BODY executes (everything after the `super()/this()` line)

## Diagram — 3-Level Hierarchy Initialization Order

```
class GrandParent { ... }  
class Parent extends GrandParent { ... }  
class Child extends Parent { ... }
```

```
new Child();
```

EXECUTION ORDER:

- [1] GrandParent static blocks (first use only)
- [2] Parent static blocks (first use only)
- [3] Child static blocks (first use only)
- 
- [4] GrandParent instance field initializers + instance blocks
- [5] GrandParent constructor body
- [6] Parent instance field initializers + instance blocks
- [7] Parent constructor body
- [8] Child instance field initializers + instance blocks
- [9] Child constructor body

**Why this matters:** This proves the ENTIRE parent object is fully constructed before the child's own constructor body runs even ONE line — a subclass can always safely assume its inherited state is ready.

## 5. Rules

| Rule                                                                                              | Detail                                                                                                    |
|---------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| <code>this(...)</code> or <code>super(...)</code> must be the FIRST statement                     | Compile-time error otherwise                                                                              |
| Only ONE of <code>this(...)</code> or <code>super(...)</code> per constructor                     | Never both — they're mutually exclusive in a single constructor                                           |
| If neither is written                                                                             | Compiler <b>implicitly inserts</b> <code>super()</code> (no-arg) as the first line                        |
| If the parent has NO no-arg constructor, and the child writes neither <code>this()/super()</code> | <b>Compile-time error</b> — the implicit <code>super()</code> fails to find a matching no-arg constructor |
| Recursive/circular chaining ( <code>A() { this(); }</code> calling back to itself)                | Compile-time error: "recursive constructor invocation"                                                    |

## 6. Code Examples

```
class Vehicle {
    String type;
    Vehicle() {
        this("Generic Vehicle"); // same-class chaining
        System.out.println("Vehicle() called");
    }
    Vehicle(String type) {
        this.type = type;
        System.out.println("Vehicle(String) called");
    }
}

class Car extends Vehicle {
    String model;
    Car(String model) {
        super("Car"); // MUST be first statement - parent chaining
        this.model = model;
        System.out.println("Car(String) called");
    }
}

public class ChainingDemo {
    public static void main(String[] args) {
        Car c = new Car("Tesla Model 3");
        // Output:
        // Vehicle(String) called <- super("Car") runs Vehicle(String) directly
        // Car(String) called
    }
}
```

## 7. Edge Cases

| Case                                                                                                | Result                                                                                                          |
|-----------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| Parent class has only a parameterized constructor, child writes no explicit <code>super(...)</code> | Compile-time error — implicit <code>super()</code> looks for a no-arg constructor that doesn't exist            |
| Both <code>this(...)</code> and <code>super(...)</code> attempted in one constructor                | Compile-time error — only one is allowed, and only as the very first statement                                  |
| Instance initializer block placed AFTER a field that reads a later-declared field                   | Uses whatever default/assigned value exists at that point in top-to-bottom order — order of declaration matters |
| Static blocks in a deep hierarchy, class used for the first time                                    | Each ancestor's static block runs exactly once, top-down, before ANY instance is created                        |

## 8. Common Mistakes

- Assuming the child constructor's body runs before the parent's — it's always the reverse: parent fully initializes first.
- Forgetting that when a parent class has no no-arg constructor, every subclass MUST explicitly call `super(args)` matching one of the parent's actual constructors.
- Writing both `this(...)` and `super(...)` in the same constructor (illegal — pick one).
- Assuming static blocks run every time an object is created — they run only ONCE, the first time the class is loaded, regardless of how many objects are later created.



## 9. Best Practices

- Use `this(...)` chaining to eliminate duplicated initialization logic across overloaded constructors — always delegate to the "most complete" constructor.
- When designing a class meant to be subclassed, consider providing a no-arg (or clearly documented) constructor to avoid forcing every subclass to chain explicitly.
- Keep constructor bodies focused on validation/assignment — avoid complex logic or calling overridable methods from a constructor (calling an overridable method during construction is a known anti-pattern, since a subclass's override might run before the subclass's own fields are initialized).

## 10. Frequently Asked Interview Questions

- 1 **What is constructor chaining?** One constructor invoking another — `this(...)` for the same class, `super(...)` for the parent class — always as the first statement.
- 2 **What happens if you write neither `this(...)` nor `super(...)`?** The compiler implicitly inserts a no-arg `super()` call as the first statement.
- 3 **Can a constructor call both `this(...)` and `super(...)`?** No — only one, and it must be the very first statement.
- 4 **In a 3-level class hierarchy, which constructor body executes first?** The topmost ancestor's (e.g., `GrandParent`) — parent initialization always fully completes before any subclass constructor body runs.
- 5 **What happens if a parent class has only a parameterized constructor and the child doesn't explicitly call `super(args)`?** Compile-time error — the compiler's implicit `super()` can't find a matching no-arg constructor in the parent.
- 6 **Do static initializer blocks run every time a new object is created?** No — they run exactly once, the first time the class is loaded, regardless of how many objects are subsequently created.
- 7 **Why is calling an overridable instance method from within a constructor considered risky?** Because if a subclass overrides that method, the override executes during the PARENT constructor's run — before the subclass's own fields have been initialized — potentially operating on incomplete/default state.

## 11. Revision Notes

### CONSTRUCTOR CHAINING – CHEAT SHEET

=====

this(...) -> same-class constructor chaining

super(...) -> parent-class constructor chaining

BOTH must be the FIRST statement; only ONE allowed per constructor

Neither written -> compiler inserts implicit super() automatically

FULL INIT ORDER (once per class-load, then once per object):

[class load, ONCE]: parent static blocks -> this class's static blocks

[per object, top-down]:

super(...) fully completes (recursively, same rules)

-> instance field initializers (in written order)

-> instance initializer blocks (in written order)

-> constructor body

GOTCHA: parent has NO no-arg constructor -> child MUST explicitly call super(args)

GOTCHA: static blocks run ONCE per class, NOT once per object

ANTI-PATTERN: calling an overridable method from a constructor

## 12. Homework

- 1 Build a 3-class hierarchy (GrandParent → Parent → Child) each printing its own constructor name, and verify the exact top-down execution order shown in Section 4.
- 2 Remove the parent's no-arg constructor (leave only a parameterized one) without adding an explicit super(...) in the child, and read the exact compile error produced.

---

(End of Topic 16: Constructor Chaining.)

---

# TOPIC 17: METHOD OVERLOADING

---

## 1. Introduction

**What is it?** Method overloading is defining multiple methods in the same class with the **same name** but **different parameter lists** (different number, types, or order of parameters). This is Java's form of **compile-time (static) polymorphism**.

**Why introduced:** Without overloading, you'd need distinctly-named methods for every input variation (addInts, addDoubles, addThreeInts) — overloading lets the same conceptual operation (add) share one intuitive name regardless of input types, resolved by the compiler at compile time based on argument types.

**Interview definition:** "Method overloading is a compile-time polymorphism mechanism where multiple methods share the same name within a class (or class hierarchy) but differ in their parameter list, with the compiler selecting the correct method to bind at compile time based on the static types of the arguments at the call site."

## 2. Real Life Analogy

- **Restaurant:** A chef's `prepare()` skill applies to `prepare(Pizza)`, `prepare(Pasta)`, `prepare(Salad)` — same conceptual action name, but the exact steps executed depend on WHICH ingredient type is handed over, decided the moment the order ticket (call) is written.
- **Bank:** `transfer(accountNumber, amount)` vs `transfer(accountNumber, amount, currency)` — same core operation name, but the bank's system picks the exact matching procedure based on how many/what type of details are provided upfront.

### 3. What Counts as a Valid Overload

| Valid difference                               | Example                                                                                                                |
|------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| Different number of parameters                 | <code>add(int a, int b)</code> vs <code>add(int a, int b, int c)</code>                                                |
| Different parameter types                      | <code>add(int a, int b)</code> vs <code>add(double a, double b)</code>                                                 |
| Different parameter order (of different types) | <code>format(String s, int n)</code> vs <code>format(int n, String s)</code>                                           |
| INVALID (does NOT count as overloading)        | Why                                                                                                                    |
| Only the return type differs                   | Return type is not part of the method signature used for overload resolution — compile error: "method already defined" |
| Only parameter NAMES differ                    | Parameter names are irrelevant to the signature; only types/order/count matter                                         |

### 4. Internal Working — Overload Resolution Priority (Critical, Frequently Tested)

When a method call is made, the compiler picks the *most specific* matching overload using this **exact priority order**:

1. EXACT MATCH (no conversion needed at all)
2. WIDENING PRIMITIVE (e.g., `int` -> `long` -> `float` -> `double`)
3. AUTOBOXING/UNBOXING (e.g., `int` -> `Integer`)
4. VARARGS (last resort - only if nothing else matches)

```
static void show(int x) { System.out.println("int"); }
static void show(long x) { System.out.println("long"); }
static void show(Integer x) { System.out.println("Integer"); }
static void show(int... x) { System.out.println("varargs"); }
```

```
show(5); // prints "int" - EXACT match wins over widening/boxing/varargs
```

If `show(int)` didn't exist, `show(5)` would print `"long"` (widening beats autoboxing). If neither `show(int)` nor `show(long)` existed, it would print `"Integer"` (autoboxing beats varargs). Varargs is **always the last resort**.

### 5. Ambiguous Overload Errors

```
static void print(int a, long b) { }
static void print(long a, int b) { }
```

```
print(5, 5); // COMPILER ERROR: "reference to print is ambiguous"
// Both overloads require exactly ONE widening conversion -
// the compiler cannot decide which is "more specific"
```

## 6. Overloading vs Overriding — Comparison Table

| Aspect                      | Overloading                                                    | Overriding                                                                                     |
|-----------------------------|----------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| Definition                  | Same name, different parameter list, same class (or hierarchy) | Same name AND same parameter list, subclass redefines parent's method                          |
| Polymorphism type           | Compile-time (static)                                          | Runtime (dynamic)                                                                              |
| Resolved by                 | Compiler, based on argument types at compile time              | JVM, based on the object's actual runtime type                                                 |
| Return type                 | Can differ freely                                              | Must be same or a covariant subtype                                                            |
| Access modifier             | No restriction                                                 | Cannot reduce visibility (e.g., can't override <code>public</code> with <code>private</code> ) |
| <code>static</code> methods | Can be overloaded                                              | Cannot be overridden (only hidden — different mechanism entirely)                              |
| Relationship required       | None — can be in the same single class                         | Requires inheritance (subclass relationship)                                                   |

(Overriding is covered in full depth in the dedicated [\[\[Method Overriding\]\]](#) topic.)

## 7. Code Examples

```
public class OverloadDemo {
    static int add(int a, int b) {
        return a + b;
    }
    static double add(double a, double b) {
        return a + b;
    }
    static int add(int a, int b, int c) {
        return a + b + c;
    }

    public static void main(String[] args) {
        System.out.println(add(2, 3)); // calls add(int, int) -> 5
        System.out.println(add(2.5, 3.5)); // calls add(double, double) -> 6.0
        System.out.println(add(1, 2, 3)); // calls add(int, int, int) -> 6
    }
}
```

## 8. Edge Cases

| Case                                                                                                                               | Result                                                                                                                                                                          |
|------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Overloads differing only by return type                                                                                            | Compile-time error — "method already defined in class"                                                                                                                          |
| <code>null</code> passed where two overloads accept different reference types (e.g., <code>String</code> and <code>Object</code> ) | Compiler picks the <b>most specific</b> applicable type ( <code>String</code> over <code>Object</code> ) — if truly ambiguous between two unrelated types, it's a compile error |
| Widening AND autoboxing both possible for a call                                                                                   | Widening always wins over autoboxing                                                                                                                                            |
| Overload resolution when a subclass and superclass parameter types are both applicable                                             | The <b>most specific type</b> in the hierarchy is chosen                                                                                                                        |

## 9. Common Mistakes

- Trying to "overload" by return type alone — doesn't compile.
- Being surprised when `show(5)` picks `int` over `Integer` or `varargs` — forgetting the exact resolution priority (exact match → widening → autoboxing → `varargs`).
- Creating genuinely ambiguous overloads (as in Section 5) and being confused by the resulting compile error.

## 10. Best Practices

- Keep overloaded methods' behavior conceptually consistent (all `add(...)` variants should genuinely "add" — don't overload the same name for unrelated behavior, which confuses readers).
- Avoid overloading with both a specific wrapper type and a matching primitive type unless truly necessary — it increases the chance of subtle resolution surprises.
- Prefer distinct, descriptive method names over excessive overloading when the parameter differences aren't obviously related in meaning.

## 11. Frequently Asked Interview Questions

- 1 **What is method overloading?** Defining multiple methods with the same name but different parameter lists within a class, resolved by the compiler at compile time.
- 2 **Can you overload a method by changing only its return type?** No — the compiler uses the method's parameter list (not return type) for signature resolution; return-type-only differences cause a compile error.
- 3 **What is the overload resolution priority order?** Exact match, then widening primitive conversion, then autoboxing/unboxing, then `varargs` (as an absolute last resort).
- 4 **Is method overloading compile-time or runtime polymorphism?** Compile-time (static) polymorphism — the exact method to call is determined by the compiler based on argument types at the call site.
- 5 **Can static methods be overloaded?** Yes — overloading has no relationship to `static`; both static and instance methods can be freely overloaded.
- 6 **What causes an "ambiguous method call" compile error?** When two or more overloads are equally applicable via the same level of conversion (e.g., both require exactly one widening conversion) and neither is more specific than the other.

## 12. Revision Notes

### METHOD OVERLOADING — CHEAT SHEET

=====

SAME name + DIFFERENT parameter list (count/type/order) = valid overload

Return-type-only difference = COMPILE ERROR (not a valid overload)

Parameter NAME differences = irrelevant to overload resolution

RESOLUTION PRIORITY (compiler picks the FIRST that matches):

1. Exact type match
2. Widening primitive conversion
3. Autoboxing/unboxing
4. Varargs (last resort)

Compile-time (STATIC) polymorphism -> resolved by compiler, not JVM at runtime

Static methods CAN be overloaded (unlike overriding, which they cannot do)

## 13. Homework

- 1 Write 4 overloaded `show()` methods (int, long, Integer, varargs) exactly like Section 4, call `show(5)`, and predict which one runs BEFORE checking.
- 2 Deliberately create two ambiguous overloads (as in Section 5) and read the exact compiler error message produced.

---

(End of Topic 17: Method Overloading.)

---

## TOPIC 18: METHOD OVERRIDING

---

### 1. Introduction

#### 1.1 What is it?

Method overriding is when a **subclass provides its own implementation** of a method that is already defined in its **superclass**, using the exact same method signature (name + parameter list). This is Java's mechanism for **runtime (dynamic) polymorphism** — the specific implementation that actually executes is decided at runtime, based on the object's real (runtime) type, not its declared (compile-time) reference type.

#### 1.2 Why was it introduced

Without overriding, a superclass reference could never behave differently for different subclass objects — you'd need explicit type-checking (`if (obj instanceof Dog) ... else if (obj instanceof Cat) ...`) scattered everywhere. Overriding lets you write `animal.makeSound()` ONCE, and have it automatically execute the RIGHT behavior for whatever actual object (Dog, Cat, Cow) is behind that reference — the core mechanism enabling extensible, polymorphic designs.

### 1.3 Interview Definition

"Method overriding is a runtime polymorphism mechanism where a subclass redefines a method inherited from its superclass with an identical signature, such that calls made through a superclass-typed reference are dispatched, at runtime, to the actual object's most specific overriding implementation — resolved via the JVM's dynamic method dispatch (virtual method table lookup), not at compile time."

---

## 2. Real Life Analogy

| Domain     | Analogy                                                                                                                                                                                                                                                                                                                                                                                             |
|------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Bank       | A generic <code>Account.calculateInterest()</code> method is overridden differently by <code>SavingsAccount</code> (4% simple) and <code>FixedDepositAccount</code> (7% compounded) — code that just calls <code>account.calculateInterest()</code> automatically gets the correct calculation for whichever specific account type it's actually holding, without needing to check the type itself. |
| Uber       | A generic <code>Vehicle.calculateFare()</code> is overridden differently by <code>UberGo</code> , <code>UberXL</code> , and <code>UberPremier</code> — the app calls the same method name on a <code>Vehicle</code> reference, and the CORRECT fare logic runs automatically based on the actual vehicle type assigned.                                                                             |
| Restaurant | An <code>Employee.calculateSalary()</code> method is overridden by <code>Chef</code> (fixed + bonus) and <code>Waiter</code> (fixed + tips) — payroll code calling <code>employee.calculateSalary()</code> in a loop over ALL employees gets the right calculation for each, automatically.                                                                                                         |

---

### 3. Rules for Valid Overriding

| Rule                | Detail                                                                                                                                                                              |
|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Same method name    | Mandatory                                                                                                                                                                           |
| Same parameter list | Mandatory — exact match (this is what distinguishes it from overloading)                                                                                                            |
| Return type         | Must be the same, OR a <b>covariant</b> (subtype) return type (allowed since Java 5)                                                                                                |
| Access modifier     | Cannot be MORE restrictive than the parent's (can be same or wider — e.g., <code>protected</code> → <code>public</code> is fine, <code>public</code> → <code>private</code> is NOT) |
| Checked exceptions  | The overriding method cannot throw new/broader checked exceptions than the parent method declares (can throw the same, narrower, fewer, or none at all)                             |
| static methods      | <b>Cannot be overridden</b> — they can only be "hidden" (a completely different mechanism, resolved at compile time based on reference type, not runtime type)                      |
| final methods       | Cannot be overridden at all — compile-time error if attempted                                                                                                                       |
| private methods     | Cannot be overridden — they aren't even visible/inherited by subclasses in the polymorphic sense                                                                                    |
| Constructors        | Cannot be overridden (they aren't inherited at all)                                                                                                                                 |

### 4. Internal Working — Dynamic Method Dispatch (Virtual Method Table)

#### 4.1 Step-by-Step

```
class Animal {  
    void makeSound() { System.out.println("Some generic sound"); }  
}  
class Dog extends Animal {  
    @Override  
    void makeSound() { System.out.println("Bark"); }  
}  
  
Animal a = new Dog(); // reference type: Animal, ACTUAL/runtime type: Dog  
a.makeSound(); // prints "Bark" - NOT "Some generic sound"
```

- 1 At **compile time**, the compiler only checks that `Animal` has a `makeSound()` method (so the call is legal) — it does NOT decide which implementation to run.
- 2 At **runtime**, the JVM looks at the object's actual class (`Dog`), consults that class's **virtual method table (vtable)** — an internal per-class array mapping method signatures to their most-specific implementation — and invokes `Dog.makeSound()`.
- 3 This is why the bytecode instruction used for instance method calls is `invokevirtual` (see [[Java Compilation Process|Topic 3]]) — "virtual" specifically refers to this runtime-resolved dispatch behavior.



## 4.2 Diagram — Virtual Method Table Concept

```
Animal's vtable: Dog's vtable (INHERITS + OVERRIDES):
+-----+ +-----+
| makeSound() -> Animal | | makeSound() -> Dog | <- overridden entry replaces parent's
| eat() -> Animal | | eat() -> Animal | <- not overridden, inherited as-is
+-----+ +-----+

Animal a = new Dog();
a.makeSound() --> JVM checks a's ACTUAL class (Dog) --> looks up Dog's vtable --> runs Dog.makeSound()
```

## 4.3 `super.methodName()` — Calling the Parent's Version Explicitly

```
class Dog extends Animal {
@Override
void makeSound() {
super.makeSound(); // explicitly calls Animal's version first
System.out.println("Bark"); // then adds Dog-specific behavior
}
}
```

## 5. The `@Override` Annotation

```
@Override
void makeSound() { ... }
```

- **Purely a compile-time safety check** — has zero effect on runtime behavior.
- If the annotated method does NOT actually match a parent method's signature (e.g., a typo in the name, or a slightly wrong parameter type), the compiler raises an error immediately, instead of silently creating an unrelated overload — this catches a very common real bug (accidentally overloading instead of overriding due to a subtle signature mismatch).

```
class Animal {
void makeSound() { }
}
class Dog extends Animal {
@Override
void makeSond() { } // TYPO! Compile error: "method does not override a method from its superclass"
// WITHOUT @Override, this would silently compile as an unrelated NEW method!
}
```

## 6. Covariant Return Types (Java 5+)

```
class Animal {
Animal reproduce() { return new Animal(); }
}
class Dog extends Animal {
@Override
Dog reproduce() { return new Dog(); } // narrower return type (Dog, a subtype of Animal) - LEGAL
}
```

## 7. Static Method "Hiding" vs Overriding — Critical Distinction

```
class Animal {
    static void identify() { System.out.println("Animal"); }
}
class Dog extends Animal {
    static void identify() { System.out.println("Dog"); } // this HIDES, does NOT override
}

Animal a = new Dog();
a.identify(); // prints "Animal" - resolved by REFERENCE TYPE (compile-time), NOT runtime type!
```

**Interview gold:** For instance methods, the runtime object's type decides which version runs. For `static` methods, the **compile-time reference type** decides — because static methods are resolved via `invokestatic` at compile time, with no dynamic dispatch at all.

## 8. Code Examples

```
public class OverrideDemo {
    public static void main(String[] args) {
        Animal[] animals = { new Dog(), new Cat(), new Animal() };
        for (Animal a : animals) {
            a.makeSound(); // polymorphic call - correct override runs for each actual type
        }
    }
}

class Animal {
    void makeSound() { System.out.println("Some generic animal sound"); }
}
class Dog extends Animal {
    @Override
    void makeSound() { System.out.println("Bark"); }
}
class Cat extends Animal {
    @Override
    void makeSound() { System.out.println("Meow"); }
}
```

### Output:

```
Bark
Meow
Some generic animal sound
```

## 9. Dry Run

```
Animal a = new Dog();
a.makeSound();
```

| Step         | What Happens                                                                                                                                                              |
|--------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Compile time | Compiler checks <code>Animal</code> class declares <code>makeSound()</code> — call is syntactically legal. Bytecode emits <code>invokevirtual Animal.makeSound()</code> . |

| Step      | What Happens                                                                                                             |
|-----------|--------------------------------------------------------------------------------------------------------------------------|
| Runtime   | JVM inspects a's <b>actual object type</b> on the Heap (Dog), NOT the declared reference type (Animal).                  |
| Dispatch  | JVM looks up Dog's vtable entry for makeSound() (which points to Dog's overridden version, since it was replaced there). |
| Execution | Dog.makeSound() executes, printing "Bark".                                                                               |

## 10. Edge Cases

| Case                                                                        | Result                                                                                                                                                                                                              |
|-----------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Overriding method throws a broader checked exception than the parent's      | Compile-time error                                                                                                                                                                                                  |
| Overriding method reduces visibility (parent public, child tries protected) | Compile-time error                                                                                                                                                                                                  |
| Attempting to override a final method                                       | Compile-time error                                                                                                                                                                                                  |
| Attempting to override a private method                                     | Doesn't actually override — silently creates an unrelated new method in the subclass (since private methods aren't inherited/visible)                                                                               |
| Calling an overridden method from within the PARENT's constructor           | The SUBCLASS's overridden version runs (since dispatch is always based on actual runtime type) — dangerous if that override relies on subclass fields not yet initialized at that point (see [[Constructor Chaining |
| Overriding with a covariant return type NOT actually a subtype              | Compile-time error                                                                                                                                                                                                  |

## 11. Common Mistakes

- Forgetting `@Override` and accidentally creating an unrelated overload due to a typo or wrong parameter type — silently breaks polymorphism without any warning.
- Confusing static method "hiding" with true overriding — assuming static methods are polymorphic when they are resolved at compile time by reference type.
- Calling an overridable method from a constructor, unaware the subclass's override may run before subclass fields are initialized.
- Trying to reduce an overriding method's access modifier (a common compile-error trigger for beginners refactoring inherited methods).

## 12. Best Practices

- **Always** use `@Override` when overriding — it's free compile-time safety against silent signature mismatches.
- Avoid calling overridable instance methods from constructors; if unavoidable, mark the method `final` so subclasses cannot alter its behavior mid-construction.

- Keep overridden method behavior consistent with the parent's documented contract (Liskov Substitution Principle) — an override shouldn't violate the expectations set by the base class's method.
- 

## 13. Production Usage

- The **entire Spring MVC / Servlet lifecycle** relies on overriding (`doGet()`, `doPost()` in `HttpServlet` subclasses; controller methods implementing interface contracts).
  - **JPA/Hibernate entity** `equals()`/`hashCode()`/`toString()` **overrides** are fundamental to correct behavior in collections and persistence contexts.
  - **Strategy design pattern** and most extensible framework "plugin" architectures are built entirely on overriding an interface/abstract method.
- 

## 14. Frequently Asked Interview Questions

- 1 **What is method overriding?** A subclass providing its own implementation of a method already defined in its superclass, with an identical signature, resolved at runtime based on the object's actual type.
  - 2 **What is dynamic method dispatch?** The JVM mechanism (via `invokevirtual` + vtable lookup) that determines, at runtime, which overridden method implementation to actually execute, based on the object's real class.
  - 3 **Can you override a static method?** No — static methods can only be "hidden," resolved at compile time by the reference's declared type, not the object's runtime type.
  - 4 **Can you override a private or final method?** No to both — `private` methods aren't inherited/visible for overriding purposes; `final` methods explicitly forbid overriding.
  - 5 **What does `@Override` actually do at runtime?** Nothing — it's a compile-time-only annotation that makes the compiler verify the method genuinely matches a parent method's signature, catching accidental overload-instead-of-override typos.
  - 6 **Can an overriding method's return type differ from the parent's?** Only via a covariant return type (a subtype of the original return type) — otherwise it must match exactly.
  - 7 **Can an overriding method widen an access modifier? Narrow it?** It can widen (`protected` → `public`) but CANNOT narrow (`public` → `protected`) — a compile-time error.
  - 8 **What happens if you call an overridable method from a constructor?** The subclass's OVERRIDDEN version runs (dispatch is always based on runtime type), potentially operating on subclass fields that haven't been initialized yet — a well-known anti-pattern.
  - 9 **How do checked exceptions interact with overriding?** An overriding method cannot throw new or broader checked exceptions than the parent method declares — it can throw the same, fewer, narrower, or none.
-

## 15. Revision Notes

### METHOD OVERRIDING – CHEAT SHEET

=====

SAME name + SAME parameter list, subclass redefines parent's method

RUNTIME (dynamic) polymorphism -> resolved by JVM via `invokevirtual` + vtable lookup

#### RULES:

return type -> same OR covariant (subtype) - Java 5+

access modifier -> same or WIDER, never narrower

checked exceptions -> same, fewer, or narrower only - never new/broader

static / final / private methods -> CANNOT be overridden

static -> can only be HIDDEN (resolved by reference type, compile-time!)

final -> forbidden entirely

private -> not inherited/visible, "override" attempt = new unrelated method

@Override annotation -> compile-time-only safety net against signature typos

`super.method()` -> explicitly calls the PARENT's version

GOTCHA: static method "override" resolved by REFERENCE type, NOT object's actual type

GOTCHA: calling overridable method from constructor -> subclass override runs

BEFORE subclass fields are initialized

## 16. Homework

- 1 Build the `Animal/Dog/Cat` hierarchy from Section 8, add a `static identify()` method to prove static "overriding" is resolved by reference type, not runtime type.
- 2 Deliberately introduce a typo in an `@Override`-annotated method and observe the exact compile error, then remove `@Override` and observe it silently compiles as an unrelated overload instead.
- 3 Write a parent class that calls an overridable method from its constructor, override it in a subclass that reads a subclass field, and observe the resulting bug (field not yet initialized).

(End of Topic 18: Method Overriding.)

## TOPIC 19: INHERITANCE

### 1. Introduction

#### 1.1 What is it?

Inheritance is an OOP mechanism where a new class (**subclass/child**) acquires the fields and methods of an existing class (**superclass/parent**), modeling an **"IS-A" relationship** (`Dog IS-A Animal`), enabling code reuse and establishing a type hierarchy.

## 1.2 Why was it introduced

Without inheritance, common behavior shared by related types (Dog, Cat, Cow all "eat" and "sleep") would need to be duplicated in every single class — a maintenance nightmare (fixing a bug means fixing it in N places). Inheritance lets shared structure/behavior be defined **once**, in a common ancestor, and automatically reused by every descendant.

## 1.3 Real-world problem it solves

- Eliminates code duplication across conceptually related classes.
- Enables **polymorphism** (see [[Method Overriding|Topic 18]]) — code can operate on a superclass reference and correctly handle any subclass object.
- Models natural real-world "is-a" taxonomies directly in code structure (an Employee hierarchy, a Shape hierarchy).

## 1.4 Interview Definition

"Inheritance is the OOP mechanism by which a subclass acquires the accessible fields and methods of a superclass via the `extends` keyword, establishing an IS-A relationship and enabling both code reuse and runtime polymorphism through method overriding."

## 2. Real Life Analogy

| Domain     | Analogy                                                                                                                                                                                                              |
|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| College    | GraduateStudent and UndergraduateStudent both inherit common fields/behavior from Student (name, rollNumber, attendClass()), while each adds its own specifics (thesis vs. no thesis).                               |
| Bank       | SavingsAccount and CurrentAccount both inherit from Account (accountNumber, balance, deposit(), withdraw()), each overriding calculateInterest() differently.                                                        |
| Restaurant | Chef and Waiter both inherit from Employee (name, salary, clockIn()), each adding role-specific behavior (cookDish() vs takeOrder()).                                                                                |
| Amazon     | Book, Electronics, and Clothing product types all inherit shared fields (price, seller, rating) from a base Product class, while adding category-specific fields (author for books, warrantyPeriod for electronics). |

## 3. Types of Inheritance in Java

1. SINGLE INHERITANCE A extends B (fully supported)
2. MULTILEVEL INHERITANCE A extends B, B extends C (fully supported)
3. HIERARCHICAL INHERITANCE B extends A, C extends A (fully supported - multiple children, one parent)
4. MULTIPLE INHERITANCE (classes) A extends B, C (NOT SUPPORTED for classes!)
5. MULTIPLE INHERITANCE (interfaces) class A implements X, Y (SUPPORTED via interfaces)

## Why Java does NOT support multiple class inheritance — The Diamond Problem

```
Animal
/ \
Dog Bird
\ /
(???) - if a class could extend BOTH Dog and Bird,
and both defined a conflicting move() method,
WHICH implementation would it inherit? AMBIGUOUS!
```

Java's designers deliberately avoided this ambiguity by allowing a class to extend only **ONE** other class. Java achieves the *benefits* of multiple inheritance safely through **interfaces** (implements multiple interfaces), which historically had no conflicting field state, and even with Java 8's default methods, the compiler **FORCES** you to explicitly resolve any conflicting default method (a compile-time error, not silent ambiguity) — solving the Diamond Problem with a deliberate, explicit choice rather than an implicit, ambiguous one.

## 4. Internal Working — What a Subclass Object Actually Contains

```
class Animal {
    String name;
    void eat() { System.out.println(name + " is eating"); }
}
class Dog extends Animal {
    String breed;
    void bark() { System.out.println(name + " says Woof!"); }
}

Dog d = new Dog();
```

### Memory layout of the Dog object on the Heap:

```
+-----+
| Dog object |
| --- inherited from Animal --- |
| name (from Animal) |
| --- Dog's own fields --- |
| breed (declared in Dog) |
+-----+
```

A Dog object contains BOTH Animal's fields AND Dog's own fields, in ONE single, unified object on the Heap - there is no separate "Animal part" floating elsewhere; it's all one object.

- Method bytecode is NOT duplicated — Dog doesn't get its own copy of `eat()`; it simply **INHERITS** the reference to `Animal.eat()` in its effective method table (see [\[\[Method Overriding|Topic 18\]\]](#)'s vtable diagram) unless it overrides it.
- Every class in Java implicitly extends `java.lang.Object` if no other superclass is specified — meaning EVERY class has (and inherits) methods like `toString()`, `equals()`, `hashCode()` (full detail in the [\[\[Object Class\]\]](#) topic).

## 5. Syntax

```
class Animal { // superclass / parent / base class
protected String name; // 'protected' - accessible to subclasses (see Access Modifiers topic)

public Animal(String name) {
this.name = name;
}

public void eat() {
System.out.println(name + " is eating.");
}
}

class Dog extends Animal { // 'extends' - establishes inheritance (subclass / child / derived class)
private String breed;

public Dog(String name, String breed) {
super(name); // MUST call parent constructor first (see Constructor Chaining, Topic 16)
this.breed = breed;
}

public void bark() {
System.out.println(name + " (" + breed + ") says Woof!");
}
}
```

---

## 6. `super` Keyword — Three Uses

```
class Dog extends Animal {
private String breed;

Dog(String name, String breed) {
super(name); // (1) super(...) - call parent's CONSTRUCTOR
this.breed = breed;
}

void describe() {
super.eat(); // (2) super.method() - call parent's METHOD explicitly
System.out.println(super.name); // (3) super.field - access parent's FIELD explicitly (rarely needed)
}
}
```

---



## 7. Code Examples

### 7.1 Hierarchical Inheritance + Polymorphism

```
public class InheritanceDemo {
    public static void main(String[] args) {
        Animal[] animals = { new Dog("Rex", "Labrador"), new Cat("Whiskers") };
        for (Animal a : animals) {
            a.eat(); // inherited method, same for all
        }
        ((Dog) animals[0]).bark(); // Dog-specific method, requires downcast
    }
}

class Animal {
    protected String name;
    Animal(String name) { this.name = name; }
    void eat() { System.out.println(name + " is eating."); }
}

class Dog extends Animal {
    private String breed;
    Dog(String name, String breed) {
        super(name);
        this.breed = breed;
    }
    void bark() { System.out.println(name + " says Woof!"); }
}

class Cat extends Animal {
    Cat(String name) { super(name); }
}
```

### 7.2 Multiple Interface "Inheritance" (Diamond Problem Solved Explicitly)

```
interface Flyer {
    default void move() { System.out.println("Flying"); }
}

interface Swimmer {
    default void move() { System.out.println("Swimming"); }
}

class Duck implements Flyer, Swimmer {
    @Override
    public void move() { // MUST override - compiler forces explicit resolution
        Flyer.super.move(); // explicitly choose Flyer's version
        Swimmer.super.move(); // and/or Swimmer's version
        System.out.println("Duck decides how to move");
    }
}
```

## 8. Edge Cases

| Case                                                                                               | Result                                                                                                                                          |
|----------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| A class extends two classes ( <code>class C extends A, B</code> )                                  | <b>Compile-time error</b> — Java does not support multiple class inheritance                                                                    |
| Two implemented interfaces have a conflicting default method with the same signature               | <b>Compile-time error</b> unless the implementing class explicitly overrides and resolves it (as in Section 7.2)                                |
| Subclass constructor doesn't call <code>super ( args )</code> and parent has no no-arg constructor | Compile-time error (see [[Constructor Chaining                                                                                                  |
| Accessing a <code>private</code> field of the parent from the subclass                             | Compile-time error — <code>private</code> members are NOT inherited/visible, even though they technically exist in memory as part of the object |
| A <code>final</code> class                                                                         | Cannot be extended at all — compile-time error if attempted                                                                                     |

## 9. Common Mistakes

- Overusing inheritance for code reuse where the relationship isn't a genuine "IS-A" (e.g., making `Stack` extends `Vector` historically in the JDK — now considered a design mistake) — should often be **composition** ("HAS-A") instead.
- Forgetting `private` fields aren't inherited/accessible directly by subclasses — use `protected` or provide protected/public accessor methods if subclass access is genuinely needed.
- Assuming Java supports multiple class inheritance — it does not; only single class inheritance plus multiple interface implementation.
- Deep inheritance hierarchies (5+ levels) that become fragile and hard to reason about — a well-known real-world anti-pattern ("fragile base class problem").

## 10. Best Practices

- **Favor composition over inheritance** when the relationship is "HAS-A" rather than truly "IS-A" (e.g., a `Car` HAS-A `Engine`, it is NOT an `Engine`) — reduces fragile coupling.
- Keep inheritance hierarchies shallow (2-3 levels) — deep hierarchies are hard to understand and maintain.
- Mark classes `final` when they aren't designed to be extended, preventing unintended/fragile subclassing.
- Use `protected` (not `public`) for fields/methods that subclasses genuinely need but external code shouldn't touch directly.

## 11. Production Usage

- Framework class hierarchies: `HttpServlet` extends `GenericServlet`, exception hierarchies (`IOException` extends `Exception` extends `Throwable`), Spring's `AbstractController` patterns.

- ORM entity hierarchies: JPA's `@Inheritance` mapping strategies (`SINGLE_TABLE`, `JOINED`, `TABLE_PER_CLASS`) directly model Java class inheritance onto relational database tables.
  - Exception class hierarchies (custom exceptions extending `RuntimeException/Exception`) are a near-universal production pattern (see the Exception Handling topic).
- 

## 12. Frequently Asked Interview Questions

- 1 **What is inheritance?** An OOP mechanism allowing a subclass to acquire the fields and methods of a superclass, modeling an IS-A relationship.
  - 2 **Does Java support multiple inheritance of classes?** No — a class can `extends` only one other class, to avoid the Diamond Problem's ambiguity; multiple inheritance of *type* (behavior contracts) is achieved via implementing multiple interfaces instead.
  - 3 **How does Java resolve the Diamond Problem for interface default methods?** By forcing a compile-time error if two implemented interfaces provide conflicting default methods with the same signature, requiring the implementing class to explicitly override and resolve the conflict (optionally calling `InterfaceName.super.method()` to pick a specific one).
  - 4 **What are the three uses of the `super` keyword?** Calling the parent's constructor (`super(args)`), calling the parent's overridden method explicitly (`super.method()`), and accessing the parent's field explicitly (`super.field`).
  - 5 **Does every class in Java have a superclass, even if none is declared?** Yes — every class implicitly extends `java.lang.Object` if no other superclass is specified.
  - 6 **Can a `private` member be inherited?** It exists in memory as part of the subclass object, but it is NOT accessible/visible to the subclass directly — inheritance and accessibility are separate concepts.
  - 7 **Why is "favor composition over inheritance" considered a best practice?** Because inheritance creates tight coupling between subclass and superclass implementation details (the "fragile base class problem") — composition (HAS-A, via object references) is more flexible and doesn't break when the "contained" class's internals change.
  - 8 **Can a `final` class be extended?** No — attempting to extend a `final` class is a compile-time error.
-

## 13. Revision Notes

### INHERITANCE – CHEAT SHEET

=====

`extends` -> class inheritance (IS-A), single inheritance ONLY for classes  
`implements` -> interface inheritance, MULTIPLE allowed

SUPPORTED: single, multilevel, hierarchical, multiple-via-interfaces  
NOT SUPPORTED: multiple CLASS inheritance (Diamond Problem avoidance)

DIAMOND PROBLEM (interfaces): conflicting default methods -> COMPILE ERROR,  
must explicitly override + optionally use `Interface.super.method()`

super KEYWORD - 3 uses:

`super(args)` -> parent constructor call  
`super.method()` -> parent method call  
`super.field` -> parent field access

EVERY class implicitly extends `java.lang.Object` if nothing else specified  
private members: NOT inherited/visible to subclass (still exist in memory, just inaccessible)  
final class -> cannot be extended at all

BEST PRACTICE: favor COMPOSITION (HAS-A) over inheritance (IS-A) when unsure

---

## 14. Homework

- 1 Build a Shape -> Circle/Rectangle/Triangle hierarchy with a shared `area()` method overridden per shape, and iterate over a `Shape[]` array calling `area()` polymorphically.
  - 2 Create two interfaces with a conflicting default method, implement both in one class, observe the compile error, then fix it using `Interface.super.method()`.
  - 3 Attempt to extend a class marked `final` and record the exact compiler error.
-

## 15. Mini Project — Employee Payroll Hierarchy

```
package com.jobsradar.payroll;

abstract class Employee { // (abstract classes covered fully in next topic)
    protected String name;
    protected double baseSalary;

    Employee(String name, double baseSalary) {
        this.name = name;
        this.baseSalary = baseSalary;
    }

    abstract double calculatePay();

    void printPaySlip() {
        System.out.printf("%s's pay: %.2f%n", name, calculatePay());
    }
}

class Chef extends Employee {
    private double bonus;
    Chef(String name, double baseSalary, double bonus) {
        super(name, baseSalary);
        this.bonus = bonus;
    }
    @Override
    double calculatePay() { return baseSalary + bonus; }
}

class Waiter extends Employee {
    private double tips;
    Waiter(String name, double baseSalary, double tips) {
        super(name, baseSalary);
        this.tips = tips;
    }
    @Override
    double calculatePay() { return baseSalary + tips; }
}

public class PayrollSystem {
    public static void main(String[] args) {
        Employee[] staff = {
            new Chef("Ramesh", 30000, 5000),
            new Waiter("Suresh", 20000, 3000)
        };
        for (Employee e : staff) {
            e.printPaySlip(); // polymorphic - correct calculatePay() runs for each
        }
    }
}
```

## 16. Final Summary

- Inheritance establishes an **IS-A** relationship via `extends`, letting a subclass reuse a superclass's fields/methods and enabling runtime polymorphism.

- Java supports single, multilevel, and hierarchical class inheritance, but **NOT multiple class inheritance** — solved instead via multiple interface implementation, with the compiler forcing explicit resolution of any Diamond-Problem-style default method conflicts.
  - A subclass object is ONE unified Heap object containing both inherited and its own fields — method bytecode isn't duplicated, only referenced via the effective vtable.
  - `super` has three distinct uses: constructor chaining, explicit parent method calls, and explicit parent field access.
  - Every class implicitly extends `java.lang.Object`.
  - **Favor composition over inheritance** when the relationship isn't a genuine IS-A — this is one of the most repeated pieces of senior-engineer design guidance in the industry.
- 

(End of Topic 19: Inheritance.)

---

## TOPIC 20: POLYMORPHISM

---

### 1. Introduction

**What is it?** Polymorphism ("many forms") is the OOP principle that lets a single interface, method name, or reference type represent multiple underlying implementations/behaviors. Java achieves this through two distinct mechanisms, both already covered mechanically in earlier topics:

- **Compile-time (static) polymorphism** → [[Method Overloading|Topic 17]] — resolved by the compiler based on argument types.
- **Runtime (dynamic) polymorphism** → [[Method Overriding|Topic 18]] — resolved by the JVM based on the object's actual type.

This topic ties both together as ONE unifying concept and covers the additional polymorphic patterns (upcasting, polymorphic collections, the `instanceof` pattern) that weren't the primary focus of those two topics.

**Interview definition:** "Polymorphism is the ability of a reference type, method, or operator to take on multiple forms/behaviors depending on the actual object or argument types involved, realized in Java through compile-time overload resolution and runtime dynamic method dispatch."

### 2. Real Life Analogy

- **Amazon:** A single "Add to Cart" button (one interface) behaves differently depending on the actual product type behind it — a book adds a fixed unit, a subscription starts a recurring billing cycle — same button, different underlying behavior (runtime polymorphism).
- **Uber:** The single `requestRide()` action produces different underlying dispatch logic depending on whether the actual vehicle type is `UberGo`, `UberXL`, or `UberPremier` — the caller doesn't need to know which.

### 3. The Two Types — Side-by-Side Comparison

| Aspect                | Compile-time (Overloading)                                   | Runtime (Overriding)                                   |
|-----------------------|--------------------------------------------------------------|--------------------------------------------------------|
| Resolved by           | Compiler, at compile time                                    | JVM, at runtime                                        |
| Based on              | Static (declared) argument types                             | Actual (runtime) object type                           |
| Requires inheritance? | No                                                           | Yes (subclass relationship)                            |
| Bytecode instruction  | Resolved directly to a specific method reference             | <code>invokevirtual</code> + vtable lookup             |
| Also called           | Static binding, early binding                                | Dynamic binding, late binding                          |
| Example               | <code>add(int,int)</code> vs <code>add(double,double)</code> | <code>animal.makeSound()</code> dispatching to Dog/Cat |

(Full mechanics of each are in [\[\[Method Overloading|Topic 17\]\]](#) and [\[\[Method Overriding|Topic 18\]\]](#) — this table exists purely to connect the two under the single "polymorphism" umbrella, a very common interview framing.)

### 4. Polymorphism via Upcasting — The Core Pattern

```
Animal a = new Dog(); // UPCASTING - Dog treated as its superclass type Animal
a.makeSound(); // "Bark" - runtime dispatch, NOT compile-time reference type
```

This single pattern — declaring a variable of a **general (super)type** but assigning a **specific (sub)type** object — is the foundation of virtually all polymorphic Java code: method parameters typed as an interface/abstract superclass, collections of a common supertype, and framework extension points.

### 5. Polymorphism with Collections — The Most Common Real Pattern

```
List<Shape> shapes = new ArrayList<>();
shapes.add(new Circle(5));
shapes.add(new Rectangle(4, 6));
shapes.add(new Triangle(3, 8));

double totalArea = 0;
for (Shape s : shapes) {
    totalArea += s.area(); // each call polymorphically dispatches to the CORRECT area() override
}
```

This is exactly how real frameworks work: a `List<PaymentMethod>` might hold `CreditCard`, `UPI`, and `wallet` objects, all processed identically via `paymentMethod.pay(amount)`, each executing its own actual logic.

### 6. Polymorphism and Interfaces (The Preferred Modern Style)

```
interface Shape {
    double area();
}

class Circle implements Shape {
    private double radius;
    Circle(double radius) { this.radius = radius; }
    public double area() { return Math.PI * radius * radius; }
}

class Rectangle implements Shape {
    private double w, h;
    Rectangle(double w, double h) { this.w = w; this.h = h; }
    public double area() { return w * h; }
}
```

**Best practice:** Modern Java code typically favors **programming to an interface** (`Shape shape = new Circle(5);`) over a concrete or abstract superclass reference, for maximum flexibility (see the upcoming [\[\[Interfaces\]\]](#) and [\[\[Abstract Classes\]\]](#) topics).

## 7. Edge Cases

| Case                                                                                           | Result                                                                                                                      |
|------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| Calling a method that exists <b>ONLY</b> on the subclass, through a superclass-typed reference | Compile-time error — you must downcast first ( <code>((Dog) a).bark()</code> ), even though the actual object supports it   |
| <code>instanceof</code> check before downcasting                                               | Prevents <code>ClassCastException</code> — the standard safe pattern (see <a href="#">[[Type Casting]]</a> )                |
| Overloaded method call with an <code>null</code> literal argument matching multiple overloads  | Compiler picks the <b>MOST SPECIFIC</b> applicable reference type; genuinely ambiguous cases are a compile error            |
| Static method called through a polymorphic (superclass-typed) reference                        | Resolved by the <b>REFERENCE's</b> declared type, <b>NOT</b> the actual object (see <a href="#">[[Method Overriding]]</a> ) |

## 8. Common Mistakes

- Believing **ALL** method calls in Java are polymorphic — only **instance methods** are dynamically dispatched; `static`, `private`, and `final` methods, plus all fields, are resolved statically (by reference type), **NOT** polymorphically.
- Forgetting that fields (unlike methods) are **NEVER** polymorphic in Java — accessing a field through a superclass reference always uses the superclass's field, even if the subclass declares a same-named field (a lesser-known but real gotcha called "field hiding").

```
class Animal { String category = "Generic"; }
class Dog extends Animal { String category = "Canine"; }

Animal a = new Dog();
System.out.println(a.category); // prints "Generic" - FIELDS are NOT polymorphic!
System.out.println(((Dog) a).category); // prints "Canine" - only via explicit downcast
```

## 9. Best Practices

- Program to an interface/abstract type wherever practical (`List<Shape>`, not `ArrayList<Circle>`), to maximize the benefit of polymorphism.
- Never rely on field access through a polymorphic reference expecting subclass behavior — always use methods (getters) for polymorphic field-like access, since fields don't participate in dynamic dispatch.
- Use `instanceof` pattern matching (Java 16+) before any downcast needed to reach subclass-specific behavior.

## 10. Production Usage

- Nearly every extensible framework API is built on polymorphism: Spring's `List<PaymentStrategy>` auto-wiring, JDBC's `Driver` interface implemented differently per database vendor, the Strategy/Template Method/Visitor design patterns.



- REST controllers processing a `List<Event>` where each concrete `Event` subtype (`OrderCreated`, `OrderCancelled`) handles itself polymorphically via an overridden `process()` method — a common event-driven microservices pattern.

## 11. Frequently Asked Interview Questions

- 1 **What is polymorphism?** The ability of a method call or reference to behave differently depending on the actual object/argument types involved — realized via overloading (compile-time) and overriding (runtime).
- 2 **What are the two types of polymorphism in Java?** Compile-time (static, via overloading) and runtime (dynamic, via overriding).
- 3 **Are fields polymorphic in Java?** No — fields are resolved by the reference's declared (compile-time) type, never dynamically dispatched, even if a subclass declares a same-named field ("field hiding").
- 4 **Are static methods polymorphic?** No — static methods are resolved at compile time based on the reference's declared type, not the object's actual runtime type.
- 5 **What is upcasting, and why is it central to polymorphism?** Treating a subclass reference as its superclass type — it's the foundational pattern enabling one method/collection to operate uniformly across many different concrete subclass behaviors.
- 6 **Why is "program to an interface, not an implementation" repeated so often in interviews?** Because it maximizes polymorphic flexibility — code depending on an interface/abstract type can work with ANY current or future implementation without modification, a core tenet of extensible, maintainable design (directly related to the Open/Closed Principle in SOLID).

## 12. Revision Notes

```
POLYMORPHISM — CHEAT SHEET
=====
"Many forms" - ONE interface/method name, MULTIPLE behaviors

TWO TYPES:
Compile-time (overloading) -> resolved by COMPILER, argument types (Topic 17)
Runtime (overriding) -> resolved by JVM, object's actual type (Topic 18)

POLYMORPHIC: instance methods (via invokevirtual + vtable)
NOT POLYMORPHIC: fields, static methods, private methods, final methods
-> all resolved by REFERENCE's declared (compile-time) type

CORE PATTERN: Animal a = new Dog(); a.makeSound(); -> runs Dog's version
BEST PRACTICE: program to an interface/abstract type, not a concrete class
```

## 13. Homework

- 1 Write the field-hiding example from Section 8 yourself, predicting the output BEFORE running it.
- 2 Build a `PaymentMethod` interface with `CreditCard`, `UPI`, `Wallet` implementations, and process a `List<PaymentMethod>` polymorphically in a single loop.

## TOPIC 21: ABSTRACTION

---

### 1. Introduction

**What is it?** Abstraction is the OOP principle of exposing only **essential, relevant behavior** to the user of a class/interface, while hiding the **internal implementation details** of how that behavior is actually carried out.

**Why introduced:** Complex systems become unmanageable if every consumer of a class needs to understand its full internal workings. Abstraction lets you interact with a `List` by calling `.add()`, `.get()`, `.remove()` without ever needing to know whether it's internally backed by an array, a linked structure, or something else — the "how" is hidden behind the "what."

**Interview definition:** "Abstraction is the process of exposing only essential operations and characteristics of an object through a well-defined contract (interface or abstract class), while concealing the underlying implementation details, achieved in Java primarily via `abstract` classes and `interface` declarations."

### 2. Real Life Analogy

- **Bank:** When you use an ATM (`withdraw(amount)`), you don't see (or need to see) the internal ledger reconciliation, fraud-check algorithms, or physical cash-dispensing mechanics — you interact with a simple, abstracted contract: insert card, enter amount, receive cash.
- **Uber:** The rider app exposes `requestRide()` — the rider has zero visibility into the actual driver-matching algorithm, surge-pricing computation, or routing engine running behind that single action.
- **Restaurant:** A customer orders "Pasta" from the menu (the abstraction) without knowing the chef's exact recipe, ingredient sourcing, or cooking technique (the hidden implementation).

### 3. Abstraction vs Encapsulation — A Critical Distinction (Frequently Confused in Interviews)

| Aspect             | Abstraction                                                                                                    | Encapsulation                                                                                                                                              |
|--------------------|----------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Focus              | <b>WHAT</b> a class does (hiding implementation complexity)                                                    | <b>HOW</b> data is protected (hiding internal state/data)                                                                                                  |
| Achieved via       | <code>abstract</code> classes, <code>interfaces</code>                                                         | <code>private</code> fields + public getters/setters                                                                                                       |
| Level              | Design-level (contract vs implementation)                                                                      | Code-level (data access control)                                                                                                                           |
| Real-world framing | "Hiding the HOW, showing only the WHAT"                                                                        | "Hiding the DATA, controlling access to it"                                                                                                                |
| Example            | <code>List</code> interface hides whether it's an <code>ArrayList</code> or <code>LinkedList</code> underneath | A <code>BankAccount</code> 's <code>balance</code> field is <code>private</code> , only modifiable via validated <code>deposit()/withdraw()</code> methods |

**Interview one-liner to remember:** *Abstraction hides complexity/implementation; Encapsulation hides data.*  
They're complementary, not the same thing, and often used together in the same class.

## 4. How Java Achieves Abstraction

TWO MECHANISMS (each has its own full dedicated topic next):  
1. Abstract classes -> partial abstraction (can mix abstract + concrete methods)  
2. Interfaces -> traditionally full abstraction (Java 8+ allows default/static methods too)

```
abstract class PaymentProcessor {  
    abstract void processPayment(double amount); // WHAT to do - no implementation here  
  
    void logTransaction(double amount) { // concrete, shared implementation  
        System.out.println("Logging transaction: " + amount);  
    }  
}  
  
class CreditCardProcessor extends PaymentProcessor {  
    @Override  
    void processPayment(double amount) { // HOW it's actually done - hidden from callers  
        System.out.println("Charging credit card: " + amount);  
    }  
}
```

A caller using `PaymentProcessor p = new CreditCardProcessor(); p.processPayment(100);` never needs to know HOW the charge is processed internally — only that calling `processPayment()` achieves the intended result.

## 5. Levels of Abstraction

0% abstraction -> concrete class, all methods fully implemented  
Partial abstraction -> abstract class, MIX of abstract + implemented methods  
100% abstraction (traditional) -> interface, ALL methods abstract (pre-Java 8)

Since Java 8, interfaces can include default and static methods with actual bodies, blurring this once-strict line — but the core PURPOSE (defining a contract, hiding implementation from the caller) remains the defining characteristic of abstraction either way.

## 6. Code Example — Abstraction in a Real-World-Style Design

```
interface NotificationService {
    void send(String recipient, String message); // the CONTRACT - the "what"
}

class EmailNotificationService implements NotificationService {
    @Override
    public void send(String recipient, String message) {
        System.out.println("Sending EMAIL to " + recipient + ": " + message);
        // actual SMTP client logic hidden here
    }
}

class SmsNotificationService implements NotificationService {
    @Override
    public void send(String recipient, String message) {
        System.out.println("Sending SMS to " + recipient + ": " + message);
        // actual SMS gateway logic hidden here
    }
}

public class AbstractionDemo {
    public static void main(String[] args) {
        NotificationService service = new EmailNotificationService(); // caller only knows the CONTRACT
        service.send("user@example.com", "Your order has shipped!");
    }
}
```

## 7. Edge Cases

| Case                                                                            | Result                                                                                                                  |
|---------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| Instantiating an abstract class directly (new <code>PaymentProcessor()</code> ) | Compile-time error — abstract classes cannot be instantiated                                                            |
| A concrete subclass fails to implement ALL inherited abstract methods           | Compile-time error — the subclass itself must also be declared <code>abstract</code> if it doesn't implement everything |
| An interface method with no <code>default/static</code> modifier                | Implicitly <code>public abstract</code> — must be implemented by any concrete implementing class                        |

## 8. Common Mistakes

- Confusing abstraction with encapsulation in interviews — they solve related but distinct problems (implementation-hiding vs data-hiding).
- Over-abstracting simple programs (creating interfaces/abstract classes for things that will only ever have one implementation) — unnecessary indirection that hurts readability without real benefit ("speculative generality" anti-pattern).
- Exposing implementation details through method names/return types that leak internal structure (e.g., returning a concrete `ArrayList` instead of the `List` interface from a public API method).

## 9. Best Practices

- Design public APIs around interfaces/abstract contracts, not concrete implementation classes — enables swapping implementations later without breaking callers.
- Don't introduce abstraction prematurely for code that has (and is only ever expected to have) one implementation — add it when a genuine second implementation or clear extension point actually emerges.
- Keep abstractions focused on a single clear responsibility (Interface Segregation Principle, covered in the System Design topic).

## 10. Production Usage

- Nearly every enterprise Java layer is built on abstraction: the JDBC `Driver/Connection` interfaces abstract away vendor-specific database implementations; Spring's `PlatformTransactionManager` interface abstracts different transaction strategies (JDBC, JPA, JTA); the SLF4J `Logger` interface abstracts the actual logging framework (Logback, Log4j2) underneath.

## 11. Frequently Asked Interview Questions

- 1 **What is abstraction?** Exposing only essential behavior through a well-defined contract while hiding internal implementation details.
- 2 **What's the difference between abstraction and encapsulation?** Abstraction hides implementation complexity (the HOW); encapsulation hides internal data/state (protecting it via private fields + controlled access) — they're complementary, not identical.
- 3 **Can you instantiate an abstract class?** No — directly instantiating an `abstract` class is a compile-time error; you must instantiate a concrete subclass instead.
- 4 **What are the two ways Java achieves abstraction?** Abstract classes (partial abstraction, can mix abstract and concrete methods) and interfaces (traditionally full abstraction, though Java 8+ permits `default/static` method bodies too).
- 5 **Give a real production example of abstraction.** The JDBC API — application code calls standard `Connection/Statement` methods without knowing or caring whether the underlying database is PostgreSQL, MySQL, or Oracle.

## 12. Revision Notes

### ABSTRACTION – CHEAT SHEET

=====

Hides the HOW (implementation), exposes only the WHAT (contract/behavior)

vs ENCAPSULATION: encapsulation hides DATA (private fields), abstraction hides IMPLEMENTATION COMPLEXITY (via abstract class/interface)

#### ACHIEVED VIA:

abstract class -> PARTIAL abstraction (mix of abstract + concrete methods)

interface -> traditionally FULL abstraction (Java 8+ allows default/static bodies)

Cannot instantiate an abstract class or interface directly.

Concrete subclass MUST implement all inherited abstract methods, or itself be declared abstract.

## 13. Homework

- 1 Design a `Shape` abstraction (interface or abstract class — your choice) with `Circle` and `Square` implementations, and explain in your own words which parts are "abstracted away" from a caller using `Shape`  
`shape = new Circle(5);`.
- 2 Write one paragraph in your own words distinguishing abstraction from encapsulation, using a real-world example NOT already used in this topic.

---

*(End of Topic 21: Abstraction.)*

---

## TOPIC 22: ENCAPSULATION

---

### 1. Introduction

**What is it?** Encapsulation is the OOP principle of bundling an object's data (fields) together with the methods that operate on it, while restricting direct external access to that data — typically by making fields `private` and exposing controlled access via public getter/setter methods.

**Why introduced:** Without encapsulation, any external code could freely read AND modify an object's internal fields directly, bypassing any validation or invariants the class is supposed to maintain (e.g., setting a bank balance to a negative number directly). Encapsulation forces all interaction through a controlled, validated API surface.

**Interview definition:** "Encapsulation is the mechanism of restricting direct access to an object's internal state by declaring fields `private` and exposing controlled, validated access through public methods, protecting the object's invariants and enabling its internal implementation to change without breaking external callers."

### 2. Real Life Analogy

- **Bank:** You cannot walk into the vault and change your account balance directly — you must go through the teller/ATM (public methods: `deposit()`, `withdraw()`), which validates the operation (sufficient funds, correct PIN) before applying it.
- **Hospital:** A patient's medical record isn't directly editable by anyone walking by — only authorized staff, through defined procedures (methods), can update specific fields, ensuring data integrity.

### 3. How to Achieve Encapsulation in Java

```
public class BankAccount {
    private double balance; // 1. Make fields PRIVATE

    public double getBalance() { // 2. Provide public GETTER
        return balance;
    }

    public void deposit(double amount) { // 3. Provide public, VALIDATED setter-like methods
        if (amount <= 0) {
            throw new IllegalArgumentException("Deposit must be positive");
        }
        balance += amount;
    }

    public void withdraw(double amount) {
        if (amount > balance) {
            throw new IllegalStateException("Insufficient funds");
        }
        balance -= amount;
    }
}
```

Notice there is **no direct** `setBalance()` — withdrawals/deposits are the only sanctioned ways to change `balance`, each enforcing business rules. This is a stronger, more deliberate form of encapsulation than blindly generating a setter for every field.

### 4. Levels of Encapsulation (Getter/Setter Patterns)

| Pattern                    | Example                                     | When to use                                                                                                      |
|----------------------------|---------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| Read-only                  | Only a getter, no setter                    | Immutable-after-construction fields (e.g., an ID)                                                                |
| Read-write with validation | Getter + setter that validates input        | Mutable fields with business rules (e.g., age must be positive)                                                  |
| Write-only (rare)          | Only a setter, no getter                    | Sensitive fields you never want read back directly (e.g., a password field, though typically you'd store a hash) |
| Fully immutable            | No setters at all, only set via constructor | Value objects, DTOs designed to never change after creation                                                      |

## 5. Code Examples

```
public class Employee {
    private String name;
    private int age;

    public Employee(String name, int age) {
        setName(name); // reuse validation logic even in the constructor
        setAge(age);
    }

    public String getName() { return name; }

    public void setName(String name) {
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("Name cannot be blank");
        }
        this.name = name;
    }

    public int getAge() { return age; }

    public void setAge(int age) {
        if (age < 18 || age > 65) {
            throw new IllegalArgumentException("Age must be between 18 and 65");
        }
        this.age = age;
    }
}
```

## 6. Edge Cases

| Case                                                                                    | Result                                                                                                                                                                                                                                      |
|-----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Public mutable field directly exposed ( <code>public List&lt;String&gt; items;</code> ) | Even though the reference itself might be "protected," callers can still directly mutate the LIST's contents ( <code>obj.items.add(...)</code> ) — a common <b>encapsulation leak</b> even when the field reference itself looks controlled |
| Returning a mutable internal collection directly from a getter                          | External code can mutate it, silently bypassing the class's own validation — must return a defensive copy or an unmodifiable view instead                                                                                                   |
| Providing a setter for every single field "just in case"                                | Defeats the purpose of encapsulation — turns the class into a glorified struct with no real invariant protection                                                                                                                            |

```
// ENCAPSULATION LEAK EXAMPLE:
class Team {
    private List<String> members = new ArrayList<>();
    public List<String> getMembers() { return members; } // LEAK! caller can now do
    team.getMembers().clear()
}

// FIX:
public List<String> getMembers() {
    return Collections.unmodifiableList(members); // or return new ArrayList<>(members) for a defensive
    copy
}
```

## 7. Common Mistakes



- Auto-generating getters AND setters for every field without thinking about whether each one should truly be mutable from outside — a widespread anti-pattern that produces "anemic" classes with no real behavior or protection.
- Returning direct references to internal mutable collections/objects from getters, allowing external code to bypass all validation.
- Confusing encapsulation with abstraction (see [[Abstraction|Topic 21]]'s comparison table) — encapsulation is specifically about protecting DATA, not about hiding implementation complexity in general.

## 8. Best Practices

- Default to `private` fields; only add a getter/setter when there's a genuine need for external read/write access.
- Validate all input inside setters (or dedicated methods like `deposit()`/`withdraw()`) — never trust that callers will only pass valid values.
- Return defensive copies or unmodifiable views for any getter exposing a mutable internal collection or object.
- Prefer immutable objects (only settable via constructor, no setters at all) wherever the object's state genuinely shouldn't change after creation.

## 9. Production Usage

- Every JavaBean-style DTO/Entity class used across Spring, Hibernate, and Jackson (JSON serialization) relies on the getter/setter encapsulation convention.
- Domain-driven design (DDD) strongly emphasizes encapsulation — rich domain objects expose only meaningful business operations (`order.cancel()`, `account.withdraw(amount)`), never raw field setters, to protect business invariants.

## 10. Frequently Asked Interview Questions

- 1 **What is encapsulation?** Bundling data with the methods that operate on it, restricting direct external access to that data via `private` fields and controlled public methods.
- 2 **How do you achieve encapsulation in Java?** Declare fields `private`, and expose access only through public getter/setter methods (ideally with validation).
- 3 **What's the difference between encapsulation and abstraction?** Encapsulation protects/hides DATA; abstraction hides implementation COMPLEXITY behind a contract — related but distinct concepts (see [[Abstraction|Topic 21]]).
- 4 **Is it always correct to generate a getter AND setter for every field?** No — this is a common anti-pattern; fields that shouldn't change after construction should have only a getter (or no accessor at all), preserving genuine encapsulation rather than defeating its purpose.

- 5 **What is an "encapsulation leak," and give an example?** When a getter returns a direct reference to a mutable internal object/collection, allowing external code to bypass validation and mutate internal state directly — fixed by returning a defensive copy or unmodifiable view.

## 11. Revision Notes

### ENCAPSULATION — CHEAT SHEET

=====

Bundle DATA + METHODS together; restrict direct external access to data

HOW: private fields + public getters/(validated) setters

GOOD PRACTICE: NOT every field needs a setter - only add one where mutation is genuinely valid, and always validate input

LEAK: returning a mutable internal collection/object directly from a getter

FIX: `Collections.unmodifiableList(...)` or a defensive copy

vs ABSTRACTION: encapsulation hides DATA; abstraction hides IMPLEMENTATION

## 12. Homework

- 1 Write a `Person` class with `private` fields and validating setters (age must be non-negative, name must be non-blank), and prove invalid input throws `IllegalArgumentException`.
- 2 Create an "encapsulation leak" example (getter returning a mutable `List` directly), demonstrate the leak with a caller mutating it, then fix it using `Collections.unmodifiableList()`.

---

(End of Topic 22: Encapsulation — this concludes the "four pillars" of OOP: Inheritance, Polymorphism, Abstraction, Encapsulation.)

---

# TOPIC 23: INTERFACES

---

## 1. Introduction

**What is it?** An interface is a fully abstract type (traditionally) that defines a **contract** — a set of method signatures that any implementing class must provide — without dictating HOW those methods are implemented. Since Java 8, interfaces can also include `default` and `static` methods with actual bodies, and since Java 9, `private` interface methods for internal code reuse.

**Why introduced:** Java needed a way to achieve "multiple inheritance of type" (a class conforming to multiple contracts) without the ambiguity of multiple class inheritance (the Diamond Problem — see [\[\[Inheritance|Topic 19\]\]](#)). Interfaces let unrelated classes agree to honor the same contract, enabling polymorphism across otherwise unrelated class hierarchies.

**Interview definition:** "An interface is a reference type in Java, similar to a class, that can contain only constants, method signatures, default methods, static methods, private methods, and nested types; interfaces

cannot contain instance fields or constructors, and a class implements an interface by providing concrete bodies for all its abstract methods."

## 2. Real Life Analogy

- **Restaurant:** A `Deliverable` interface (contract: `deliver()`) can be implemented by completely unrelated things — a `Pizza`, a `Grocery`, a `Medicine` — none of which share a common parent class, but all agree to honor the same "can be delivered" contract, letting a single delivery-dispatch system handle any of them uniformly.
- **Amazon:** A `Payable` interface (`processPayment()`) is implemented by `CreditCard`, `UPI`, `GiftCard` — entirely unrelated concrete types unified only by their shared contract.

## 3. Syntax and Evolution Across Java Versions

```
interface Vehicle {  
    int MAX_SPEED = 200; // implicitly public static final (a CONSTANT)  
  
    void start(); // implicitly public abstract  
  
    default void honk() { // Java 8+ : DEFAULT method, has a body  
        System.out.println("Beep beep!");  
    }  
  
    static Vehicle createDefault() { // Java 8+ : STATIC method, has a body  
        return new Car();  
    }  
  
    private void logStart() { // Java 9+ : PRIVATE method, internal reuse only  
        System.out.println("Starting vehicle...");  
    }  
}
```

| Member Type                  | Since    | Implicit Modifiers                          | Purpose                                                      |
|------------------------------|----------|---------------------------------------------|--------------------------------------------------------------|
| Constants (fields)           | Java 1.0 | <code>public static final</code>            | Shared, immutable contract-level constants                   |
| Abstract methods             | Java 1.0 | <code>public abstract</code>                | The core contract every implementer MUST fulfill             |
| <code>default</code> methods | Java 8   | <code>public</code> (has a body)            | Shared implementation, optionally overridden by implementers |
| <code>static</code> methods  | Java 8   | (has a body, NOT inherited by implementers) | Utility/factory methods related to the interface             |
| <code>private</code> methods | Java 9   | (has a body, internal only)                 | Code reuse BETWEEN default methods, not exposed externally   |

## 4. Why `default` Methods Were Added (Java 8) — A Real Historical Problem

Before Java 8, adding a NEW method to an existing interface would **break every single class that already implemented it** (all of them would fail to compile, missing the new method). This became a critical problem when Java 8 needed to add methods like `forEach()` to the `Collection` interface WITHOUT breaking the millions of existing classes implementing it across the entire Java ecosystem. `default methods` solved this: an interface can add a new method WITH a default body, and all EXISTING implementers automatically inherit that default behavior without any code changes required — full backward compatibility.

## 5. Multiple Interface Implementation & Diamond Resolution

*(Fully covered with code in [\[\[Inheritance|Topic 19\]\]](#), Section 7.2 — a class can `implements` multiple interfaces; conflicting `default` methods force an explicit, compiler-enforced resolution via `Interface.super.method()`.)*

## 6. Functional Interfaces (Preview — Full Depth in Java 8 Features Topic)

```
@FunctionalInterface // exactly ONE abstract method - enables lambda expressions
interface Calculator {
    int calculate(int a, int b);
}
Calculator add = (a, b) -> a + b; // lambda expression implementing the interface's single method
```

## 7. Code Examples

```
interface Shape {
    double area(); // abstract - the contract
    default void describe() { // default - shared behavior
        System.out.println("This shape has area: " + area());
    }
}

class Circle implements Shape {
    private double radius;
    Circle(double radius) { this.radius = radius; }
    @Override
    public double area() { return Math.PI * radius * radius; }
}

public class InterfaceDemo {
    public static void main(String[] args) {
        Shape s = new Circle(5);
        s.describe(); // uses the INHERITED default method, calling the overridden area()
    }
}
```

## 8. Edge Cases

| Case                                                                                        | Result                                                                                                                                                                                       |
|---------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Implementing class doesn't implement an abstract interface method                           | Compile-time error, unless the implementing class is itself declared <code>abstract</code>                                                                                                   |
| Two interfaces with conflicting <code>default</code> methods, both implemented by one class | Compile-time error unless explicitly resolved (see [[Inheritance                                                                                                                             |
| Trying to instantiate an interface directly ( <code>new Shape()</code> )                    | Compile-time error — interfaces cannot be instantiated                                                                                                                                       |
| Interface field NOT explicitly marked <code>final/static</code>                             | Still implicitly IS <code>public static final</code> — attempting to reassign it is a compile-time error                                                                                     |
| <code>static</code> interface method called via an implementing class's reference           | Compile-time error — unlike <code>default</code> methods, <code>static</code> interface methods are NOT inherited by implementing classes; must call via <code>InterfaceName.method()</code> |

## 9. Common Mistakes

- Forgetting interface fields are implicitly `public static final` — attempting to treat them as regular mutable fields.
- Assuming `static` interface methods are inherited/callable via an implementing class reference — they are not; call them via the interface name directly.
- Overusing `default` methods to add substantial business logic directly into interfaces, blurring the line between "contract" and "implementation" — keep default methods focused on genuinely shared, simple behavior.

## 10. Best Practices

- Keep interfaces focused on a single, cohesive responsibility (Interface Segregation Principle — many small, focused interfaces beat one large "fat" interface).
- Use `default` methods primarily for backward-compatible API evolution, not as a substitute for proper abstract class design.
- Mark functional interfaces with `@FunctionalInterface` — the annotation causes a compile-time error if a second abstract method is accidentally added, protecting lambda compatibility.

## 11. Production Usage

- The ENTIRE Java Collections Framework is built on interfaces (`List`, `Set`, `Map`, `Collection`, `Iterable`) — enabling `ArrayList/LinkedList` to be swapped interchangeably wherever `List` is the declared type.
- Spring's core is interface-driven: `Repository`, `Service` layers are typically defined as interfaces with concrete implementations wired via dependency injection, enabling easy testing (mock implementations) and swappable backends.
- JDBC's `Driver`, `Connection`, `Statement` are all interfaces — allowing any database vendor to provide a compliant implementation.

## 12. Frequently Asked Interview Questions

- 1 **What is an interface?** A reference type defining a contract of method signatures (plus constants, and since Java 8, default/static methods) that implementing classes must fulfill.
- 2 **Can an interface have instance fields?** No — interface fields are implicitly `public static final` (constants), never mutable instance state.
- 3 **Why were default methods introduced in Java 8?** To allow adding new methods to existing interfaces (like `Collection.forEach()`) without breaking all existing implementing classes — full backward compatibility.
- 4 **Are static interface methods inherited by implementing classes?** No — unlike default methods, static interface methods must be called via the interface name directly, not through an implementing class's reference.
- 5 **Can a class implement multiple interfaces?** Yes — this is how Java achieves "multiple inheritance of type" safely, with the compiler forcing explicit resolution of any conflicting default methods.
- 6 **What is a functional interface?** An interface with exactly one abstract method, enabling it to be implemented via a lambda expression; conventionally annotated with `@FunctionalInterface`.
- 7 **What was the historical problem default methods solved?** Adding any new abstract method to a widely-implemented interface would break every existing implementer at compile time — default methods allow safe, backward-compatible interface evolution.

## 13. Revision Notes

### INTERFACES – CHEAT SHEET

=====

Contract only - defines WHAT, not HOW (traditionally)

#### MEMBERS:

fields -> implicitly `public static final` (constants)

abstract methods -> implicitly `public abstract` (the core contract)

default methods -> Java 8+, HAS a body, INHERITED by implementers, overridable

static methods -> Java 8+, HAS a body, NOT inherited - call via `InterfaceName.method()`

private methods -> Java 9+, internal code reuse between default methods only

WHY default METHODS EXIST: backward-compatible interface evolution

(adding `Collection.forEach()` without breaking every existing implementer)

Cannot instantiate an interface directly.

Multiple interface implementation IS supported (unlike multiple class inheritance).

`@FunctionalInterface` -> exactly ONE abstract method -> enables lambda expressions

## 14. Homework

- 1 Create an interface with one abstract method and one default method; implement it in two unrelated classes and call both methods on each.
- 2 Add a static factory method to an interface and call it correctly (via the interface name) versus incorrectly (via an implementing instance), observing the compile error for the latter.

## TOPIC 24: ABSTRACT CLASSES

---

### 1. Introduction

**What is it?** An abstract class is a class declared with the `abstract` keyword that **cannot be instantiated directly** and may contain a mix of fully-implemented (concrete) methods and unimplemented (abstract) methods that subclasses must provide.

**Why introduced:** Sometimes a base type needs to provide SOME shared, ready-to-use implementation (unlike a pure interface) while still forcing subclasses to fill in certain type-specific behavior. Abstract classes fill the gap between "fully concrete class" and "fully abstract interface."

**Interview definition:** "An abstract class is a class that cannot be instantiated on its own, may declare zero or more abstract methods (without a body, to be implemented by subclasses) alongside regular concrete methods and instance fields, and is extended (not implemented) by subclasses using `extends`."

### 2. Real Life Analogy

- **College:** An abstract `Employee` class might provide a fully-implemented `clockIn()` method (identical for everyone) but declare `calculateSalary()` as abstract, since HOW salary is calculated genuinely differs between a `Professor` and an `AdminStaff` subclass.
- **Restaurant:** An abstract `MenuItem` class implements `printLabel()` identically for all items but leaves `calculatePrice()` abstract, since pricing logic differs meaningfully between a `Pizza` (per-topping) and a `Drink` (fixed price).

### 3. Syntax

```
abstract class Employee {
    protected String name;
    protected double baseSalary;

    Employee(String name, double baseSalary) { // abstract classes CAN have constructors
        this.name = name;
        this.baseSalary = baseSalary;
    }

    abstract double calculateSalary(); // no body - MUST be implemented by subclasses

    void printPaySlip() { // concrete, shared implementation
        System.out.printf("%s: %.2f%n", name, calculateSalary());
    }
}
```

## 4. Abstract Class vs Interface — Full Comparison (Critical Interview Table)

| Aspect                      | Abstract Class                                                                            | Interface                                                                                           |
|-----------------------------|-------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| Keyword                     | <code>abstract class</code> , extended via <code>extends</code>                           | <code>interface</code> , implemented via <code>implements</code>                                    |
| Multiple inheritance        | A class can extend only ONE abstract class                                                | A class can implement MULTIPLE interfaces                                                           |
| Constructors                | CAN have constructors (called via subclass's <code>super()</code> )                       | CANNOT have constructors                                                                            |
| Instance fields             | CAN have regular (mutable) instance fields                                                | CANNOT — fields are implicitly <code>public static final</code> constants only                      |
| Access modifiers on methods | Can be <code>public</code> , <code>protected</code> , <code>private</code> , etc.         | Methods are implicitly <code>public</code> (except <code>private</code> interface methods, Java 9+) |
| Method implementation       | Mix of abstract AND fully concrete methods freely                                         | Traditionally all abstract; Java 8+ allows default/static bodies too                                |
| When to use                 | Related classes sharing significant common state/implementation ("IS-A" with shared code) | Unrelated classes needing to honor a common contract/capability                                     |
| Real example                | <code>AbstractList</code> , <code>HttpServlet</code>                                      | <code>Runnable</code> , <code>Comparable</code> , <code>List</code>                                 |

**The single most repeated interview guidance:** Use an abstract class when subclasses share significant common state/implementation and a clear IS-A hierarchy exists; use an interface when you need multiple, possibly unrelated classes to honor a shared capability/contract, or need multiple inheritance of type.

## 5. Code Examples

```
abstract class Shape {
    abstract double area(); // must be implemented
    void printArea() { // shared, concrete
        System.out.println("Area: " + area());
    }
}

class Circle extends Shape {
    private double radius;
    Circle(double radius) { this.radius = radius; }
    @Override
    double area() { return Math.PI * radius * radius; }
}

public class AbstractClassDemo {
    public static void main(String[] args) {
        Shape s = new Circle(4);
        s.printArea(); // "Area: 50.26..."
    }
}
```



## 6. Edge Cases

| Case                                                                                         | Result                                                                                                      |
|----------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| Instantiating an abstract class directly ( <code>new Shape()</code> )                        | Compile-time error                                                                                          |
| A subclass doesn't implement all inherited abstract methods                                  | Compile-time error, UNLESS that subclass is itself declared <code>abstract</code>                           |
| An abstract class with ZERO abstract methods (all concrete)                                  | Perfectly legal — still cannot be instantiated, purely to force subclassing/prevent direct use              |
| An abstract method marked <code>private</code> , <code>static</code> , or <code>final</code> | Compile-time error — these modifiers are fundamentally incompatible with "must be overridden by a subclass" |

## 7. Common Mistakes

- Trying to instantiate an abstract class directly.
- Forgetting a concrete subclass must implement EVERY inherited abstract method, not just some.
- Choosing an abstract class when an interface would have been more flexible (blocking a class from also extending something else, since only one class can ever be extended).

## 8. Best Practices

- Use abstract classes when subclasses genuinely share meaningful state/behavior beyond just a method signature contract.
- Provide sensible concrete "template" methods in the abstract class that call the abstract methods internally (the **Template Method design pattern**) to maximize code reuse.
- Prefer interfaces by default for pure contracts; reach for an abstract class specifically when shared state/constructors are genuinely needed.

## 9. Production Usage

- `HttpServlet` (provides concrete `service()` dispatch logic, but relies on subclasses overriding `doGet()/doPost()`) is a textbook abstract class example used in virtually every traditional Java web application.
- `AbstractList`, `AbstractMap` in the Collections Framework provide significant shared skeletal implementation that concrete collection classes build upon.

## 10. Frequently Asked Interview Questions

- 1 **Can an abstract class have a constructor?** Yes — called implicitly via `super()` when a subclass object is created, useful for initializing shared fields.
- 2 **Can an abstract class have zero abstract methods?** Yes — it's still legal and still cannot be instantiated; sometimes done purely to prevent direct instantiation.

- 3 **Can a class extend more than one abstract class?** No — Java allows extending only one class (abstract or not); use interfaces for multiple-contract needs.
- 4 **When would you choose an abstract class over an interface?** When subclasses genuinely share significant common state/implementation and constructors, forming a clear IS-A hierarchy — not just a behavioral contract.
- 5 **Can an abstract method be `private` or `static`?** No — both are incompatible with the requirement that subclasses must be able to override/implement it.

## 11. Revision Notes

### ABSTRACT CLASSES — CHEAT SHEET

=====

Cannot be instantiated. CAN have: constructors, instance fields, concrete methods, AND abstract methods (mix freely)

vs INTERFACE:

abstract class -> single inheritance, CAN have state+constructors

interface -> multiple inheritance, traditionally no state

USE ABSTRACT CLASS WHEN: subclasses share real implementation/state (IS-A)

USE INTERFACE WHEN: unrelated classes need a shared CAPABILITY/contract

Concrete subclass MUST implement ALL abstract methods, or be abstract itself.

abstract method CANNOT be `private`/`static`/`final`.

## 12. Homework

- 1 Build an abstract `Vehicle` class with a concrete `honk()` method and an abstract `move()` method; implement `Car` and `Boat` subclasses.
- 2 Write one paragraph explaining, with a NEW example, when you'd choose an abstract class over an interface for a real design.

---

(End of Topic 24: Abstract Classes.)

---

# TOPIC 25: OBJECT CLASS

---

## 1. Introduction

**What is it?** `java.lang.Object` is the **root of the entire Java class hierarchy** — every class in Java, whether you explicitly write `extends Object` or not, ultimately inherits from it.

**Why introduced:** Java needed a universal common type so that fundamental, universally-needed operations (equality checking, hashing, string representation, thread coordination) are guaranteed to exist on EVERY object, enabling generic APIs (like `Object[]`, `equals()`-based collections) to work uniformly across all types.

**Interview definition:** "Object is the implicit superclass of every class in Java that declares no explicit superclass, providing a baseline contract of methods — `equals()`, `hashCode()`, `toString()`, `getClass()`, `clone()`, `wait()`/`notify()`/`notifyAll()` — that every Java object inherits and may override."

## 2. Key Methods of `Object`

| Method                                                                 | Purpose                                                  | Default Behavior                                                                        |
|------------------------------------------------------------------------|----------------------------------------------------------|-----------------------------------------------------------------------------------------|
| <code>equals(Object o)</code>                                          | Logical equality check                                   | Default: reference equality (same as <code>==</code> )                                  |
| <code>hashCode()</code>                                                | Integer hash used by hash-based collections              | Default: JVM-implementation-specific (often related to memory address)                  |
| <code>toString()</code>                                                | Human-readable string representation                     | Default: <code>ClassName@hexHashCode</code>                                             |
| <code>getClass()</code>                                                | Returns the runtime <code>Class</code> object            | Cannot be overridden (implicitly <code>final</code> )                                   |
| <code>clone()</code>                                                   | Creates a copy of the object                             | Requires implementing <code>Cloneable</code> ; default does a shallow field copy        |
| <code>wait()</code> , <code>notify()</code> , <code>notifyAll()</code> | Thread coordination (monitor methods)                    | Used with synchronized blocks (full detail in Multithreading topic)                     |
| <code>finalize()</code>                                                | Called by GC before reclaiming (deprecated since Java 9) | Discouraged — unpredictable timing, replaced by <code>try-with-resources/Cleaner</code> |

## 3. `equals()` and `hashCode()` — The Critical Contract

### 3.1 Why Override Both Together (Never Just One)

```
class Point {
    int x, y;
    Point(int x, int y) { this.x = x; this.y = y; }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Point)) return false;
        Point p = (Point) o;
        return this.x == p.x && this.y == p.y;
    }

    @Override
    public int hashCode() {
        return Objects.hash(x, y); // combines fields into one consistent hash
    }
}
```

### 3.2 The `equals()`/`hashCode()` Contract (Interview Gold)

- 1 If `a.equals(b)` is true, then `a.hashCode() == b.hashCode()` **MUST** also be true.
- 2 The reverse is NOT required — two unequal objects CAN share the same hash code (a "hash collision," which is legal and expected occasionally).
- 3 **Violating rule 1** breaks `HashMap/HashSet` silently — two "equal" objects could end up in different hash buckets, making lookups fail unpredictably (a very common, hard-to-debug real-world bug).

```
Point p1 = new Point(1, 2);
Point p2 = new Point(1, 2);
```

```

System.out.println(p1 == p2); // false - different objects (references)
System.out.println(p1.equals(p2)); // true - equal CONTENT (custom equals())
System.out.println(p1.hashCode() == p2.hashCode()); // true - REQUIRED by the contract

Set<Point> points = new HashSet<>();
points.add(p1);
System.out.println(points.contains(p2)); // true - ONLY works because equals()+hashCode() are
correctly overridden

```

#### 4. toString()

```

class Point {
    int x, y;
    @Override
    public String toString() {
        return "Point(" + x + ", " + y + ")";
    }
}
System.out.println(new Point(1, 2)); // calls toString() automatically -> "Point(1, 2)"

```

Without an override, printing an object gives the unhelpful default: `Point@1b6d3586` (class name + @ + hex hash code).

#### 5. getClass()

```

Object obj = "Hello";
System.out.println(obj.getClass().getName()); // "java.lang.String"
System.out.println(obj.getClass().getSimpleName()); // "String"

```

Used heavily in reflection (see the dedicated Reflection API topic) and for runtime type identification without needing `instanceof` chains.

### 6. Edge Cases

| Case                                                                                  | Result                                                                                                                                      |
|---------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| Overriding <code>equals()</code> but NOT <code>hashCode()</code>                      | Breaks the contract — object behaves inconsistently in <code>HashMap/HashSet</code> (a very common real bug)                                |
| <code>equals(null)</code>                                                             | Must return <code>false</code> (never throw <code>NullPointerException</code> ) — a well-defined part of the <code>equals()</code> contract |
| <code>obj.equals(obj)</code> (comparing to itself)                                    | Must always return <code>true</code> (reflexivity, part of the contract)                                                                    |
| Two objects with the SAME hash code but different <code>equals()</code> result        | Perfectly legal (a hash collision) — hash-based collections handle this via bucket chaining                                                 |
| Calling <code>clone()</code> on a class that doesn't implement <code>Cloneable</code> | Throws <code>CloneNotSupportedException</code>                                                                                              |

### 7. Common Mistakes

- Overriding `equals()` without `hashCode()` (or vice versa) — breaking the fundamental contract, causing silent `HashMap/HashSet` bugs.

- Using mutable fields in `hashCode()/equals()` for objects stored as `HashMap` keys — if the field changes AFTER insertion, the object becomes "lost" in its original bucket (a classic real-world bug).
- Forgetting `toString()` isn't automatically meaningful — relying on the default `ClassName@hash` output in production logs, making debugging harder than necessary.

## 8. Best Practices

- ALWAYS override `equals()` and `hashCode()` together, using the SAME set of fields in both, generated via IDE tools or `java.util.Objects.hash(...)/Objects.equals(...)` helpers.
- Override `toString()` for any class you expect to log or debug — massively improves developer experience.
- Never use mutable fields as `HashMap/HashSet` keys' identity basis unless you can guarantee they won't change after insertion.
- Prefer records (Java 16+, covered in its own topic) for simple data classes — they auto-generate correct `equals()`, `hashCode()`, and `toString()`.

## 9. Production Usage

- JPA/Hibernate entities require carefully-considered `equals()/hashCode()` overrides (often based on a stable business key, NOT the auto-generated database ID, due to subtleties around detached/transient entity states).
- Every DTO/domain object used in `Sets`, `Map` keys, or compared for business equality across Spring/enterprise codebases relies on correct `Object` method overrides.

## 10. Frequently Asked Interview Questions

- 1 **What is the root of the Java class hierarchy?** `java.lang.Object` — every class implicitly extends it if no other superclass is specified.
- 2 **What is the `equals()/hashCode()` contract?** If two objects are equal via `equals()`, they MUST have the same `hashCode()`; the converse isn't required (unequal objects can share a hash code).
- 3 **What happens if you override `equals()` but not `hashCode()`?** You violate the contract, causing unpredictable behavior in hash-based collections (`HashMap/HashSet`) — an object might not be found even though an "equal" one was inserted.
- 4 **What does `toString()` return by default?** `ClassName@hexHashCode` — not useful for debugging unless explicitly overridden.
- 5 **Can `getClass()` be overridden?** No — it's implicitly `final`.
- 6 **Why shouldn't mutable fields be used in `hashCode()` for `HashMap` keys?** If the field changes after the object is inserted, its hash code changes too, but the map still looks in the ORIGINAL bucket — making the object effectively "lost," even though it's technically still in the map.
- 7 **What replaced `finalize()`, and why?** `try-with-resources` (for deterministic cleanup) and `java.lang.ref.Cleaner` (Java 9+) — `finalize()` was deprecated due to unpredictable timing and potential to resurrect objects, causing GC and correctness issues.

## 11. Revision Notes

### OBJECT CLASS – CHEAT SHEET

=====

`java.lang.Object` = root of ALL Java classes (implicit superclass)

KEY METHODS: `equals()`, `hashCode()`, `toString()`, `getClass()`, `clone()`,  
`wait()/notify()/notifyAll()`, `finalize()` (deprecated)

CONTRACT: `a.equals(b) == true => a.hashCode() == b.hashCode()` (MANDATORY)  
(reverse not required - hash collisions are legal)

ALWAYS override `equals()` + `hashCode()` TOGETHER, using the SAME fields

NEVER use mutable fields for `HashMap/HashSet` key identity

`getClass()` -> implicitly final, cannot be overridden

Records (Java 16+) auto-generate correct `equals/hashCode/toString`

## 12. Homework

- 1 Write a class overriding `equals()` and `hashCode()` correctly, then prove `HashSet.contains()` works correctly for logically-equal-but-different-reference objects.
- 2 Deliberately override ONLY `equals()` (not `hashCode()`) and demonstrate the resulting `HashSet` bug where a logically-equal object isn't found.

---

*(End of Topic 25: Object Class.)*

---

# TOPIC 26: PACKAGES

---

## 1. Introduction

**What is it?** A package is a namespace mechanism that groups related classes/interfaces together, both organizing code logically and preventing naming collisions between classes with the same simple name in different parts of a large codebase or across different libraries.

**Interview definition:** "A package is a namespace that organizes a set of related classes and interfaces, declared via the `package` statement at the top of a source file, mapping conventionally to a directory structure on disk, and forming part of a class's fully-qualified name."

## 2. Real Life Analogy

- **College:** Packages are like departmental building/room numbering — "Computer Science, Room 101" versus "Mechanical Engineering, Room 101" — the same room NUMBER (101, akin to a simple class name) can exist without conflict, distinguished by department (package).

- **Library:** The Dewey Decimal System groups books by subject — a package is like a specific numbered section, letting two different books both be called "Volume 1" without ambiguity, since their FULL classification differs.

### 3. Syntax and Naming Conventions

```
package com.jobsradar.banking; // MUST be the first non-comment line in the file

import java.util.List; // import a SPECIFIC class
import java.util.*; // import ALL classes in a package (avoid in production - unclear)
import static java.lang.Math.PI; // static import - use PI directly, not Math.PI

public class Account {
    // ...
}
```

- **Naming convention:** reverse domain name (`com.company.project.module`), all lowercase — avoids collisions with other organizations' code globally.
- Directory structure MUST mirror the package name: `com.jobsradar.banking.Account` lives at `com/jobsradar/banking/Account.java`.

### 4. Types of Packages

| Type           | Example                                                                      | Note                                                                                                                                     |
|----------------|------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| Built-in (JDK) | <code>java.lang</code> , <code>java.util</code> , <code>java.io</code>       | <code>java.lang</code> is auto-imported into every file — no explicit import needed for <code>String</code> , <code>System</code> , etc. |
| User-defined   | <code>com.jobsradar.banking</code>                                           | Your own organized code                                                                                                                  |
| Third-party    | <code>org.springframework.*</code> ,<br><code>com.fasterxml.jackson.*</code> | From external libraries/dependencies                                                                                                     |

### 5. Fully Qualified Names

```
java.util.List<String> list = new java.util.ArrayList<>(); // fully qualified, no import needed
```

Useful when two imported classes share the same simple name (e.g., `java.util.Date` vs `java.sql.Date`) — you import one normally and refer to the other by its fully qualified name to disambiguate.

## 6. Code Example

```
// File: com/jobsradar/util/MathHelper.java
package com.jobsradar.util;

public class MathHelper {
    public static int square(int x) { return x * x; }
}

// File: com/jobsradar/app/Main.java
package com.jobsradar.app;

import com.jobsradar.util.MathHelper;

public class Main {
    public static void main(String[] args) {
        System.out.println(MathHelper.square(5)); // 25
    }
}
```

## 7. Edge Cases

| Case                                                                                | Result                                                                                                                                                                   |
|-------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Class file's directory doesn't match its package declaration                        | Compile/runtime error — package structure must mirror directory structure                                                                                                |
| Two classes with the same simple name, different packages, both imported explicitly | Compile-time error ("ambiguous") — must use at least one fully-qualified name                                                                                            |
| No <code>package</code> statement at all                                            | Class belongs to the <b>default (unnamed) package</b> — legal for small scripts, strongly discouraged for real projects (can't be imported by classes in named packages) |
| <code>import java.util.*;</code>                                                    | Imports classes directly in <code>java.util</code> , but NOT sub-packages like <code>java.util.concurrent</code> (which needs its own explicit import)                   |

## 8. Common Mistakes

- Leaving real project code in the default (unnamed) package — breaks importability and most build tool conventions.
- Using wildcard imports (`import java.util.*;`) in production code, reducing clarity about exactly which classes are actually used (most style guides/IDEs discourage this).
- Assuming `import java.util.*` also imports sub-packages — it does not.

## 9. Best Practices

- Follow the reverse-domain-name convention consistently (`com.company.project.module`).
- Prefer explicit, specific imports over wildcard imports for clarity (though many teams/IDEs differ on this stylistically — consistency matters more than the specific choice).



- Organize packages by feature/domain (`com.company.orders`, `com.company.payments`) rather than purely by technical layer, for larger applications — a common modern architectural preference (though layered packaging by `controller/service/repository` remains widespread and valid too).

## 10. Frequently Asked Interview Questions

- 1 **What is a package?** A namespace mechanism grouping related classes/interfaces, preventing naming collisions and organizing code logically.
- 2 **Which package is automatically imported into every Java file?** `java.lang` — classes like `String`, `System`, `Object`, `Math` need no explicit import.
- 3 **What's the naming convention for packages?** Reverse domain name, all lowercase (e.g., `com.company.project`).
- 4 **Does `import java.util.*` import sub-packages like `java.util.concurrent`?** No — wildcard imports only cover classes directly within the specified package, not its sub-packages.
- 5 **What happens if you don't declare a package at all?** The class belongs to the default (unnamed) package — legal, but strongly discouraged for real projects since it can't be properly imported by named-package classes.

## 11. Revision Notes

### PACKAGES – CHEAT SHEET

=====

Namespace grouping related classes; prevents naming collisions

``package`` statement MUST be the first line (aside from comments) in a file

Directory structure MUST mirror package name (`com.a.b` -> `com/a/b/`)

`java.lang` -> auto-imported everywhere (`String`, `System`, `Math`, `Object`, ...)

`import X.*` -> imports classes directly in X, NOT its sub-packages

Fully qualified name -> resolves ambiguity between same-named classes

in different packages (e.g., `java.util.Date` vs `java.sql.Date`)

No package declared -> default (unnamed) package - AVOID in real projects

## 12. Homework

- 1 Create two classes with the identical simple name (`Report`) in two different packages, import both in a third file, and resolve the resulting ambiguity using a fully-qualified name.
- 2 Organize a small 3-class mini-project into a proper `com.yourname.project` package structure matching the correct directory layout.

---

(End of Topic 26: Packages.)

---

# TOPIC 27: ACCESS MODIFIERS

## 1. Introduction

**What is it?** Access modifiers (`public`, `protected`, default/package-private, `private`) control the visibility/accessibility of classes, fields, methods, and constructors from other code, forming the enforcement mechanism behind encapsulation (see [[Encapsulation|Topic 22]]).

**Interview definition:** "Access modifiers are keywords that restrict the scope of visibility for a class member or top-level class, controlling which other classes or packages may access it, ranging from `private` (most restrictive) to `public` (least restrictive)."

## 2. The Four Levels — Full Visibility Table

| Modifier                                | Same Class | Same Package | Subclass (different package) | Different Package (non-subclass) |
|-----------------------------------------|------------|--------------|------------------------------|----------------------------------|
| <code>private</code>                    | ■ Yes      | ■ No         | ■ No                         | ■ No                             |
| (default / package-private, no keyword) | ■ Yes      | ■ Yes        | ■ No                         | ■ No                             |
| <code>protected</code>                  | ■ Yes      | ■ Yes        | ■ Yes                        | ■ No                             |
| <code>public</code>                     | ■ Yes      | ■ Yes        | ■ Yes                        | ■ Yes                            |

## 3. Real Life Analogy

- **Bank:** `private` fields are like the bank's internal ledger — only that specific department (class) can touch it. `protected` is like information shared with all branch managers group-wide (subclasses), even in other cities (packages), but not the general public. `public` is like the bank's published interest rates — anyone, anywhere, can see them.
- **College:** `private` = a professor's personal notes (only they can access). Default/package-private = shared department resources (only accessible within that department/package). `protected` = resources shared with affiliated colleges (subclasses) even off-campus. `public` = the college's public course catalog (accessible to anyone).

## 4. Applying Access Modifiers — Where Each Can Be Used

| Target                      | Allowed Modifiers                                                                                            |
|-----------------------------|--------------------------------------------------------------------------------------------------------------|
| Top-level class             | <code>public</code> or default (package-private) ONLY — never <code>private</code> or <code>protected</code> |
| Fields/methods/constructors | All four: <code>public</code> , <code>protected</code> , default, <code>private</code>                       |
| Inner/nested classes        | All four (since they behave more like a class member)                                                        |

## 5. Code Examples

```
package com.jobsradar.banking;

public class Account {
    private double balance; // only this class
    protected String accountType; // this package + subclasses anywhere
    String branchCode; // default - only this package (no modifier written)
    public String accountNumber; // accessible from anywhere

    private void logInternal() { } // internal implementation detail only
    protected void applyInterest() { } // meant to be used/overridden by subclasses
    public void deposit(double amt) { } // the public API
}
```

## 6. Edge Cases

| Case                                                                                                   | Result                                                                                                                         |
|--------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| Top-level class marked <code>private</code>                                                            | Compile-time error — not a legal modifier for a top-level class                                                                |
| Overriding method tries to narrow access (parent <code>public</code> , child <code>private</code> )    | Compile-time error (see [[Method Overriding                                                                                    |
| <code>protected</code> member accessed from an unrelated class in a DIFFERENT package (not a subclass) | Compile-time error — <code>protected</code> grants subclass + same-package access only, NOT arbitrary different-package access |
| Default (package-private) member accessed from a subclass in a DIFFERENT package                       | Compile-time error — default access does NOT extend to subclasses outside the package, unlike <code>protected</code>           |

## 7. Common Mistakes

- Confusing `protected` with "accessible to all subclasses everywhere unconditionally" — it also requires either same-package OR an actual subclass relationship; a `protected` member accessed via a supertype reference from an unrelated subclass instance in a different package has subtle additional restrictions in the JLS that trip up even experienced developers.
- Making fields `public` for convenience instead of `private` + accessors — breaks encapsulation.
- Forgetting default (no modifier) is package-private, NOT "fully public" — a common assumption error for developers coming from languages where omitting a modifier means public.

## 8. Best Practices

- Default to the MOST restrictive access level that still satisfies the design (`private` first, widen only when genuinely needed) — a core encapsulation principle.
- Use `protected` deliberately for members specifically intended to be used/extended by subclasses, not as a vague "semi-public" choice.
- Keep top-level classes package-private (default) unless they're genuinely part of your public API surface.

## 9. Production Usage

- Library/framework API design relies heavily on careful access modifier choices — public classes/methods form the supported "contract" with consumers, while `private`/package-private implementation details can be freely changed between versions without breaking anyone.
- Module systems (Java 9+ JPMS) add an additional, stronger layer of encapsulation on top of access modifiers, restricting access even to `public` classes unless their containing package is explicitly `exports`-ed by the module.

## 10. Frequently Asked Interview Questions

- 1 **What are the four access modifier levels in Java?** `private`, default (package-private, no keyword), `protected`, `public` — from most to least restrictive.
- 2 **Can a top-level class be `private` or `protected`?** No — only `public` or default (package-private) are legal for top-level classes.
- 3 **What's the difference between `protected` and default access?** Default is limited to the same package only; `protected` extends that to also include subclasses even in different packages.
- 4 **Can an overriding method reduce the access level of the method it overrides?** No — it can only keep the same or widen it, never narrow it.
- 5 **If no access modifier is written, what is the default?** Package-private — accessible only within the same package, NOT truly "public" as some developers mistakenly assume.

## 11. Revision Notes

ACCESS MODIFIERS – CHEAT SHEET  
=====

`private` -> same class ONLY  
 (default) -> same class + same package  
`protected` -> same class + same package + subclasses (even other packages)  
`public` -> everywhere

TOP-LEVEL CLASS: only `public` or default allowed (never `private`/`protected`)  
 FIELDS/METHODS: all four allowed

RULE: default to the MOST restrictive level that still works; widen only when needed  
 Overriding method CANNOT narrow access, only keep same or widen

## 12. Homework

- 1 Create a class with one field of each access level (`private`, default, `protected`, `public`) and a second class in a DIFFERENT package attempting to access each — record which succeed and which fail to compile.
- 2 Attempt to declare a top-level class as `private`, and record the exact compiler error.

---

*(End of Topic 27: Access Modifiers — this concludes the Object-Oriented Programming sub-section of Core Java!)*

---

# ADVANCED CORE JAVA

---

## TOPIC 28: WRAPPER CLASSES

---

### 1. Introduction

**What is it?** Wrapper classes provide an **object representation** for each of Java's 8 primitive types (`Integer` for `int`, `Double` for `double`, `Character` for `char`, `Boolean` for `boolean`, etc.), allowing primitives to be used wherever an **object** is required (Collections, Generics).

**Why introduced:** Java's Collections Framework (`List`, `Map`, etc.) works only with objects/references, not raw primitives (a design decision predating generics that avoided complicating the JVM with primitive-specialized containers). Wrapper classes bridge this gap, letting primitives participate in generic, object-oriented APIs.

**Interview definition:** "Wrapper classes are immutable, final classes in `java.lang` that encapsulate a primitive value as an object, providing utility methods (parsing, conversion) and enabling primitives to be used in Collections and Generics via autoboxing/unboxing."

### 2. The Complete Mapping

| Primitive            | Wrapper Class          |
|----------------------|------------------------|
| <code>byte</code>    | <code>Byte</code>      |
| <code>short</code>   | <code>Short</code>     |
| <code>int</code>     | <code>Integer</code>   |
| <code>long</code>    | <code>Long</code>      |
| <code>float</code>   | <code>Float</code>     |
| <code>double</code>  | <code>Double</code>    |
| <code>char</code>    | <code>Character</code> |
| <code>boolean</code> | <code>Boolean</code>   |

### 3. Autoboxing and Unboxing (Java 5+)

```
int primitive = 10;
Integer wrapper = primitive; // AUTOBOXING - compiler inserts Integer.valueOf(primitive)
int back = wrapper; // AUTO-UNBOXING - compiler inserts wrapper.intValue()

List<Integer> list = new ArrayList<>();
list.add(5); // 5 (int) autoboxed to Integer to satisfy List<Integer>
int x = list.get(0); // Integer auto-unboxed back to int
```

## 4. Internal Working — Integer Caching (A Major Interview Trap)

```
Integer a = 100;
Integer b = 100;
System.out.println(a == b); // TRUE! (both from the cache)

Integer c = 200;
Integer d = 200;
System.out.println(c == d); // FALSE! (outside the cache range, two different objects)
```

**Why:** `Integer.valueOf(int)` (used internally by autoboxing) maintains a **cache of Integer objects for values -128 to 127** (per the JLS, to optimize common small-value usage). Values in that range return the SAME cached object; values outside it create a brand-new Integer object each time. This is a classic, frequently-asked interview trap — **always use `.equals()`, never `==`, to compare wrapper objects for value equality.**

## 5. Key Methods

| Method                                                                 | Purpose                                                                                                        |
|------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| <code>Integer.parseInt(String s)</code>                                | Converts a String to a primitive <code>int</code> , throws <code>NumberFormatException</code> on invalid input |
| <code>Integer.valueOf(String s) / valueOf(int i)</code>                | Returns an Integer OBJECT (uses the cache for small values)                                                    |
| <code>Integer.toString(int i)</code>                                   | Converts an <code>int</code> to its String representation                                                      |
| <code>Integer.MAX_VALUE / MIN_VALUE</code>                             | Constants for the type's range                                                                                 |
| <code>Integer.compare(int a, int b)</code>                             | Compares two <code>ints</code> , returns negative/zero/positive                                                |
| <code>Double.isNaN(double d)</code>                                    | Checks for "Not a Number"                                                                                      |
| <code>Character.isDigit(char c) / isLetter(c) / isWhitespace(c)</code> | Character classification utilities                                                                             |

## 6. Code Examples

```
public class WrapperDemo {
    public static void main(String[] args) {
        Integer a = 100, b = 100;
        Integer c = 200, d = 200;
        System.out.println(a == b); // true - cached
        System.out.println(c == d); // false - NOT cached
        System.out.println(c.equals(d)); // true - always use equals() for wrapper value comparison

        String numStr = "42";
        int parsed = Integer.parseInt(numStr);
        System.out.println(parsed + 8); // 50
    }
}
```

## 7. Edge Cases

| Case                                                                                        | Result                                                                                                                      |
|---------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| <code>Integer a = 127; Integer b = 127; a == b</code>                                       | <code>true</code> (within cache range -128 to 127)                                                                          |
| <code>Integer a = 128; Integer b = 128; a == b</code>                                       | <code>false</code> (outside cache range)                                                                                    |
| Unboxing a <code>null</code> <code>Integer</code> ( <code>int x = someNullInteger;</code> ) | <code>NullPointerException</code> — a very common, sneaky production bug                                                    |
| <code>Integer.parseInt("abc")</code>                                                        | <code>NumberFormatException</code>                                                                                          |
| Comparing <code>Double.NaN == Double.NaN</code>                                             | <code>false</code> — <code>NaN</code> is never equal to anything, including itself; use <code>Double.isNaN()</code> instead |

## 8. Common Mistakes

- Using `==` to compare wrapper objects, relying on the `Integer` cache behavior — works "by accident" for small values, breaks for larger ones.
- Unboxing a `null` wrapper reference, causing an unexpected `NullPointerException` (common when a `Map<String, Integer>` returns `null` for a missing key and the caller auto-unboxes it directly).
- Excessive autoboxing/unboxing in performance-critical loops — causes unnecessary object creation/GC pressure compared to using primitive arrays directly.

## 9. Best Practices

- Always use `.equals()` (or `Objects.equals()`) for wrapper value comparison, never `==`.
- Guard against `null` before unboxing, especially values retrieved from `Maps` or nullable fields.
- Prefer primitive types for tight, performance-sensitive loops; only box when a `Collection`/`Generic` API genuinely requires it.

## 10. Frequently Asked Interview Questions

- What is autoboxing?** Automatic conversion of a primitive value to its corresponding wrapper object, inserted by the compiler.
- Why does `Integer a = 100; Integer b = 100; a == b` return `true`, but the same with `200` returns `false`?** Java caches `Integer` objects for values -128 to 127 via `Integer.valueOf()`; values in that range share the same cached object reference, values outside create new objects each time.
- What exception occurs when unboxing a `null` wrapper?** `NullPointerException`.
- How should you compare two wrapper objects for value equality?** Always use `.equals()`, never `==` (which compares references, unreliable due to caching behavior).
- Why does Java have wrapper classes at all?** To let primitives participate in object-oriented APIs (`Collections`, `Generics`) that require `Object` references, since Java's primitives aren't objects themselves.

## 11. Revision Notes

### WRAPPER CLASSES — CHEAT SHEET

=====

byte->Byte short->Short int->Integer long->Long  
float->Float double->Double char->Character boolean->Boolean

Autoboxing: primitive -> wrapper (compiler-inserted valueOf())

Unboxing: wrapper -> primitive (compiler-inserted xxxValue())

INTEGER CACHE: -128 to 127 -> == returns true (same cached object)

outside range -> == returns false (new objects each time)

ALWAYS use .equals() for wrapper value comparison, NEVER ==

GOTCHA: unboxing a null wrapper -> NullPointerException

## 12. Homework

- 1 Prove the Integer cache boundary experimentally: test == for values 127/128 and -128/-129, and explain each result.
- 2 Write code that retrieves a missing key from a Map<String, Integer> and unboxes it directly into an int, observing the resulting NullPointerException.

---

(End of Topic 28: Wrapper Classes.)

---

## TOPIC 29: STRING

---

### 1. Introduction

**What is it?** String is a final, immutable class in java.lang representing a sequence of characters — one of the most heavily used types in all of Java, with special JVM-level optimizations (the String Constant Pool) due to its ubiquity.

**Why immutability:** Strings are used as HashMap keys, shared across threads, and cached extensively (class names, file paths). If String were mutable, changing one reference's content could corrupt every other reference sharing that same object, break hash-based collections, and introduce serious thread-safety and security vulnerabilities (e.g., a mutable String passed as a file path or SQL query could be changed after a security check but before use).

**Interview definition:** "String is an immutable, final class representing a sequence of char values, where any apparent modification operation actually returns a brand-new String object, and literal instances are interned in a JVM-managed String Constant Pool to enable safe sharing and memory efficiency."



## 2. The String Constant Pool (SCP)

```
String a = "Hello"; // literal -> placed in the String Constant Pool (SCP)
String b = "Hello"; // literal -> REUSES the SAME pooled object
String c = new String("Hello"); // 'new' -> FORCES a brand-new object on the regular Heap, NOT pooled

System.out.println(a == b); // true - both reference the SAME pooled object
System.out.println(a == c); // false - c is a distinct Heap object
System.out.println(a.equals(c)); // true - same CONTENT

STRING CONSTANT POOL (special area, part of the Heap since Java 7+)
+-----+
| "Hello" | <--- a and b BOTH point here
+-----+

REGULAR HEAP
+-----+
| "Hello" | <--- c points here (separate object, from 'new')
+-----+
```

**Note:** The String Constant Pool moved from PermGen into the main Heap starting Java 7, enabling it to be properly garbage collected like any other Heap content.

## 3. Why String is Immutable — Deep Reasons

- 1 **Security:** Strings are used for file paths, network connections, class names, and SQL queries — immutability prevents them from being altered after a validation check but before use (a "time-of-check to time-of-use" attack vector).
- 2 **Thread-safety:** An immutable object can be freely shared across threads with zero synchronization needed — no risk of one thread modifying it while another reads it.
- 3 **HashCode caching:** Since `String`'s content never changes, its `hashCode()` can be computed ONCE and cached, making `String` extremely fast as a `HashMap` key.
- 4 **String pooling safety:** Pooling (sharing one object across many references) is only safe if that shared object can never be mutated by one reference and unexpectedly affect all the others.

## 4. Key Methods

| Method                                                | Purpose                                                                                                  | Time Complexity                   |
|-------------------------------------------------------|----------------------------------------------------------------------------------------------------------|-----------------------------------|
| <code>length()</code>                                 | Number of characters                                                                                     | O(1)                              |
| <code>charAt(int i)</code>                            | Character at index                                                                                       | O(1)                              |
| <code>substring(int begin, int end)</code>            | New String from a range                                                                                  | O(n) (copies characters, Java 7+) |
| <code>concat(String s) / +</code>                     | Concatenation (creates a NEW String)                                                                     | O(n)                              |
| <code>equals(Object o)</code>                         | Content equality                                                                                         | O(n)                              |
| <code>equalsIgnoreCase(String s)</code>               | Case-insensitive content equality                                                                        | O(n)                              |
| <code>compareTo(String s)</code>                      | Lexicographic comparison                                                                                 | O(n)                              |
| <code>indexOf(String s)</code>                        | First occurrence index, -1 if not found                                                                  | O(n×m) naive                      |
| <code>replace(CharSequence a, CharSequence b)</code>  | Returns a new String with replacements                                                                   | O(n)                              |
| <code>split(String regex)</code>                      | Splits into a <code>String[]</code> by a regex pattern                                                   | O(n)                              |
| <code>trim()</code> / <code>strip()</code> (Java 11+) | Removes leading/trailing whitespace ( <code>strip()</code> is Unicode-aware, <code>trim()</code> is not) | O(n)                              |
| <code>toCharArray()</code>                            | Converts to a <code>char[]</code>                                                                        | O(n)                              |
| <code>String.format(...)</code>                       | Formatted string construction (like <code>printf</code> )                                                | O(n)                              |
| <code>String.join(delimiter, elements)</code>         | Joins multiple strings with a delimiter                                                                  | O(n)                              |
| <code>isBlank()</code> (Java 11+)                     | True if empty OR only whitespace                                                                         | O(n)                              |
| <code>intern()</code>                                 | Forces a String into the String Constant Pool, returning the pooled reference                            | O(n) (pool lookup)                |

## 5. Code Examples

```
public class StringDemo {
    public static void main(String[] args) {
        String s1 = "Hello";
        String s2 = "Hello";
        String s3 = new String("Hello");
        String s4 = s3.intern(); // forces s3's content into the pool

        System.out.println(s1 == s2); // true
        System.out.println(s1 == s3); // false
        System.out.println(s1 == s4); // true - intern() returns the pooled reference

        String result = s1.concat(" World"); // creates a NEW String, s1 itself is UNCHANGED
        System.out.println(s1); // still "Hello"
        System.out.println(result); // "Hello World"
    }
}
```

## 6. Edge Cases

| Case                                                                            | Result                                                                                                                                                                                                             |
|---------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| String concatenation in a loop using <code>+</code>                             | Creates a NEW String object on EVERY iteration — $O(n^2)$ total cost for $n$ concatenations; use <code>StringBuilder</code> instead                                                                                |
| <code>"" == new String("")</code>                                               | <code>false</code> — different objects despite identical (empty) content                                                                                                                                           |
| Comparing Strings with <code>==</code> across compile-time-constant expressions | <code>"a" + "b" == "ab"</code> is actually <code>true</code> — the compiler performs <b>constant folding</b> at COMPILE time, producing the same pooled literal; this is NOT true for runtime-concatenated strings |
| <code>null</code> String concatenation ( <code>"Value: " + null</code> )        | Produces the literal text <code>"Value: null"</code> — no exception                                                                                                                                                |
| Very long strings (millions of characters)                                      | Can consume significant Heap memory since each character (Java 9+ uses compact Strings — Latin-1 byte array when possible, UTF-16 otherwise) still adds up                                                         |

## 7. Common Mistakes

- Using `==` to compare String content instead of `.equals()` — extremely common beginner bug, works "by accident" for literals due to pooling but fails for `new String(...)`.
- Concatenating Strings with `+` inside loops — a well-known performance anti-pattern; use `StringBuilder` (next topic).
- Assuming `trim()` and `strip()` are identical — `strip()` (Java 11+) is Unicode-whitespace-aware, `trim()` only handles characters `≤`.

## 8. Best Practices

- Always use `.equals()` for String content comparison.
- Use `StringBuilder` for any String building involving loops or many concatenations.
- Use text blocks (`"""..."""`, Java 15+) for multi-line String literals instead of manual `+` concatenation with `\n`.
- Prefer `String.format()` or text blocks over complex chained concatenation for readability.

## 9. Production Usage

- SQL query construction (though `PreparedStatement` parameterization, NOT string concatenation, is the correct production practice to avoid SQL injection).
- Configuration values, log messages, JSON/XML payloads — Strings are the universal currency of nearly all I/O and serialization in Java applications.

## 10. Frequently Asked Interview Questions

- 1 **Is String mutable or immutable?** Immutable — every apparent modification returns a brand-new `String` object; the original is never changed.

- 2 **Why is String immutable in Java?** For security (prevents tampering between validation and use), thread-safety (safe sharing without synchronization), hashCode caching (fast HashMap key performance), and safe String pooling.
- 3 **What is the String Constant Pool?** A special area (within the Heap since Java 7) where String literals are stored and reused, so identical literals share the same object reference.
- 4 **What's the difference between "Hello" and new String("Hello")?** The literal is pooled/reused; new String(...) always forces a brand-new, non-pooled object on the Heap, even if identical content already exists in the pool.
- 5 **What does intern() do?** Forces a String's content into the String Constant Pool (or returns the existing pooled reference if already present), useful for reclaiming memory-sharing benefits for dynamically-built strings.
- 6 **Why is concatenating Strings with + in a loop inefficient?** Each + creates a brand-new String object (since Strings are immutable), leading to  $O(n^2)$  total time/memory for  $n$  concatenations — `StringBuilder` avoids this by mutating an internal buffer.
- 7 **Is "a" + "b" == "ab" true or false, and why?** true — because both are compile-time constants, the compiler performs constant folding, producing the identical pooled literal; this does NOT hold for runtime-computed concatenations.

## 11. Revision Notes

### STRING – CHEAT SHEET

=====

IMMUTABLE, final class. Every "modification" returns a NEW String object.

STRING CONSTANT POOL (SCP): literals are pooled/reused  
"Hello" == "Hello" -> true (same pooled object)  
"Hello" == new String("Hello") -> false (forced new object)  
.intern() -> forces/retrieves the pooled reference

WHY IMMUTABLE: security, thread-safety, hashCode caching, safe pooling

ALWAYS use .equals() for content comparison, NEVER ==  
String + in a LOOP ->  $O(n^2)$ , use `StringBuilder` instead  
compile-time constant "a"+"b" == "ab" -> true (constant folding)

## 12. Homework

- 1 Prove "Hello" == "Hello" is true but "Hello" == new String("Hello") is false, then fix the second case using `.intern()`.
- 2 Time (roughly, using `System.nanoTime()`) concatenating 10,000 strings with + in a loop versus using `StringBuilder`, and compare.

---

(End of Topic 29: String.)

---

# TOPIC 30: STRINGBUILDER

## 1. Introduction

**What is it?** `StringBuilder` is a **mutable** sequence of characters, designed specifically to efficiently build/modify String-like content without creating a new object on every change — the standard fix for `String`'s  $O(n^2)$  concatenation-in-a-loop problem.

**Interview definition:** "`StringBuilder` is a mutable, non-thread-safe character sequence class backed by an internal resizable `char[]` (or `byte[]` for compact Strings) buffer, providing efficient in-place append/insert/delete operations, typically converted to an immutable `String` via `toString()` once building is complete."

## 2. Internal Working

```
StringBuilder sb = new StringBuilder(); // default initial capacity: 16 characters
sb.append("Hello"); // mutates the INTERNAL buffer, no new object created
sb.append(" World");
String result = sb.toString(); // creates ONE final immutable String
```

- Internally backed by a resizable array — when capacity is exceeded, a new, larger array is allocated (typically  $2 \times \text{currentCapacity} + 2$ ) and existing characters are copied over (an amortized  $O(1)$  per-append cost overall, similar to `ArrayList`'s growth strategy).
- `append()` operations modify the SAME object in place — no new `StringBuilder` object per call, unlike `String`'s `+`.

## 3. Key Methods

| Method                                             | Purpose                                                 | Time Complexity                       |
|----------------------------------------------------|---------------------------------------------------------|---------------------------------------|
| <code>append(...)</code>                           | Adds content to the end (overloaded for all types)      | $O(1)$ amortized                      |
| <code>insert(int offset, ...)</code>               | Inserts content at a specific position                  | $O(n)$ (shifts subsequent characters) |
| <code>delete(int start, int end)</code>            | Removes a range of characters                           | $O(n)$                                |
| <code>deleteCharAt(int index)</code>               | Removes one character                                   | $O(n)$                                |
| <code>reverse()</code>                             | Reverses the entire sequence in place                   | $O(n)$                                |
| <code>replace(int start, int end, String s)</code> | Replaces a range with new content                       | $O(n)$                                |
| <code>toString()</code>                            | Converts to an immutable <code>String</code>            | $O(n)$                                |
| <code>capacity()</code>                            | Current internal buffer size                            | $O(1)$                                |
| <code>setLength(int n)</code>                      | Truncates or pads the sequence to length <code>n</code> | $O(1)/O(n)$                           |

## 4. Code Examples

```
public class StringBuilderDemo {
    public static void main(String[] args) {
        StringBuilder sb = new StringBuilder("Hello");
        sb.append(" World");
        sb.insert(5, ",");
        System.out.println(sb); // "Hello, World"

        sb.reverse();
        System.out.println(sb); // "dlrow ,olleH"

        StringBuilder loopBuilder = new StringBuilder();
        for (int i = 0; i < 5; i++) {
            loopBuilder.append(i).append(","); // efficient - mutates ONE object
        }
        System.out.println(loopBuilder); // "0,1,2,3,4,"
    }
}
```

## 5. `StringBuilder` vs `String` vs `StringBuffer` — Comparison Table

| Aspect        | <code>String</code>                                   | <code>StringBuilder</code>                                 | <code>StringBuffer</code>                                                                                                  |
|---------------|-------------------------------------------------------|------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| Mutability    | Immutable                                             | Mutable                                                    | Mutable                                                                                                                    |
| Thread-safety | N/A (immutable, inherently safe)                      | <b>NOT</b> thread-safe                                     | Thread-safe (synchronized methods)                                                                                         |
| Performance   | Slow for repeated modification (new object each time) | Fast (no synchronization overhead)                         | Slower than <code>StringBuilder</code> (synchronization overhead), even in single-threaded use                             |
| Introduced    | Java 1.0                                              | Java 5                                                     | Java 1.0                                                                                                                   |
| When to use   | Fixed/rarely-changing text, HashMap keys              | Single-threaded string building (the DEFAULT choice today) | Multi-threaded string building (rare in practice — usually better to synchronize externally or avoid shared mutable state) |

(Full `StringBuffer`-specific detail in the next topic.)

## 6. Edge Cases

| Case                                                                                       | Result                                                                                                                          |
|--------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| <code>StringBuilder</code> shared across multiple threads without external synchronization | Race conditions possible (lost appends, corrupted content) — NOT thread-safe by design                                          |
| <code>sb.charAt(sb.length())</code>                                                        | <code>StringIndexOutOfBoundsException</code>                                                                                    |
| Chained method calls<br>( <code>sb.append("a").append("b").reverse()</code> )              | Fully supported — each method returns <code>this</code> (the same <code>StringBuilder</code> ), enabling fluent chaining        |
| Very large capacity growth in a tight loop                                                 | Still efficient overall (amortized $O(1)$ append) due to the doubling growth strategy, same principle as <code>ArrayList</code> |

## 7. Common Mistakes

- Using `StringBuilder` across multiple threads assuming it's safe (it is NOT) — use `StringBuffer` or proper external synchronization if genuinely needed in a concurrent context.
- Calling `toString()` repeatedly inside a loop instead of once at the very end — defeats the purpose of using `StringBuilder` in the first place.
- Forgetting method chaining works because each mutator method returns `this`.

## 8. Best Practices

- Default to `StringBuilder` (not `StringBuffer`) for all single-threaded string building — which is the vast majority of real-world use cases.
- Pre-size the initial capacity (`new StringBuilder(expectedSize)`) when the approximate final length is known, avoiding repeated internal resizing.
- Call `.toString()` exactly once, after all building is complete.

## 9. Production Usage

- The Java compiler itself automatically uses `StringBuilder` internally to implement `String +` concatenation involving variables (see [[Java Compilation Process|Topic 3]]) — proving it's the officially sanctioned, performant mechanism.
- Building dynamic SQL fragments, JSON payloads, log messages, and report text in loops are all standard `StringBuilder` use cases across virtually every Java codebase.

## 10. Frequently Asked Interview Questions

- 1 **What is `StringBuilder`?** A mutable character sequence class designed for efficient `String` building/modification without creating a new object per change.
- 2 **Is `StringBuilder` thread-safe?** No — it has zero internal synchronization; `StringBuffer` is the thread-safe equivalent (at a performance cost).
- 3 **Why is `StringBuilder` faster than repeated `String` concatenation?** It mutates one internal buffer in place (amortized  $O(1)$  per append) instead of allocating a brand-new `String` object on every single concatenation.
- 4 **How does the compiler use `StringBuilder` internally?** It automatically rewrites `String +` concatenation involving variables into chained `StringBuilder.append()` calls behind the scenes.
- 5 **Why do `StringBuilder` methods like `append()` return `this`?** To enable fluent method chaining (`sb.append("a").append("b")`).

## 11. Revision Notes

### STRINGBUILDER — CHEAT SHEET

=====

MUTABLE character sequence, backed by a resizable internal buffer  
NOT thread-safe (use `StringBuffer` if genuinely needed across threads)

`append()/insert()/delete()/reverse()` -> all mutate the SAME object in place  
`toString()` -> converts to an immutable `String`, call ONCE at the end

Amortized  $O(1)$  per append (doubling growth strategy, like `ArrayList`)  
Compiler AUTOMATICALLY uses `StringBuilder` for `String` + concatenation involving variables  
DEFAULT choice for string building in single-threaded code

## 12. Homework

- 1 Build a comma-separated string from an array of 1000 integers using `StringBuilder`, then compare its speed/simplicity against manual `String` + concatenation.
- 2 Chain at least 4 `StringBuilder` methods (`append`, `insert`, `reverse`, `delete`) in one fluent statement and predict the final output before running it.

---

(End of Topic 30: `StringBuilder`.)

---

# TOPIC 31: STRINGBUFFER

---

## 1. Introduction

**What is it?** `StringBuffer` is the original (Java 1.0) mutable character sequence class — functionally almost identical to `StringBuilder`, but with every method marked `synchronized`, making it safe for concurrent access by multiple threads at the cost of synchronization overhead.

**Why it still exists alongside `StringBuilder`:** `StringBuffer` predates `StringBuilder` by nearly a decade (Java 1.0 vs Java 5). When Sun added `StringBuilder` in Java 5, they kept `StringBuffer` for backward compatibility and for the (relatively rare) genuine cross-thread mutable-buffer use case.

## 2. `StringBuffer` vs `StringBuilder` — The Only Real Difference

```
// StringBuffer's internal method (conceptually):
public synchronized StringBuffer append(String s) { ... }

// StringBuilder's internal method (conceptually):
public StringBuilder append(String s) { ... } // NO synchronized keyword
```

Every single method on `StringBuffer` acquires the object's intrinsic lock before executing — meaning even in a purely single-threaded program, `StringBuffer` pays for lock acquisition/release overhead it doesn't actually need, making it consistently **slower** than `StringBuilder` outside genuinely concurrent scenarios.



### 3. Code Example

```
public class StringBufferDemo {
    public static void main(String[] args) {
        StringBuffer sb = new StringBuffer("Hello");
        sb.append(" World");
        System.out.println(sb); // "Hello World" - same API surface as StringBuilder
    }
}
```

### 4. When `StringBuffer` Is Actually Justified

- Multiple threads genuinely need to append/modify the SAME shared buffer concurrently, and you specifically want per-METHOD-CALL atomicity (note: even then, `StringBuffer`'s synchronization only guarantees each INDIVIDUAL method call is atomic — a sequence of multiple calls like `sb.append(a); sb.append(b);` from one thread can still interleave unpredictably with another thread's calls in between, so it does NOT automatically guarantee full compound-operation safety).
- In practice, most modern concurrent designs prefer building strings independently per thread (e.g., using `StringBuilder` locally, or a thread-safe `StringJoiner/Collectors.joining()` in streams) rather than sharing one mutable `StringBuffer` — making genuine `StringBuffer` use fairly rare in modern code.

### 5. Edge Cases

| Case                                                                                           | Result                                                                                                                                                            |
|------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Two threads calling <code>append()</code> on the same <code>StringBuffer</code> simultaneously | Each individual <code>append()</code> call is atomic (no corrupted internal state), but the ORDER of interleaved calls between threads is still non-deterministic |
| Using <code>StringBuffer</code> in a purely single-threaded method                             | Works correctly, just measurably slower than <code>StringBuilder</code> due to unnecessary lock overhead                                                          |

### 6. Common Mistakes

- Using `StringBuffer` by habit/legacy convention in new single-threaded code where `StringBuilder` would be strictly better — a common outdated-code-style leftover.
- Assuming `StringBuffer` makes an entire multi-step string-building SEQUENCE thread-safe — it only guarantees each individual method call is atomic, not a whole compound sequence of calls across threads.

### 7. Best Practices

- Default to `StringBuilder` for virtually all new code; reach for `StringBuffer` only when you've specifically confirmed genuine concurrent shared-buffer access is required AND per-call atomicity is sufficient for your correctness needs.

### 8. Frequently Asked Interview Questions

- 1 **What's the difference between `StringBuffer` and `StringBuilder`?** `StringBuffer`'s methods are `synchronized` (thread-safe but slower); `StringBuilder`'s are not (faster, but not thread-safe) — otherwise nearly identical APIs.
- 2 **Which one should you use by default?** `StringBuilder`, for the vast majority of single-threaded use cases — it's strictly faster with no correctness downside outside genuine concurrency.
- 3 **Does `StringBuffer` guarantee a whole sequence of operations is thread-safe?** No — only each INDIVIDUAL method call is atomic; a multi-call sequence from one thread can still interleave unpredictably with another thread's calls.
- 4 **Why does `StringBuffer` still exist if `StringBuilder` is usually better?** Backward compatibility (it predates `StringBuilder` by years) and the narrower genuine use case of needing per-call thread-safety on a shared buffer.

## 9. Revision Notes

STRINGBUFFER – CHEAT SHEET

=====

Same API as `StringBuilder`, but EVERY method is ``synchronized`` (thread-safe, slower due to lock overhead)

DEFAULT to `StringBuilder` unless you specifically need thread-safe shared buffer access (rare in modern code - prefer thread-local building instead)

GOTCHA: `synchronized` per-METHOD only - does NOT make a multi-call SEQUENCE atomic across threads

## 10. Homework

- 1 Benchmark (roughly) appending 100,000 strings using `StringBuilder` vs `StringBuffer` in a single-threaded loop and compare timings.
- 2 Explain, in your own words, why `StringBuffer`'s synchronization does NOT protect a multi-step sequence of operations across threads, only each individual call.

---

(End of Topic 31: `StringBuffer`.)

---

# TOPIC 32: IMMUTABLE OBJECTS

---

## 1. Introduction

**What is it?** An immutable object is one whose observable state **cannot change** after construction — every field is fixed for the lifetime of the object. `String` (see [\[\[String|Topic 29\]\]](#)) is Java's most famous built-in example; wrapper classes, `LocalDate`, and `BigInteger`/`BigDecimal` are all immutable too.

**Why it matters:** Immutable objects are inherently **thread-safe** (no synchronization needed — nothing can change, so there's nothing to race over), simpler to reason about (no "who changed this and when" bugs), and safe to use freely as `HashMap` keys or in `Sets`.

**Interview definition:** "An immutable object is one whose entire observable state is fixed at construction time and cannot be modified afterward, typically implemented via `final` fields, no setters, defensive copying of mutable inputs/outputs, and a `final` class to prevent subclasses from breaking the immutability contract."

## 2. Rules to Create a Truly Immutable Class

```
public final class ImmutablePoint { // 1. class is FINAL (prevents subclass from adding mutability)

    private final int x; // 2. all fields are PRIVATE and FINAL
    private final int y;

    public ImmutablePoint(int x, int y) { // 3. set values only in the constructor
        this.x = x;
        this.y = y;
    }

    public int getX() { return x; } // 4. only GETTERS, NO setters
    public int getY() { return y; }

    public ImmutablePoint translate(int dx, int dy) { // 5. "modifier" methods return a NEW object
        return new ImmutablePoint(this.x + dx, this.y + dy);
    }
}
```

### *The 5 Rules (Full Checklist)*

- 1 Make the class `final` (prevents a subclass from adding mutable behavior or breaking invariants).
- 2 Make all fields `private` and `final`.
- 3 Don't provide any setters.
- 4 If a field is a mutable object/array/collection, make a **defensive copy** on both input (constructor) AND output (getter) — otherwise external code can still mutate the "immutable" object's internal state indirectly.
- 5 Ensure no method allows the object's internal state to be modified after construction — any transformation returns a NEW object instead.

### *Defensive Copying Example (Rule 4 — Frequently Tested)*

```
public final class ImmutableEvent {
    private final String name;
    private final List<String> attendees;

    public ImmutableEvent(String name, List<String> attendees) {
        this.name = name;
        this.attendees = new ArrayList<>(attendees); // DEFENSIVE COPY on input!
    }

    public List<String> getAttendees() {
        return Collections.unmodifiableList(attendees); // DEFENSIVE (unmodifiable) view on output!
    }
}
```

Without BOTH defensive copies, external code could mutate the caller's original list (affecting the "immutable" object after construction) OR mutate the list returned by the getter (breaking immutability from the outside) — a very commonly missed detail even by experienced developers.

### 3. Real Life Analogy

- **Bank:** A cleared, finalized transaction receipt is immutable — once issued, its amount/date/reference number can never be altered; any "correction" requires issuing an entirely NEW receipt (a reversal + new transaction), never editing the original.
- **Library:** A published book edition is immutable once printed — if an error is found, the publisher issues a NEW edition (a new object), rather than modifying every already-printed physical copy.

### 4. Edge Cases

| Case                                                                                               | Result                                                                                                                                                                   |
|----------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Immutable class holds a reference to a mutable object WITHOUT defensive copying                    | NOT truly immutable — external code can still mutate the contained object's state through the shared reference                                                           |
| Reflection ( <code>setAccessible(true)</code> ) used to forcibly modify a <code>final</code> field | Technically POSSIBLE via reflection, bypassing normal immutability guarantees — a known, deliberate JVM security-manager-dependent edge case, not a normal usage concern |
| An immutable class extended by a NON-final subclass adding mutable state                           | Breaks the immutability contract for that "impostor" subtype — this is exactly why the class itself must be <code>final</code>                                           |

### 5. Common Mistakes

- Making fields `final` but forgetting that a `final` reference to a MUTABLE object (like a `List`) still allows that object's CONTENTS to change — `final` only prevents reassigning the reference, not mutating what it points to.
- Skipping defensive copies for constructor inputs and getter outputs — the single most common way "immutable" classes turn out not to actually be immutable.
- Not marking the class `final`, allowing a subclass to add mutable fields/methods that violate the parent's immutability guarantee.

### 6. Best Practices

- Follow all 5 rules above rigorously for any class intended to be truly immutable.
- Prefer Java 16+ **Records** (covered in their own topic) for simple immutable data carriers — they handle most of this boilerplate (`final` fields, constructor, accessors, `equals/hashCode/toString`) automatically, though defensive copying of mutable fields inside a record still needs to be done manually in a compact constructor if needed.
- Use immutable objects by default for value types (money, coordinates, dates) unless there's a specific, deliberate reason for mutability.

## 7. Production Usage

- `String`, all wrapper classes, `java.time.*` (`LocalDate`, `Instant`, etc.), `BigDecimal`/`BigInteger` are all immutable in the JDK itself.
- DTOs and value objects in well-designed enterprise systems are increasingly built as immutable (often via Records since Java 16) specifically to eliminate an entire class of concurrency and accidental-mutation bugs.

## 8. Frequently Asked Interview Questions

- 1 **What makes an object immutable?** Its entire observable state is fixed at construction and cannot change afterward — no setters, `final` fields, defensive copying of any mutable inputs/outputs, and typically a `final` class.
- 2 **Why are immutable objects inherently thread-safe?** Because there's no mutable state to race over — multiple threads can freely read the same immutable object with zero synchronization needed.
- 3 **Does making a field `final` alone guarantee immutability?** No — `final` only prevents REASSIGNING the reference; if that reference points to a mutable object (like a `List`), its internal contents can still be changed unless defensively copied.
- 4 **Why must a defensive copy happen on BOTH constructor input and getter output?** Without the constructor copy, the caller's original mutable object could be changed after construction, altering the "immutable" object's state; without the getter copy, external code could mutate the internal object directly through the returned reference.
- 5 **Why should an immutable class be declared `final`?** To prevent a subclass from adding mutable fields/behavior or overriding methods in a way that breaks the immutability guarantee the base class promises.

## 9. Revision Notes

### IMMUTABLE OBJECTS – CHEAT SHEET

=====

#### 5 RULES:

1. class is `FINAL`
2. all fields `PRIVATE + FINAL`
3. NO setters
4. DEFENSIVE COPY any mutable field on BOTH constructor input AND getter output
5. "modifying" methods return a NEW object instead of mutating

WHY: thread-safety (no sync needed), safe `HashMap` keys, simpler reasoning

GOTCHA: `final` reference != immutable contents (a `final List` reference can still have its `ELEMENTS` changed unless defensively copied)

JDK EXAMPLES: `String`, Integer/wrapper classes, `LocalDate`, `BigDecimal`

MODERN SHORTCUT: Records (Java 16+) auto-generate most immutability boilerplate

## 10. Homework

- 1 Build an immutable `ImmutableEvent` class (as in Section 2) WITHOUT defensive copying first, prove externally that its "immutable" state can still be mutated via the shared list reference, then fix it with proper defensive copies.
- 2 List 3 classes from the JDK (other than `String`) that are immutable, and explain why immutability makes sense for each.

---

(End of Topic 32: Immutable Objects.)

---

## TOPIC 33: ENUMS

---

### 1. Introduction

**What is it?** An `enum` (enumeration) is a special class type representing a **fixed set of named constants** — e.g., days of the week, order statuses, HTTP methods — providing type safety that plain `int/String` constants cannot.

**Why introduced:** Before enums (Java 5), developers used `int` or `String` constants (`public static final int MONDAY = 0;`) — this "typed as `int`" approach allowed **any** `int` to be passed where a day was expected (no compile-time safety), had no meaningful `toString()`, and allowed invalid values entirely. Enums fix all of this by making the set of valid values a genuine, closed, type-checked set.

**Interview definition:** "An `enum` is a special Java type whose instances are restricted to a fixed set of predefined, named constants, implicitly extending `java.lang.Enum`, providing type safety, and capable of having fields, constructors, and methods like any other class."

### 2. Real Life Analogy

- **Restaurant:** An `OrderStatus` enum (`RECEIVED`, `PREPARING`, `READY`, `DELIVERED`) represents the **ONLY** valid states an order can be in — unlike a raw `String status = "Delvired";` (typo!), the compiler simply won't allow an invalid enum value to exist.
- **Bank:** An `AccountType` enum (`SAVINGS`, `CURRENT`, `FIXED_DEPOSIT`) ensures every account is unambiguously one of exactly these types, with no possibility of an invalid/misspelled type slipping through.

### 3. Syntax — From Simple to Advanced

#### 3.1 Simple Enum

```
enum Day { MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY, SUNDAY }
```

## 3.2 Enum with Fields, Constructor, and Methods

```
enum Planet {  
    MERCURY(3.303e+23, 2.4397e6),  
    VENUS(4.869e+24, 6.0518e6),  
    EARTH(5.976e+24, 6.37814e6);  
  
    private final double mass; // kg  
    private final double radius; // meters  
  
    Planet(double mass, double radius) { // enum constructors are IMPLICITLY private  
        this.mass = mass;  
        this.radius = radius;  
    }  
  
    double surfaceGravity() {  
        final double G = 6.67300E-11;  
        return G * mass / (radius * radius);  
    }  
}
```

## 3.3 Enum with Abstract Methods (Constant-Specific Bodies)

```
enum Operation {  
    ADD { public int apply(int a, int b) { return a + b; } },  
    SUBTRACT { public int apply(int a, int b) { return a - b; } };  
  
    public abstract int apply(int a, int b); // each constant provides its OWN implementation  
}
```

## 4. Internal Working

- Every enum implicitly extends `java.lang.Enum<T>` (so an enum CANNOT extend any other class — single inheritance rule still applies, but enums already "use up" their one extends slot).
- Every enum constant is internally a `public static final` instance of the enum type — created exactly ONCE, at class loading time.
- Enum constructors are implicitly `private` — you can NEVER call `new Planet(...)` externally; only the constants declared inside the enum body exist.

## 5. Key Methods (inherited from `java.lang.Enum`)

| Method                            | Purpose                                                                                                                        |
|-----------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| <code>name()</code>               | Returns the exact constant name as declared (e.g., "MONDAY")                                                                   |
| <code>ordinal()</code>            | Returns the constant's position (0-indexed) in declaration order                                                               |
| <code>values()</code>             | Static method (compiler-generated) returning an array of ALL constants, in declaration order                                   |
| <code>valueOf(String name)</code> | Static method converting a String back to its matching enum constant, throws <code>IllegalArgumentException</code> if no match |
| <code>compareTo(E other)</code>   | Compares based on <code>ordinal()</code>                                                                                       |

## 6. Enums in `switch` Statements

```
Day today = Day.WEDNESDAY;
switch (today) {
case MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY -> System.out.println("Weekday");
case SATURDAY, SUNDAY -> System.out.println("Weekend");
}
```

## 7. Code Examples

```
public class EnumDemo {
public static void main(String[] args) {
Day d = Day.valueOf("MONDAY");
System.out.println(d.name()); // "MONDAY"
System.out.println(d.ordinal()); // 0

for (Day day : Day.values()) { // values() - iterate all constants
System.out.println(day);
}
}
enum Day { MONDAY, TUESDAY, WEDNESDAY }
}
```

## 8. Edge Cases

| Case                                                             | Result                                                                                                                                                                             |
|------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>Day.valueOf("monday")</code> (wrong case)                  | <code>IllegalArgumentException</code> — matching is case-sensitive                                                                                                                 |
| <code>Day.valueOf("INVALID")</code>                              | <code>IllegalArgumentException: No enum constant Day.INVALID</code>                                                                                                                |
| Enum used in a <code>HashMap/HashSet</code>                      | Works correctly and efficiently — <code>EnumMap/EnumSet</code> (specialized, ordinal-array-backed) are even MORE efficient alternatives                                            |
| Reordering enum constants in source code                         | Changes every subsequent constant's <code>ordinal()</code> value — DANGEROUS if <code>ordinal()</code> is persisted/serialized anywhere, since re-ordering silently shifts meaning |
| Relying on <code>ordinal()</code> for business logic/persistence | Fragile — adding/reordering constants breaks previously stored ordinal values; use <code>name()</code> or an explicit field instead for anything persisted                         |

## 9. Common Mistakes

- Using `ordinal()` for persistence/serialization or business logic — extremely fragile, since adding or reordering constants silently changes meaning for all subsequent values.
- Comparing enum values with `.equals()` instead of `==` — while `.equals()` works correctly too, `==` is actually the conventional, safe, and more efficient choice for enums specifically (since each constant is a single, unique, cached instance — unlike general objects where `==` is usually wrong).
- Forgetting enum constructors are implicitly `private`, then trying (and failing) to instantiate additional instances externally.



## 10. Best Practices

- Use `name()` (or an explicit dedicated field) rather than `ordinal()` for anything persisted to a database or serialized externally.
- Use `EnumMap/EnumSet` instead of general `HashMap/HashSet` when keys are enum-typed — more memory-efficient and faster (backed by simple arrays indexed by ordinal internally).
- Prefer constant-specific method bodies (Section 3.3) over external `switch/if-else` chains scattered across the codebase when behavior genuinely differs per constant.

## 11. Production Usage

- Representing fixed domain states: order statuses, HTTP methods, day-of-week scheduling rules, permission levels/roles — enums are the standard, type-safe representation across virtually every enterprise Java codebase.
- JPA/Hibernate `@Enumerated` mapping persists enums to database columns (by name is strongly preferred over by ordinal, for exactly the fragility reason above).

## 12. Frequently Asked Interview Questions

- 1 **What is an enum?** A special type representing a fixed set of named constants, providing type safety over raw `int/String` constants.
- 2 **What class does every enum implicitly extend?** `java.lang.Enum<T>` — meaning an enum cannot extend any other class.
- 3 **Are enum constructors public?** No — they are implicitly `private`; you can never instantiate additional enum instances externally with `new`.
- 4 **What's the difference between `name()` and `ordinal()`?** `name()` returns the constant's declared identifier as a `String`; `ordinal()` returns its zero-indexed declaration position — `ordinal()` is fragile for persistence since reordering constants silently shifts its meaning.
- 5 **Can enums have abstract methods with constant-specific implementations?** Yes — each constant can provide its own method body, a powerful pattern avoiding scattered `switch` statements.
- 6 **Can enums implement interfaces?** Yes — enums can implement interfaces (though they cannot extend another class, since they already implicitly extend `Enum`).
- 7 **Why is `ordinal()` considered risky for persistence?** Because adding, removing, or reordering enum constants in source code silently changes the ordinal values of subsequent constants, corrupting any previously stored data that relied on the old ordinal meanings.

## 13. Revision Notes

### ENUMS — CHEAT SHEET

=====

Fixed set of named constants, type-safe (vs raw int/String constants)  
Implicitly extends `java.lang.Enum<T>` (cannot extend anything else)  
Constructors are **IMPLICITLY PRIVATE** - cannot 'new' additional instances

KEY METHODS: `name()`, `ordinal()`, `values()` (static), `valueOf(String)` (static)

GOTCHA: `ordinal()` is **FRAGILE** for persistence - reordering constants shifts meaning silently. Use `name()` or an explicit field instead.

`EnumMap` / `EnumSet` -> more efficient than `HashMap`/`HashSet` for enum keys  
Constant-specific method bodies -> each constant can override abstract methods

## 14. Homework

- 1 Build an `OrderStatus` enum with a constructor field (e.g., a display label) and a method using it, then iterate all constants with `values()`.
- 2 Demonstrate why relying on `ordinal()` for persistence is dangerous by reordering enum constants and showing previously "saved" ordinal values now point to different constants.

---

(End of Topic 33: Enums.)

---

# TOPIC 34: ANNOTATIONS

---

## 1. Introduction

**What is it?** An annotation is a form of metadata added to Java code (classes, methods, fields, parameters) that provides information to the compiler, tooling, or runtime frameworks, without directly affecting the annotated code's own execution logic.

**Why introduced:** Before annotations (Java 5), metadata was often expressed through separate XML configuration files (e.g., early EJB/Hibernate config) — verbose, error-prone, and disconnected from the actual code they described. Annotations let metadata live directly alongside the code it describes, closer to the source of truth, enabling frameworks to use reflection to discover and act on it at runtime or compile time.

**Interview definition:** "An annotation is a special interface-like construct, declared with `@interface`, that attaches structured metadata to program elements, which can be processed at compile time (by the compiler or annotation processors) or at runtime (via reflection), without altering the annotated code's own semantics directly."

## 2. Real Life Analogy

- **Restaurant:** A sticky note on a dish ("Vegan", "Contains Nuts") is metadata ABOUT the dish — it doesn't change how the dish is cooked, but tells the KITCHEN STAFF (the framework/tooling) important information to act on.
- **College:** A "Fragile — Handle With Care" label on exam papers is metadata that tells the delivery process (framework) how to treat the item, without being part of the exam's actual content.

### 3. Built-in Annotations

| Annotation                            | Purpose                                                                                       |
|---------------------------------------|-----------------------------------------------------------------------------------------------|
| <code>@Override</code>                | Compile-time check that a method genuinely overrides a parent method (see [[Method Overriding |
| <code>@Deprecated</code>              | Marks an element as obsolete, triggering compiler warnings when used                          |
| <code>@SuppressWarnings("...")</code> | Suppresses specific compiler warnings (e.g., "unchecked")                                     |
| <code>@FunctionalInterface</code>     | Compile-time check that an interface has exactly one abstract method                          |
| <code>@SafeVarargs</code>             | Suppresses "unsafe varargs" warnings for methods known to be safe with generic varargs        |

### 4. Meta-Annotations (Annotations That Annotate Other Annotations)

| Meta-Annotation          | Purpose                                                                                                                                                                                                       |
|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>@Retention</code>  | Controls how long the annotation is retained: <code>SOURCE</code> (discarded by compiler), <code>CLASS</code> (in bytecode, not visible at runtime), <code>RUNTIME</code> (visible via reflection at runtime) |
| <code>@Target</code>     | Restricts where the annotation can be applied ( <code>TYPE</code> , <code>METHOD</code> , <code>FIELD</code> , <code>PARAMETER</code> , etc.)                                                                 |
| <code>@Documented</code> | Includes the annotation in generated Javadoc                                                                                                                                                                  |
| <code>@Inherited</code>  | Allows a class-level annotation to be inherited by subclasses                                                                                                                                                 |

### 5. Creating a Custom Annotation

```
import java.lang.annotation.*;

@Retention(RetentionPolicy.RUNTIME) // must be visible at RUNTIME to be read via reflection
@Target(ElementType.METHOD) // can only be applied to METHODS
public @interface TestCase {
    String author() default "Unknown"; // element with a DEFAULT value
    int priority() default 1;
}

class Calculator {
    @TestCase(author = "Alice", priority = 2)
    void testAddition() { }
}
```

## 6. Reading Annotations via Reflection

```
import java.lang.reflect.Method;

public class AnnotationReader {
    public static void main(String[] args) throws Exception {
        Method method = Calculator.class.getMethod("testAddition");
        if (method.isAnnotationPresent(TestCase.class)) {
            TestCase tc = method.getAnnotation(TestCase.class);
            System.out.println("Author: " + tc.author() + ", Priority: " + tc.priority());
        }
    }
}
```

This is EXACTLY how frameworks like JUnit (`@Test`), Spring (`@Component`, `@Autowired`), and Jackson (`@JsonProperty`) work internally — they scan classes via reflection at startup, find their specific annotations, and act accordingly (registering test methods, wiring dependencies, mapping JSON fields).

## 7. Edge Cases

| Case                                                                                                | Result                                                                                                                            |
|-----------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Annotation with <code>@Retention(SOURCE)</code> accessed via reflection at runtime                  | Returns nothing/absent — <code>SOURCE</code> -retention annotations don't even make it into the compiled <code>.class</code> file |
| Annotation applied to a target NOT listed in <code>@Target</code>                                   | Compile-time error                                                                                                                |
| Custom annotation element without a <code>default</code> value, used without providing that element | Compile-time error — non-default elements are mandatory when using the annotation                                                 |
| <code>@Deprecated</code> method still called in new code                                            | Compiles successfully, but triggers a compiler warning (not an error)                                                             |

## 8. Common Mistakes

- Forgetting `@Retention(RetentionPolicy.RUNTIME)` on a custom annotation meant to be read via reflection — without it, the annotation silently won't be visible at runtime, and reflection-based lookups will find nothing.
- Assuming annotations execute code themselves — they are purely metadata; SOME other mechanism (annotation processor, reflection-based framework code) must explicitly read and act on them for anything to actually happen.
- Overusing custom annotations for logic that would be clearer as a plain method call or configuration — annotations are best for cross-cutting, declarative metadata, not core business logic itself.

## 9. Best Practices

- Always specify the correct `@Retention` and `@Target` for custom annotations, matching their intended actual use (compile-time-only vs runtime-reflected).
- Provide sensible `default` values for optional annotation elements to keep usage concise.
- Document custom annotations clearly (using `@Documented`) since their effect is often "invisible" without understanding the framework/processor that consumes them.

## 10. Production Usage

- **Spring:** `@Component`, `@Service`, `@Autowired`, `@RequestMapping` — the ENTIRE Spring dependency injection and MVC routing system is driven by annotations read via reflection/classpath scanning at startup.
- **JPA/Hibernate:** `@Entity`, `@Table`, `@Column`, `@Id` — object-relational mapping metadata lives directly on entity classes.
- **JUnit:** `@Test`, `@BeforeEach`, `@AfterEach` — the test runner discovers and invokes annotated methods via reflection.
- **Jackson:** `@JsonProperty`, `@JsonIgnore` — controls JSON serialization/deserialization mapping.

## 11. Frequently Asked Interview Questions

- 1 **What is an annotation?** Metadata attached to code elements, processed by the compiler, annotation processors, or reflection-based frameworks, without directly altering the annotated code's own runtime logic.
- 2 **What are the three `@Retention` policies?** `SOURCE` (compiler-only, discarded after compilation), `CLASS` (kept in bytecode but not visible via reflection), `RUNTIME` (visible via reflection at runtime).
- 3 **How would a framework like Spring discover which classes are annotated `@Component`?** By scanning the classpath at startup and using reflection to check each candidate class for the presence of that annotation (which requires `RUNTIME` retention).
- 4 **Does an annotation do anything by itself?** No — it's purely metadata; something else (an annotation processor at compile time, or reflection-based code at runtime) must explicitly read and act on it.
- 5 **What does `@Target` control?** Which kinds of program elements (methods, fields, types, parameters, etc.) an annotation is legally allowed to be applied to.
- 6 **Why must a custom annotation intended for framework use (like Spring's `@Autowired`-style annotations) use `RUNTIME` retention?** Because the framework discovers and processes such annotations via reflection AFTER the program is compiled and running — `SOURCE` or `CLASS` retention would make them invisible at that point.

## 12. Revision Notes

### ANNOTATIONS — CHEAT SHEET

=====

Metadata attached to code, read by compiler/reflection/annotation processors  
DOES NOTHING by itself - needs a consumer (framework, reflection code, etc.)

BUILT-IN: `@Override`, `@Deprecated`, `@SuppressWarnings`, `@FunctionalInterface`

META-ANNOTATIONS (annotate annotations):  
`@Retention(SOURCE/CLASS/RUNTIME)` -> how long it's kept  
`@Target(...)` -> where it can be applied  
`@Documented` / `@Inherited`

CUSTOM ANNOTATION: `@interface Name { Type element() default val; }`  
MUST use `@Retention(RUNTIME)` if you need to read it via reflection at runtime

PRODUCTION: Spring (`@Component`, `@Autowired`), JPA (`@Entity`), JUnit (`@Test`)  
ALL work by reflection-scanning for these annotations at startup/runtime

## 13. Homework

- 1 Create a custom `@RUNTIME`-retained annotation with two elements (one with a default value), apply it to a method, and read its values via reflection.
- 2 Deliberately use `@Retention(RetentionPolicy.SOURCE)` on a custom annotation and prove via reflection that it's invisible at runtime.

---

(End of Topic 34: Annotations.)

---

## TOPIC 35: GENERICS

---

### 1. Introduction

**What is it?** Generics allow classes, interfaces, and methods to operate on **type parameters** — placeholders for actual types specified when the generic construct is used — enabling compile-time type safety without duplicating code for every specific type.

**Why introduced (Java 5, 2004):** Before generics, Collections stored raw `Object` references — meaning `List list = new ArrayList(); list.add("hello"); list.add(42);` compiled fine, but retrieving an element required an explicit, error-prone cast (`String s = (String) list.get(0);`), and a wrong cast only failed at `RUNTIME` (`ClassCastException`), not compile time. Generics move this type-checking to compile time, catching an entire class of bugs before the program ever runs.

**Interview definition:** "Generics are a Java 5 language feature enabling classes, interfaces, and methods to be parameterized over types, providing compile-time type safety and eliminating explicit casting, implemented via compiler-enforced type checking followed by type erasure (see [[Java Compilation Process|Topic 3]]) — meaning generic type information does not exist in the compiled bytecode at runtime."

### 2. Real Life Analogy

- **Restaurant:** A generic `Container<T>` is like a labeled storage box that can hold "vegetables only" (`Container<Vegetable>`) or "drinks only" (`Container<Drink>`) — the box shape/mechanism is identical, but a strict label prevents accidentally putting the wrong item type inside, checked at packing time (compile time), not when unpacking (runtime).
- **Library:** A generic `Shelf<T>` can be a `Shelf<Fiction>` or `Shelf<NonFiction>` — same physical shelf design reused, but each specific shelf instance is restricted to exactly one book category.

### 3. Generic Classes

```
class Box<T> { // T is a TYPE PARAMETER (placeholder)
    private T content;
    public void set(T content) { this.content = content; }
    public T get() { return content; }
}

Box<String> stringBox = new Box<>(); // T is bound to String for THIS instance
stringBox.set("Hello");
String s = stringBox.get(); // NO CAST needed - compiler already knows it's a String

Box<Integer> intBox = new Box<>();
intBox.set(42);
```

### 4. Generic Methods

```
static <T> void printArray(T[] array) { // <T> before return type declares a generic METHOD
    for (T item : array) {
        System.out.println(item);
    }
}

printArray(new String[]{"a", "b"}); // T inferred as String
printArray(new Integer[]{1, 2, 3}); // T inferred as Integer
```

### 5. Bounded Type Parameters

```
static <T extends Number> double sumOf(List<T> list) { // T must be Number OR a subtype of Number
    double sum = 0;
    for (T item : list) {
        sum += item.doubleValue(); // safe - guaranteed to have doubleValue() since T extends Number
    }
    return sum;
}
```

- `<T extends Number>` restricts `T` to `Number` or its subtypes (`Integer`, `Double`, etc.) — even though it uses the keyword `extends`, this works for BOTH classes AND interfaces in a generic bound context.
- Multiple bounds: `<T extends Number & Comparable<T>>` — `T` must satisfy BOTH constraints.

### 6. Wildcards — `?`, `? extends`, `? super`

```
void printList(List<?> list) { } // UNBOUNDED wildcard - accepts List of ANY type

void printNumbers(List<? extends Number> list) { } // UPPER BOUNDED - accepts List<Integer>,
List<Double>, etc.
// (read-only safe - "Producer Extends")

void addNumbers(List<? super Integer> list) { // LOWER BOUNDED - accepts List<Integer>, List<Number>,
List<Object>
    list.add(10); // write-safe - "Consumer Super"
}
```

#### ***The PECS Rule (Producer Extends, Consumer Super) — Critical Interview Concept***

If a generic parameter is a PRODUCER (you only READ from it) -> use `? extends T`  
If a generic parameter is a CONSUMER (you only WRITE to it) -> use `? super T`  
If you need BOTH read AND write -> use an exact type, no wildcard

```
List<? extends Number> producer = List.of(1, 2, 3);
Number n = producer.get(0); // READING is fine
// producer.add(5); // COMPILE ERROR! Can't add - compiler doesn't know the EXACT subtype

List<? super Integer> consumer = new ArrayList<Number>();
consumer.add(5); // WRITING is fine (5 is definitely compatible with Integer or any supertype)
// Integer i = consumer.get(0); // COMPILE ERROR-ish (returns Object, needs a cast) - reading isn't
// type-safe
```

## 7. Type Erasure Recap (Full Detail in [[Java Compilation Process|Topic 3]])

```
List<String> stringList = new ArrayList<>();
List<Integer> intList = new ArrayList<>();
System.out.println(stringList.getClass() == intList.getClass()); // true! generics erased at runtime
```

Generics provide **compile-time-only** safety — the JVM has zero knowledge of `<String>` or `<Integer>` at runtime; it all becomes raw `List` with compiler-inserted casts.

## 8. Code Examples

```
import java.util.*;

public class GenericsDemo {
    static <T extends Comparable<T>> T max(List<T> list) {
        T result = list.get(0);
        for (T item : list) {
            if (item.compareTo(result) > 0) {
                result = item;
            }
        }
        return result;
    }

    public static void main(String[] args) {
        List<Integer> nums = List.of(3, 7, 2, 9, 4);
        System.out.println(max(nums)); // 9

        List<String> words = List.of("banana", "apple", "cherry");
        System.out.println(max(words)); // "cherry"
    }
}
```



## 9. Edge Cases

| Case                                                                                    | Result                                                                                                                                                                                    |
|-----------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>new T()</code> inside a generic class                                             | Compile-time error — cannot instantiate a type parameter directly (type erasure means the JVM doesn't know what <code>T</code> actually is at runtime)                                    |
| <code>T[] array = new T[10];</code>                                                     | Compile-time error — cannot create a generic array directly (same erasure reason); must use <code>Object[]</code> internally with an unchecked cast, or <code>(T[]) new Object[10]</code> |
| Raw type usage ( <code>List list = new ArrayList();</code> , no <code>&lt;&gt;</code> ) | Compiles with an "unchecked" warning — legal for backward compatibility, but loses all generic type safety                                                                                |
| <code>list instanceof List&lt;String&gt;</code>                                         | Compile-time error — generic type info doesn't exist at runtime to check against                                                                                                          |
| Two overloaded methods that erase to the identical signature                            | Compile-time error (see <a href="#">[[Java Compilation Process</a>                                                                                                                        |

## 10. Common Mistakes

- Using raw types (`List` instead of `List<String>`) — loses all compile-time type safety, generates "unchecked" warnings.
- Confusing `<? extends T>` (read-only, "Producer Extends") with `<? super T>` (write-only, "Consumer Super") — a very common PECS mix-up.
- Trying to create a generic array (`new T[10]`) directly — not allowed due to type erasure.

## 11. Best Practices

- Avoid raw types entirely in new code — always parameterize.
- Apply the PECS rule (Producer Extends, Consumer Super) when designing methods accepting generic collection parameters, to maximize flexibility for callers.
- Prefer bounded type parameters (`<T extends Comparable<T>>`) when your generic logic genuinely needs specific capabilities from `T`.

## 12. Production Usage

- The ENTIRE Collections Framework (`List<E>`, `Map<K, V>`, `Optional<T>`) is generic.
- Spring's `RestTemplate/WebClient` (`ResponseEntity<T>`), repository interfaces (`JpaRepository<T, ID>`), and virtually all modern API design leverage generics extensively for type-safe, reusable abstractions.

## 13. Frequently Asked Interview Questions

- What are generics?** A language feature allowing classes/methods/interfaces to be parameterized over types, providing compile-time type safety without code duplication per type.

- 2 **What is type erasure?** The process by which generic type information is removed at compile time, replaced by raw types and compiler-inserted casts — meaning generics provide zero runtime type information (see [[Java Compilation Process|Topic 3]]).
- 3 **What is the PECS rule?** "Producer Extends, Consumer Super" — use `? extends T` when only reading from a generic parameter, `? super T` when only writing to it.
- 4 **Why can't you create a generic array (`new T[10]`)?** Because of type erasure — the JVM has no runtime knowledge of what `T` actually is, so it cannot safely allocate an array of that specific erased type.
- 5 **What's a bounded type parameter?** A type parameter restricted to a specific type or its subtypes (e.g., `<T extends Number>`), allowing the generic code to safely call methods defined on that bound.
- 6 **Why does `List<String>.getClass() == List<Integer>.getClass()` return `true`?** Because generic type parameters are erased at compile time — both are simply raw `ArrayList` at runtime, indistinguishable by `getClass()`.

## 14. Revision Notes

### GENERICS — CHEAT SHEET

=====

Parameterize classes/methods over TYPES -> compile-time type safety, no casts

Generic class: `class Box<T> { T content; }`

Generic method: `static <T> void method(T param) { }`

Bounded: `<T extends Number>` (works for interfaces too, despite 'extends')

Wildcards: `? (any)`, `? extends T` (read-only), `? super T` (write-only)

PECS RULE: Producer Extends, Consumer Super

read-only -> `? extends T`

write-only -> `? super T`

both -> exact type, no wildcard

TYPE ERASURE: generics exist ONLY at compile time, erased in bytecode

-> cannot: `new T()`, `new T[10]`, `instanceof List<String>`

-> ALL generic instantiations of a class share ONE class object at runtime

## 15. Homework

- 1 Write a generic `Pair<A, B>` class holding two values of potentially different types, with getters for each.
- 2 Write a method using `<? extends Number>` that sums a `List` of any `Number` subtype, and a separate method using `<? super Integer>` that adds integers to a list — explain why each wildcard direction was necessary.

---

(End of Topic 35: Generics.)

---

## TOPIC 36: REFLECTION API

---

## 1. Introduction

**What is it?** Reflection is Java's capability to **inspect and manipulate classes, methods, fields, and constructors at runtime**, even ones not known at compile time — enabling frameworks to work generically across arbitrary user-defined classes without the framework author needing to know those classes in advance.

**Why introduced:** Frameworks like Spring, Hibernate, and JUnit need to discover annotations (see [[Annotations]Topic 34]]), instantiate classes, inject dependencies, and invoke methods on types they've never seen at compile time — impossible without a way to inspect and interact with arbitrary classes purely at runtime.

**Interview definition:** "Reflection is the `java.lang.reflect` API that allows a running program to examine or modify the structure and behavior (classes, fields, methods, constructors, annotations) of itself or other classes at runtime, bypassing normal compile-time type checking, at the cost of performance and some type safety."

## 2. Real Life Analogy

- **Hospital:** Reflection is like a medical scanner that can examine ANY patient's internal structure (organs, bones) without needing to know that specific patient in advance — a generic diagnostic tool that works on any body, discovering its structure dynamically rather than requiring prior knowledge of exactly that patient.
- **Bank auditor:** An auditor (reflection) can inspect ANY department's internal records/procedures at runtime, regardless of which specific department it is, without needing pre-existing, department-specific knowledge coded in advance.

## 3. Getting a `Class` Object — Three Ways

```
Class<?> c1 = String.class; // (1) via .class literal - compile-time known type
Class<?> c2 = "hello".getClass(); // (2) via an instance's getClass()
Class<?> c3 = Class.forName("java.lang.String"); // (3) via fully-qualified name - purely runtime,
throws ClassNotFoundException
```

## 4. Key Reflection Operations

```
import java.lang.reflect.*;

public class ReflectionDemo {
    public static void main(String[] args) throws Exception {
        Class<?> clazz = Person.class;

        System.out.println("Class name: " + clazz.getName());
        System.out.println("Simple name: " + clazz.getSimpleName());

        // Inspect fields
        for (Field field : clazz.getDeclaredFields()) {
            System.out.println("Field: " + field.getName() + " : " + field.getType());
        }

        // Inspect methods
        for (Method method : clazz.getDeclaredMethods()) {
            System.out.println("Method: " + method.getName());
        }

        // Create an instance dynamically (no compile-time 'new Person()' needed)
        Object instance = clazz.getDeclaredConstructor(String.class).newInstance("Alice");

        // Invoke a method dynamically
        Method greetMethod = clazz.getMethod("greet");
        greetMethod.invoke(instance);

        // Access a PRIVATE field, bypassing normal access control
        Field nameField = clazz.getDeclaredField("name");
        nameField.setAccessible(true); // breaks encapsulation deliberately!
        System.out.println(nameField.get(instance));
    }
}

class Person {
    private String name;
    public Person(String name) { this.name = name; }
    public void greet() { System.out.println("Hi, I'm " + name); }
}
```

## 5. Edge Cases

| Case                                                                                                     | Result                                                                                                                                                                                                      |
|----------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>Class.forName("nonexistent.Class")</code>                                                          | <code>ClassNotFoundException</code>                                                                                                                                                                         |
| Accessing a private field/method without <code>setAccessible(true)</code>                                | <code>IllegalAccessException</code>                                                                                                                                                                         |
| Calling <code>setAccessible(true)</code> when a <code>SecurityManager</code> /module system restricts it | <code>InaccessibleObjectException</code> (Java 9+ strong encapsulation) or <code>SecurityException</code>                                                                                                   |
| Invoking a method with wrong argument types via reflection                                               | <code>IllegalArgumentException</code>                                                                                                                                                                       |
| Reflection performance vs direct calls                                                                   | Reflective calls are notably SLOWER (bypasses JIT optimizations more easily, extra security/type checks per call) — frameworks often cache <code>Method/Field</code> objects to reduce repeated lookup cost |

## 6. Common Mistakes

- Using reflection casually in application business logic instead of normal method calls — reflection should be reserved for framework/library/tooling code, not everyday application logic, due to performance cost and lost compile-time safety.
- Forgetting `setAccessible(true)` is required to access `private` members, and that Java 9+ module encapsulation may block it entirely for JDK-internal classes.
- Not caching `Method/Field/Constructor` objects when reflection is used repeatedly (e.g., in a loop or hot path) — repeated lookups add unnecessary overhead.

## 7. Best Practices

- Reserve reflection for framework/library-level code (dependency injection, ORM, serialization, testing tools) — not everyday application logic.
- Cache reflective `Method/Field` lookups when used repeatedly, rather than re-resolving them on every call.
- Be aware that `setAccessible(true)` deliberately breaks encapsulation — use it sparingly and only where genuinely justified (e.g., testing frameworks, serialization libraries).

## 8. Production Usage

- **Spring:** Uses reflection extensively to instantiate beans, inject dependencies (`@Autowired`), and invoke annotated methods.
- **Hibernate/JPA:** Uses reflection to read/write entity fields directly (bypassing getters/setters in some strategies) and to instantiate entities from database rows.
- **JUnit:** Uses reflection to discover and invoke `@Test`-annotated methods.
- **Jackson/Gson:** Use reflection to serialize/deserialize objects to/from JSON without requiring manual mapping code per class.

## 9. Frequently Asked Interview Questions

- 1 **What is reflection?** The capability to inspect and manipulate classes, fields, methods, and constructors at runtime, even for types not known at compile time.
- 2 **What are the three ways to get a `Class` object?** `.class` literal, an instance's `getClass()`, and `Class.forName("fully.qualified.Name")`.
- 3 **How do you access a `private` field via reflection?** Retrieve it with `getDeclaredField()`, then call `setAccessible(true)` before reading/writing it — deliberately bypassing normal access control.
- 4 **Why is reflection slower than direct method calls?** It involves additional runtime type/security checks and generally bypasses some JIT compiler optimizations compared to a direct, statically-resolved method call.
- 5 **Name a real framework mechanism built on reflection.** Spring's dependency injection (scanning for `@Component/@Autowired` and instantiating/wiring beans reflectively) or Hibernate's entity instantiation and field mapping.

- 6 What exception is thrown if a class isn't found via `Class.forName()`? `ClassNotFoundException` (a checked exception).

## 10. Revision Notes

### REFLECTION API – CHEAT SHEET

=====

Inspect/manipulate classes, fields, methods, constructors AT RUNTIME,  
even for types unknown at compile time

GET A Class OBJECT: `Type.class` | `instance.getClass()` | `Class.forName("...")`

KEY OPERATIONS: `getDeclaredFields()`, `getDeclaredMethods()`, `newInstance()`  
(via Constructor), `invoke()` (via Method), `setAccessible(true)` for private members

DOWNSIDES: slower than direct calls, breaks encapsulation, loses compile-time  
type safety

PRODUCTION USE: Spring (DI), Hibernate (ORM), JUnit (test discovery),  
Jackson/Gson (JSON mapping) - all framework-level, not typical app logic

## 11. Homework

- 1 Use reflection to list all fields and methods of a class you didn't write yourself (e.g., `java.util.ArrayList`), and print their names/types.
- 2 Use reflection to access and print a `private` field's value from an object, using `setAccessible(true)`.

---

(End of Topic 36: Reflection API.)

---

# TOPIC 37: INNER CLASSES

---

## 1. Introduction

**What is it?** An inner class is a **non-static class defined inside another class**, holding an implicit reference to an instance of its enclosing class, allowing it tight access to the outer class's instance members (including `private` ones).

**Why introduced:** Some helper classes are only meaningful in the context of a specific outer object (e.g., an `Iterator` that only makes sense tied to a specific `LinkedList` instance's internal nodes) — inner classes let such helper types live logically inside their owner, with automatic access to its private state, without needing to pass that state around explicitly.

## 2. Syntax and Internal Working

```
class Outer {
    private int outerField = 10;

    class Inner { // INNER (non-static) class
        void display() {
            System.out.println("Outer field: " + outerField); // direct access to Outer's private field!
        }
    }
}

Outer outer = new Outer();
Outer.Inner inner = outer.new Inner(); // must create via an OUTER INSTANCE
inner.display(); // "Outer field: 10"
```

- Internally, the compiler gives Inner a hidden, synthetic field (`Outer.this` reference) automatically, letting it silently access the enclosing instance's members.
- An inner (non-static) class object **cannot exist without an enclosing outer object** — it's created via `outer.new Inner()`, not a bare `new Inner()`.

## 3. Real Life Analogy

- Bank:** A `LoanApplication.Reviewer` inner class is only meaningful tied to a SPECIFIC `LoanApplication` instance — it needs direct access to that specific application's private details, and creating a "reviewer" without an associated application makes no sense.

## 4. Code Example

```
class Car {
    private String model = "Tesla Model 3";

    class Engine { // inner class
        void start() {
            System.out.println("Starting engine of " + model);
        }
    }
}

public class InnerClassDemo {
    public static void main(String[] args) {
        Car car = new Car();
        Car.Engine engine = car.new Engine(); // requires an outer Car instance
        engine.start();
    }
}
```

## 5. Edge Cases

| Case                                                                                                                              | Result                                                                                                                                                                               |
|-----------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Creating an inner class instance WITHOUT an outer instance ( <code>new Inner()</code> directly, from outside <code>Outer</code> ) | Compile-time error — must use <code>outer.new Inner()</code>                                                                                                                         |
| Inner class declared <code>static</code> fields (non-constant)                                                                    | Compile-time error (prior to Java 16) — non-static inner classes cannot have static members other than constants; relaxed somewhat in later versions but still generally discouraged |
| Multiple <code>Outer</code> instances, each creating their own <code>Inner</code>                                                 | Each <code>Inner</code> instance is bound to its OWN specific <code>Outer</code> instance — accessing <code>outerField</code> gives that specific outer object's value               |

## 6. Common Mistakes

- Forgetting inner classes require an outer instance to be created (`outer.new Inner()`), unlike static nested classes.
- Overusing inner classes for logic that doesn't genuinely need access to the outer instance's state — adds unnecessary coupling; a `static` nested class (next topic) is usually the better choice if outer-instance access isn't truly needed.

## 7. Best Practices

- Use a (non-static) inner class specifically when the helper genuinely needs tight access to a specific outer instance's state.
- If outer-instance access isn't needed, prefer a `static` nested class instead — simpler, more decoupled, no implicit outer reference held (which could otherwise cause memory leaks by unintentionally keeping the outer object alive).

## 8. Frequently Asked Interview Questions

- 1 **What is an inner class?** A non-static class defined inside another class, holding an implicit reference to a specific instance of the enclosing class.
- 2 **How do you create an instance of an inner class from outside the outer class?** `outer.new Inner()` — requires an existing outer instance.
- 3 **Why might holding an inner class reference unintentionally cause a memory leak?** Because every inner class instance implicitly holds a reference to its enclosing outer instance — if the inner object outlives its intended scope (e.g., stored somewhere long-lived), it prevents the outer object from being garbage collected too.
- 4 **What's the key difference between an inner class and a static nested class?** An inner class holds an implicit reference to a specific outer instance and can access its instance members directly; a static nested class does not, and can be instantiated without any outer instance at all.



## 9. Revision Notes

### INNER CLASSES – CHEAT SHEET

=====

Non-static class INSIDE another class - implicitly holds a reference to a SPECIFIC outer instance

CREATE VIA: `outer.new Inner()` (NOT a bare `'new Inner()'`)

Can access outer's PRIVATE instance members directly

GOTCHA: implicit outer reference can cause memory leaks if the inner object outlives its intended scope

USE WHEN: helper genuinely needs tight access to a specific outer instance's state

## 10. Homework

- 1 Build an `Outer/Inner` pair where `Inner` reads and modifies a `private` field of `Outer`, and create two separate `Outer` instances to prove each `Inner` instance is bound to its own specific outer object.

---

(End of Topic 37: Inner Classes.)

---

# TOPIC 38: NESTED CLASSES

---

## 1. Introduction

**What is it?** "Nested class" is the umbrella term for ANY class defined inside another class. It splits into two categories: **static nested classes** and **(non-static) inner classes** (covered in [\[\[Inner Classes|Topic 37\]\]](#)). This topic focuses specifically on **static nested classes** and the full classification.

## 2. Full Classification of Nested Classes

NESTED CLASSES (any class defined inside another)

■■■ STATIC nested class -> does NOT hold an outer instance reference

■■■ INNER (non-static) class -> HOLDS an implicit outer instance reference (Topic 37)

■■■ Local inner class -> defined INSIDE a method body

■■■ Anonymous inner class -> no name, defined + instantiated in one expression

### 3. Static Nested Class

```
class Outer {
    private static int staticOuterField = 100;

    static class StaticNested { // STATIC - no outer instance needed
        void display() {
            System.out.println("Static outer field: " + staticOuterField); // can access outer's STATIC members only
        }
    }
}

Outer.StaticNested nested = new Outer.StaticNested(); // NO outer instance required!
nested.display();
```

- Behaves like a regular top-level class that simply happens to be namespaced inside another class — can be instantiated WITHOUT any `Outer` instance.
- Can only access the outer class's `static` members directly (no implicit reference to any specific instance exists).

### 4. Local Inner Class (Defined Inside a Method)

```
class Outer {
    void processOrder() {
        class OrderValidator { // LOCAL class - only visible within this method
            boolean isValid(int amount) { return amount > 0; }
        }
        OrderValidator validator = new OrderValidator();
        System.out.println(validator.isValid(50));
    }
}
```

### 5. Anonymous Inner Class

```
interface Greeting {
    void greet();
}

Greeting g = new Greeting() { // ANONYMOUS class - no name, defined + instantiated together
    @Override
    public void greet() {
        System.out.println("Hello from an anonymous class!");
    }
};
g.greet();
```

**Note:** Since Java 8, lambda expressions largely replace anonymous classes for functional interfaces (single abstract method) — but anonymous classes are still needed for interfaces/abstract classes with MORE than one method to implement, or when you need instance state beyond what a lambda naturally supports.

## 6. Static Nested Class vs Inner Class — Comparison Table

| Aspect                             | Static Nested Class                                                                                              | Inner (Non-Static) Class                                                    |
|------------------------------------|------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| Needs outer instance to create?    | No                                                                                                               | Yes ( <code>outer.new Inner()</code> )                                      |
| Access to outer's instance members | No (only outer's static members)                                                                                 | Yes (full access, including private)                                        |
| Memory                             | No implicit outer reference held                                                                                 | Holds an implicit outer reference (potential leak risk)                     |
| Typical use                        | Logically grouping a helper class with its "owner" without needing instance ties (e.g., <code>Map.Entry</code> ) | Helper genuinely tied to and dependent on a specific outer instance's state |

## 7. Real Life Analogy

- **College:** A `University.Department` static nested class represents a department that exists independently of any specific university `PERSON` instance — just logically organized under the `University` namespace. A `University.EnrollmentOffice` (inner class) that needs direct access to THIS SPECIFIC university's specific enrolled-student list would be a genuine (non-static) inner class instead.

## 8. Edge Cases

| Case                                                                              | Result                                                                                                                                  |
|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| Static nested class trying to access outer's INSTANCE (non-static) field directly | Compile-time error — no implicit outer instance exists                                                                                  |
| Local inner class referencing a local variable of its enclosing method            | Allowed, but that local variable must be effectively <code>final</code> (never reassigned after initialization) — enforced since Java 8 |
| Anonymous class implementing an interface with 2+ abstract methods                | Fully supported (unlike lambdas, which only work for single-abstract-method functional interfaces)                                      |

## 9. Common Mistakes

- Making a nested class an inner (non-static) class when it doesn't actually need outer-instance access — unnecessarily couples it and risks memory-leak-via-retained-outer-reference.
- Trying to modify a local variable captured by a local/anonymous inner class after using it — violates the "effectively final" requirement, causing a compile error.

## 10. Best Practices

- Default to `static` nested classes unless outer-instance access is genuinely required — safer, simpler, no implicit coupling.
- Prefer lambda expressions over anonymous classes for functional interfaces (Java 8+) — more concise and readable; reserve anonymous classes for multi-method interfaces/abstract classes.

## 11. Production Usage

- `Map.Entry` is a well-known JDK static nested interface.
- Builder pattern implementations often use static nested `Builder` classes (`Person.Builder`).
- Anonymous classes remain common for one-off event listener/callback implementations in UI frameworks and older-style APIs predating lambdas.

## 12. Frequently Asked Interview Questions

- 1 **What are the four kinds of nested classes in Java?** Static nested class, (non-static) inner class, local inner class, anonymous inner class.
- 2 **Can a static nested class access the outer class's instance fields directly?** No — only the outer class's `static` members, since no implicit outer instance reference exists.
- 3 **Why must a local variable captured by a local/anonymous class be effectively final?** Because the inner class actually captures a COPY of the variable's value at the time of creation (for correctness across potentially different lifetimes/threads) — allowing it to change afterward would create ambiguity about which value the inner class should see.
- 4 **When would you use an anonymous class instead of a lambda?** When implementing an interface/abstract class with MORE than one abstract method, or when you need to maintain additional instance state beyond a lambda's simple functional body.
- 5 **What's a common real JDK example of a static nested class?** `Map.Entry` — represents a key-value pair without needing any specific `Map` instance tied to it.

## 13. Revision Notes

### NESTED CLASSES — CHEAT SHEET

=====

STATIC NESTED CLASS: no outer instance needed, only accesses outer's `STATIC` members

INNER (non-static) CLASS: needs outer instance, full access to outer's instance members

LOCAL CLASS: defined inside a method body, can capture effectively-final locals

ANONYMOUS CLASS: no name, defined + instantiated in one expression

DEFAULT to `STATIC` nested unless outer-instance access is genuinely needed

Java 8+ LAMBDA replace most anonymous-class use for single-method interfaces

Effectively-final rule: captured local variables cannot change after capture

## 14. Homework

- 1 Build a `static` nested `Builder` class for a `Person` class (Builder pattern), demonstrating it needs no outer `Person` instance to construct one.
- 2 Write both an anonymous class AND a lambda implementing the same single-method functional interface, and compare the resulting code.

---

*(End of Topic 38: Nested Classes.)*

---

# TOPIC 39: RECORDS (JAVA 17)

---

## 1. Introduction

**What is it?** A **record** (finalized in Java 16, a headline feature of the Java 17 LTS toolset) is a special, concise class form specifically designed for **immutable data carriers** — automatically generating the constructor, accessors, `equals()`, `hashCode()`, and `toString()` that a hand-written `[[Immutable Objects|Topic 32]]` class would otherwise require you to write manually.

**Why introduced:** Writing a correct, fully immutable "plain data" class (DTO, value object) manually requires significant repetitive boilerplate (private final fields, a constructor, getters, and carefully-matched `equals()/hashCode()/toString()` overrides) — easy to get subtly wrong (see `[[Object Class|Topic 25]]`'s `equals/hashCode` contract). Records eliminate this boilerplate entirely for the extremely common "just a bag of immutable values" case.

**Interview definition:** "A record is a special, implicitly `final` class declaration that defines an immutable data aggregate via a concise header listing its components, for which the compiler automatically generates private final fields, a canonical constructor, public accessor methods, and correctly-paired `equals()`, `hashCode()`, and `toString()` implementations."

## 2. Syntax — Before and After

### *Before Records (Manual, Java 8-style)*

```
public final class Point {
    private final int x;
    private final int y;
    public Point(int x, int y) { this.x = x; this.y = y; }
    public int x() { return x; }
    public int y() { return y; }
    @Override public boolean equals(Object o) { /* ~10 lines */ }
    @Override public int hashCode() { /* ~3 lines */ }
    @Override public String toString() { /* ~1 line */ }
}
```

### *With Records (Java 16+)*

```
public record Point(int x, int y) { } // ONE LINE - generates everything above automatically!

Point p = new Point(3, 4);
System.out.println(p.x()); // 3 - accessor method named exactly `x()`, NOT `getX()`
System.out.println(p.y()); // 4
System.out.println(p); // "Point[x=3, y=4]" - auto-generated toString()
System.out.println(p.equals(new Point(3, 4))); // true - auto-generated, field-based equals()
```

### 3. What the Compiler Auto-Generates

| Generated               | Detail                                                                                                                            |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Private final fields    | One per record component, exactly as declared                                                                                     |
| Canonical constructor   | Takes all components in declared order, assigns each to its field                                                                 |
| Accessor methods        | Named EXACTLY like the component ( <code>x()</code> , not <code>getX()</code> ) — a deliberate departure from JavaBean convention |
| <code>equals()</code>   | Compares ALL components for equality                                                                                              |
| <code>hashCode()</code> | Combines ALL components' hash codes                                                                                               |
| <code>toString()</code> | <code>RecordName[component1=value1, component2=value2, ...]</code> format                                                         |

### 4. Compact Constructors — Adding Validation

```
public record Point(int x, int y) {
    public Point { // COMPACT constructor - no parameter list repeated, no explicit field assignment!
        if (x < 0 || y < 0) {
            throw new IllegalArgumentException("Coordinates must be non-negative");
        }
        // field assignment (this.x = x; this.y = y;) happens AUTOMATICALLY after this block
    }
}
```

This is the standard place to add validation or normalization logic — you write ONLY the validation, and the compiler still handles the actual field assignment afterward automatically.

### 5. Records Can Have Additional Members

```
public record Point(int x, int y) {
    static final Point ORIGIN = new Point(0, 0); // static fields allowed

    double distanceFromOrigin() { // additional instance methods allowed
        return Math.sqrt(x * x + y * y);
    }

    public Point { // compact constructor for validation
        if (x < 0 || y < 0) throw new IllegalArgumentException("Must be non-negative");
    }
}
```

## 6. Rules and Restrictions

| Rule                                                               | Detail                                                                                                                                                                                                                                                            |
|--------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Implicitly <code>final</code>                                      | A record can NEVER be extended/subclassed                                                                                                                                                                                                                         |
| Cannot extend another class                                        | Records already implicitly extend <code>java.lang.Record</code> (similar to how enums implicitly extend <code>Enum</code> )                                                                                                                                       |
| CAN implement interfaces                                           | Fully supported — a common pattern for giving records a shared contract                                                                                                                                                                                           |
| All fields are implicitly <code>private final</code>               | Cannot be reassigned after construction — records are inherently immutable at the field level (though a component's <code>REFERENCE</code> type could still be internally mutable unless you defensively copy it, same caveat as <code>[[Immutable Objects</code> |
| Cannot declare additional instance fields beyond the record header | Only the declared components become instance fields; you CAN add <code>static</code> fields freely                                                                                                                                                                |

## 7. Real Life Analogy

- **Bank:** A `TransactionReceipt(String id, double amount, LocalDateTime timestamp)` record is a perfect fit — a receipt is fundamentally a fixed, immutable bundle of values with no independent identity or lifecycle beyond just representing that data.
- **Restaurant:** An `OrderLine(String itemName, int quantity, double price)` record — a clean, immutable data carrier used to pass order details around, exactly the kind of "plain data" record is designed for.

## 8. Code Examples

```
public record Employee(String name, double salary) implements Comparable<Employee> {
    @Override
    public int compareTo(Employee other) {
        return Double.compare(this.salary, other.salary);
    }
}

public class RecordDemo {
    public static void main(String[] args) {
        List<Employee> employees = new ArrayList<>(List.of(
            new Employee("Alice", 75000),
            new Employee("Bob", 60000)
        ));
        Collections.sort(employees);
        employees.forEach(System.out::println);
        // Employee[name=Bob, salary=60000.0]
        // Employee[name=Alice, salary=75000.0]
    }
}
```

## 9. Edge Cases

| Case                                                                              | Result                                                                                                                                                                                                  |
|-----------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Trying to <code>extends</code> a record                                           | Compile-time error — records are implicitly <code>final</code>                                                                                                                                          |
| Trying to add a mutable instance field NOT declared in the record header          | Compile-time error — only header components become instance fields                                                                                                                                      |
| A record component that's a mutable type (e.g., <code>List&lt;String&gt;</code> ) | The record itself is still "shallow" immutable — the LIST's contents can still be externally mutated unless you defensively copy it in a compact constructor (same caveat as regular immutable classes) |
| Two records with identical component values                                       | <code>equals()</code> returns <code>true</code> automatically (field-based comparison), unlike default <code>Object.equals()</code> which would compare references                                      |

## 10. Common Mistakes

- Assuming a record with a mutable component type (like a `List`) is FULLY immutable — the record's own fields can't be reassigned, but a mutable component's CONTENTS can still change unless explicitly defensively copied in a compact constructor.
- Trying to add extra instance fields beyond the declared record components — not allowed; only `static` fields can be added freely.
- Forgetting accessor methods are named exactly like the component (`x()`), not the JavaBean-style `getX()` — can cause confusion/compile errors when integrating with tools expecting strict JavaBean conventions (some libraries offer Records support explicitly to bridge this).

## 11. Best Practices

- Use records for DTOs, value objects, and simple immutable data aggregates — the ideal, most common use case.
- Add a compact constructor for validation/normalization whenever the record's components have real invariants to enforce.
- Defensively copy any mutable component types (like `List/Map`) inside a compact constructor to achieve TRUE deep immutability, not just shallow field immutability.

## 12. Production Usage

- Modern (Java 17+) Spring Boot REST APIs increasingly use records for request/response DTOs — concise, immutable, and automatically correct `equals()/hashCode()/toString()`.
- Pattern matching for switch (Java 21, preview in 17-19) pairs powerfully with records via "record patterns," destructuring record components directly in `switch` expressions.

## 13. Frequently Asked Interview Questions



- 1 **What is a record?** A concise class form (Java 16+) for immutable data aggregates, auto-generating the constructor, accessors, `equals()`, `hashCode()`, and `toString()`.
- 2 **What are a record's accessor methods named?** Exactly like the component (`x()` for a component named `x`), NOT the JavaBean-style `getX()`.
- 3 **Can a record extend another class?** No — records are implicitly `final` and already implicitly extend `java.lang.Record`.
- 4 **Can a record implement interfaces?** Yes — fully supported, a common and encouraged pattern.
- 5 **What is a compact constructor, and why use one?** A constructor form without a repeated parameter list or explicit field assignments, used for adding validation/normalization logic — the actual field assignment still happens automatically afterward.
- 6 **Is a record fully immutable if one of its components is a `List`?** Not automatically — the record's own field reference can't be reassigned, but the `LIST`'s contents can still be mutated externally unless you defensively copy it in a compact constructor.
- 7 **Why were records introduced given classes already existed?** To eliminate substantial, error-prone, repetitive boilerplate for the extremely common "immutable data carrier" pattern, ensuring correct `equals()/hashCode()/toString()` by construction rather than by careful manual coding.

## 14. Revision Notes

RECORDS (Java 16+) – CHEAT SHEET

=====

```
record Point(int x, int y) { }
```

-> auto-generates: private final fields, canonical constructor, accessors `x()/y()` (NOT `getX()/getY()`), `equals()`, `hashCode()`, `toString()`

Implicitly `FINAL` (cannot be extended). Implicitly extends `java.lang.Record`.

CAN implement interfaces. CANNOT add extra instance fields (only static ones).

COMPACT CONSTRUCTOR: ``public Point { if(...) throw ...; }``

-> for validation; field assignment still happens automatically after

GOTCHA: mutable component (e.g., `List`) -> record itself is immutable, but the `LIST'S CONTENTS` can still be mutated unless defensively copied

USE FOR: DTOs, value objects, simple immutable data aggregates

## 15. Homework

- 1 Convert a hand-written immutable `Point` class (fields, constructor, getters, `equals()/hashCode()/toString()`) into a one-line `record`, and verify identical behavior.
- 2 Create a record with a `List<String>` component, prove its contents can still be mutated externally, then fix it using a compact constructor with a defensive copy.

---

(End of Topic 39: Records.)

---

# TOPIC 40: SEALED CLASSES (JAVA 17)

## 1. Introduction

**What is it?** A sealed class (or interface) explicitly restricts WHICH other classes/interfaces are allowed to extend/implement it, via a `permits` clause — giving the class author precise, compiler-enforced control over its entire inheritance hierarchy, finalized as a standard feature in Java 17.

**Why introduced:** Before sealed classes, an author of an extensible base type had only two blunt options: `final` (no subclassing AT ALL) or a fully open class (ANY class, anywhere, could extend it). Neither option let an author say "exactly these 3 specific subclasses are allowed, and no others" — a common, genuine design need (e.g., modeling a fixed, closed set of shape types, or a fixed set of payment methods) that sealed classes finally address directly.

**Interview definition:** "A sealed class or interface, declared with the `sealed` modifier and a `permits` clause, restricts its set of direct subclasses/implementers to an explicitly named, closed list, with each permitted subclass required to declare itself `final`, `sealed`, or `non-sealed`, enabling the compiler to reason exhaustively about the type's complete hierarchy (e.g., for exhaustive switch pattern matching)."

## 2. Syntax

```
public sealed interface Shape permits Circle, Rectangle, Triangle { }

public final class Circle implements Shape { // FINAL - no further subclassing allowed
    double radius;
}
public final class Rectangle implements Shape {
    double width, height;
}
public non-sealed class Triangle implements Shape { // NON-SEALED - opens it back up for further
    extension
    double base, height;
}
```

## 3. The Three Required Modifiers for Permitted Subclasses

| Modifier                | Meaning                                                                                                                           |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| <code>final</code>      | This subclass cannot be extended further — the hierarchy definitively ends here                                                   |
| <code>sealed</code>     | This subclass can be extended, but ONLY by its own explicitly permitted subclasses (the restriction continues down the hierarchy) |
| <code>non-sealed</code> | This subclass "opens the hierarchy back up" — ANY class can now extend IT freely, escaping the original sealing                   |

Every class listed in a `permits` clause MUST use exactly one of these three modifiers — the compiler enforces this, refusing to compile otherwise.

## 4. Why Sealed Classes Matter — Exhaustive Pattern Matching

```
static double area(Shape shape) {  
    return switch (shape) {  
        case Circle c -> Math.PI * c.radius * c.radius;  
        case Rectangle r -> r.width * r.height;  
        case Triangle t -> 0.5 * t.base * t.height;  
        // NO 'default' NEEDED! Compiler KNOWS these are the ONLY possible subtypes  
    };  
}
```

Because `Shape` is sealed with an EXACT, closed list of permitted subtypes, the compiler can PROVE the `switch` covers every possible case — no `default` branch is needed, and if a new permitted subtype is EVER added later, the compiler will immediately flag every `switch` statement across the entire codebase that now needs updating — a powerful safety net non-sealed hierarchies can never provide.

## 5. Real Life Analogy

- **Bank:** A sealed `PaymentMethod` (permits `CreditCard`, `DebitCard`, `UPI`) models a genuinely CLOSED, deliberately fixed set of payment types the bank supports — unlike a normal open interface, no external code can secretly add a rogue new payment type the bank's processing logic wasn't designed to handle.
- **Restaurant:** A sealed `OrderStatus` type (permits `Placed`, `Preparing`, `Delivered`) guarantees the kitchen dashboard's status-handling logic can exhaustively account for every possible state, with the compiler catching any gap immediately.

## 6. Sealed Classes vs Enums — When to Use Which

| Aspect                                           | Enum                                                                          | Sealed Class Hierarchy                                                                     |
|--------------------------------------------------|-------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| Each "case" carries different data/fields        | No — all enum constants share the SAME fields                                 | Yes — each permitted subclass can have entirely different fields                           |
| Each "case" needs different behavior/state shape | Limited (constant-specific method bodies help, but structure is still shared) | Full flexibility — each subclass is a genuinely distinct type                              |
| Exhaustive switch checking                       | Yes                                                                           | Yes (Java 17+ with sealed + pattern matching)                                              |
| Best for                                         | A closed, uniform set of simple named constants                               | A closed set of structurally DIFFERENT variants (a "sum type"/algebraic data type pattern) |

## 7. Code Examples

```
public sealed interface PaymentMethod permits CreditCard, UPI { }

public record CreditCard(String cardNumber, String expiryDate) implements PaymentMethod { }
public record UPI(String upiId) implements PaymentMethod { } // sealed + records = powerful combo!

public class PaymentProcessor {
    static String describe(PaymentMethod method) {
        return switch (method) {
            case CreditCard c -> "Card ending in " + c.cardNumber().substring(c.cardNumber().length() - 4);
            case UPI u -> "UPI ID: " + u.upiId();
        };
    }
}
```

**Sealed interfaces + records together** form Java's closest equivalent to "algebraic data types" from functional languages — a closed set of structurally distinct variants, exhaustively pattern-matchable, all with automatic immutability and correct `equals()`/`hashCode()`.

## 8. Edge Cases

| Case                                                                                                              | Result                                                                                                                                                                           |
|-------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| A class NOT listed in <code>permits</code> tries to extend the sealed class                                       | Compile-time error                                                                                                                                                               |
| A permitted subclass doesn't declare itself <code>final</code> , <code>sealed</code> , or <code>non-sealed</code> | Compile-time error — one of the three is mandatory                                                                                                                               |
| Switch over a sealed type missing a case for one permitted subtype, with NO default                               | Compile-time error — the compiler enforces exhaustiveness                                                                                                                        |
| Adding a new permitted subtype to the <code>permits</code> list later                                             | Immediately surfaces a compile error in every exhaustive switch across the codebase that needs updating — a deliberate, valuable "forcing function" for safe hierarchy evolution |

## 9. Common Mistakes

- Forgetting each permitted subclass MUST explicitly declare `final`, `sealed`, or `non-sealed` — omitting this is a compile-time error, not a silent default.
- Using `non-sealed` carelessly, which fully reopens the hierarchy for that branch — defeating the purpose of sealing if overused throughout the hierarchy.
- Choosing a sealed hierarchy when a simple enum would suffice (i.e., when all "cases" genuinely share identical structure/fields) — unnecessary complexity for a uniform case.

## 10. Best Practices

- Use sealed classes/interfaces specifically when you have a genuinely CLOSED, deliberately fixed set of structurally DIFFERENT subtypes that should never silently expand.
- Combine sealed interfaces with records for a clean, modern "algebraic data type" style design in Java 17+.

- Avoid overusing `non-sealed` — it undermines the guarantees sealing is meant to provide; use it deliberately only where a specific extension point is genuinely intended.

## 11. Production Usage

- Modeling closed API response/result types (`sealed interface Result` permits `Success`, `Failure`), closed domain event hierarchies, and closed expression/AST node hierarchies (compilers, rule engines) are common, powerful real-world sealed class use cases in modern (Java 17+) codebases.

## 12. Frequently Asked Interview Questions

- 1 **What is a sealed class?** A class/interface that explicitly restricts which other classes can extend/implement it, via a `permits` clause listing exactly the allowed subtypes.
- 2 **What three modifiers can a permitted subclass use, and what does each mean?** `final` (no further subclassing), `sealed` (further restricted subclassing continues), `non-sealed` (freely reopens extension for that branch).
- 3 **Why do sealed classes enable exhaustive switch expressions without a `default` branch?** Because the compiler knows the COMPLETE, closed set of possible subtypes and can statically verify every case is handled.
- 4 **When would you choose a sealed class hierarchy over an enum?** When different "cases" need to carry structurally different fields/data, not just act as simple uniform named constants.
- 5 **What's a powerful modern Java combination with sealed interfaces?** Records — together they form a clean, immutable, exhaustively pattern-matchable "algebraic data type" style design.
- 6 **What happens if you add a new permitted subtype to an existing sealed hierarchy?** Every exhaustive `switch` over that sealed type elsewhere in the codebase that doesn't yet handle the new subtype will immediately fail to compile — a deliberate safety mechanism forcing complete, safe hierarchy evolution.

## 13. Revision Notes

```
SEALED CLASSES (Java 17) – CHEAT SHEET
=====
sealed [class|interface] X permits A, B, C { }
-> ONLY A, B, C may extend/implement X - closed, compiler-enforced hierarchy

EVERY permitted subclass MUST be exactly one of:
final -> hierarchy ends here
sealed -> further restricted extension continues
non-sealed -> hierarchy reopens freely from here

BENEFIT: exhaustive switch/pattern matching WITHOUT needing a default branch
- compiler catches every un-updated switch if a new permitted type is added

BEST COMBO: sealed interface + records = Java's "algebraic data type" pattern
USE OVER ENUM WHEN: cases need structurally DIFFERENT fields/data
```

## 14. Homework

- 1 Build a sealed interface `Result` permits `Success`, `Failure` with `Success/Failure` as records carrying different fields, and write an exhaustive switch expression over it.
- 2 Add a `THIRD` permitted subtype later, and observe the compiler immediately flag your existing switch expression as no longer exhaustive.

---

*(End of Topic 40: Sealed Classes — this concludes the Advanced Core Java sub-section!)*

---

## EXCEPTION HANDLING

---

### TOPIC 41: EXCEPTION HIERARCHY, CHECKED & UNCHECKED EXCEPTIONS

---

#### 1. Introduction

**What is it?** An exception is an event that disrupts a program's normal instruction flow, typically representing an error condition. Java's exception handling mechanism lets you detect, propagate, and respond to such events in a structured way, separating error-handling code from normal business logic.

**Why introduced:** Without structured exception handling, error conditions would need to be checked via return codes at every single call site (the C-style approach) — easy to forget, and it clutters normal logic with constant error-checking. Java's `try/catch/throw` model lets errors propagate automatically up the call stack until something is equipped to handle them, keeping normal-path code clean.

#### 2. The Full Exception Hierarchy

```
Throwable
/ \
Error Exception
(JVM-level, / \
unrecoverable) RuntimeException (all other Exceptions)
e.g. | |
OutOfMemoryError, UNCHECKED CHECKED EXCEPTIONS
StackOverflowError (NullPointerException, (IOException,
ArithmeticException, SQLException,
ArrayIndexOutOfBoundsException ClassNotFoundException)
Exception, ClassCast
Exception, ...)
```

### 3. Checked vs Unchecked — The Core Distinction

| Aspect                  | Checked Exceptions                                                                       | Unchecked Exceptions (RuntimeException + subclasses)                                                                     |
|-------------------------|------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| Compiler enforcement    | MUST be either caught (try-catch) or declared (throws) — compile-time error otherwise    | No compiler enforcement — can be thrown/ignored freely                                                                   |
| Represents              | Recoverable, EXPECTED external conditions (file not found, network failure, invalid SQL) | Programming errors/bugs (null dereference, invalid array index, bad cast)                                                |
| Examples                | IOException, SQLException, ClassNotFoundException                                        | NullPointerException, ArithmeticException, ArrayIndexOutOfBoundsException, IllegalArgumentException                      |
| Typical response        | Caught and handled meaningfully (retry, fallback, user message)                          | Usually indicates a bug to FIX in the code, not "handle" gracefully at runtime                                           |
| Error (separate branch) | Neither checked nor meant to be caught typically                                         | Represents serious JVM-level problems (OutOfMemoryError, StackOverflowError) — recovery is usually not possible/sensible |

### 4. Real Life Analogy

- **Bank:** A checked exception is like an ATM correctly detecting "insufficient funds" and REQUIRING the software to handle that expected scenario before proceeding — a normal, anticipated business condition. An unchecked exception is like the ATM's internal wiring short-circuiting due to a manufacturing defect — a genuine BUG, not an expected business scenario to "handle" gracefully.
- **Restaurant:** A checked exception is like "Out of Stock" for a menu item — an expected, recoverable condition the ordering system MUST account for. An unchecked exception is like a waiter accidentally dropping a tray due to a coding error in the serving robot — a bug, not a normal business condition.

### 5. Code Examples

```
import java.io.*;

public class ExceptionTypesDemo {
    public static void main(String[] args) {
        // UNCHECKED - compiler doesn't force a try-catch, but it will still crash at runtime if uncaught
        int[] arr = new int[3];
        System.out.println(arr[5]); // ArrayIndexOutOfBoundsException (unchecked)
    }

    // CHECKED - compiler FORCES you to either catch or declare via throws
    static void readFile(String path) throws IOException {
        FileReader fr = new FileReader(path); // can throw FileNotFoundException (a checked IOException subtype)
    }
}
```

## 6. Edge Cases

| Case                                                                                | Result                                                                                                                                                                                |
|-------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| A checked exception thrown but neither caught nor declared with <code>throws</code> | Compile-time error                                                                                                                                                                    |
| An unchecked exception thrown and never caught anywhere                             | Propagates up, eventually printed as a stack trace, terminating that thread (or the whole program if it's the main thread)                                                            |
| Catching <code>Exception</code> (the broad parent) instead of a specific type       | Compiles and "works," but hides genuinely different error types under one generic handler — a common anti-pattern                                                                     |
| Catching <code>Throwable</code>                                                     | Technically legal, catches EVERYTHING including <code>Errors</code> — almost always a bad practice, since most <code>Errors</code> genuinely shouldn't be "handled" as if recoverable |

## 7. Common Mistakes

- Catching overly broad exception types (`catch (Exception e)`) instead of the specific exception actually expected, masking real bugs.
- Treating unchecked exceptions as "acceptable to ignore" simply because the compiler doesn't force handling — they still crash the program if genuinely uncaught.
- Trying to "handle" `Error` subtypes (`OutOfMemoryError`) as if they were recoverable business conditions — usually futile and can mask a genuinely critical system problem.

## 8. Best Practices

- Catch the MOST SPECIFIC exception type that's actually meaningful to handle differently.
- Use checked exceptions for genuinely recoverable, expected conditions callers should be forced to consider; use unchecked (custom `RuntimeException` subclasses) for programming errors/invariant violations.
- Never silently swallow exceptions (`catch (Exception e) { }` with an empty body) — at minimum, log them.

## 9. Frequently Asked Interview Questions

- 1 **What's the difference between checked and unchecked exceptions?** Checked exceptions must be caught or declared (compiler-enforced), representing expected/recoverable conditions; unchecked exceptions (`RuntimeException` and its subclasses) aren't compiler-enforced, typically representing programming bugs.
- 2 **Where does `Error` fit in the hierarchy?** A sibling of `Exception` under `Throwable`, representing serious JVM-level problems not meant to be handled as recoverable business logic.
- 3 **Give two examples each of checked and unchecked exceptions.** Checked: `IOException`, `SQLException`. Unchecked: `NullPointerException`, `ArrayIndexOutOfBoundsException`.
- 4 **Is `RuntimeException` checked or unchecked?** Unchecked — it and all its subclasses are exempt from the compiler's catch-or-declare requirement.



- 5 **Why shouldn't you routinely catch Throwable?** It catches `Errors` too, which typically represent unrecoverable JVM-level problems that shouldn't be treated as normal, handleable business conditions.

## 10. Revision Notes

### EXCEPTION HIERARCHY – CHEAT SHEET

=====

Throwable

- Error (JVM-level, unrecoverable: `OutOfMemoryError`, `StackOverflowError`)
- Exception
  - `RuntimeException` (UNCHECKED: `NullPointerException`, `ArrayIndexOutOfBoundsException`, `ArithmeticException`)
  - (everything else) (CHECKED: `IOException`, `SQLException`, `ClassNotFoundException`)

CHECKED: compiler FORCES catch or `throws` declaration

UNCHECKED: no compiler enforcement, but STILL crashes if genuinely uncaught

CHECKED = expected/recoverable conditions

UNCHECKED = programming bugs/invariant violations

## TOPIC 42: TRY-CATCH-FINALLY

### 1. Introduction

**What is it?** `try-catch-finally` is Java's core syntax for handling exceptions: code that might throw is wrapped in `try`, matching handlers in `catch` blocks, and `finally` guarantees cleanup code runs regardless of whether an exception occurred.

### 2. Syntax and Execution Order

```
try {
    riskyOperation();
} catch (IOException e) {
    System.out.println("Handled IOException: " + e.getMessage());
} catch (RuntimeException e) {
    System.out.println("Handled RuntimeException: " + e.getMessage());
} finally {
    System.out.println("This ALWAYS runs - cleanup code goes here");
}
```

- Multiple `catch` blocks are checked TOP TO BOTTOM — the FIRST matching type handles it; order matters (more specific exceptions must come BEFORE more general ones, or it's a compile-time "unreachable catch block" error).
- `finally` runs in ALL cases: normal completion, an exception was caught, an exception was NOT caught (propagating further), or even a `return` inside `try/catch`.

### 3. Multi-Catch (Java 7+)

```
try {
    riskyOperation();
} catch (IOException | SQLException e) { // single block handles BOTH types
    System.out.println("I/O or SQL problem: " + e.getMessage());
}
```

### 4. Try-With-Resources (Java 7+) — Automatic Resource Cleanup

```
try (FileReader fr = new FileReader("data.txt");
    BufferedReader br = new BufferedReader(fr)) {
    System.out.println(br.readLine());
} // both fr and br are AUTOMATICALLY closed here, in REVERSE order, even if an exception occurs
```

- Any resource implementing `AutoCloseable` (or `Closeable`) can be used here — `close()` is guaranteed to be called, eliminating the classic bug of forgetting to close streams/connections in a `finally` block manually.

### 5. Internal Working — Desugaring (Recap from [[Java Compilation Process|Topic 3]])

```
// try-with-resources SOURCE:
try (FileReader fr = new FileReader("f")) { use(fr); }

// DESUGARS TO (conceptually):
FileReader fr = new FileReader("f");
Throwable primaryException = null;
try {
    use(fr);
} catch (Throwable t) {
    primaryException = t;
    throw t;
} finally {
    if (fr != null) {
        if (primaryException != null) {
            try { fr.close(); } catch (Throwable suppressed) { primaryException.addSuppressed(suppressed); }
        } else {
            fr.close();
        }
    }
}
```

## 6. Edge Cases

| Case                                                                                               | Result                                                                                                                                       |
|----------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| return inside try, ALSO a finally block with its own return                                        | The finally block's return WINS, silently discarding the try block's return value — a well-known, dangerous anti-pattern                     |
| Exception thrown inside finally itself                                                             | Replaces/masks whatever exception was propagating from try/catch — another reason to keep finally blocks simple                              |
| Multiple catch blocks, a subclass exception caught by a listed SUPERCLASS type earlier in the list | Compile-time error: "exception X has already been caught" if a more specific catch is placed AFTER a broader one that would already catch it |
| Resources in try-with-resources fail to close (their close() itself throws)                        | That exception is added as a SUPPRESSED exception on the primary one (retrievable via getSuppressed()), not silently lost                    |

## 7. Common Mistakes

- Returning from within a finally block — silently overrides any exception or return value from the try/catch, a serious and confusing anti-pattern.
- Manually closing resources in a finally block instead of using try-with-resources — more verbose and error-prone (forgetting null-checks, nested resource closing order).
- Ordering catch blocks from general to specific — causes a compile error since the specific catch becomes unreachable.

## 8. Best Practices

- Always prefer try-with-resources over manual finally-based resource cleanup for anything implementing AutoCloseable.
- Never put a return statement inside a finally block.
- Order catch blocks from MOST specific to LEAST specific.

## 9. Frequently Asked Interview Questions

- 1 **Does finally always run?** Yes — regardless of whether an exception was thrown, caught, or a return occurred in try/catch (the only real exceptions are JVM shutdown via System.exit() or a crash).
- 2 **What happens if both try and finally have a return statement?** The finally block's return wins, discarding the try block's return value — a well-known anti-pattern to avoid.
- 3 **What is try-with-resources, and what interface must a resource implement?** A Java 7+ syntax that automatically calls close() on resources implementing AutoCloseable (or Closeable), in reverse declaration order, even if an exception occurs.

- 4 **What happens if `close()` itself throws inside try-with-resources, while another exception is already propagating?** The `close()` exception is added as a SUPPRESSED exception on the primary one, retrievable via `getSuppressed()`, rather than replacing/losing the original.
- 5 **Can you catch multiple exception types in one `catch` block?** Yes, using multi-catch syntax: `catch (IOException | SQLException e)`.

## 10. Revision Notes

### TRY-CATCH-FINALLY – CHEAT SHEET

=====

catch blocks checked TOP TO BOTTOM - most specific FIRST (else compile error)  
finally ALWAYS runs (except `System.exit()` or JVM crash)

GOTCHA: return in finally OVERRIDES try/catch's return - NEVER do this  
try-with-resources (Java 7+): auto-closes `AutoCloseable` resources, reverse order, even on exception; `close()`-time exceptions become SUPPRESSED, not lost (`getSuppressed()`)  
multi-catch: `catch (TypeA | TypeB e) { }`

## TOPIC 43: THROW AND THROWS

### 1. Introduction

**What is it?** `throw` is a statement that explicitly raises an exception instance at the point of execution. `throws` is a method declaration keyword that declares which checked exceptions a method MIGHT propagate to its caller, without handling them itself.

### 2. `throw` vs `throws` — Comparison Table

| Aspect         | <code>throw</code>                                                                         | <code>throws</code>                                                                 |
|----------------|--------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| Used where     | Inside a method BODY                                                                       | In a method SIGNATURE                                                               |
| Purpose        | Actually RAISES an exception instance                                                      | DECLARES that a method might propagate a checked exception                          |
| Followed by    | A single exception INSTANCE ( <code>throw new IOException("msg")</code> )                  | One or more exception CLASS names ( <code>throws IOException, SQLException</code> ) |
| Number allowed | Exactly one <code>throw</code> per statement (though reachable multiple times in a method) | Multiple exception types in one <code>throws</code> clause                          |

### 3. Code Examples

```
public class ThrowThrowsDemo {

    static void validateAge(int age) {
        if (age < 0) {
            throw new IllegalArgumentException("Age cannot be negative"); // THROW - raises immediately
        }
    }

    static void readConfig(String path) throws IOException { // THROWS - declares propagation
        if (path == null) {
            throw new FileNotFoundException("Path is null");
        }
        // ... actual file reading logic that might also throw IOException
    }

    public static void main(String[] args) {
        try {
            readConfig(null);
        } catch (IOException e) {
            System.out.println("Caught: " + e.getMessage());
        }
    }
}
```

### 4. Rethrowing and Exception Chaining

```
try {
    riskyDatabaseCall();
} catch (SQLException e) {
    throw new RuntimeException("Failed to process order", e); // WRAPS the original as the 'cause'
}
```

- The second constructor argument (e) becomes the new exception's **cause**, preserving the full original stack trace, retrievable later via `getCause()` — critical for real debugging, since the ORIGINAL low-level error (a specific SQL failure) shouldn't be lost just because a higher-level, more meaningful exception wraps it.

### 5. Edge Cases

| Case                                                                                                        | Result                                                                                 |
|-------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| A method declares <code>throws IOException</code> but never actually throws it                              | Perfectly legal — <code>throws</code> is a declaration of POSSIBILITY, not a guarantee |
| Overriding a method that declares <code>throws IOException</code> , adding a NEW, broader checked exception | Compile-time error (see <a href="#">[[Method Overriding</a>                            |
| <code>throw null;</code>                                                                                    | Throws a <code>NullPointerException</code> (not literally "nothing")                   |
| Method body has unreachable code after an unconditional <code>throw</code>                                  | Compile-time error: "unreachable statement"                                            |

### 6. Common Mistakes

- Confusing `throw` (a statement, raises an instance) with `throws` (a declaration, lists possible checked exception types).

- Losing the original exception's context when wrapping/rethrowing — always pass the original as the `cause` argument, never discard it.
- Declaring overly broad `throws Exception` on methods, forcing every caller to handle a vague, uninformative catch-all instead of specific, meaningful exception types.

## 7. Best Practices

- Always chain the original exception as the `cause` when wrapping/rethrowing a different exception type.
- Declare the MOST SPECIFIC checked exception types a method can actually throw, not a broad `throws Exception`.
- Reserve `throw` for genuinely exceptional conditions, not routine control flow (exceptions have real performance cost compared to normal conditional logic, and abusing them for control flow harms readability).

## 8. Frequently Asked Interview Questions

- 1 **What's the difference between `throw` and `throws`?** `throw` is a statement that actually raises an exception instance inside a method body; `throws` is a method-signature declaration listing checked exception types that MIGHT propagate.
- 2 **Can a method declare `throws` for an exception it never actually throws?** Yes — it's legal; `throws` declares a possibility, not a certainty.
- 3 **How do you preserve the original exception when wrapping it in a new one?** Pass the original exception as the second constructor argument (the `cause`), retrievable later via `getCause()`.
- 4 **What does `throw null;` do?** Throws a `NullPointerException`, since there's no actual exception object to throw.
- 5 **Why is it discouraged to declare `throws Exception` broadly on a method?** It forces every caller into a vague, uninformative catch-all, hiding which SPECIFIC checked exceptions could actually occur and how to meaningfully handle each.

## 9. Revision Notes

### THROW vs THROWS — CHEAT SHEET

=====

`throw` -> STATEMENT, raises a SPECIFIC exception INSTANCE, in a method BODY

`throws` -> DECLARATION, lists possible checked exception TYPES, in the SIGNATURE

CHAINING: `new RuntimeException("context msg", originalException)`

-> preserves original via `getCause()`, critical for real debugging

`throws` declares POSSIBILITY, not a guarantee the exception will occur

# TOPIC 44: CUSTOM EXCEPTIONS

---

## 1. Introduction

**What is it?** A custom exception is a user-defined class extending `Exception` (checked) or `RuntimeException` (unchecked) to represent a specific, meaningful, application/domain-level error condition not adequately captured by the JDK's built-in exception types.

**Why introduced:** Generic exceptions (`Exception`, `RuntimeException`, `IllegalArgumentException`) don't convey domain-specific meaning — a `InsufficientFundsException` is far more expressive, self-documenting, and catchable-specifically than a generic `IllegalStateException` for the exact same underlying business condition.

## 2. Syntax — Checked vs Unchecked Custom Exception

```
// CHECKED custom exception - callers are FORCED to handle it
public class InsufficientFundsException extends Exception {
    public InsufficientFundsException(String message) {
        super(message);
    }
    public InsufficientFundsException(String message, Throwable cause) {
        super(message, cause);
    }
}

// UNCHECKED custom exception - represents a programming error/invariant violation
public class InvalidAccountStateException extends RuntimeException {
    public InvalidAccountStateException(String message) {
        super(message);
    }
}
```

## 3. Real Life Analogy

- **Bank:** `InsufficientFundsException` (checked — an EXPECTED business scenario every withdrawal-processing caller MUST consider) versus `CorruptedLedgerException` (unchecked — represents an internal system BUG/invariant violation, not a normal business scenario to handle gracefully).

## 4. Code Examples

```
class InsufficientFundsException extends Exception {
    private final double shortfall;
    public InsufficientFundsException(double shortfall) {
        super("Insufficient funds - short by: " + shortfall);
        this.shortfall = shortfall;
    }
    public double getShortfall() { return shortfall; }
}

class BankAccount {
    private double balance;
    BankAccount(double balance) { this.balance = balance; }

    void withdraw(double amount) throws InsufficientFundsException {
        if (amount > balance) {
            throw new InsufficientFundsException(amount - balance);
        }
        balance -= amount;
    }
}

public class CustomExceptionDemo {
    public static void main(String[] args) {
        BankAccount acc = new BankAccount(100);
        try {
            acc.withdraw(150);
        } catch (InsufficientFundsException e) {
            System.out.println(e.getMessage() + " | Shortfall: " + e.getShortfall());
        }
    }
}
```

## 5. Best Practices for Custom Exceptions

- Extend `Exception` for genuinely recoverable, expected business conditions; extend `RuntimeException` for programming errors/invariant violations.
- Always provide constructors matching the standard pattern: `(String message)` and `(String message, Throwable cause)`, for proper exception chaining support.
- Add meaningful additional fields/methods (like `getShortfall()` above) when extra structured context genuinely helps callers handle the error more precisely, rather than just parsing a message string.
- Name custom exceptions clearly and specifically, ending in `Exception` by strong convention.

## 6. Common Mistakes

- Creating custom exceptions that add no real value over an existing built-in type (essentially just renaming `IllegalArgumentException` with zero additional context) — unnecessary proliferation.
- Forgetting to provide a `(message, cause)` constructor, breaking the ability to properly chain/wrap underlying causes.
- Extending `Exception` when the condition is actually a programming bug (should be `RuntimeException`), forcing unnecessary `try-catch` boilerplate on every caller for something that should simply be fixed in code, not "handled."



## 7. Production Usage

- Nearly every enterprise Java codebase has a rich hierarchy of custom exceptions (`OrderNotFoundException`, `PaymentDeclinedException`, `UserAlreadyExistsException`) mapped to specific HTTP status codes in REST APIs via Spring's `@ExceptionHandler/@ControllerAdvice` mechanisms (covered in the REST APIs topic).

## 8. Frequently Asked Interview Questions

- When would you create a custom exception instead of using a built-in one?** When a specific, domain-meaningful error condition isn't adequately captured by generic built-in types, and additional structured context (custom fields/methods) or specific catchability genuinely helps callers.
- Should a custom exception extend `Exception` or `RuntimeException`?** `Exception` for genuinely expected, recoverable business conditions callers should be forced to consider; `RuntimeException` for programming errors/invariant violations.
- What constructors should every custom exception typically provide?** At minimum, `(String message)` and `(String message, Throwable cause)`, to support proper exception chaining.
- Give a real production example of a custom exception hierarchy.** `OrderNotFoundException`, `PaymentDeclinedException`, `UserAlreadyExistsException` — each mapped to specific HTTP responses in a Spring Boot REST API.

## 9. Revision Notes

### CUSTOM EXCEPTIONS – CHEAT SHEET

=====

extends `Exception` -> CHECKED, for expected/recoverable business conditions

extends `RuntimeException` -> UNCHECKED, for programming errors/invariant violations

ALWAYS provide: `(String message)` AND `(String message, Throwable cause)` constructors

Add extra fields/methods when structured context genuinely helps callers

Name clearly, ending in "Exception" by convention

PRODUCTION: custom exception hierarchies map directly to REST API error responses via Spring's `@ExceptionHandler/@ControllerAdvice`

## 10. Homework (Covers Topics 41-44)

- Write a method that throws a checked custom exception, catch it with proper multi-catch alongside a built-in exception type, and print both the message and cause chain.
- Build a try-with-resources example using two chained `AutoCloseable` resources, deliberately make the second one's `close()` throw, and observe the suppressed exception via `getSuppressed()`.
- Design a 3-exception custom hierarchy for a hypothetical "Library" system (e.g., `BookNotFoundException`, `BookAlreadyBorrowedException`), deciding checked vs unchecked for each with justification.

# COLLECTIONS FRAMEWORK

## TOPIC 45: THE COLLECTIONS FRAMEWORK & THE `List` INTERFACE

### 1. Introduction

**What is it?** The Java Collections Framework (JCF) is a unified architecture of interfaces and classes for storing and manipulating groups of objects — `List`, `Set`, `Map`, `Queue`, and their many implementations — introduced in Java 2 (1998) to replace the earlier inconsistent, non-generic `Vector`/`Hashtable`/`Enumeration` classes.

**Why introduced:** Before the JCF, each container type (`Vector`, `Hashtable`, arrays) had its own inconsistent API (different method names for adding/removing elements, no shared interfaces), making generic, reusable algorithms impossible. The JCF unifies everything under common interfaces (`Collection`, `List`, `Set`, `Map`), enabling one `Collections.sort()` to work on ANY `List` implementation, for example.

### 2. The Core Hierarchy

```
Iterable<E>
|
Collection<E>
/ | \
List<E> Set<E> Queue<E>
| | |
ArrayList, HashSet, PriorityQueue,
LinkedList, LinkedHashSet, ArrayDeque
Vector, Stack TreeSet

(Map<K,V> is SEPARATE - does NOT extend Collection, since it deals
in key-value PAIRS, not single elements)
HashMap, LinkedHashMap, TreeMap, Hashtable
```

### 3. The `List` Interface

**What is it?** `List` is an ORDERED collection (also called a sequence) that allows **duplicate elements** and provides **index-based access**.

| Aspect              | Detail                                                                 |
|---------------------|------------------------------------------------------------------------|
| Ordering            | Maintains insertion order                                              |
| Duplicates          | Allowed                                                                |
| Index access        | Yes — <code>get(int)</code> , <code>set(int, E)</code>                 |
| Key implementations | <code>ArrayList</code> , <code>LinkedList</code> , <code>Vector</code> |

## 4. List vs Set vs Map — High-Level Comparison (Preview)

| Aspect     | List                     | Set                               | Map                                  |
|------------|--------------------------|-----------------------------------|--------------------------------------|
| Duplicates | Allowed                  | NOT allowed                       | Keys NOT allowed (values can repeat) |
| Ordering   | Insertion order, indexed | Depends on implementation         | Depends on implementation            |
| Access     | By index                 | By value (via iteration/contains) | By key                               |

## 5. Real Life Analogy

- **Restaurant:** A `List<Order>` for a table's queue of orders — ordered exactly as placed, duplicates allowed (same dish ordered twice), and you can ask for "the 3rd order placed" directly by index.
- **Bank:** A `Set<AccountNumber>` of registered accounts — no duplicates allowed (an account number can't be registered twice), order isn't the primary concern.

## 6. Code Example — Common List Operations

```
import java.util.*;

public class ListDemo {
    public static void main(String[] args) {
        List<String> fruits = new ArrayList<>();
        fruits.add("Apple");
        fruits.add("Banana");
        fruits.add(1, "Mango"); // insert at specific index
        fruits.add("Apple"); // duplicates ARE allowed

        System.out.println(fruits); // [Apple, Mango, Banana, Apple]
        System.out.println(fruits.get(0)); // "Apple"
        fruits.remove("Banana"); // removes FIRST matching element
        fruits.remove(0); // removes by INDEX (overloaded method - careful!)
        System.out.println(fruits);
    }
}
```

## 7. Edge Cases

| Case                                                                                                              | Result                                                                                                                                 |
|-------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| <code>list.remove(1)</code> vs <code>list.remove(Integer.valueOf(1))</code> on a <code>List&lt;Integer&gt;</code> | The first removes by INDEX; the second removes the OBJECT 1 — a classic overload-ambiguity gotcha for <code>List&lt;Integer&gt;</code> |
| <code>list.get(list.size())</code>                                                                                | <code>IndexOutOfBoundsException</code>                                                                                                 |
| Modifying a <code>List</code> while iterating with a plain for-each loop                                          | <code>ConcurrentModificationException</code> (see <code>Iterator</code> )                                                              |
| <code>List.of(...)</code> (Java 9+) elements                                                                      | Returns an IMMUTABLE list — <code>add()</code> / <code>remove()</code> throw <code>UnsupportedOperationException</code>                |

## 8. Common Mistakes

- Calling `list.remove(1)` on a `List<Integer>` expecting to remove the VALUE 1, but it actually removes the element at INDEX 1 — must use `list.remove(Integer.valueOf(1))` to remove by value.
- Modifying a list directly while iterating over it with a for-each loop, causing `ConcurrentModificationException`.
- Treating `List.of(...)` results as mutable — they are immutable and throw on any modification attempt.

## 9. Frequently Asked Interview Questions

- 1 **What is the `List` interface?** An ordered collection allowing duplicates, with index-based access.
- 2 **What's the ambiguity with `list.remove(1)` for `List<Integer>`?** It's interpreted as removing by INDEX (since `remove(int)` is a distinct overload from `remove(Object)`) — to remove the VALUE 1, you must call `list.remove(Integer.valueOf(1))`.
- 3 **Does `List` allow duplicates?** Yes.
- 4 **What does `List.of(...)` return?** An immutable list — any mutation attempt throws `UnsupportedOperationException`.

---

# TOPIC 46: ARRAYLIST

---

## 1. Introduction

**What is it?** `ArrayList` is a resizable-array implementation of the `List` interface — the most commonly used `List` implementation in Java, backed internally by a dynamically-growing `Object[]` array.

## 2. Internal Working

```
ArrayList<Integer> list = new ArrayList<>(); // default initial capacity: 10
```

- Backed by an `Object[] elementData` internally.
- When capacity is exceeded, a NEW array of size `oldCapacity + (oldCapacity >> 1)` (i.e., **1.5× growth**) is allocated, and ALL existing elements are copied over — an  $O(n)$  operation, but happening infrequently enough that `add()` is **amortized  $O(1)$** .

```
Initial: [_, _, _, _, _, _, _, _, _, _] (capacity 10)
... after 10 adds, capacity exceeded ...
Grow to: [_, _, _, _, _, _, _, _, _, _, _, _, _, _, _] (capacity 15, all 10 old elements copied)
```

### 3. Key Methods with Time Complexity

| Method                                 | Time Complexity  | Note                                      |
|----------------------------------------|------------------|-------------------------------------------|
| <code>get(int index)</code>            | $O(1)$           | Direct array offset access                |
| <code>add(E element)</code> (at end)   | $O(1)$ amortized | Occasional $O(n)$ resize                  |
| <code>add(int index, E element)</code> | $O(n)$           | Must shift all subsequent elements        |
| <code>remove(int index)</code>         | $O(n)$           | Must shift all subsequent elements down   |
| <code>contains(Object o)</code>        | $O(n)$           | Linear search, uses <code>equals()</code> |
| <code>size()</code>                    | $O(1)$           | Maintained as a field, not recomputed     |

### 4. Code Example

```
import java.util.*;

public class ArrayListDemo {
    public static void main(String[] args) {
        List<String> list = new ArrayList<>(20); // pre-sized capacity, avoids early resizing
        list.add("A");
        list.add("B");
        list.add(0, "Z"); // O(n) - shifts A, B right
        System.out.println(list); // [Z, A, B]
        System.out.println(list.get(1)); // O(1) - "A"
    }
}
```

### 5. Edge Cases

| Case                                                                              | Result                                                                                                                                |
|-----------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| Adding elements to a pre-sized <code>ArrayList</code> beyond its initial capacity | Still works correctly — just triggers the normal resize-and-copy growth                                                               |
| <code>ArrayList</code> of primitives (e.g., <code>ArrayList&lt;int&gt;</code> )   | Not allowed directly — must use the wrapper type ( <code>ArrayList&lt;Integer&gt;</code> ), incurring autoboxing overhead per element |
| Frequent insertions/removals at the FRONT of a large <code>ArrayList</code>       | $O(n)$ per operation — a <code>LinkedList</code> (next topic) is more efficient for this specific access pattern                      |

### 6. Best Practices

- Pre-size the initial capacity (`new ArrayList<>(expectedSize)`) when the approximate final size is known, avoiding repeated resize-and-copy operations.
- Use `ArrayList` as the DEFAULT List choice — its  $O(1)$  random access and generally good cache locality make it the best fit for most real-world use cases.

### 7. Frequently Asked Interview Questions

- 1 What's the default initial capacity of `ArrayList`? 10.
- 2 What's `ArrayList`'s growth factor?  $1.5\times$  (new capacity = old + old/2), unlike `Vector`'s  $2\times$ .

- 3 **What's the time complexity of `get(index)`?**  $O(1)$  — direct array access.
- 4 **What's the time complexity of inserting at the beginning?**  $O(n)$  — every subsequent element must shift right.
- 5 **Why is `add()` considered "amortized  $O(1)$ " rather than strictly  $O(1)$ ?** Because most calls are  $O(1)$ , but occasionally a resize-and-copy operation costs  $O(n)$  — averaged (amortized) across many calls, the cost per call is still  $O(1)$ .

## TOPIC 47: LINKEDLIST

### 1. Introduction

**What is it?** `LinkedList` is a doubly-linked-list implementation of BOTH the `List` and `Deque` interfaces — each element is a node holding references to the PREVIOUS and NEXT nodes, rather than being stored in one contiguous array like `ArrayList`.

### 2. Internal Working

```

null <- [prev|A|next] <-> [prev|B|next] <-> [prev|C|next] -> null
head tail

```

- Insertion/removal at the head or tail is  $O(1)$  — just re-link a few pointers, no shifting needed.
- Random access (`get(index)`) requires traversing from the head OR tail (whichever is closer) —  $O(n)$ .

### 3. `ArrayList` vs `LinkedList` — Full Comparison Table

| Aspect                                   | <code>ArrayList</code>                        | <code>LinkedList</code>                             |
|------------------------------------------|-----------------------------------------------|-----------------------------------------------------|
| Backing structure                        | Dynamic array                                 | Doubly-linked list of nodes                         |
| <code>get(index)</code>                  | $O(1)$                                        | $O(n)$                                              |
| <code>add()/remove()</code> at END       | $O(1)$ amortized                              | $O(1)$                                              |
| <code>add()/remove()</code> at BEGINNING | $O(n)$ (shift all elements)                   | $O(1)$                                              |
| <code>add()/remove()</code> in MIDDLE    | $O(n)$ (shift)                                | $O(n)$ (traversal to find position) + $O(1)$ relink |
| Memory overhead                          | Lower (just the array + some slack capacity)  | Higher (each node stores 2 extra references)        |
| Cache locality                           | Better (contiguous memory)                    | Worse (scattered node objects)                      |
| Implements                               | <code>List</code> , <code>RandomAccess</code> | <code>List</code> , <code>Deque</code>              |

**Interview reality check:** `LinkedList` is very rarely the right choice in modern Java — `ArrayList` (or `ArrayDeque` for genuine queue/stack needs) usually outperforms it in practice due to better cache locality, even for "insert at front" patterns, unless the list is extremely large and insertions are heavily concentrated at known node positions.

## 4. Code Example

```
import java.util.*;

public class LinkedListDemo {
    public static void main(String[] args) {
        LinkedList<String> list = new LinkedList<>();
        list.addFirst("B");
        list.addFirst("A"); // O(1) - just relinks the head pointer
        list.addLast("C"); // O(1) - just relinks the tail pointer
        System.out.println(list); // [A, B, C]

        list.removeFirst(); // O(1)
        System.out.println(list); // [B, C]
    }
}
```

## 5. Frequently Asked Interview Questions

- 1 **What data structure backs `LinkedList`?** A doubly-linked list of nodes, each holding references to the previous and next nodes.
  - 2 **What's the time complexity of `get(index)` on a `LinkedList`?**  $O(n)$  — requires traversal from the head or tail.
  - 3 **When is `LinkedList` actually a good choice over `ArrayList`?** When insertions/removals happen frequently at the FRONT (or at an already-held node reference) and random-access-by-index is rarely needed — though in practice, `ArrayDeque` often outperforms it even here due to better cache locality.
  - 4 **Which interfaces does `LinkedList` implement besides `List`?** `Deque` — allowing it to function as a double-ended queue, stack, or queue directly.
- 

# TOPIC 48: VECTOR

---

## 1. Introduction

**What is it?** `Vector` is a legacy (Java 1.0), synchronized, resizable-array `List` implementation — functionally very similar to `ArrayList`, but with every method marked `synchronized`, and a DOUBLING (2×) growth strategy instead of `ArrayList`'s 1.5×.

## 2. Vector vs ArrayList — Comparison Table

| Aspect                | Vector                                                                                                                                                           | ArrayList                       |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------|
| Thread-safety         | Synchronized (thread-safe, but with overhead)                                                                                                                    | NOT synchronized                |
| Growth factor         | 2× (doubles capacity)                                                                                                                                            | 1.5×                            |
| Introduced            | Java 1.0 (legacy)                                                                                                                                                | Java 2 (part of the modern JCF) |
| Modern recommendation | Avoid — use <code>Collections.synchronizedList(new ArrayList&lt;&gt;())</code> or <code>CopyOnWriteArrayList</code> instead if thread-safety is genuinely needed | Default choice                  |

## 3. Code Example

```
import java.util.Vector;

Vector<String> vector = new Vector<>();
vector.add("A");
vector.add("B");
System.out.println(vector); // [A, B] - same API surface as ArrayList/List
```

## 4. Frequently Asked Interview Questions

- 1 **Is Vector thread-safe?** Yes — all its methods are `synchronized`, unlike `ArrayList`.
- 2 **Why is Vector considered a legacy class today?** It predates the modern Collections Framework, and its blanket per-method synchronization is usually unnecessary overhead for single-threaded use — `ArrayList` (or specific concurrent collections when genuinely needed) is preferred today.
- 3 **What's Vector's growth factor compared to ArrayList's?** `Vector` doubles (2×) capacity; `ArrayList` grows by 1.5×.
- 4 **What modern alternative should be used instead of Vector for thread-safe lists?** `Collections.synchronizedList(new ArrayList<>())` for coarse-grained synchronization, or `CopyOnWriteArrayList` for read-heavy, rarely-modified concurrent scenarios.

---

(End of Topics 45-48: List Family — List, ArrayList, LinkedList, Vector.)

---

# TOPIC 49: STACK

---

## 1. Introduction

**What is it?** `Stack` is a legacy (Java 1.0) class extending `Vector`, implementing a classic **LIFO (Last-In-First-Out)** data structure — the element added most recently is the first one removed.



**Interview note:** `Stack` extends `Vector` is now widely considered a JDK design mistake — it inherits ALL of `Vector`'s list-like methods (`get`, `add(index, e)`, etc.), which breaks the pure LIFO contract a stack should enforce (you could bypass `push/pop` and directly insert/remove from the middle). The JDK documentation itself recommends using `Deque` (specifically `ArrayDeque`) instead for stack behavior in modern code.

## 2. Key Methods

| Method                    | Purpose                                                                                      |
|---------------------------|----------------------------------------------------------------------------------------------|
| <code>push(E item)</code> | Adds to the TOP of the stack                                                                 |
| <code>pop()</code>        | Removes and returns the TOP element; throws <code>EmptyStackException</code> if empty        |
| <code>peek()</code>       | Returns (without removing) the TOP element; throws <code>EmptyStackException</code> if empty |
| <code>isEmpty()</code>    | Checks if the stack has no elements                                                          |

## 3. Code Example

```
import java.util.Stack;

Stack<Integer> stack = new Stack<>();
stack.push(1);
stack.push(2);
stack.push(3);
System.out.println(stack.pop()); // 3 (LIFO - last pushed, first popped)
System.out.println(stack.peek()); // 2 (top, not removed)
```

## 4. Real Life Analogy

- **Restaurant:** A stack of clean plates — you always take the TOP plate (most recently placed), and new clean plates go on TOP too — classic LIFO.

## 5. Modern Alternative — `ArrayDeque` as a Stack

```
Deque<Integer> stack = new ArrayDeque<>();
stack.push(1);
stack.push(2);
System.out.println(stack.pop()); // 2 - same LIFO behavior, WITHOUT Vector's baggage
```

## 6. Frequently Asked Interview Questions

- 1 **What data structure does `Stack` implement?** LIFO (Last-In-First-Out).
- 2 **Why is `Stack` extends `Vector` considered a design flaw?** Because it inherits list-like methods that let you bypass the LIFO discipline (insert/access at arbitrary positions), and inherits `Vector`'s synchronization overhead unconditionally.
- 3 **What's the modern recommended replacement for `Stack`?** `ArrayDeque`, used via its `push()`/`pop()`/`peek()` methods.
- 4 **What exception does `pop()` throw on an empty stack?** `EmptyStackException`.

# TOPIC 50: QUEUE

## 1. Introduction

**What is it?** `Queue` is an interface representing a **FIFO (First-In-First-Out)** collection — elements are added at the tail (`offer/add`) and removed from the head (`poll/remove`), modeling a waiting line.

## 2. Key Methods — The "Safe" vs "Throwing" Pairs

| Operation | Throws on failure                                                                                   | Returns special value on failure                   |
|-----------|-----------------------------------------------------------------------------------------------------|----------------------------------------------------|
| Insert    | <code>add(e)</code> — throws <code>IllegalStateException</code> if full (rare for unbounded queues) | <code>offer(e)</code> — returns <code>false</code> |
| Remove    | <code>remove()</code> — throws <code>NoSuchElementException</code> if empty                         | <code>poll()</code> — returns <code>null</code>    |
| Examine   | <code>element()</code> — throws <code>NoSuchElementException</code> if empty                        | <code>peek()</code> — returns <code>null</code>    |

**Best practice:** Prefer the "safe" methods (`offer/poll/peek`) over the throwing ones in most application code — they let you handle an empty/full queue via a simple `null/false` check instead of exception-based control flow.

## 3. Real Life Analogy

- **Bank:** A physical queue (line) of customers — the first person to join is the first one served (FIFO), new customers join at the back.

## 4. Code Example

```
import java.util.*;

Queue<String> queue = new LinkedList<>(); // LinkedList implements Queue too
queue.offer("Customer1");
queue.offer("Customer2");
System.out.println(queue.poll()); // "Customer1" - FIFO, removes from the FRONT
System.out.println(queue.peek()); // "Customer2" - examines without removing
```

## 5. Frequently Asked Interview Questions

- 1 **What ordering does `Queue` follow?** FIFO (First-In-First-Out).
- 2 **What's the difference between `add()/remove()/element()` and `offer()/poll()/peek()`?** The former THROW exceptions on failure conditions (full/empty); the latter return a special value (`false/null`) instead — generally preferred for cleaner control flow.
- 3 **Name a class that implements `Queue`.** `LinkedList`, `ArrayDeque`, `PriorityQueue`.

---

# TOPIC 51: PRIORITYQUEUE

---

## 1. Introduction

**What is it?** `PriorityQueue` is a `Queue` implementation backed internally by a **binary heap**, where elements are dequeued in **PRIORITY** order (smallest/highest priority first by default, per natural ordering or a supplied `Comparator`) rather than strict insertion order.

## 2. Internal Working — The Binary Heap

- Internally backed by an `ARRAY` representing a complete binary tree, where the parent at index `i` has children at `2i+1` and `2i+2`.
- The **MINIMUM** element (by default, natural ordering — a **MIN-heap**) is always at index 0 (the root), retrievable in  $O(1)$  via `peek()`.
- `offer()/poll()` maintain the heap property via "sift up"/"sift down" operations — both  $O(\log n)$ .

Min-Heap example (natural ordering of Integers):

```
1
 / \
3 5
 / \ /
9 7 6
```

Array representation: [1, 3, 5, 9, 7, 6]

## 3. Key Methods with Time Complexity

| Method                | Time Complexity                                                                       |
|-----------------------|---------------------------------------------------------------------------------------|
| <code>offer(e)</code> | $O(\log n)$                                                                           |
| <code>poll()</code>   | $O(\log n)$ — removes and returns the highest-priority (smallest, by default) element |
| <code>peek()</code>   | $O(1)$ — examines the root without removing                                           |

## 4. Custom Priority via `Comparator`

```
PriorityQueue<Integer> minHeap = new PriorityQueue<>(); // natural order (min-heap by default)
PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Comparator.reverseOrder()); // max-heap
maxHeap.offer(5); maxHeap.offer(1); maxHeap.offer(9);
System.out.println(maxHeap.poll()); // 9 - highest priority first with reverseOrder()
```

## 5. Real Life Analogy

- **Hospital:** An emergency room's patient queue isn't FIFO — it's PRIORITY-based (most critical patient treated first, regardless of arrival order) — exactly `PriorityQueue`'s behavior with a custom "severity" `Comparator`.

## 6. Edge Cases

| Case                                                                             | Result                                                                                                                 |
|----------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| Iterating a <code>PriorityQueue</code> directly (via its <code>Iterator</code> ) | Does NOT guarantee any particular (e.g., sorted) order — only <code>poll()</code> repeatedly guarantees priority order |
| Elements without natural ordering AND no <code>Comparator</code> supplied        | <code>ClassCastException</code> at runtime when comparison is attempted                                                |

## 7. Frequently Asked Interview Questions

- 1 **What data structure backs `PriorityQueue`?** A binary heap (array-based).
- 2 **What's the time complexity of `offer()/poll()`?**  $O(\log n)$  each; `peek()` is  $O(1)$ .
- 3 **Does iterating a `PriorityQueue` return elements in sorted order?** No — only repeated `poll()` calls guarantee priority order; direct iteration order is unspecified.
- 4 **How do you create a max-heap using `PriorityQueue`?** Supply `Comparator.reverseOrder()` (or an equivalent custom comparator) to the constructor, since it's a min-heap by default.

# TOPIC 52: DEQUE

## 1. Introduction

**What is it?** `Deque` (Double-Ended Queue, pronounced "deck") is an interface supporting insertion and removal at BOTH ends — functioning as a `Queue`, a `Stack`, or both simultaneously. `ArrayDeque` is its primary, high-performance implementation.

## 2. Key Methods

| Method                                    | Purpose                                                                    |
|-------------------------------------------|----------------------------------------------------------------------------|
| <code>addFirst(e) / addLast(e)</code>     | Insert at front/back                                                       |
| <code>removeFirst() / removeLast()</code> | Remove from front/back                                                     |
| <code>peekFirst() / peekLast()</code>     | Examine front/back without removing                                        |
| <code>push(e) / pop()</code>              | Stack-style LIFO operations (alias for <code>addFirst/removeFirst</code> ) |
| <code>offer(e) / poll()</code>            | Queue-style FIFO operations (alias for <code>addLast/removeFirst</code> )  |

### 3. `ArrayDeque` vs `LinkedList` (as a `Deque`) vs `Stack`

| Aspect            | <code>ArrayDeque</code>                                                                    | <code>LinkedList</code> (as <code>Deque</code> ) | <code>Stack</code> (legacy)                    |
|-------------------|--------------------------------------------------------------------------------------------|--------------------------------------------------|------------------------------------------------|
| Backing structure | Resizable circular array                                                                   | Doubly-linked nodes                              | Array (via <code>Vector</code> )               |
| Performance       | Generally FASTEST for stack/queue operations (better cache locality, no per-node overhead) | Slower due to node overhead                      | Slower (synchronized overhead)                 |
| Thread-safety     | NOT thread-safe                                                                            | NOT thread-safe                                  | Thread-safe (but usually unnecessary overhead) |
| Recommended for   | Stack AND queue use cases (modern default)                                                 | Rarely the best choice                           | Avoid — legacy                                 |

### 4. Code Example

```
import java.util.*;

Deque<Integer> deque = new ArrayDeque<>();
deque.addFirst(1);
deque.addLast(2);
deque.addFirst(0);
System.out.println(deque); // [0, 1, 2]

deque.push(-1); // stack-style push (adds to front)
System.out.println(deque.pop()); // -1 (LIFO)
System.out.println(deque.poll()); // 0 (FIFO, from front)
```

### 5. Real Life Analogy

- **Restaurant:** A deque models a buffet line where people can join or leave from EITHER end — unlike a strict single-direction queue.

### 6. Frequently Asked Interview Questions

- 1 **What does `Deque` stand for?** Double-Ended Queue.
- 2 **Can `Deque` function as both a stack and a queue?** Yes — via `push()`/`pop()` for stack (LIFO) behavior, and `offer()`/`poll()` for queue (FIFO) behavior.
- 3 **Why is `ArrayDeque` generally preferred over `LinkedList` for stack/queue use cases today?** Better cache locality and lower per-element memory overhead (no node objects with extra reference fields), typically making it faster in practice.
- 4 **Is `ArrayDeque` thread-safe?** No.

---

(End of Topics 49-52: Queue Family — `Stack`, `Queue`, `PriorityQueue`, `Deque`.)

---

# TOPIC 53: SET

---

## 1. Introduction

**What is it?** `Set` is a collection that contains **no duplicate elements**, modeling the mathematical concept of a set. It extends `Collection` but adds no new methods — its entire contract is defined by the DUPLICATE-FREE guarantee.

## 2. Real Life Analogy

- **College:** A `Set<String>` of enrolled student IDs — each student can only be enrolled ONCE; attempting to "add" an already-present ID has no effect.

## 3. The Three Main Implementations (Preview)

| Implementation             | Ordering                  | Backing Structure        |
|----------------------------|---------------------------|--------------------------|
| <code>HashSet</code>       | No guaranteed order       | Hash table               |
| <code>LinkedHashSet</code> | Insertion order preserved | Hash table + linked list |
| <code>TreeSet</code>       | Sorted order              | Red-black tree           |

(Full detail on each in the next three topics.)

## 4. Frequently Asked Interview Questions

- 1 **What is the defining characteristic of `Set`?** No duplicate elements allowed.
- 2 **Does `Set` extend `List`?** No — both extend `Collection` independently; `Set` adds no index-based access.
- 3 **Name the three main `Set` implementations and their ordering guarantee.** `HashSet` (no order), `LinkedHashSet` (insertion order), `TreeSet` (sorted order).

---

# TOPIC 54: HASHSET

---

## 1. Introduction

**What is it?** `HashSet` is a `Set` implementation backed internally by a `HashMap` (each element is stored as a KEY in that internal map, with a shared dummy constant object as the value) — providing O(1) average-case add/remove/contains, with NO guaranteed iteration order.

## 2. Internal Working

```
public class HashSet<E> {
    private transient HashMap<E, Object> map; // internally backed by a HashMap!
    private static final Object PRESENT = new Object();

    public boolean add(E e) {
        return map.put(e, PRESENT) == null; // element becomes the KEY; value is just a dummy placeholder
    }
}
```

- Requires correctly-implemented `equals()`/`hashCode()` on stored elements (see [\[\[Object Class|Topic 25\]\]](#)'s contract) — a `HashSet` of custom objects with a broken `hashCode()`/`equals()` pair will silently allow duplicates or fail `contains()` checks.

## 3. Code Example

```
import java.util.*;

Set<String> set = new HashSet<>();
set.add("Apple");
set.add("Banana");
set.add("Apple"); // duplicate - silently ignored, add() returns false
System.out.println(set.size()); // 2
System.out.println(set.contains("Apple")); // true - O(1) average case
```

## 4. Frequently Asked Interview Questions

- 1 **What backs `HashSet` internally?** A `HashMap`, where set elements become map keys and a shared dummy object is used as the value.
- 2 **What's the time complexity of `add()`/`contains()`?**  $O(1)$  average case (assuming a good hash distribution),  $O(n)$  worst case (many hash collisions).
- 3 **Does `HashSet` maintain any order?** No — iteration order is unspecified and can even change between runs.
- 4 **What's required of elements stored in a `HashSet`?** Correctly-implemented, consistent `equals()` and `hashCode()` — see the [\[\[Object Class|Topic 25\]\]](#) contract.

---

# TOPIC 55: LINKEDHASHSET

---

## 1. Introduction

**What is it?** `LinkedHashSet` extends `HashSet`, adding a doubly-linked list running through all entries to **preserve insertion order** during iteration — giving the  $O(1)$  performance of hashing PLUS predictable iteration order, at a modest extra memory cost per entry.

## 2. Code Example

```
Set<String> set = new LinkedHashSet<>();
set.add("Banana");
set.add("Apple");
set.add("Cherry");
System.out.println(set); // [Banana, Apple, Cherry] - PRESERVES insertion order (unlike plain HashSet)
```

## 3. HashSet VS LinkedHashSet VS TreeSet — Comparison Table

| Aspect              | HashSet                       | LinkedHashSet                    | TreeSet                                             |
|---------------------|-------------------------------|----------------------------------|-----------------------------------------------------|
| Ordering            | None guaranteed               | Insertion order                  | Sorted (natural or Comparator)                      |
| add/contains/remove | O(1) average                  | O(1) average                     | O(log n)                                            |
| Backing structure   | Hash table                    | Hash table + linked list         | Red-black tree                                      |
| Allows null?        | One null allowed              | One null allowed                 | No null (throws NullPointerException when compared) |
| When to use         | Fastest, order doesn't matter | Need predictable iteration order | Need sorted order                                   |

## 4. Frequently Asked Interview Questions

- 1 **What does LinkedHashSet add over HashSet?** A linked list running through entries, preserving insertion order during iteration, while keeping O(1) average performance.
- 2 **What's the memory trade-off of LinkedHashSet vs HashSet?** Slightly higher — each entry additionally stores prev/next pointers for the linked list.

# TOPIC 56: TREESSET

## 1. Introduction

**What is it?** `TreeSet` is a `Set` implementation backed by a **red-black tree** (a self-balancing binary search tree), maintaining elements in **sorted order** (natural ordering via `Comparable`, or a supplied `Comparator`) at all times.

## 2. Key Methods (Beyond Standard Set)

| Method                                            | Purpose                                                          |
|---------------------------------------------------|------------------------------------------------------------------|
| <code>first()</code> / <code>last()</code>        | Smallest / largest element                                       |
| <code>higher(e)</code> / <code>lower(e)</code>    | Smallest element STRICTLY greater / largest STRICTLY less than e |
| <code>ceiling(e)</code> / <code>floor(e)</code>   | Smallest element $\geq e$ / largest element $\leq e$             |
| <code>headSet(e)</code> / <code>tailSet(e)</code> | Sub-set of elements less than / greater than or equal to e       |



### 3. Code Example

```
TreeSet<Integer> set = new TreeSet<>(Arrays.asList(5, 1, 9, 3));
System.out.println(set); // [1, 3, 5, 9] - ALWAYS sorted
System.out.println(set.first()); // 1
System.out.println(set.higher(3)); // 5 (strictly greater than 3)
System.out.println(set.ceiling(4)); // 5 (smallest >= 4)
```

### 4. Edge Cases

| Case                                                                                 | Result                                                                                                                              |
|--------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| Adding a <code>null</code> element                                                   | <code>NullPointerException</code> — the tree needs to compare elements to place them, and comparing against <code>null</code> fails |
| Elements without natural ordering AND no <code>Comparator</code> supplied            | <code>ClassCastException</code> at insertion time                                                                                   |
| <code>add()</code> / <code>remove()</code> / <code>contains()</code> time complexity | $O(\log n)$ — tree traversal, NOT $O(1)$ like <code>HashSet</code>                                                                  |

### 5. Frequently Asked Interview Questions

- 1 **What backs `TreeSet`?** A red-black tree (self-balancing BST).
- 2 **What's the time complexity of `add()`/`contains()` on `TreeSet`?**  $O(\log n)$  — slower than `HashSet`'s  $O(1)$  average, but maintains sorted order.
- 3 **Does `TreeSet` allow `null`?** No — throws `NullPointerException`, since comparisons are needed to maintain sort order.
- 4 **What must elements satisfy to be stored in a `TreeSet`?** They must be mutually comparable — either implementing `Comparable`, or a `Comparator` must be supplied to the `TreeSet` constructor.
- 5 **When would you choose `TreeSet` over `HashSet`?** When you need elements maintained in sorted order at all times, and can accept  $O(\log n)$  instead of  $O(1)$  average performance.

---

(End of Topics 53-56: Set Family — Set, HashSet, LinkedHashSet, TreeSet.)

---

## TOPIC 57: MAP

---

### 1. Introduction

**What is it?** `Map<K, V>` is an interface representing a collection of **key-value pairs**, where each key maps to exactly ONE value, and keys must be unique (adding a duplicate key **OVERWRITES** the previous value). Notably, `Map` does **NOT** extend `Collection` — it's a separate root interface in the JCF, since it deals in pairs, not single elements.

## 2. Real Life Analogy

- **Bank:** A `Map<AccountNumber, Account>` — each account number (key) maps to exactly one account object (value); looking up by account number is instant, and re-"adding" the same account number just replaces/updates its associated account.

## 3. Core Methods

| Method                                                   | Purpose                                                                                        |
|----------------------------------------------------------|------------------------------------------------------------------------------------------------|
| <code>put(K key, V value)</code>                         | Inserts or UPDATES a key-value pair, returns the PREVIOUS value (or null)                      |
| <code>get(K key)</code>                                  | Returns the value for a key, or null if absent                                                 |
| <code>getOrDefault(K key, V default)</code>              | Returns the value, or a fallback if the key is absent                                          |
| <code>containsKey(K key) / containsValue(V value)</code> | Existence checks                                                                               |
| <code>remove(K key)</code>                               | Removes a mapping                                                                              |
| <code>keySet() / values() / entrySet()</code>            | Views for iterating keys, values, or key-value pairs                                           |
| <code>putIfAbsent(K, V)</code>                           | Inserts only if the key isn't already present                                                  |
| <code>computeIfAbsent(K, Function)</code>                | Computes and inserts a value only if absent (common for building nested/aggregated structures) |
| <code>merge(K, V, BiFunction)</code>                     | Combines an existing value with a new one via a function (common for counting/aggregation)     |

## 4. Frequently Asked Interview Questions

- 1 **Does `Map` extend `Collection`?** No — it's a separate root interface, since it manages key-value pairs, not single elements.
- 2 **What happens when you `put()` a key that already exists?** The previous value is overwritten (replaced) with the new one, and the OLD value is returned by `put()`.
- 3 **What are the three "view" methods for iterating a `Map`?** `keySet()`, `values()`, `entrySet()`.

---

# TOPIC 58: HASHMAP

---

## 1. Introduction

**What is it?** `HashMap<K, V>` is the most widely used `Map` implementation, providing  $O(1)$  average-case `get/put/remove`, backed internally by an ARRAY OF BUCKETS, where each bucket holds entries whose keys hash to that bucket index.

## 2. Internal Working — Deep Dive (Critical, Frequently Tested)

### 2.1 The Bucket Array

```
HashMap<String, Integer> map = new HashMap<>(); // default initial capacity: 16, load factor: 0.75

Bucket Array (capacity 16):
[0] -> (empty)
[1] -> ("apple", 5) -> ("grape", 3) <- HASH COLLISION: both hash to bucket 1, chained via linked list
[2] -> (empty)
...
[15] -> ("zebra", 1)
```

### 2.2 Step-by-Step: `put("apple", 5)`

- 1 Compute `hashCode()` of the key `"apple"`.
- 2 Apply HotSpot's **hash spreading function** (`hash ^ (hash >>> 16)`) to reduce collision clustering from poor `hashCode()` implementations.
- 3 Compute the bucket index: `(capacity - 1) & spreadHash` — an efficient equivalent of `spreadHash % capacity` when capacity is a power of 2.
- 4 If the bucket is empty, insert directly.
- 5 If the bucket already has entries (a **hash collision**), traverse the chain, comparing keys via `equals()`:
  - If an equal key is found, UPDATE its value (and return the old value).
  - Otherwise, APPEND the new entry to the chain (as a linked list node, OR into a red-black TREE if the chain has grown very long — see 2.4).

### 2.3 Load Factor and Resizing

- **Load factor** (default 0.75) determines WHEN the map resizes: when `size > capacity × loadFactor` (e.g.,  $16 \times 0.75 = 12$  entries), the map **doubles its capacity** (to 32) and **rehashes every existing entry** into the new, larger bucket array — an  $O(n)$  operation, but infrequent enough that `put()` remains amortized  $O(1)$ .
- A LOWER load factor means more memory used but fewer collisions (faster lookups); a HIGHER load factor means less memory but more collisions.

### 2.4 Treeification (Java 8+ Optimization)

If a SINGLE bucket's chain grows to **8 or more entries** (`TREEIFY_THRESHOLD`) AND the overall table capacity is at least 64, that bucket's linked list is converted into a **red-black tree**, changing worst-case lookup within that bucket from  $O(n)$  to  $O(\log n)$  — a defense against maliciously crafted or pathologically hash-colliding keys degrading `HashMap` to linear-list performance. If entries are later removed and the bucket shrinks below 6 entries (`UNTREEIFY_THRESHOLD`), it converts back to a linked list.

## 2.5 Diagram — Full `put()` Flow

```
put(key, value)
|
v
hashCode(key) -> spread hash -> bucket index = (capacity-1) & hash
|
v
Bucket empty? --Yes--> insert new Node directly
|No
v
Traverse chain/tree, compare via equals()
|
v
Key found? --Yes--> UPDATE value, return OLD value
|No
v
APPEND new Node (as linked list, or insert into tree if already treeified)
|
v
size > capacity * loadFactor? --Yes--> RESIZE (double capacity, rehash all entries)
```

## 3. Key Methods with Time Complexity

| Method                       | Average Case     | Worst Case (before treeification) | Worst Case (with treeification, Java 8+) |
|------------------------------|------------------|-----------------------------------|------------------------------------------|
| <code>get(key)</code>        | $O(1)$           | $O(n)$                            | $O(\log n)$                              |
| <code>put(key, value)</code> | $O(1)$ amortized | $O(n)$                            | $O(\log n)$                              |
| <code>remove(key)</code>     | $O(1)$           | $O(n)$                            | $O(\log n)$                              |

## 4. Code Examples

```
import java.util.*;

public class HashMapDemo {
    public static void main(String[] args) {
        Map<String, Integer> wordCount = new HashMap<>();
        String[] words = {"apple", "banana", "apple", "cherry", "banana", "apple"};

        for (String word : words) {
            wordCount.merge(word, 1, Integer::sum); // classic counting pattern
        }
        System.out.println(wordCount); // {banana=2, cherry=1, apple=3} (order not guaranteed)

        wordCount.computeIfAbsent("date", k -> 0); // inserts "date"->0 only if absent
        System.out.println(wordCount.getOrDefault("kiwi", -1)); // -1, key absent
    }
}
```

## 5. `HashMap` VS `Hashtable` VS `LinkedHashMap` VS `TreeMap` — Full Comparison Table

| Aspect          | <code>HashMap</code>                       | <code>Hashtable</code>                    | <code>LinkedHashMap</code>          | <code>TreeMap</code>                                |
|-----------------|--------------------------------------------|-------------------------------------------|-------------------------------------|-----------------------------------------------------|
| Thread-safety   | No                                         | Yes (synchronized, legacy)                | No                                  | No                                                  |
| Ordering        | None guaranteed                            | None guaranteed                           | Insertion (or access) order         | Sorted (natural/ <code>Comparator</code> )          |
| null key/values | ONE null key, multiple null values allowed | NO null key or values allowed             | Same as <code>HashMap</code>        | No null key (throws <code>NPE</code> on comparison) |
| Performance     | $O(1)$ average                             | $O(1)$ average, but synchronized overhead | $O(1)$ average, slight extra memory | $O(\log n)$                                         |
| Introduced      | Java 2                                     | Java 1.0 (legacy)                         | Java 4                              | Java 2                                              |

## 6. Edge Cases

| Case                                                                                                      | Result                                                                                                                                                     |
|-----------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>put(null, value)</code>                                                                             | Allowed — <code>HashMap</code> permits exactly ONE null key (special-cased internally, hash treated as 0)                                                  |
| Custom object used as a key with a poorly-implemented/missing <code>hashCode()</code>                     | All instances may hash to the same bucket, degrading performance toward $O(n)/O(\log n)$ instead of $O(1)$                                                 |
| Mutating a key's fields (used in <code>hashCode()</code> ) AFTER inserting it into a <code>HashMap</code> | The entry becomes "lost" — the map still searches the ORIGINAL bucket, which no longer matches the mutated key's new hash (see <code>Object Class</code> ) |
| Iterating a <code>HashMap</code> while modifying it (not via <code>Iterator.remove()</code> )             | <code>ConcurrentModificationException</code>                                                                                                               |

## 7. Common Mistakes

- Using a mutable object (whose `hashCode()` depends on mutable fields) as a `HashMap` key, then mutating it after insertion — silently "loses" the entry.
- Assuming `HashMap` iteration order is predictable or stable — it is NOT (use `LinkedHashMap` if insertion order matters).
- Forgetting `HashMap` is NOT thread-safe — using it from multiple threads without external synchronization can cause data corruption or even infinite loops (a well-documented historical bug in pre-Java-8 resizing under concurrent modification).

## 8. Best Practices

- Never use mutable fields for `hashCode()/equals()` when an object will be used as a `HashMap` key.
- Use `ConcurrentHashMap` (covered in Multithreading) instead of synchronizing a plain `HashMap` externally, for genuinely concurrent use cases.
- Pre-size the initial capacity when the approximate number of entries is known, to minimize resizing operations.

## 9. Frequently Asked Interview Questions

- 1 **What is the internal data structure of `HashMap`?** An array of buckets, each holding a linked list (or red-black tree, if sufficiently long) of entries whose keys hash to that bucket index.
- 2 **What is the default initial capacity and load factor?** Capacity 16, load factor 0.75.
- 3 **What triggers a `HashMap` resize?** When `size` exceeds `capacity × loadFactor`, the capacity doubles and all entries are rehashed into the new array.
- 4 **What is treeification, and when does it happen?** Since Java 8, if a single bucket's chain grows to 8+ entries (with overall capacity  $\geq 64$ ), it converts from a linked list to a red-black tree, improving worst-case lookup from  $O(n)$  to  $O(\log n)$ .
- 5 **How many `null` keys does `HashMap` allow?** Exactly one; `Hashtable` allows none at all.
- 6 **Is `HashMap` thread-safe?** No — use `ConcurrentHashMap` or `Collections.synchronizedMap()` for concurrent access.
- 7 **What happens if you mutate a key's hash-relevant fields after inserting it?** The entry becomes effectively "lost" — the map still looks for it in its ORIGINAL bucket based on the OLD hash, which no longer matches.
- 8 **Why does `HotSpot`'s `HashMap` apply an extra "hash spreading" step instead of just using `hashCode()` directly?** To reduce clustering/collisions caused by poorly-distributed `hashCode()` implementations, by mixing higher bits into the lower bits before computing the bucket index.

## 10. Revision Notes

### HASHMAP – CHEAT SHEET

=====

Backed by: array of buckets, each a linked list (or red-black tree if long)

Default capacity: 16, load factor: 0.75 (resize threshold = `capacity * loadFactor`)

`put()/get()/remove()`:  $O(1)$  average,  $O(\log n)$  worst-case (Java 8+ treeification at 8+ entries per bucket & `capacity >= 64`),  $O(n)$  worst-case pre-treeification

RESIZE: `size > capacity * loadFactor` -> DOUBLE capacity, REHASH everything ( $O(n)$ , but amortized  $O(1)$  per put overall)

ONE null key allowed, multiple null values allowed

NOT thread-safe -> use `ConcurrentHashMap` for concurrent access

GOTCHA: mutating a key's `hashCode()`-relevant fields after insertion "loses" the entry

## TOPIC 59: LINKEDHASHMAP

## 1. Introduction

**What is it?** `LinkedHashMap` extends `HashMap`, adding a doubly-linked list running through all entries to preserve **insertion order** (or optionally **access order**, useful for building an LRU cache) during iteration.

## 2. Building an LRU Cache — A Classic Production Pattern

```
import java.util.*;

class LRUCache<K, V> extends LinkedHashMap<K, V> {
    private final int capacity;

    LRUCache(int capacity) {
        super(16, 0.75f, true); // 'true' -> ACCESS order (not insertion order!)
        this.capacity = capacity;
    }

    @Override
    protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
        return size() > capacity; // automatically evict the LEAST recently used entry
    }
}

LRUCache<Integer, String> cache = new LRUCache<>(3);
cache.put(1, "A"); cache.put(2, "B"); cache.put(3, "C");
cache.get(1); // accessing 1 moves it to "most recently used"
cache.put(4, "D"); // evicts 2 (the least recently used), NOT 1
System.out.println(cache.keySet()); // [3, 1, 4]
```

This `removeEldestEntry()` override + access-order constructor combination is a well-known, elegant way to implement a fully-working LRU cache in just a few lines, leveraging `LinkedHashMap`'s built-in linked-list ordering machinery.

## 3. Frequently Asked Interview Questions

- 1 **What does `LinkedHashMap` add over `HashMap`?** A linked list preserving insertion (or optionally access) order during iteration.
- 2 **How do you enable access-order mode?** Pass `true` as the third constructor argument: `new LinkedHashMap<>(initialCapacity, loadFactor, true)`.
- 3 **How is `LinkedHashMap` commonly used to build an LRU cache?** By enabling access order and overriding `removeEldestEntry()` to return `true` once the cache exceeds its capacity, automatically evicting the least recently used entry.

---

## TOPIC 60: TREEMAP

---

## 1. Introduction

**What is it?** `TreeMap<K, V>` is a `Map` implementation backed by a **red-black tree**, maintaining keys in sorted order at all times (natural ordering via `Comparable`, or a supplied `Comparator`).

## 2. Key Methods (via `NavigableMap`)

| Method                                                | Purpose                                                      |
|-------------------------------------------------------|--------------------------------------------------------------|
| <code>firstKey()</code> / <code>lastKey()</code>      | Smallest / largest key                                       |
| <code>higherKey(k)</code> / <code>lowerKey(k)</code>  | Smallest key strictly greater / largest strictly less than k |
| <code>ceilingKey(k)</code> / <code>floorKey(k)</code> | Smallest key $\geq k$ / largest key $\leq k$                 |
| <code>headMap(k)</code> / <code>tailMap(k)</code>     | Sub-map of entries with keys less than / $\geq k$            |

## 3. Code Example

```
TreeMap<String, Integer> map = new TreeMap<>();
map.put("banana", 3);
map.put("apple", 5);
map.put("cherry", 2);
System.out.println(map); // {apple=5, banana=3, cherry=2} - ALWAYS sorted by key
System.out.println(map.firstKey()); // "apple"
```

## 4. Frequently Asked Interview Questions

- 1 **What backs `TreeMap`?** A red-black tree.
- 2 **What's the time complexity of `get()`/`put()` on `TreeMap`?**  $O(\log n)$ , versus `HashMap`'s  $O(1)$  average.
- 3 **When would you use `TreeMap` over `HashMap`?** When keys must be maintained in sorted order, or when range-based queries (`headMap`, `tailMap`, `ceilingKey`) are needed.

---

# TOPIC 61: HASHTABLE

---

## 1. Introduction

**What is it?** `Hashtable` is a legacy (Java 1.0), synchronized `Map` implementation — the historical predecessor to `HashMap`, now largely superseded by `HashMap` (single-threaded) or `ConcurrentHashMap` (multi-threaded).



## 2. Hashtable vs HashMap — Key Differences

| Aspect                | Hashtable                                                      | HashMap                                    |
|-----------------------|----------------------------------------------------------------|--------------------------------------------|
| Thread-safety         | Synchronized (thread-safe, coarse-grained locking)             | Not synchronized                           |
| null key/values       | NOT allowed at all (throws NullPointerException)               | One null key, multiple null values allowed |
| Performance           | Slower (blanket synchronization, even for single-threaded use) | Faster                                     |
| Modern recommendation | Avoid — use ConcurrentHashMap for genuine thread-safety needs  | Default choice for single-threaded use     |

## 3. Frequently Asked Interview Questions

- 1 **Is Hashtable thread-safe?** Yes — via coarse-grained method-level synchronization.
- 2 **Does Hashtable allow null keys or values?** No — throws NullPointerException for either.
- 3 **What's the modern recommended replacement for concurrent use cases?** ConcurrentHashMap — offers much better concurrent performance via finer-grained locking (covered in the Multithreading topic).

---

(End of Topics 57-61: Map Family — Map, HashMap, LinkedHashMap, TreeMap, Hashtable.)

---

# TOPIC 62: COMPARABLE

---

## 1. Introduction

**What is it?** Comparable<T> is an interface a class implements to define its **natural ordering** — a single, class-intrinsic way of comparing its instances, used automatically by Collections.sort(), TreeSet, and TreeMap when no separate Comparator is supplied.

## 2. Syntax

```
class Employee implements Comparable<Employee> {
    String name;
    double salary;

    Employee(String name, double salary) { this.name = name; this.salary = salary; }

    @Override
    public int compareTo(Employee other) {
        return Double.compare(this.salary, other.salary); // natural order: by salary, ascending
    }
}
```

### 3. The `compareTo()` Contract

| Return Value | Meaning                                                   |
|--------------|-----------------------------------------------------------|
| Negative     | this comes BEFORE other                                   |
| Zero         | this is considered EQUAL to other (for ordering purposes) |
| Positive     | this comes AFTER other                                    |

**Important:** `compareTo() == 0` should ideally be CONSISTENT with `equals()` (i.e., if `compareTo()` returns 0, `equals()` should also return true) — inconsistency is legal but can cause surprising behavior in sorted collections like `TreeSet` (which uses `compareTo()`, not `equals()`, to determine "duplicates").

### 4. Code Example

```
import java.util.*;

public class ComparableDemo {
    public static void main(String[] args) {
        List<Employee> employees = new ArrayList<>(List.of(
            new Employee("Alice", 75000),
            new Employee("Bob", 60000)
        ));
        Collections.sort(employees); // uses Employee's compareTo() automatically
        System.out.println(employees.get(0).name); // "Bob" (lower salary first)
    }
}
```

### 5. Frequently Asked Interview Questions

- 1 **What is `Comparable` used for?** Defining a class's single, natural ordering, used by default by `Collections.sort()` and sorted collections (`TreeSet/TreeMap`).
- 2 **How many `compareTo()` implementations can a class have?** Exactly one — it's the class's ONE natural ordering; use `Comparator` for alternative orderings.
- 3 **What should `compareTo() == 0` ideally be consistent with?** `equals()` — inconsistency is legal but can cause confusing behavior in `TreeSet/TreeMap`, which treat `compareTo() == 0` as "equal/duplicate."

---

## TOPIC 63: COMPARATOR

---

### 1. Introduction

**What is it?** `Comparator<T>` is a functional interface defining an **external, alternative ordering** for objects — unlike `Comparable`'s single "natural" ordering baked INTO the class, a `Comparator` can be written separately, and multiple different `Comparators` can exist for the same class (e.g., sort employees by name, OR by salary, OR by hire date).

## 2. Comparable vs Comparator — Full Comparison Table

| Aspect              | Comparable                                           | Comparator                                                      |
|---------------------|------------------------------------------------------|-----------------------------------------------------------------|
| Defined             | INSIDE the class itself ( <code>compareTo()</code> ) | EXTERNALLY, as a separate object/lambda                         |
| Number per class    | Exactly one (the natural ordering)                   | Unlimited — as many different orderings as needed               |
| Method              | <code>compareTo(T other)</code>                      | <code>compare(T a, T b)</code>                                  |
| Modifies the class? | Yes — the class itself must implement it             | No — works with ANY class, even ones you don't own/can't modify |
| Use case            | The single "obvious" default ordering                | Alternative/multiple/situational orderings                      |

## 3. Syntax — Old Style vs Modern (Java 8+) Style

```
// OLD STYLE (anonymous class, pre-Java 8)
Comparator<Employee> byNameOld = new Comparator<Employee>() {
    @Override
    public int compare(Employee a, Employee b) {
        return a.name.compareTo(b.name);
    }
};

// MODERN STYLE (lambda, Java 8+)
Comparator<Employee> byName = (a, b) -> a.name.compareTo(b.name);

// EVEN MORE MODERN (method reference + comparing() helper, Java 8+)
Comparator<Employee> bySalary = Comparator.comparing(e -> e.salary);
```

## 4. Chaining Comparators (Java 8+)

```
Comparator<Employee> byDeptThenSalary = Comparator
    .comparing((Employee e) -> e.department)
    .thenComparing(e -> e.salary); // secondary sort key if department ties

Comparator<Employee> reversed = Comparator
    .comparing((Employee e) -> e.salary)
    .reversed(); // descending order
```

## 5. Code Example

```
import java.util.*;

public class ComparatorDemo {
    public static void main(String[] args) {
        List<Employee> employees = new ArrayList<>(List.of(
            new Employee("Bob", 60000),
            new Employee("Alice", 75000)
        ));

        employees.sort(Comparator.comparing((Employee e) -> e.name)); // sort by NAME (not the class's natural order)
        System.out.println(employees.get(0).name); // "Alice"

        employees.sort(Comparator.comparingDouble((Employee e) -> e.salary).reversed());
        System.out.println(employees.get(0).name); // "Alice" (highest salary first)
    }
}
```

## 6. Edge Cases

| Case                                                                                      | Result                                                                                               |
|-------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| Comparator returns inconsistent results for the same pair (e.g., non-deterministic logic) | Undefined/incorrect sort behavior — the comparator MUST be consistent for correct sorting            |
| Sorting with a null-unsafe Comparator on a list containing null elements                  | NullPointerException; use Comparator.nullsFirst()/nullsLast() wrappers if null elements are expected |

## 7. Frequently Asked Interview Questions

- 1 **What's the key difference between Comparable and Comparator?** Comparable defines a SINGLE natural ordering inside the class itself; Comparator defines an EXTERNAL, alternative ordering, and you can have as many different Comparators as needed for the same class.
- 2 **Which interface would you use to sort a class you don't own/can't modify?** Comparator — since it doesn't require modifying the target class.
- 3 **How do you chain multiple sort criteria with Comparator?** Using thenComparing() for secondary/tertiary sort keys.
- 4 **How do you reverse a Comparator's order?** Call .reversed() on it.

---

## TOPIC 64: ITERATOR

---

## 1. Introduction

**What is it?** `Iterator<E>` is an interface providing a standard way to traverse a collection's elements ONE AT A TIME, without exposing the collection's internal structure, and — critically — allowing SAFE element removal DURING iteration (unlike a plain for-each loop).

## 2. Key Methods

| Method                 | Purpose                                                                                                  |
|------------------------|----------------------------------------------------------------------------------------------------------|
| <code>hasNext()</code> | Returns <code>true</code> if there are more elements to visit                                            |
| <code>next()</code>    | Returns the next element, advances the cursor; throws <code>NoSuchElementException</code> if none remain |
| <code>remove()</code>  | Removes the LAST element returned by <code>next()</code> — the ONLY safe way to remove during iteration  |

## 3. `ConcurrentModificationException` — Why It Happens

```
List<String> list = new ArrayList<>(List.of("A", "B", "C"));
for (String s : list) {
    if (s.equals("B")) {
        list.remove(s); // MODIFYING the list directly during for-each iteration!
    }
}
// Throws ConcurrentModificationException on the NEXT hasNext()/next() call
```

**Why:** The enhanced for-loop uses an `Iterator` internally. Most JCF collections maintain an internal **modification count** (`modCount`), incremented on every structural change. The `Iterator` checks this count on each `next()` call — if it detects the count changed UNEXPECTEDLY (i.e., not through the iterator's OWN `remove()`), it throws `ConcurrentModificationException` as a **fail-fast** safety mechanism, rather than allowing silently corrupted iteration.

### *The Correct, Safe Way*

```
Iterator<String> it = list.iterator();
while (it.hasNext()) {
    String s = it.next();
    if (s.equals("B")) {
        it.remove(); // SAFE - updates modCount consistently, iterator stays valid
    }
}
```

## 4. Real Life Analogy

- **Library:** An `Iterator` is like a librarian walking down a specific shelf, one book at a time — if someone else (another thread, or the for-each loop's underlying collection modification) rearranges the shelf WHILE the librarian is mid-walk, the librarian immediately notices something is wrong (fail-fast) rather than continuing to hand out potentially wrong/skipped books.

## 5. Frequently Asked Interview Questions

- 1 **What is `Iterator`?** An interface for traversing a collection's elements one at a time, supporting safe removal during traversal.
  - 2 **Why does modifying a list directly during a for-each loop throw `ConcurrentModificationException`?** The for-each loop uses an `Iterator` internally, which tracks a modification count (`modCount`); any structural change NOT made through the iterator's own `remove()` is detected as unexpected on the next `hasNext()/next()` call, triggering a fail-fast exception.
  - 3 **What's the ONLY safe way to remove elements while iterating?** Using the `Iterator`'s own `remove()` method, never the collection's `remove()` directly.
  - 4 **What does "fail-fast" mean in this context?** The iterator immediately detects and reports unexpected concurrent structural modification via an exception, rather than risking silently incorrect/corrupted iteration behavior.
- 

## TOPIC 65: LISTITERATOR

---

### 1. Introduction

**What is it?** `ListIterator<E>` extends `Iterator`, adding **bidirectional traversal** (forward AND backward) and the ability to ADD or REPLACE elements during iteration, specifically for `List` implementations.

### 2. Key Additional Methods (Beyond `Iterator`)

| Method                                     | Purpose                                                              |
|--------------------------------------------|----------------------------------------------------------------------|
| <code>hasPrevious() / previous()</code>    | Traverse BACKWARD                                                    |
| <code>nextIndex() / previousIndex()</code> | Get the index of the next/previous element                           |
| <code>set(E e)</code>                      | REPLACES the last element returned by <code>next()/previous()</code> |
| <code>add(E e)</code>                      | Inserts a new element at the current cursor position                 |

### 3. Code Example

```
import java.util.*;

List<String> list = new ArrayList<>(List.of("A", "B", "C"));
ListIterator<String> it = list.listIterator();

while (it.hasNext()) {
    String value = it.next();
    if (value.equals("B")) {
        it.set("B-Modified"); // safely REPLACES "B" during iteration
    }
}
System.out.println(list); // [A, B-Modified, C]

while (it.hasPrevious()) { // traverse BACKWARD from where we ended
    System.out.println(it.previous());
}
```

### 4. `Iterator` vs `ListIterator` — Comparison Table

| Aspect                | <code>Iterator</code> | <code>ListIterator</code> |
|-----------------------|-----------------------|---------------------------|
| Direction             | Forward only          | Forward AND backward      |
| Applicable to         | Any Collection        | Only List                 |
| Can add elements?     | No                    | Yes — <code>add(e)</code> |
| Can replace elements? | No                    | Yes — <code>set(e)</code> |

### 5. Frequently Asked Interview Questions

- What does `ListIterator` add over `Iterator`?** Bidirectional traversal, plus the ability to `add()` and `set()` (replace) elements during iteration.
- Which collections support `ListIterator`?** Only `List` implementations (via `list.listIterator()`), not arbitrary `Collection`s.
- How do you safely replace an element during iteration?** Using `ListIterator`'s `set()` method — directly modifying the underlying list via index during iteration risks the same `ConcurrentModificationException` issues as with a plain `Iterator`.

---

(End of Topics 62-65 — this concludes the entire Collections Framework sub-section of Core Java!)

---

## JAVA 8 FEATURES

---

## TOPIC 66: LAMBDA EXPRESSIONS

---

## 1. Introduction

**What is it?** A lambda expression is a concise, anonymous, unnamed block of code representing an implementation of a **functional interface** (an interface with exactly one abstract method) — Java's introduction of functional-programming-style syntax, landmark feature of Java 8 (2014).

**Why introduced:** Before Java 8, implementing a simple single-method interface (like `Runnable` or a custom `Comparator`) required verbose anonymous inner class syntax (see [\[\[Nested Classes|Topic 38\]\]](#)). Lambdas eliminate this boilerplate, making functional-style code (passing behavior as data) dramatically more concise and readable, and were essential groundwork for the Streams API.

**Interview definition:** "A lambda expression is a compact representation of an anonymous function — a block of code with parameters and a body — that implements the single abstract method of a functional interface, compiled via `invokedynamic` and `LambdaMetafactory` to lazily generate the implementing class at runtime (see [\[\[Java Compilation Process|Topic 3\]\]](#))."

## 2. Syntax Evolution

```
// Anonymous class (PRE-Java 8)
Runnable r1 = new Runnable() {
    @Override
    public void run() { System.out.println("Running..."); }
};

// Lambda expression (Java 8+)
Runnable r2 = () -> System.out.println("Running...");

// With parameters
Comparator<Integer> cmp = (a, b) -> a - b;

// Multi-statement body (needs braces + explicit return)
Comparator<Integer> cmp2 = (a, b) -> {
    System.out.println("Comparing " + a + " and " + b);
    return a - b;
};
```

## 3. Real Life Analogy

- **Restaurant:** A lambda is like handing a chef a quick, unwritten verbal instruction ("just plate it") instead of a full, formal written recipe card (an anonymous class) — same effective result, far less ceremony, for a simple one-off action.

## 4. Variable Capture — "Effectively Final"

```
int multiplier = 3; // must be effectively final (never reassigned)
Function<Integer, Integer> multiply = x -> x * multiplier; // captures 'multiplier' by VALUE
```

Lambdas can only capture local variables that are **effectively final** — they're captured by VALUE at creation time, not by live reference, so allowing reassignment afterward would create ambiguity about which value the lambda should see (same rule as anonymous/local classes, see [\[\[Nested Classes|Topic 38\]\]](#)).



## 5. Edge Cases

| Case                                                                                            | Result                                                                                                                                       |
|-------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| Lambda targeting an interface with 2+ abstract methods                                          | Compile-time error — lambdas ONLY work for functional interfaces (exactly one abstract method)                                               |
| Reassigning a captured local variable after lambda creation                                     | Compile-time error — violates the effectively-final requirement                                                                              |
| A checked exception thrown inside a lambda whose functional interface method doesn't declare it | Compile-time error — must handle the exception inside the lambda, or use a functional interface whose method declares that checked exception |

## 6. Common Mistakes

- Trying to use a lambda for a 2+ method interface — only works for genuine functional interfaces.
- Attempting to modify a captured local variable from inside a lambda — violates effective finality.
- Overusing complex, multi-line lambda bodies where a properly named method reference or regular method would be clearer.

## 7. Frequently Asked Interview Questions

- 1 **What is a lambda expression?** A concise, anonymous implementation of a functional interface's single abstract method.
- 2 **What must be true of an interface for a lambda to implement it?** It must be a functional interface — exactly one abstract method (though it may have any number of default/static methods too).
- 3 **Do lambdas compile to anonymous inner classes?** No — they use `invokedynamic` + `LambdaMetafactory` to lazily generate the implementation class at runtime (see [\[\[Java Compilation Process|Topic 3\]\]](#)).
- 4 **What does "effectively final" mean for captured variables?** The variable is never reassigned after its initial assignment, even without the explicit `final` keyword — required because lambdas capture local variables by value at creation time.

---

# TOPIC 67: FUNCTIONAL INTERFACES

---

## 1. Introduction

**What is it?** A functional interface is any interface with **exactly one abstract method** (it may have any number of default/static methods too), making it eligible to be implemented via a lambda expression or method reference. `@FunctionalInterface` is an optional but recommended annotation enforcing this at compile time.

*(Full interface mechanics already covered in [\[\[Interfaces|Topic 23\]\]](#) — this topic focuses on the JDK's built-in functional interfaces used pervasively with lambdas/Streams.)*

## 2. The Four Core Built-in Functional Interfaces (`java.util.function`)

| Interface                         | Abstract Method                | Purpose                                            |
|-----------------------------------|--------------------------------|----------------------------------------------------|
| <code>Predicate&lt;T&gt;</code>   | <code>boolean test(T t)</code> | A condition/filter check                           |
| <code>Function&lt;T, R&gt;</code> | <code>R apply(T t)</code>      | Transforms a <code>T</code> into an <code>R</code> |
| <code>Consumer&lt;T&gt;</code>    | <code>void accept(T t)</code>  | Consumes a value, no return (side-effect action)   |
| <code>Supplier&lt;T&gt;</code>    | <code>T get()</code>           | Produces/supplies a value, no input                |

(Each covered in full depth in its own dedicated topic below.)

## 3. Other Notable Built-in Functional Interfaces

| Interface                              | Signature                                                                          | Purpose                                                       |
|----------------------------------------|------------------------------------------------------------------------------------|---------------------------------------------------------------|
| <code>BiFunction&lt;T, U, R&gt;</code> | <code>R apply(T t, U u)</code>                                                     | Two-argument transformation                                   |
| <code>UnaryOperator&lt;T&gt;</code>    | <code>T apply(T t)</code> (extends <code>Function&lt;T, T&gt;</code> )             | Same-type transformation                                      |
| <code>BinaryOperator&lt;T&gt;</code>   | <code>T apply(T t1, T t2)</code> (extends <code>BiFunction&lt;T, T, T&gt;</code> ) | Combines two same-type values into one                        |
| <code>Runnable</code>                  | <code>void run()</code>                                                            | No input, no output (an action)                               |
| <code>Callable&lt;V&gt;</code>         | <code>V call()</code> throws <code>Exception</code>                                | Like <code>Supplier</code> , but can throw checked exceptions |

## 4. Code Example

```
import java.util.function.*;

Predicate<Integer> isEven = n -> n % 2 == 0;
System.out.println(isEven.test(4)); // true

BinaryOperator<Integer> add = (a, b) -> a + b;
System.out.println(add.apply(3, 5)); // 8
```

## 5. Frequently Asked Interview Questions

- What defines a functional interface?** Exactly one abstract method (any number of `default/static` methods are allowed too).
- Name the four core `java.util.function` interfaces.** `Predicate`, `Function`, `Consumer`, `Supplier`.
- What's the difference between `Function<T, R>` and `UnaryOperator<T>`?** `UnaryOperator<T>` is a specialization of `Function<T, T>` where the input and output types are the same.
- What does `@FunctionalInterface` actually enforce?** A compile-time error if the annotated interface does NOT have exactly one abstract method — purely a safety-net annotation, optional but strongly recommended.

# TOPIC 68: METHOD REFERENCES

## 1. Introduction

**What is it?** A method reference (::) is shorthand syntax for a lambda that does nothing but call an EXISTING method — a more concise, often more readable alternative when the lambda body would just be a direct delegation.

## 2. The Four Kinds of Method References

| Kind                                                          | Syntax                    | Equivalent Lambda                 | Example             |
|---------------------------------------------------------------|---------------------------|-----------------------------------|---------------------|
| Static method                                                 | ClassName::staticMethod   | x -> ClassName.staticMethod(x)    | Integer::parseInt   |
| Instance method of a PARTICULAR object                        | object::instanceMethod    | x -> object.instanceMethod(x)     | System.out::println |
| Instance method of an ARBITRARY object (of a particular type) | ClassName::instanceMethod | (obj, x) -> obj.instanceMethod(x) | String::toUpperCase |
| Constructor reference                                         | ClassName::new            | x -> new ClassName(x)             | ArrayList::new      |

## 3. Code Examples

```
import java.util.*;
import java.util.function.*;

List<String> names = List.of("charlie", "alice", "bob");

names.forEach(System.out::println); // instance method of a PARTICULAR object

List<String> upper = names.stream()
    .map(String::toUpperCase) // instance method of an ARBITRARY object
    .toList();

Function<String, Integer> parse = Integer::parseInt; // static method reference
System.out.println(parse.apply("42")); // 42

Supplier<ArrayList<String>> listFactory = ArrayList::new; // constructor reference
List<String> newList = listFactory.get();
```

## 4. Frequently Asked Interview Questions

- What are the four kinds of method references?** Static method, instance method of a particular object, instance method of an arbitrary object of a particular type, and constructor reference.
- When would you prefer a method reference over a lambda?** When the lambda body would do nothing but directly delegate to an existing method — improves readability by naming the exact operation instead of restating it as a lambda.
- What does `String::toUpperCase` mean as a method reference?** An instance method reference on an arbitrary `String` object — equivalent to the lambda `s -> s.toUpperCase()`.

- 4 **What does** `ArrayList::new` **represent?** A constructor reference, equivalent to the lambda `() -> new ArrayList<>()`.
- 

## TOPIC 69: STREAMS API

---

### 1. Introduction

**What is it?** The Streams API (`java.util.stream`) provides a **declarative, functional-style** way to process sequences of elements (from Collections, arrays, I/O) — describing WHAT transformation/aggregation you want, rather than manually writing HOW to loop through and accumulate results imperatively.

**Why introduced:** Traditional imperative loops over Collections are verbose and don't parallelize easily. Streams let you express a pipeline of operations (`filter` → `map` → `collect`) declaratively, which the JVM can execute sequentially OR in parallel with a single method call change, and which reads closer to the actual INTENT of the computation.

**Interview definition:** "A Stream is a sequence of elements supporting a pipeline of functional-style, LAZY intermediate operations (`filter`, `map`, `sorted`, etc.) followed by exactly one terminal operation (`collect`, `forEach`, `reduce`, etc.) that triggers actual execution; a Stream is a one-time-use, non-storage data-processing abstraction, NOT a data structure itself."

### 2. Real Life Analogy

- **Restaurant:** A stream pipeline is like an assembly line for orders: `filter` (only vegetarian orders) → `map` (convert each order into a plated dish) → `collect` (gather all plated dishes onto a tray) — each station only processes what's needed, and nothing happens until the FINAL station (terminal operation) actually starts the line moving.

### 3. Stream Pipeline Structure

SOURCE -> INTERMEDIATE OPERATIONS (lazy, return a NEW stream) -> TERMINAL OPERATION (triggers execution)

```
List.of(1,2,3,4,5).stream()
    .filter(n -> n % 2 == 0) // intermediate - LAZY, not executed yet
    .map(n -> n * n) // intermediate - LAZY, not executed yet
    .collect(Collectors.toList()); // TERMINAL - NOW the entire pipeline actually runs
```

## 4. Key Intermediate Operations

| Operation                                               | Purpose                                                                             |
|---------------------------------------------------------|-------------------------------------------------------------------------------------|
| <code>filter(Predicate)</code>                          | Keeps elements matching a condition                                                 |
| <code>map(Function)</code>                              | Transforms each element                                                             |
| <code>flatMap(Function)</code>                          | Transforms each element into a stream, then FLATTENS all resulting streams into one |
| <code>sorted()</code> / <code>sorted(Comparator)</code> | Sorts elements                                                                      |
| <code>distinct()</code>                                 | Removes duplicates (using <code>equals()</code> )                                   |
| <code>limit(n)</code> / <code>skip(n)</code>            | Truncates / skips the first n elements                                              |
| <code>peek(Consumer)</code>                             | Performs a side-effect action without altering the stream (mainly for debugging)    |

## 5. Key Terminal Operations

| Operation                                                                    | Purpose                                                            |
|------------------------------------------------------------------------------|--------------------------------------------------------------------|
| <code>collect(Collector)</code>                                              | Accumulates into a List, Set, Map, or custom structure             |
| <code>forEach(Consumer)</code>                                               | Performs an action on each element                                 |
| <code>reduce(...)</code>                                                     | Combines all elements into a single result                         |
| <code>count()</code>                                                         | Counts elements                                                    |
| <code>anyMatch()</code> / <code>allMatch()</code> / <code>noneMatch()</code> | Boolean condition checks (short-circuiting)                        |
| <code>toList()</code> (Java 16+)                                             | Convenience shortcut for <code>collect(Collectors.toList())</code> |
| <code>findFirst()</code> / <code>findAny()</code>                            | Returns an Optional wrapping the first/any matching element        |

## 6. Code Examples

### 6.1 Basic Pipeline

```
import java.util.*;
import java.util.stream.*;

List<String> names = List.of("Alice", "Bob", "Charlie", "Dave");

List<String> result = names.stream()
    .filter(n -> n.length() > 3)
    .map(String::toUpperCase)
    .sorted()
    .toList();
System.out.println(result); // [ALICE, CHARLIE, DAVE]
```

### 6.2 flatMap — Flattening Nested Structures

```
List<List<Integer>> nested = List.of(List.of(1, 2), List.of(3, 4), List.of(5));
List<Integer> flat = nested.stream()
    .flatMap(List::stream) // each inner List<Integer> -> stream, all merged into ONE stream
    .toList();
System.out.println(flat); // [1, 2, 3, 4, 5]
```

### 6.3 `reduce` — *Aggregation*

```
List<Integer> numbers = List.of(1, 2, 3, 4, 5);
int sum = numbers.stream().reduce(0, Integer::sum); // 15 (identity value 0, then accumulate)
Optional<Integer> max = numbers.stream().reduce(Integer::max); // no identity - returns Optional
```

### 6.4 Collectors — *The Aggregation Toolkit*

```
import java.util.stream.Collectors;

List<String> names = List.of("Alice", "Bob", "Charlie", "Anna");

Map<Character, List<String>> byFirstLetter = names.stream()
    .collect(Collectors.groupingBy(n -> n.charAt(0)));
System.out.println(byFirstLetter); // {A=[Alice, Anna], B=[Bob], C=[Charlie]}

String joined = names.stream().collect(Collectors.joining(", "));
System.out.println(joined); // "Alice, Bob, Charlie, Anna"

long count = names.stream().collect(Collectors.counting());
```

### 6.5 Parallel Streams

```
long sum = numbers.parallelStream() // splits work across multiple threads automatically
    .mapToLong(Integer::longValue)
    .sum();
```

## 7. Internal Working — Laziness (Critical Interview Point)

```
Stream<String> stream = names.stream()
    .filter(n -> {
        System.out.println("Filtering: " + n); // nothing prints yet!
        return n.length() > 3;
    });
// NOTHING has been printed - the pipeline hasn't executed at all!

long count = stream.count(); // ONLY NOW does the pipeline actually execute
```

Intermediate operations build up a PIPELINE DESCRIPTION lazily — nothing actually processes any elements until a TERMINAL operation is invoked, at which point the entire pipeline executes element-by-element (not stage-by-stage) for efficiency.

## 8. Edge Cases

| Case                                                                                | Result                                                                                                                                                                                     |
|-------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Reusing a <code>Stream</code> after a terminal operation has already run on it      | <code>IllegalStateException: stream has already been operated upon or closed</code>                                                                                                        |
| An empty source stream                                                              | All operations complete without error; terminal operations return sensible empty results ( <code>Optional.empty()</code> , empty <code>List</code> , 0 for <code>count()</code> )          |
| A stream with <code>null</code> elements passed through <code>map()/filter()</code> | Depends on the lambda's own null-safety — Streams themselves don't automatically guard against <code>null</code> , individual operations can still throw <code>NullPointerException</code> |
| Modifying the SOURCE collection while a stream is mid-pipeline                      | Can cause <code>ConcurrentModificationException</code> , same underlying reason as <code>[[Iterator</code>                                                                                 |

## 9. Common Mistakes

- Trying to reuse a `Stream` object after a terminal operation has consumed it — Streams are single-use.
- Using `parallelStream()` by default without measuring — parallel overhead can make SMALL datasets actually SLOWER than sequential processing; only genuinely helps for large datasets with substantial per-element work.
- Using `peek()` for actual business logic (rather than debugging) — its execution is not guaranteed under all circumstances, and it's intended purely as a diagnostic tool.

## 10. Best Practices

- Prefer method references over lambdas in stream pipelines when a direct delegation reads more clearly (`String::toUpperCase` vs `s -> s.toUpperCase()`).
- Keep stream pipelines readable — extremely long chained pipelines can hurt clarity; consider breaking into named intermediate variables/methods.
- Measure before reaching for `parallelStream()` — it's not automatically faster.

## 11. Production Usage

- Data transformation/aggregation in service layers (converting entity lists to DTOs, grouping/summarizing data) is one of the most common real-world uses of Streams across virtually every modern Java codebase.
- REST API response shaping (`entities.stream().map(EntityMapper::toDto).toList()`) is an extremely common pattern in Spring Boot services.

## 12. Frequently Asked Interview Questions

- 1 **What is a Stream?** A sequence supporting a pipeline of lazy intermediate operations and one terminal operation, NOT a data structure itself — it doesn't store elements.
- 2 **What's the difference between intermediate and terminal operations?** Intermediate operations (`filter`, `map`) are LAZY and return a new stream; terminal operations (`collect`, `forEach`, `reduce`) trigger actual pipeline execution and produce a final result (or side effect).
- 3 **Can a Stream be reused after a terminal operation?** No — attempting to reuse it throws `IllegalStateException`.
- 4 **What does `flatMap` do differently from `map`?** `map` transforms each element 1-to-1; `flatMap` transforms each element into its OWN stream, then flattens ALL those resulting streams into a single combined stream.
- 5 **Why are stream operations "lazy"?** So the JVM can build an efficient, single-pass execution plan across the whole pipeline once a terminal operation is finally invoked, rather than materializing intermediate collections at every stage.
- 6 **Is `parallelStream()` always faster?** No — for small datasets or lightweight per-element operations, the parallelization overhead can make it SLOWER than a sequential stream; benefits emerge mainly with large datasets and substantial per-element work.

- 7 **What does** `Collectors.groupingBy()` **do?** Groups stream elements into a `Map` keyed by a classifier function's result, with values collected into lists (by default) per group.

## 13. Revision Notes

### STREAMS API – CHEAT SHEET

=====

Stream = pipeline of LAZY intermediate ops + ONE terminal op. NOT a data structure.

SINGLE-USE: reusing after a terminal op -> `IllegalStateException`

INTERMEDIATE (lazy, return new stream): `filter`, `map`, `flatMap`, `sorted`, `distinct`, `limit`, `skip`, `peek`

TERMINAL (triggers execution): `collect`, `forEach`, `reduce`, `count`, `anyMatch`/  
`allMatch`/`noneMatch`, `toList()`, `findFirst`/`findAny`

`flatMap`: element -> STREAM, then FLATTENS all into one combined stream

Collectors: `toList()`, `joining()`, `groupingBy()`, `counting()`, `partitioningBy()`

`parallelStream()` -> NOT automatically faster, measure before using

LAZINESS: nothing runs until a terminal operation is called

## TOPIC 70: OPTIONAL

### 1. Introduction

**What is it?** `Optional<T>` is a container object that MAY or MAY NOT hold a non-null value — Java 8's answer to the pervasive `NullPointerException` problem, forcing callers to explicitly acknowledge and handle the "value might be absent" case rather than silently risking an NPE.

**Why introduced:** Before `Optional`, a method returning "no result" typically returned `null`, and callers frequently forgot to check for it, causing the single most common runtime exception in Java (`NullPointerException`). `Optional` makes absence an explicit, visible part of a method's return TYPE, encouraging (though not strictly forcing) safer handling.

### 2. Creating Optionals

```
Optional<String> present = Optional.of("Hello"); // throws NPE if the argument IS null
```

```
Optional<String> empty = Optional.empty(); // explicitly empty
```

```
Optional<String> nullable = Optional.ofNullable(maybeNullValue); // safely wraps a POSSIBLY-null value
```



### 3. Key Methods

| Method                                                     | Purpose                                                                                                                     |
|------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| <code>isPresent() / isEmpty()</code> (Java 11+)            | Checks presence/absence                                                                                                     |
| <code>get()</code>                                         | Returns the value; throws <code>NoSuchElementException</code> if empty — AVOID calling this without first checking presence |
| <code>orElse(T default)</code>                             | Returns the value, or a default if empty                                                                                    |
| <code>orElseGet(Supplier)</code>                           | Returns the value, or LAZILY computes a default only if actually needed                                                     |
| <code>orElseThrow(...)</code>                              | Returns the value, or throws a custom exception if empty                                                                    |
| <code>map(Function)</code>                                 | Transforms the wrapped value IF present, otherwise stays empty                                                              |
| <code>filter(Predicate)</code>                             | Keeps the value only if it matches a condition, otherwise becomes empty                                                     |
| <code>ifPresent(Consumer)</code>                           | Executes an action only if a value is present                                                                               |
| <code>ifPresentOrElse(Consumer, Runnable)</code> (Java 9+) | Executes one action if present, another if empty                                                                            |

### 4. Code Examples

```
import java.util.*;

public class OptionalDemo {
    public static void main(String[] args) {
        Optional<String> name = findUserName(42);

        // BAD - defeats the purpose of Optional
        // String result = name.get();

        // GOOD - safe handling
        String result = name.orElse("Unknown User");
        System.out.println(result);

        name.ifPresentOrElse(
            n -> System.out.println("Found: " + n),
            () -> System.out.println("Not found")
        );

        Optional<Integer> length = name.map(String::length);
    }

    static Optional<String> findUserName(int id) {
        return id == 42 ? Optional.of("Alice") : Optional.empty();
    }
}
```

## 5. Edge Cases

| Case                                                                                       | Result                                                                                                                                                                                                                                                |
|--------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>Optional.of(null)</code>                                                             | <code>NullPointerException</code> immediately — use <code>ofNullable()</code> for genuinely nullable sources                                                                                                                                          |
| Calling <code>.get()</code> on an empty <code>Optional</code>                              | <code>NoSuchElementException</code>                                                                                                                                                                                                                   |
| Using <code>Optional</code> as a class FIELD type                                          | Strongly discouraged — <code>Optional</code> is designed as a RETURN type for methods, not for fields, parameters, or collection elements                                                                                                             |
| <code>orElse()</code> vs <code>orElseGet()</code> when the default is expensive to compute | <code>orElse()</code> ALWAYS evaluates its argument eagerly (even if the <code>Optional</code> IS present); <code>orElseGet()</code> only evaluates its <code>Supplier</code> lazily, when actually needed — a subtle but real performance difference |

## 6. Common Mistakes

- Calling `.get()` without checking `isPresent()` first — completely defeats the purpose of `Optional`, just relocating the same NPE-like risk (`NoSuchElementException`) elsewhere.
- Using `Optional` for method PARAMETERS or class FIELDS — its intended, idiomatic use is exclusively as a method RETURN type.
- Using `orElse()` with an expensive default computation — it's evaluated EAGERLY even when not needed; use `orElseGet()` for lazy evaluation.

## 7. Best Practices

- Use `Optional` only as a method return type, signaling "this might not have a value" explicitly to callers.
- Prefer `map()/filter()/orElseGet()` functional-style chaining over manual `isPresent() + get()` checks.
- Never use `Optional` for fields, parameters, or inside collections (`List<Optional<T>>` is a design smell).

## 8. Frequently Asked Interview Questions

- 1 **What problem does `Optional` solve?** It makes the possibility of a "missing value" explicit in a method's return type, encouraging safer handling instead of relying on `null` and risking `NullPointerException`.
- 2 **What's the difference between `orElse()` and `orElseGet()`?** `orElse()` always eagerly evaluates its default argument; `orElseGet()` only lazily computes its `Supplier`-based default when actually needed — matters for expensive default computations.
- 3 **Should `Optional` be used for method parameters or fields?** No — its idiomatic, intended use is exclusively as a method RETURN type.
- 4 **What happens calling `Optional.of(null)`?** Throws `NullPointerException` immediately — use `Optional.ofNullable()` instead for genuinely nullable sources.
- 5 **What exception does `.get()` throw on an empty `Optional`?** `NoSuchElementException`.

# TOPIC 71: DATE & TIME API

## 1. Introduction

**What is it?** The `java.time` package (Java 8+) is a complete redesign of Java's date/time handling, replacing the old, notoriously flawed, MUTABLE `java.util.Date/Calendar` classes with a clean, **immutable**, thread-safe, ISO-8601-based API.

**Why introduced:** The old `Date/Calendar` classes were mutable (thread-unsafe), had confusing zero-indexed months, poor API design (date had both date AND time conflated awkwardly), and were widely regarded as one of Java's worst-designed early APIs. `java.time` (heavily inspired by the popular third-party Joda-Time library) fixes all of this.

## 2. Key Classes

| Class                          | Represents                                                             |
|--------------------------------|------------------------------------------------------------------------|
| <code>LocalDate</code>         | A date only (year, month, day) — no time, no timezone                  |
| <code>LocalTime</code>         | A time only (hour, minute, second) — no date, no timezone              |
| <code>LocalDateTime</code>     | Date AND time combined — no timezone                                   |
| <code>ZonedDateTime</code>     | Date, time, AND timezone                                               |
| <code>Instant</code>           | A single, precise point on the timeline (machine timestamp, UTC-based) |
| <code>Duration</code>          | A time-based amount (hours, minutes, seconds)                          |
| <code>Period</code>            | A date-based amount (years, months, days)                              |
| <code>DateTimeFormatter</code> | Formatting/parsing dates and times to/from Strings                     |

## 3. Code Examples

```
import java.time.*;
import java.time.format.DateTimeFormatter;

// ...

// Example 1: Working with LocalDate
// Create today's date
LocalDate today = LocalDate.now();
// Create a birthday (1995-08-15)
LocalDate birthday = LocalDate.of(1995, Month.AUGUST, 15); // note: real Month enum, no zero-indexing confusion!

// Calculate age
Period age = Period.between(birthday, today);
System.out.println(age.getYears() + " years old");

// Example 2: Working with LocalDateTime and DateTimeFormatter
// Create a meeting time (2026-08-06 14:30)
LocalDateTime meeting = LocalDateTime.of(2026, 8, 6, 14, 30);
// Create a formatter for 'dd-MMM-yyyy HH:mm'
DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd-MMM-yyyy HH:mm");
// Format the meeting time
System.out.println(meeting.format(formatter)); // "06-Aug-2026 14:30"

// Example 3: Working with LocalDate and Duration
// Create next week's date
LocalDate nextWeek = today.plusWeeks(1); // IMMUTABLE - returns a NEW LocalDate
// Create a duration of 90 minutes
Duration duration = Duration.ofMinutes(90);
```

## 4. `java.util.Date` VS `java.time.LocalDate` — Comparison Table

| Aspect         | <code>java.util.Date/Calendar</code> (legacy)  | <code>java.time.*</code> (Java 8+)                                                             |
|----------------|------------------------------------------------|------------------------------------------------------------------------------------------------|
| Mutability     | Mutable (thread-unsafe)                        | Immutable (thread-safe)                                                                        |
| Month indexing | 0-indexed (January = 0) — a classic bug source | Uses a proper <code>Month</code> enum, no off-by-one confusion                                 |
| API clarity    | Confusing, conflates date/time/timezone        | Clear separation: <code>LocalDate</code> , <code>LocalTime</code> , <code>ZonedDateTime</code> |
| Recommendation | Avoid in new code                              | Use exclusively for new code                                                                   |

## 5. Frequently Asked Interview Questions

- 1 **Why was `java.time` introduced given `Date/Calendar` already existed?** The old classes were mutable (thread-unsafe), had confusing zero-indexed months, and poor API design — `java.time` fixes all of this with an immutable, clearer, ISO-8601-based design.
- 2 **What's the difference between `LocalDateTime` and `ZonedDateTime`?** `LocalDateTime` has no timezone information; `ZonedDateTime` additionally carries a specific timezone.
- 3 **What's the difference between `Duration` and `Period`?** `Duration` measures TIME-based amounts (hours/minutes/seconds); `Period` measures DATE-based amounts (years/months/days).
- 4 **Is `LocalDate` mutable?** No — like all `java.time` classes, it's immutable; methods like `plusDays()` return a NEW instance rather than modifying the original.
- 5 **What represents a precise machine timestamp in `java.time`?** `Instant` — a point on the UTC timeline, independent of any human calendar/timezone representation.

# TOPIC 72: PREDICATE, FUNCTION, CONSUMER, SUPPLIER

## 1. Introduction

These four interfaces (from `java.util.function`, introduced alongside Lambdas in Java 8) are the fundamental building blocks used throughout the Streams API and functional-style Java code. (Already introduced briefly in [\[\[Functional Interfaces|Topic 67\]\]](#) — this topic covers each in depth with its own examples.)

## 2. `Predicate<T>` — A Condition

```
Predicate<Integer> isPositive = n -> n > 0;
System.out.println(isPositive.test(-5)); // false

Predicate<Integer> isEven = n -> n % 2 == 0;
Predicate<Integer> combined = isPositive.and(isEven); // combinator methods: and(), or(), negate()
System.out.println(combined.test(4)); // true
```

### 3. `Function<T, R>` — A Transformation

```
Function<String, Integer> length = String::length;
System.out.println(length.apply("Hello")); // 5

Function<Integer, Integer> square = x -> x * x;
Function<Integer, Integer> combined = length.andThen(square); // chains: length THEN square
System.out.println(combined.apply("Hello")); // 25 (5*5)
```

### 4. `Consumer<T>` — A Side-Effect Action

```
Consumer<String> print = System.out::println;
Consumer<String> printUpper = s -> System.out.println(s.toUpperCase());
Consumer<String> both = print.andThen(printUpper); // runs BOTH, in order
both.accept("hello");
// prints: "hello" then "HELLO"
```

### 5. `Supplier<T>` — A Value Producer (No Input)

```
Supplier<Double> randomValue = Math::random;
System.out.println(randomValue.get());

Supplier<List<String>> listFactory = ArrayList::new;
```

## 6. Comparison Table

| Interface                         | Input          | Output                     | Method                 | Use Case                        |
|-----------------------------------|----------------|----------------------------|------------------------|---------------------------------|
| <code>Predicate&lt;T&gt;</code>   | <code>T</code> | boolean                    | <code>test(T)</code>   | Filtering/condition checks      |
| <code>Function&lt;T, R&gt;</code> | <code>T</code> | <code>R</code>             | <code>apply(T)</code>  | Transformation                  |
| <code>Consumer&lt;T&gt;</code>    | <code>T</code> | none ( <code>void</code> ) | <code>accept(T)</code> | Side effects (printing, saving) |
| <code>Supplier&lt;T&gt;</code>    | none           | <code>T</code>             | <code>get()</code>     | Producing/generating values     |

## 7. Frequently Asked Interview Questions

- 1 **What does `Predicate<T>` represent?** A condition/boolean-valued function, via `test(T)`.
- 2 **What does `Function<T, R>` represent?** A transformation from type `T` to type `R`, via `apply(T)`.
- 3 **What does `Consumer<T>` represent, and why does it return `void`?** An action performed on a value with no result returned — it exists purely for its SIDE EFFECT (printing, saving, logging).
- 4 **What does `Supplier<T>` represent?** A value producer taking no input, via `get()` — often used for lazy computation or object factories.
- 5 **What combinator methods does `Predicate` provide?** `and()`, `or()`, `negate()` — for composing multiple conditions.

# TOPIC 73: DEFAULT METHODS & STATIC INTERFACE METHODS

---

## 1. Recap

(Full mechanics, syntax, and the historical "why" already covered in [\[\[Interfaces|Topic 23\]\]](#), Sections 3-4 — this topic is a focused summary specifically in the "Java 8 Features" context, since these were headline Java 8 additions alongside Lambdas and Streams.)

## 2. Why They Were Essential for Java 8 Specifically

Java 8 needed to add NEW methods (like `forEach()`, `stream()`, `splitterator()`) to the existing `Collection/Iterable` interfaces to support the new Streams API — WITHOUT breaking the millions of existing classes across the Java ecosystem that already implemented those interfaces. **Default methods** made this possible: existing implementers automatically inherit the new method's default behavior, requiring zero code changes.

```
// This is EXACTLY how Collection.forEach() was added in Java 8 without breaking every existing List/Set:
public interface Collection<E> extends Iterable<E> {
    default void forEach(Consumer<? super E> action) {
        for (E e : this) {
            action.accept(e);
        }
    }
}
```

## 3. Static Interface Methods — Utility/Factory Role

```
public interface Comparator<T> {
    static <T extends Comparable<T>> Comparator<T> naturalOrder() { // a REAL example from the JDK itself
        return (a, b) -> a.compareTo(b);
    }
}
Comparator<Integer> cmp = Comparator.naturalOrder(); // called via the INTERFACE name, not an instance
```

## 4. Frequently Asked Interview Questions

- 1 Why were default methods introduced specifically as part of Java 8?** To let the JDK add new methods (like `Collection.forEach()`, needed for the new Streams API) to existing interfaces without breaking every class that already implemented them.
  - 2 Can a static interface method be called via an implementing class's instance?** No — it must be called via the interface name directly (`InterfaceName.method()`), unlike default methods, which ARE inherited by implementing classes.
  - 3 Give a real JDK example of a static interface method.** `Comparator.naturalOrder()`, `Comparator.comparing(...)`.
-

# MULTITHREADING

---

## TOPIC 74: THREAD CLASS & RUNNABLE

---

### 1. Introduction

**What is it?** A thread is an independent path of execution within a program. Java provides built-in, language-level multithreading support via the `Thread` class and the `Runnable` functional interface, allowing multiple tasks to execute concurrently within a single process.

**Why introduced:** Modern applications need to handle multiple things at once — serving multiple web requests, performing background computation while keeping a UI responsive, or maximizing multi-core CPU utilization. Threading lets a single JVM process do genuinely concurrent (or, on multi-core hardware, truly parallel) work.

### 2. Two Ways to Create a Thread

```
// WAY 1: Extend Thread directly (uses UP your one "extends" slot - generally discouraged)
class MyThread extends Thread {
    @Override
    public void run() {
        System.out.println("Running via Thread subclass");
    }
}
new MyThread().start();

// WAY 2: Implement Runnable (PREFERRED - doesn't consume your inheritance slot, more flexible)
class MyTask implements Runnable {
    @Override
    public void run() {
        System.out.println("Running via Runnable");
    }
}
new Thread(new MyTask()).start();

// MODERN: Runnable is a functional interface - use a lambda!
new Thread(() -> System.out.println("Running via lambda")).start();
```

### 3. `start()` vs `run()` — Critical Distinction

| Method                               | Effect                                                                                                                   |
|--------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| <code>start()</code>                 | Creates a NEW operating system thread, which THEN calls <code>run()</code> on it — genuinely CONCURRENT execution        |
| <code>run()</code> (called directly) | Just a NORMAL method call on the CURRENT thread — no new thread is created, executes SEQUENTIALLY, no concurrency at all |

```
Thread t = new Thread(() -> System.out.println(Thread.currentThread().getName()));
t.run(); // prints "main" - NO new thread created, ran on the calling thread!
t.start(); // prints "Thread-0" (or similar) - genuinely runs on a NEW thread
```

## 4. Real Life Analogy

- **Restaurant:** A single chef (single thread) can only cook one dish at a time, sequentially. Hiring multiple chefs (multiple threads) lets several dishes cook CONCURRENTLY — but calling `run()` directly is like telling a NEW chef's recipe card to the SAME existing chef instead of actually hiring them, so nothing happens in parallel.

## 5. Key Thread Methods

| Method                             | Purpose                                                                                        |
|------------------------------------|------------------------------------------------------------------------------------------------|
| <code>start()</code>               | Begins a new thread's execution                                                                |
| <code>join()</code>                | Blocks the CALLING thread until the target thread finishes                                     |
| <code>sleep(ms)</code>             | Pauses the CURRENT thread for the given duration (static method)                               |
| <code>interrupt()</code>           | Signals a thread to stop/wake from a blocking operation                                        |
| <code>setDaemon(true)</code>       | Marks a thread as a background (daemon) thread — the JVM exits once ONLY daemon threads remain |
| <code>getName() / setName()</code> | Thread identification                                                                          |
| <code>setPriority(int)</code>      | Hints scheduling priority (1-10) — NOT a strict guarantee, OS-dependent                        |

## 6. Code Example

```
public class ThreadDemo {
    public static void main(String[] args) throws InterruptedException {
        Thread worker = new Thread(() -> {
            for (int i = 1; i <= 3; i++) {
                System.out.println("Worker: " + i);
            }
        });
        worker.start();
        worker.join(); // main thread WAITS for worker to finish
        System.out.println("Main thread continues after worker is done");
    }
}
```



## 7. Edge Cases

| Case                                                                      | Result                                                                                                                                                                        |
|---------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Calling <code>start()</code> twice on the same <code>Thread</code> object | <code>IllegalThreadStateException</code> — a thread can only be started ONCE                                                                                                  |
| Calling <code>run()</code> directly instead of <code>start()</code>       | Executes on the CURRENT thread, no actual concurrency                                                                                                                         |
| An uncaught exception inside a thread's <code>run()</code>                | Terminates ONLY that thread (prints a stack trace via the default uncaught exception handler) — does NOT crash other threads or the whole JVM (unless it was the main thread) |

## 8. Common Mistakes

- Calling `run()` instead of `start()`, expecting concurrent execution but getting sequential execution instead.
- Extending `Thread` directly instead of implementing `Runnable` — unnecessarily consumes the single inheritance slot and tightly couples "being a thread" with the task's actual logic.
- Assuming `setPriority()` guarantees execution order — it's merely a scheduling HINT, heavily OS-dependent.

## 9. Best Practices

- Prefer implementing `Runnable` (or using a lambda) over extending `Thread` directly.
- In real production code, prefer the **Executor Framework** (covered soon) over manually creating raw `Thread` objects — better resource management, reuse, and control.

## 10. Frequently Asked Interview Questions

- 1 **What's the difference between `start()` and `run()`?** `start()` creates a genuinely new OS thread which then calls `run()`; calling `run()` directly just executes it as a normal method on the CURRENT thread — no concurrency occurs.
- 2 **Why is implementing `Runnable` generally preferred over extending `Thread`?** It avoids consuming Java's single-inheritance slot, and cleanly separates "the task" from "being a thread," allowing more flexible reuse (e.g., submitting the same `Runnable` to an `Executor`).
- 3 **What does `join()` do?** Blocks the calling thread until the target thread completes execution.
- 4 **Can you call `start()` twice on the same thread object?** No — throws `IllegalThreadStateException`; a thread's lifecycle only permits starting it once.
- 5 **What is a daemon thread?** A background thread that doesn't prevent JVM shutdown — the JVM exits once only daemon threads remain running.

---

# TOPIC 75: THREAD LIFECYCLE

---

## 1. Introduction

**What is it?** Every Java thread progresses through a well-defined set of states, represented by the `Thread.State` enum, from creation to termination.

## 2. The Six States

```
NEW --start()--> RUNNABLE <-----> RUNNING (conceptually, OS-scheduled subset of RUNNABLE)
| ^
| (waiting for lock/ \
I/O/sleep/join) \ (lock acquired / I/O done / woken up)
v \
BLOCKED / WAITING / TIMED_WAITING
|
v (run() completes, or uncaught exception)
TERMINATED
```

| State         | Meaning                                                                                                                                         |
|---------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| NEW           | Thread object created, <code>start()</code> not yet called                                                                                      |
| RUNNABLE      | Eligible to run (may be actively running OR waiting for CPU time from the OS scheduler — Java doesn't distinguish these two as separate states) |
| BLOCKED       | Waiting to acquire a <code>synchronized</code> lock held by another thread                                                                      |
| WAITING       | Waiting indefinitely for another thread's action (e.g., <code>Object.wait()</code> with no timeout, <code>Thread.join()</code> with no timeout) |
| TIMED_WAITING | Waiting for a BOUNDED time ( <code>Thread.sleep(ms)</code> , <code>wait(ms)</code> , <code>join(ms)</code> )                                    |
| TERMINATED    | <code>run()</code> has completed (normally or via an uncaught exception) — cannot be restarted                                                  |

## 3. Real Life Analogy

- Restaurant:** `NEW` is a hired chef who hasn't started their shift yet. `RUNNABLE` is a chef ready and cooking (or waiting for the next order, OS-scheduled). `BLOCKED` is a chef waiting for a specific stove (lock) currently used by another chef. `WAITING` is a chef waiting indefinitely for a specific ingredient delivery with no ETA given. `TIMED_WAITING` is a chef setting a timer and waiting exactly 10 minutes. `TERMINATED` is a chef who has clocked out for good.

## 4. Code Example — Observing State Transitions

```
public class LifecycleDemo {
    public static void main(String[] args) throws InterruptedException {
        Thread t = new Thread(() -> {
            try { Thread.sleep(1000); } catch (InterruptedException e) { }
        });
        System.out.println(t.getState()); // NEW
        t.start();
        System.out.println(t.getState()); // RUNNABLE
        Thread.sleep(200);
        System.out.println(t.getState()); // TIMED_WAITING (inside sleep)
        t.join();
        System.out.println(t.getState()); // TERMINATED
    }
}
```

## 5. Frequently Asked Interview Questions

- 1 **What are the six thread states in Java?** NEW, RUNNABLE, BLOCKED, WAITING, TIMED\_WAITING, TERMINATED.
- 2 **What's the difference between BLOCKED and WAITING?** BLOCKED specifically means waiting to acquire a synchronized lock; WAITING means waiting indefinitely for another thread's explicit signal (e.g., `notify()`), or an unbounded `join()`.
- 3 **Can a TERMINATED thread be restarted?** No — once terminated, a `Thread` object can never be started again; you must create a brand-new `Thread` instance.
- 4 **What state is a thread in while executing `Thread.sleep(1000)`?** TIMED\_WAITING.

---

# TOPIC 76: SYNCHRONIZATION

---

## 1. Introduction

**What is it?** Synchronization is Java's mechanism for controlling concurrent access to shared mutable state, ensuring only ONE thread at a time can execute a critical section, preventing **race conditions** (where the outcome depends unpredictably on thread timing/interleaving).

## 2. The Race Condition Problem

```
class Counter {
    private int count = 0;
    void increment() { count++; } // NOT ATOMIC! Actually 3 steps: read, add 1, write back
}
```

If two threads call `increment()` "simultaneously," both might READ the same old value before either WRITES back the incremented result — one increment gets silently LOST. This is a **race condition**, and it's exactly why `count++` (despite looking like one operation) is NOT thread-safe.

### 3. `synchronized` — Method and Block Forms

```
class Counter {
    private int count = 0;

    synchronized void increment() { // SYNCHRONIZED METHOD - locks on 'this'
        count++;
    }

    void incrementBlock() {
        synchronized (this) { // SYNCHRONIZED BLOCK - more granular, locks only this section
            count++;
        }
    }

    static synchronized void staticMethod() { // locks on the CLASS object (Counter.class), not an
        // instance
        // ...
    }
}
```

- Every Java object has an intrinsic **monitor lock**. `synchronized` acquires that lock before entering, and releases it automatically upon exit (even if an exception is thrown) — guaranteeing only ONE thread can hold that specific lock at a time.

### 4. Real Life Analogy

- **Bank:** A single ATM/teller window (the synchronized resource) can only serve ONE customer (thread) at a time — everyone else must wait in line (blocked) until the current customer finishes, preventing two customers from simultaneously modifying the SAME account balance and corrupting it.

### 5. Code Example — Fixing the Race Condition

```
class SafeCounter {
    private int count = 0;
    synchronized void increment() { count++; }
    synchronized int getCount() { return count; }
}

public class SyncDemo {
    public static void main(String[] args) throws InterruptedException {
        SafeCounter counter = new SafeCounter();
        Runnable task = () -> {
            for (int i = 0; i < 1000; i++) counter.increment();
        };
        Thread t1 = new Thread(task);
        Thread t2 = new Thread(task);
        t1.start(); t2.start();
        t1.join(); t2.join();
        System.out.println(counter.getCount()); // reliably 2000 - WITHOUT synchronized, often < 2000
    }
}
```

## 6. `volatile` — Visibility Without Mutual Exclusion

```
class FlagHolder {
    private volatile boolean running = true; // guarantees VISIBILITY across threads, NOT atomicity
    void stop() { running = false; }
    void doWork() {
        while (running) { /* work */ } // without 'volatile', another thread's stop() might
        // never become visible here (cached read)
    }
}
```

`volatile` ensures every read sees the MOST RECENT write from any thread (no stale caching), but does NOT provide mutual exclusion — it's insufficient for compound operations like `count++` (still a race condition even if `count` is `volatile`).

## 7. Edge Cases

| Case                                                                                              | Result                                                                                                            |
|---------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| Two threads calling <code>synchronized</code> methods on TWO DIFFERENT object instances           | NO blocking occurs — each object has its OWN independent lock                                                     |
| A thread already holding a lock calls another <code>synchronized</code> method on the SAME object | Allowed — Java's locks are REENTRANT (a thread can re-acquire a lock it already holds without deadlocking itself) |
| Using <code>volatile</code> for a compound operation like <code>count++</code>                    | STILL a race condition — <code>volatile</code> only guarantees visibility, not atomicity                          |
| Two threads each holding a lock the OTHER needs                                                   | <b>Deadlock</b> — both threads wait forever (see Locks topic for prevention strategies)                           |

## 8. Common Mistakes

- Assuming `volatile` alone makes compound operations (`count++`) thread-safe — it doesn't; use `synchronized` Or `AtomicInteger` instead.
- Synchronizing on different objects (e.g., one method synchronizes on `this`, another on a separate lock object) when protecting the SAME shared state — provides NO actual protection.
- Overusing coarse-grained synchronization (locking entire large methods) when a smaller, more targeted `synchronized` block would reduce contention and improve throughput.

## 9. Best Practices

- Keep `synchronized` sections as SHORT as possible — minimize the time a lock is held to reduce contention.
- Prefer higher-level concurrency utilities (`java.util.concurrent` — Locks, Atomic classes, Concurrent Collections) over raw `synchronized`/`wait`/`notify` for most modern production code.
- Always synchronize consistently on the SAME lock object for all access paths to the same shared mutable state.

## 10. Frequently Asked Interview Questions

- 1 **What is a race condition?** A bug where the outcome of concurrent operations depends unpredictably on the exact timing/interleaving of threads, typically due to unsynchronized access to shared mutable state.
- 2 **Why isn't `count++` thread-safe even though it looks like one operation?** It's actually THREE steps (read, increment, write back) — two threads can interleave these steps, causing a lost update.
- 3 **What does `synchronized` guarantee?** Mutual exclusion (only one thread executes the protected section at a time) via acquiring an object's intrinsic monitor lock, plus a memory visibility guarantee for changes made within it.
- 4 **What does `volatile` guarantee, and what does it NOT guarantee?** It guarantees visibility (every read sees the latest write across threads), but NOT atomicity — compound operations like `count++` are still race-condition-prone even on a `volatile` variable.
- 5 **Are Java's intrinsic locks reentrant?** Yes — a thread already holding a lock can re-acquire it (e.g., calling another `synchronized` method on the same object) without deadlocking itself.
- 6 **What's the difference between synchronizing on an instance method versus a static method?** An instance `synchronized` method locks on `this` (per-object); a static `synchronized` method locks on the `CLASS` object itself (`ClassName.class`), shared across ALL instances.

---

## TOPIC 77: LOCKS (`java.util.concurrent.locks`)

---

### 1. Introduction

**What is it?** The `Lock` interface (and its main implementation, `ReentrantLock`) provides a more flexible, explicit alternative to the `synchronized` keyword — supporting features `synchronized` cannot: try-locking with a timeout, interruptible lock acquisition, and fairness policies.

### 2. `synchronized` VS `ReentrantLock` — Comparison Table

| Aspect                        | <code>synchronized</code>                                      | <code>ReentrantLock</code>                                                           |
|-------------------------------|----------------------------------------------------------------|--------------------------------------------------------------------------------------|
| Lock acquisition              | Implicit (automatic)                                           | Explicit ( <code>lock()/unlock()</code> )                                            |
| Releases lock on exception    | Automatic (built into the language construct)                  | MUST manually <code>unlock()</code> in a <code>finally</code> block — easy to forget |
| Try-lock with timeout         | Not supported                                                  | <code>tryLock(timeout, unit)</code> supported                                        |
| Interruptible waiting         | Not supported                                                  | <code>lockInterruptibly()</code> supported                                           |
| Fairness (FIFO lock granting) | Not supported                                                  | Configurable via constructor ( <code>new ReentrantLock(true)</code> )                |
| Condition variables           | One implicit condition per object ( <code>wait/notify</code> ) | Multiple named <code>Condition</code> objects supported per lock                     |

### 3. Code Example

```
import java.util.concurrent.locks.ReentrantLock;

class SafeCounter {
    private int count = 0;
    private final ReentrantLock lock = new ReentrantLock();

    void increment() {
        lock.lock();
        try {
            count++;
        } finally {
            lock.unlock(); // MUST be in finally - or the lock is never released if an exception occurs!
        }
    }
}
```

### 4. Deadlock — What It Is and How to Avoid It

```
// CLASSIC DEADLOCK SETUP:
// Thread 1: locks A, then tries to lock B
// Thread 2: locks B, then tries to lock A
// -> Both wait forever for the OTHER'S lock -> DEADLOCK
```

**Prevention strategies:** always acquire multiple locks in a CONSISTENT, GLOBAL order across all threads; use `tryLock()` with a timeout instead of blocking indefinitely; minimize the number of locks held simultaneously.

### 5. Frequently Asked Interview Questions

- 1 **What advantages does `ReentrantLock` have over `synchronized`?** Try-locking with a timeout, interruptible lock acquisition, configurable fairness, and support for multiple `Condition` objects per lock.
- 2 **Why must `unlock()` always be called inside a `finally` block?** Because unlike `synchronized`, `ReentrantLock` does NOT automatically release the lock if an exception occurs — forgetting this causes the lock to be held forever, likely causing a deadlock for other threads.
- 3 **What is a deadlock?** A situation where two or more threads are each waiting indefinitely for a lock held by another, none able to proceed.
- 4 **How can you prevent deadlocks?** Always acquire multiple locks in a consistent global order, or use `tryLock()` with a timeout instead of blocking indefinitely.

---

## TOPIC 78: EXECUTOR FRAMEWORK

---

### 1. Introduction

**What is it?** The Executor Framework (`java.util.concurrent`) provides a higher-level abstraction for managing thread execution — decoupling task SUBMISSION from the details of thread creation, pooling, and lifecycle management, via `ExecutorService` and thread pools.

**Why introduced:** Manually creating a new Thread for every task is wasteful (thread creation/destruction has real overhead) and hard to control (no built-in limit on concurrent threads, risking resource exhaustion). Thread pools REUSE a fixed/bounded set of worker threads across many submitted tasks.

## 2. Creating an Executor

```
import java.util.concurrent.*;

ExecutorService fixedPool = Executors.newFixedThreadPool(4); // exactly 4 worker threads
ExecutorService cachedPool = Executors.newCachedThreadPool(); // grows/shrinks as needed, reuses idle threads
ExecutorService singleThread = Executors.newSingleThreadExecutor(); // exactly 1 worker thread, tasks run sequentially
```

## 3. Submitting Tasks

```
ExecutorService executor = Executors.newFixedThreadPool(2);

executor.submit(() -> System.out.println("Task 1 running on: " + Thread.currentThread().getName()));
executor.submit(() -> System.out.println("Task 2 running on: " + Thread.currentThread().getName()));

executor.shutdown(); // initiates orderly shutdown - stops accepting NEW tasks, finishes queued ones
```

## 4. Real Life Analogy

- **Restaurant:** A thread pool is like having a FIXED team of chefs (worker threads) — incoming orders (tasks) get QUEUED and picked up by whichever chef becomes free next, instead of hiring (creating) and firing (destroying) a brand-new chef for every single order — vastly more efficient.

## 5. Key Methods

| Method                                             | Purpose                                                                               |
|----------------------------------------------------|---------------------------------------------------------------------------------------|
| <code>submit(Runnable/Callable)</code>             | Submits a task, returns a <code>Future</code> representing its eventual result        |
| <code>execute(Runnable)</code>                     | Submits a task with NO return value tracking (fire-and-forget)                        |
| <code>shutdown()</code>                            | Orderly shutdown — finishes queued/running tasks, rejects new ones                    |
| <code>shutdownNow()</code>                         | Attempts to forcibly stop all actively executing tasks immediately                    |
| <code>awaitTermination(timeout, unit)</code>       | Blocks until all tasks complete or the timeout elapses                                |
| <code>invokeAll(Collection&lt;Callable&gt;)</code> | Submits multiple tasks, blocks until ALL complete, returns their <code>Futures</code> |



## 6. Edge Cases

| Case                                                                   | Result                                                                                               |
|------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| Submitting a task AFTER calling <code>shutdown()</code>                | <code>RejectedExecutionException</code>                                                              |
| Not calling <code>shutdown()</code> at all                             | The JVM may never exit — non-daemon pool threads keep it alive indefinitely                          |
| An exception thrown inside a task submitted via <code>execute()</code> | Printed to the default uncaught exception handler, thread pool continues functioning for other tasks |

## 7. Common Mistakes

- Forgetting to call `shutdown()`, causing the application to hang and never terminate (non-daemon executor threads keep the JVM alive).
- Using `newCachedThreadPool()` for high-volume workloads without limits — it can create an UNBOUNDED number of threads under heavy load, risking resource exhaustion.
- Manually creating raw `Thread` objects in application code instead of submitting tasks to a shared, properly-sized executor.

## 8. Best Practices

- Always call `shutdown()` (typically in a `finally` block or via try-with-resources patterns in newer JDKs) once no more tasks will be submitted.
- Size thread pools deliberately based on workload characteristics (CPU-bound vs I/O-bound tasks need very different pool sizes).
- Prefer the Executor Framework over manually managing raw `Thread` objects in virtually all production code.

## 9. Frequently Asked Interview Questions

- 1 **Why use the Executor Framework instead of creating raw `Thread` objects?** It reuses a managed pool of worker threads (avoiding per-task thread creation overhead), provides built-in task queuing, and offers controlled lifecycle management (`shutdown`, `timeout`).
  - 2 **What's the difference between `submit()` and `execute()`?** `submit()` returns a `Future` for tracking the task's result/completion/exceptions; `execute()` is fire-and-forget with no return tracking.
  - 3 **What's the difference between `shutdown()` and `shutdownNow()`?** `shutdown()` allows already-queued/running tasks to finish before stopping; `shutdownNow()` attempts to forcibly interrupt and stop everything immediately.
  - 4 **What happens if you submit a task after calling `shutdown()`?** `RejectedExecutionException` is thrown.
  - 5 **What's a risk of `Executors.newCachedThreadPool()` under heavy load?** It can create an unbounded number of threads, potentially exhausting system resources — unlike a fixed-size pool with a hard cap.
-

# TOPIC 79: CALLABLE & FUTURE

## 1. Introduction

**What is it?** `Callable<V>` is like `Runnable`, but its `call()` method CAN return a value and throw checked exceptions — designed specifically for use with the Executor Framework. `Future<V>` represents the eventual RESULT of an asynchronous computation, submitted via `ExecutorService.submit()`.

## 2. `Runnable` vs `Callable` — Comparison Table

| Aspect             | <code>Runnable</code>                   | <code>Callable&lt;V&gt;</code>                      |
|--------------------|-----------------------------------------|-----------------------------------------------------|
| Method             | <code>void run()</code>                 | <code>V call()</code> throws <code>Exception</code> |
| Return value       | None                                    | Yes — of type <code>V</code>                        |
| Checked exceptions | Cannot declare/throw checked exceptions | CAN throw checked exceptions                        |

## 3. Code Example

```
import java.util.concurrent.*;

public class FutureDemo {
    public static void main(String[] args) throws Exception {
        ExecutorService executor = Executors.newSingleThreadExecutor();

        Callable<Integer> task = () -> {
            Thread.sleep(1000); // simulate work
            return 42;
        };

        Future<Integer> future = executor.submit(task);

        System.out.println("Doing other work while task runs...");
        Integer result = future.get(); // BLOCKS until the result is ready
        System.out.println("Result: " + result);

        executor.shutdown();
    }
}
```

## 4. Key `Future` Methods

| Method                                    | Purpose                                                                                                                  |
|-------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| <code>get()</code>                        | Blocks until the result is available, returns it (or throws <code>ExecutionException</code> wrapping any task exception) |
| <code>get(timeout, unit)</code>           | Blocks up to a bounded time, throws <code>TimeoutException</code> if not ready in time                                   |
| <code>isDone()</code>                     | Checks if the computation has completed (without blocking)                                                               |
| <code>cancel(boolean mayInterrupt)</code> | Attempts to cancel the task                                                                                              |
| <code>isCancelled()</code>                | Checks if the task was cancelled                                                                                         |

## 5. Edge Cases

| Case                                                                              | Result                                                                                             |
|-----------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| Calling <code>get()</code> on a <code>Future</code> whose task threw an exception | <code>ExecutionException</code> , with the ORIGINAL exception as its <code>getCause()</code>       |
| Calling <code>get()</code> before the task completes                              | Blocks the calling thread until it does (or use <code>get(timeout, unit)</code> to bound the wait) |
| Calling <code>cancel(true)</code> on an already-completed task                    | Returns <code>false</code> — cannot cancel what's already finished                                 |

## 6. Frequently Asked Interview Questions

- 1 **What's the key difference between `Runnable` and `Callable`?** `Callable` can return a value and throw checked exceptions; `Runnable` cannot do either.
  - 2 **What does `Future.get()` do if the task isn't finished yet?** Blocks the calling thread until the result becomes available (or use the timed overload to avoid indefinite blocking).
  - 3 **What exception wraps a task's thrown exception when retrieved via `Future.get()`?** `ExecutionException`, with the original exception accessible via `getCause()`.
  - 4 **Is `Future` itself capable of registering a callback for when the result is ready?** No — plain `Future` only supports blocking `get()`; `CompletableFuture` (next topic) adds non-blocking callback-style composition.
- 

# TOPIC 80: COMPLETABLEFUTURE

---

## 1. Introduction

**What is it?** `CompletableFuture<T>` (Java 8+) is an enhanced `Future` implementation supporting **non-blocking, callback-based, composable** asynchronous programming — letting you chain what happens NEXT once a result is ready, without blocking any thread waiting for it.

**Why introduced:** Plain `Future.get()` is BLOCKING — there's no way to say "when this completes, automatically do X next" without a thread sitting idle waiting. `CompletableFuture` solves this with a fluent, functional-style API for chaining, combining, and reacting to asynchronous results.

## 2. Creating and Chaining

```
import java.util.concurrent.*;

CompletableFuture<Integer> future = CompletableFuture.supplyAsync(() -> {
// runs on a background thread (ForkJoinPool.commonPool() by default)
return 10;
});

CompletableFuture<Integer> chained = future
    .thenApply(x -> x * 2) // transforms the result, once ready
    .thenApply(x -> x + 5);

System.out.println(chained.get()); // 25 - (10*2)+5
```

## 3. Key Methods

| Method                                      | Purpose                                                                                                                     |
|---------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| <code>supplyAsync(Supplier)</code>          | Starts an async computation returning a value                                                                               |
| <code>runAsync(Runnable)</code>             | Starts an async computation with NO return value                                                                            |
| <code>thenApply(Function)</code>            | Transforms the result once available (synchronously, on the SAME completing thread by default)                              |
| <code>thenAccept(Consumer)</code>           | Consumes the result (side effect), no further value produced                                                                |
| <code>thenCompose(Function)</code>          | Chains another async operation that itself returns a <code>CompletableFuture</code> (flattens, avoiding nested futures)     |
| <code>thenCombine(other, BiFunction)</code> | Combines results from TWO independent <code>CompletableFuture</code> s                                                      |
| <code>exceptionally(Function)</code>        | Provides a fallback value if an exception occurred anywhere upstream                                                        |
| <code>join()</code>                         | Like <code>get()</code> , but throws an <code>UncheckedException</code> instead (no <code>throws</code> declaration needed) |

## 4. Real Life Analogy

- **Restaurant:** A `CompletableFuture` is like placing an order and getting a BUZZER instead of standing at the counter waiting — you go do OTHER things (non-blocking), and when the buzzer goes off (completion), a chain of follow-up actions automatically happens (plate it, add garnish, deliver to table) — no one needs to stand around blocking.

## 5. Code Example — Combining and Exception Handling

```
CompletableFuture<Integer> priceFuture = CompletableFuture.supplyAsync(() -> 100);
CompletableFuture<Double> discountFuture = CompletableFuture.supplyAsync(() -> 0.1);

CompletableFuture<Double> finalPrice = priceFuture.thenCombine(discountFuture,
(price, discount) -> price * (1 - discount));
System.out.println(finalPrice.join()); // 90.0

CompletableFuture<Integer> risky = CompletableFuture.supplyAsync(() -> {
if (true) throw new RuntimeException("Failed!");
return 5;
}).exceptionally(ex -> {
System.out.println("Recovered from: " + ex.getMessage());
return -1; // fallback value
});
System.out.println(risky.join()); // -1
```

## 6. Future VS CompletableFuture — Comparison Table

| Aspect                     | Future                              | CompletableFuture                                               |
|----------------------------|-------------------------------------|-----------------------------------------------------------------|
| Blocking retrieval         | get() (blocks)                      | get()/join() (blocks) still available                           |
| Non-blocking chaining      | Not supported                       | thenApply/thenAccept/thenCompose (callback-style, non-blocking) |
| Combining multiple futures | Manual, awkward                     | thenCombine(), allOf(), anyOf() built in                        |
| Exception handling         | Only via get()'s ExecutionException | exceptionally(), handle() for fluent recovery                   |

## 7. Frequently Asked Interview Questions

- What problem does CompletableFuture solve that plain Future doesn't?** Non-blocking, callback-based composition — chaining follow-up actions automatically once a result is ready, instead of requiring a thread to block on get().
- What's the difference between thenApply and thenCompose?** thenApply transforms a result directly; thenCompose chains another operation that ITSELF returns a CompletableFuture, flattening what would otherwise be a nested CompletableFuture<CompletableFuture<T>>.
- How do you combine two independent CompletableFutures' results?** thenCombine(other, BiFunction).
- How do you provide a fallback value if a CompletableFuture fails with an exception?** exceptionally(Function<Throwable, T>).
- What's the difference between get() and join()?** get() throws checked exceptions (ExecutionException, InterruptedException); join() throws an unchecked equivalent, avoiding the need for a throws declaration or try-catch at the call site.

# TOPIC 81: CONCURRENT COLLECTIONS

## 1. Introduction

**What is it?** The `java.util.concurrent` package provides thread-safe collection implementations specifically designed for high-performance concurrent access — a modern, far more scalable alternative to synchronizing plain `HashMap/ArrayList` externally or using the legacy `Hashtable/Vector`.

## 2. Key Concurrent Collections

| Class                                                                                                      | Purpose                                                                                                                                                                                                                            |
|------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>ConcurrentHashMap&lt;K, V&gt;</code>                                                                 | Thread-safe Map, using fine-grained internal locking (segment/bucket-level, NOT one lock for the whole map) — vastly more scalable than <code>Collections.synchronizedMap()</code> or <code>Hashtable</code> under concurrent load |
| <code>CopyOnWriteArrayList&lt;E&gt;</code>                                                                 | Thread-safe List — every WRITE creates a new internal copy of the underlying array; reads never block, ideal for READ-HEAVY, rarely-modified lists                                                                                 |
| <code>BlockingQueue&lt;E&gt;</code> ( <code>ArrayBlockingQueue</code> , <code>LinkedBlockingQueue</code> ) | A Queue where <code>put()</code> BLOCKS if full, and <code>take()</code> BLOCKS if empty — the standard building block for producer-consumer pipelines                                                                             |
| <code>ConcurrentSkipListMap/ConcurrentSkipListSet</code>                                                   | Thread-safe, SORTED equivalents of <code>TreeMap/TreeSet</code>                                                                                                                                                                    |

## 3. `ConcurrentHashMap` VS `Hashtable/synchronizedMap()`

| Aspect                             | <code>ConcurrentHashMap</code>    | <code>Hashtable / Collections.synchronizedMap()</code>                                               |
|------------------------------------|-----------------------------------|------------------------------------------------------------------------------------------------------|
| Locking granularity                | Fine-grained (per-bucket/segment) | Coarse-grained (ENTIRE map locked per operation)                                                     |
| Concurrent reads                   | Fully non-blocking                | Blocked while any write holds the lock                                                               |
| Scalability under high concurrency | Excellent                         | Poor — becomes a bottleneck                                                                          |
| <code>null</code> keys/values      | Not allowed (throws NPE)          | <code>Hashtable</code> : not allowed;<br><code>synchronizedMap()</code> : depends on the wrapped map |

## 4. Producer-Consumer with `BlockingQueue`

```
import java.util.concurrent.*;

BlockingQueue<Integer> queue = new ArrayBlockingQueue<>(10);

Thread producer = new Thread(() -> {
    for (int i = 0; i < 5; i++) {
        try { queue.put(i); System.out.println("Produced: " + i); }
        catch (InterruptedException e) { }
    }
});

Thread consumer = new Thread(() -> {
    for (int i = 0; i < 5; i++) {
        try { System.out.println("Consumed: " + queue.take()); }
        catch (InterruptedException e) { }
    }
});

producer.start();
consumer.start();
```

## 5. Real Life Analogy

- **Restaurant:** `ConcurrentHashMap` is like a large restaurant with SEPARATE stations/registers (fine-grained locks) — many customers can be served SIMULTANEOUSLY at different stations, unlike a single-register restaurant (`Hashtable`) where EVERY customer must wait in one line regardless of which station they actually need.
- **Bank:** A `BlockingQueue` is like a bank's ticket-dispensing queue system — tellers (`take()`) automatically wait if no customer tickets are queued, and the dispenser (`put()`) pauses issuing new tickets if the waiting area is already full.

## 6. Frequently Asked Interview Questions

- 1 **Why is `ConcurrentHashMap` more scalable than `Hashtable`?** It uses fine-grained, bucket/segment-level locking instead of one lock for the entire map, allowing many threads to operate on DIFFERENT parts of the map simultaneously.
- 2 **When would you use `CopyOnWriteArrayList`?** For read-heavy, rarely-modified lists accessed concurrently — reads never block (each read sees a consistent snapshot), though writes are expensive (full array copy each time).
- 3 **What does `BlockingQueue` add over a regular queue?** Blocking behavior — `put()` blocks if the queue is full, `take()` blocks if it's empty — making it the standard building block for producer-consumer patterns.
- 4 **Does `ConcurrentHashMap` allow null keys or values?** No — unlike regular `HashMap` (which allows one null key), `ConcurrentHashMap` disallows null entirely, since null would create ambiguity in concurrent contexts about whether a key is truly absent or mapped to null.

---

## Homework (Covers Topics 74-81: Multithreading)

- 1 Write a race-condition demo (unsynchronized shared counter incremented by two threads) and show the actual (incorrect) result differs from the expected 2000, then fix it with `synchronized`.
- 2 Build a small producer-consumer system using `BlockingQueue` with one producer and two consumer threads.
- 3 Submit 5 `Callable` tasks to a fixed thread pool, collect their `Futures`, and print all results once ready.
- 4 Chain 3 `CompletableFuture` stages (`supplyAsync` → `thenApply` → `thenAccept`) and add an `exceptionally()` fallback for a deliberately failing stage.

---

(End of Topics 74-81 — this concludes the Multithreading sub-section!)

---

## FILE HANDLING

---

### TOPIC 82: FILE API (`java.io.File`)

---

#### 1. Introduction

**What is it?** `java.io.File` is Java's original (Java 1.0) class representing a file or directory PATH on the filesystem — it does NOT represent the file's actual content, only metadata (existence, path, permissions) and basic operations (create, delete, rename, list directory contents).

#### 2. Key Methods

| Method                                             | Purpose                                                                              |
|----------------------------------------------------|--------------------------------------------------------------------------------------|
| <code>exists()</code>                              | Checks if the file/directory exists                                                  |
| <code>createNewFile()</code>                       | Creates a new empty file                                                             |
| <code>mkdir()</code> / <code>mkdirs()</code>       | Creates a directory ( <code>mkdirs()</code> also creates missing parent directories) |
| <code>delete()</code>                              | Deletes the file/directory (must be empty for directories)                           |
| <code>isFile()</code> / <code>isDirectory()</code> | Type checks                                                                          |
| <code>listFiles()</code>                           | Returns an array of files/directories inside a directory                             |
| <code>getAbsolutePath()</code>                     | Returns the full, absolute path                                                      |
| <code>renameTo(File dest)</code>                   | Renames/moves the file                                                               |



### 3. Code Example

```
import java.io.File;
import java.io.IOException;

public class FileApiDemo {
    public static void main(String[] args) throws IOException {
        File file = new File("data.txt");
        if (!file.exists()) {
            file.createNewFile();
        }
        System.out.println("Path: " + file.getAbsolutePath());
        System.out.println("Is file: " + file.isFile());

        File dir = new File("output");
        dir.mkdirs();
        for (File f : dir.listFiles()) {
            System.out.println(f.getName());
        }
    }
}
```

### 4. Reading/Writing Content (Streams — the OLD way, Java 1.0+)

```
import java.io.*;

// Writing
try (FileWriter writer = new FileWriter("data.txt")) {
    writer.write("Hello, File!");
}

// Reading
try (BufferedReader reader = new BufferedReader(new FileReader("data.txt"))) {
    String line;
    while ((line = reader.readLine()) != null) {
        System.out.println(line);
    }
}
```

### 5. Frequently Asked Interview Questions

- 1 **Does `java.io.File` represent file content?** No — only metadata/path information; actual reading/writing requires separate stream classes (`FileReader`, `FileWriter`, `FileInputStream`, etc.).
- 2 **What's the difference between `mkdir()` and `mkdirs()`?** `mkdir()` creates only the final directory (fails if parent directories don't exist); `mkdirs()` creates any missing parent directories too.
- 3 **Why should file streams always be used with `try-with-resources`?** To guarantee `close()` is called automatically, even if an exception occurs, preventing resource/file-handle leaks (see [\[\[Try-Catch-Finally|Topic 42\]\]](#)).

---

## TOPIC 83: NIO (NEW I/O)

---

# 1. Introduction

**What is it?** `java.nio` (New I/O, Java 1.4, significantly enhanced in Java 7's **NIO.2** with `java.nio.file`) provides a more modern, scalable I/O API — built around `Path`, `Files`, buffers, and channels — offering better performance, symbolic link support, and file-system-event watching compared to the legacy `java.io.File/stream` classes.

## 2. `Path` and `Files` — The Modern Way (Java 7+ NIO.2)

```
import java.nio.file.*;
import java.util.List;

Path path = Paths.get("data.txt"); // modern replacement for 'new File(...)'

Files.writeString(path, "Hello, NIO!"); // one-line write (Java 11+)
String content = Files.readString(path); // one-line read (Java 11+)

List<String> lines = Files.readAllLines(path);
Files.copy(path, Paths.get("backup.txt"));
Files.delete(path);

boolean exists = Files.exists(path);
long size = Files.size(path);
```

## 3. `java.io.File` VS `java.nio.file.Path/Files` — Comparison Table

| Aspect                    | <code>java.io.File</code> (legacy)                                                              | <code>java.nio.file.Path/Files</code> (NIO.2, Java 7+)                      |
|---------------------------|-------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| API style                 | Object-oriented but limited, awkward error handling (returns <code>boolean</code> for failures) | Richer, throws specific <code>IOException</code> subtypes, more informative |
| Symbolic links            | Poor/no support                                                                                 | Full support                                                                |
| Directory traversal       | Manual recursion needed                                                                         | <code>Files.walk()</code> , <code>Files.walkFileTree()</code> built in      |
| Watching for file changes | Not supported                                                                                   | <code>WatchService</code> API supported                                     |
| Recommendation            | Legacy — avoid in new code                                                                      | Preferred for all new code                                                  |

## 4. Streams-Based Directory Walking (Java 8+)

```
import java.nio.file.*;
import java.util.stream.Stream;

try (Stream<Path> paths = Files.walk(Paths.get("."))) {
    paths.filter(Files::isRegularFile)
        .forEach(System.out::println);
}
```

## 5. Frequently Asked Interview Questions

- 1 **What replaced `java.io.File` in modern Java code?** `java.nio.file.Path` combined with the `Files` utility class (NIO.2, Java 7+).

- 2 **Name two capabilities NIO.2 has that the legacy `File` API lacks.** Full symbolic link support, and a `WatchService` API for monitoring filesystem changes (also, richer recursive directory traversal via `Files.walk()`).
  - 3 **What's a quick one-line way to read/write an entire file's text content since Java 11?**  
`Files.readString(path)` and `Files.writeString(path, content)`.
- 

## TOPIC 84: SERIALIZATION & DESERIALIZATION

---

### 1. Introduction

**What is it?** Serialization is the process of converting an object's state into a byte stream (for storage or transmission); deserialization is the reverse — reconstructing an object from that byte stream. Java's built-in mechanism requires a class to implement the `Serializable` marker interface.

**Why introduced:** Objects living in memory (Heap) need a way to be saved to disk, sent over a network, or cached across JVM restarts — serialization provides a built-in, standardized byte representation for this.

### 2. `Serializable` — A Marker Interface

```
import java.io.Serializable;

class Employee implements Serializable {
    private static final long serialVersionUID = 1L; // version control for serialized form
    String name;
    transient String password; // 'transient' - EXCLUDED from serialization
    Employee(String name, String password) { this.name = name; this.password = password; }
}
```

`Serializable` has NO methods — it's a **marker interface**, purely signaling to the JVM "this class is allowed to be serialized," checked at runtime by `ObjectOutputStream`.

### 3. Serializing and Deserializing

```
import java.io.*;

// SERIALIZE
try (ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream("emp.ser"))) {
    oos.writeObject(new Employee("Alice", "secret123"));
}

// DESERIALIZE
try (ObjectInputStream ois = new ObjectInputStream(new FileInputStream("emp.ser"))) {
    Employee emp = (Employee) ois.readObject();
    System.out.println(emp.name); // "Alice"
    System.out.println(emp.password); // null - transient fields are NEVER serialized!
}
```

### 4. `serialVersionUID` — Why It Matters

- A unique version identifier for a serializable class, either explicitly declared or auto-generated by the JVM (based on class structure) if omitted.
- If a class is DESERIALIZED using a different `serialVersionUID` than the one it was SERIALIZED with (e.g., after modifying the class's fields), an `InvalidClassException` is thrown — protecting against silently loading incompatible object data.
- **Best practice:** ALWAYS declare `serialVersionUID` explicitly — relying on the auto-generated one is fragile, since it can change based on subtle compiler/JVM differences even without meaningful class changes.

## 5. Edge Cases

| Case                                                                  | Result                                                                                          |
|-----------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| Serializing a class that does NOT implement <code>Serializable</code> | <code>NotSerializableException</code> at runtime                                                |
| A field marked <code>transient</code>                                 | Skipped entirely during serialization — deserializes back to its default value (null, 0, false) |
| A field that's itself a NON-serializable object reference             | <code>NotSerializableException</code> , unless marked <code>transient</code>                    |
| Deserializing with a mismatched <code>serialVersionUID</code>         | <code>InvalidClassException</code>                                                              |
| <code>static</code> fields                                            | NEVER serialized (they belong to the CLASS, not any instance)                                   |

## 6. Common Mistakes

- Forgetting to mark sensitive fields (passwords, security tokens) as `transient` — they'd otherwise be written in PLAIN, readable form into the serialized byte stream.
- Not declaring an explicit `serialVersionUID`, risking `InvalidClassException` after even minor, compatible class changes.
- Assuming Java's built-in serialization is a good choice for cross-language or long-term data storage — it's JVM/Java-specific and fragile across class version changes; JSON/Protobuf are generally preferred for those use cases in modern systems.

## 7. Best Practices

- Always declare `serialVersionUID` explicitly on every `Serializable` class.
- Mark sensitive or genuinely non-serializable fields `transient`.
- Prefer modern data formats (JSON via Jackson, Protobuf) over Java's native serialization for APIs, cross-language communication, and long-term storage — native Java serialization is mostly reserved for internal, same-JVM-version caching/session-replication scenarios today, and has historically been a notable SECURITY risk (deserialization of untrusted data is a well-known attack vector).

## 8. Production Usage

- Session replication in some clustered application servers, distributed caches (in older architectures), and Java RMI historically relied on native serialization.
- Modern systems overwhelmingly prefer JSON (Jackson/Gson) for REST APIs and Protobuf/Avro for high-performance service-to-service communication, largely due to native Java serialization's security risks (arbitrary code execution via crafted malicious serialized streams) and cross-language limitations.

## 9. Frequently Asked Interview Questions

- 1 **What interface must a class implement to be serializable?** `Serializable` — a marker interface with no methods.
- 2 **What happens to `transient` fields during serialization?** They're skipped entirely, deserializing back to their type's default value.
- 3 **What is `serialVersionUID`, and why should you declare it explicitly?** A version identifier for a serializable class's structure; declaring it explicitly avoids fragile, JVM-generated defaults that can change even after harmless edits, preventing unnecessary `InvalidClassExceptions` during deserialization.
- 4 **What happens if you try to serialize an object whose class doesn't implement `Serializable`?** `NotSerializableException` at runtime.
- 5 **Are `static` fields ever serialized?** No — they belong to the class itself, not to any particular object instance.
- 6 **Why is native Java serialization considered a security risk?** Deserializing data from an untrusted source can trigger arbitrary code execution via maliciously crafted byte streams exploiting the deserialization process — a well-documented, serious real-world vulnerability class (leading many teams to prefer JSON/Protobuf for external-facing data exchange).

---

*(End of Topics 82-84 — this concludes the File Handling sub-section!)*

---

# JVM INTERNALS

---

## TOPIC 85: HEAP, STACK & METASPACE (RECAP)

---

### 1. Recap

*(Fully covered in [\[\[Java Memory Model|Topic 4\]\]](#) and [\[\[JDK, JRE, JVM|Topic 2\]\]](#) — this topic is a focused consolidation specifically under the "JVM Internals" umbrella, as listed in the course order.)*

- **Heap:** Shared across all threads; stores all objects/arrays; divided into Young Generation (Eden + Survivor 0/1) and Old Generation.

- **Stack:** Per-thread; stores method call frames, local primitives, and object references (NOT the objects themselves).
- **Metaspace:** Shared; stores class metadata; allocated from NATIVE (off-heap) memory since Java 8, replacing the old fixed-size PermGen.

## 2. Quick Reference Diagram

```
JVM MEMORY
+-----+
| HEAP (shared) | METASPACE (shared, |
| Young Gen: Eden, S0, S1 | off-heap, native memory) |
| Old Gen (Tenured) | - class metadata |
+-----+
| STACK (per-thread) | PC Register | Native Method Stack |
+-----+
```

## 3. Frequently Asked Interview Questions

- 1 **What replaced PermGen in Java 8, and where is it allocated?** Metaspace, allocated from native (off-heap) memory rather than a fixed-size JVM heap region.
- 2 **Which memory areas are shared across all threads, and which are per-thread?** Heap and Method Area/Metaspace are shared; Stack, PC Register, and Native Method Stack are per-thread.
- 3 **What specifically lives on the Stack versus the Heap for `Person p = new Person();`?** The reference `p` lives on the Stack; the actual `Person` object lives on the Heap.

# TOPIC 86: GARBAGE COLLECTION

## 1. Introduction

**What is it?** Garbage Collection (GC) is the JVM's automatic process of reclaiming Heap memory occupied by objects that are no longer **reachable** from any active reference (see [[Java Memory Model|Topic 4]]), freeing developers from manual memory management (`malloc/free`) and its associated bug classes (memory leaks, dangling pointers, double-frees).

## 2. The Generational Hypothesis — Why Generations Exist

Empirically, **most objects die young** (short-lived temporary objects — loop variables, intermediate calculation results) while a SMALL minority live a long time (caches, singletons, long-lived application state). The JVM exploits this observation by dividing the Heap into generations and collecting the "usually mostly-garbage" Young Generation FAR more frequently (and cheaply) than the rarely-changing Old Generation.

### 3. Minor GC vs Major/Full GC

| Type          | Scope                                                           | Frequency                                 | Typical Pause                                                              |
|---------------|-----------------------------------------------------------------|-------------------------------------------|----------------------------------------------------------------------------|
| Minor GC      | Young Generation ONLY (Eden + Survivor spaces)                  | Frequent (objects are created constantly) | Usually short                                                              |
| Major/Full GC | Old Generation (sometimes the ENTIRE heap, including Metaspace) | Infrequent                                | Usually longer — the classic source of noticeable "GC pause" in production |

#### Minor GC Step-by-Step (Simplified)

1. New objects are allocated in EDEN.
2. When Eden fills up, a Minor GC runs:
  - LIVE objects in Eden are copied to Survivor space S0 (or S1, alternating).
  - DEAD (unreachable) objects in Eden are simply left behind - their space is reclaimed.
3. Objects that survive several Minor GC cycles (tracked via an "age" counter) get PROMOTED ("tenured") into the Old Generation.
4. When the Old Generation fills up, a Major/Full GC runs, which is more expensive.

### 4. Modern Garbage Collectors — Comparison Table

| Collector                   | Introduced                          | Approach                                                                               | Best For                                                        |
|-----------------------------|-------------------------------------|----------------------------------------------------------------------------------------|-----------------------------------------------------------------|
| Serial GC                   | Java 1.0                            | Single-threaded, stop-the-world                                                        | Small applications, single-core environments                    |
| Parallel GC                 | Java 5                              | Multi-threaded, still stop-the-world (but faster due to parallelism)                   | Throughput-focused batch applications, willing to accept pauses |
| CMS (Concurrent Mark Sweep) | Java 1.4                            | Mostly concurrent, minimizes pause time                                                | Deprecated/removed (Java 14) in favor of G1                     |
| G1 (Garbage First)          | Java 7 (default since Java 9)       | Region-based, balances throughput and pause time                                       | The current DEFAULT for most general-purpose applications       |
| ZGC                         | Java 11 (production-ready Java 15+) | Concurrent, extremely low pause times (sub-millisecond), even for MULTI-TERABYTE heaps | Latency-critical applications with very large heaps             |
| Shenandoah                  | Java 12                             | Concurrent, low-pause (similar goals to ZGC, different algorithm, from Red Hat)        | Similar use case to ZGC                                         |

### 5. G1 (Garbage First) — The Modern Default

Unlike older collectors that treat the Young/Old generations as single large contiguous blocks, **G1 divides the entire heap into many small, equal-sized REGIONS** (not strictly generational-contiguous), and prioritizes collecting the regions with the MOST garbage first ("Garbage First") — aiming to meet a configurable PAUSE-TIME GOAL (`-XX:MaxGCPauseMillis`) rather than optimizing purely for throughput.

### 6. Real Life Analogy

- **Restaurant:** Minor GC is like clearing a busy lunch table quickly between fast-turnover customers (most plates/food are consumed and discarded quickly). Major/Full GC is like a full kitchen deep-clean — needed far less often, but takes much longer and disrupts service more noticeably while it happens.
- **Library:** The Generational Hypothesis is like observing that most borrowed books (objects) are returned within a week (die young), while a few reference books stay checked out for years (long-lived) — so the library checks the "just returned" shelf far more often than it audits the rarely-touched long-term-loan shelf.

## 7. `System.gc()` Revisited

`System.gc();` // merely a HINT/SUGGESTION to the JVM - NEVER a guarantee (see Topic 4)

## 8. Edge Cases

| Case                                                                                            | Result                                                                                                                               |
|-------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| An application with a genuine memory leak (objects unintentionally kept reachable forever)      | GC cannot help — it only reclaims UNREACHABLE objects; reachable-but-unused objects are, by definition, not garbage to the collector |
| Extremely large, long-lived object graphs                                                       | Can cause frequent, expensive Major/Full GC pauses if the Old Generation fills up repeatedly                                         |
| Choosing the wrong GC algorithm for the workload (e.g., Serial GC on a large multi-core server) | Poor throughput/pause-time characteristics — GC algorithm choice IS a real production tuning decision                                |

## 9. Common Mistakes

- Confusing "has no more references" (garbage) with "the developer thinks it's no longer needed" — GC only ever cares about REACHABILITY, not intent; a genuine memory leak (unintentional lingering references) is invisible to and unfixable by the GC itself.
- Calling `System.gc()` in application code expecting deterministic, immediate collection — it's advisory only.
- Not tuning/choosing an appropriate GC algorithm for large-scale, latency-sensitive production workloads, leaving default settings that may not fit the actual traffic pattern.

## 10. Best Practices

- Fix memory leaks at the SOURCE (release unnecessary references, use bounded caches) — GC tuning cannot compensate for a genuine leak.
- Choose G1 (the modern default) for most general-purpose applications; consider ZGC/Shenandoah specifically for very large heaps with strict low-latency requirements.
- Monitor GC behavior in production (via `-Xlog:gc*`, JFR, or APM tooling) rather than guessing at tuning parameters blindly.

## 11. Production Usage



- Large-scale services (trading systems, real-time bidding platforms) often explicitly tune GC algorithm choice and pause-time goals as a critical part of their performance/latency engineering.
- Container-based deployments (Kubernetes) benefit significantly from Java 10+'s cgroup-awareness (see [\[\[Java Memory Model|Topic 4\]\]](#)) combined with an appropriately-sized, tuned GC configuration.

## 12. Frequently Asked Interview Questions

- 1 **What is the Generational Hypothesis?** The empirical observation that most objects die young (short-lived) while few live long — justifying the JVM's generational Heap division and more frequent, cheaper collection of the Young Generation.
- 2 **What's the difference between Minor GC and Major/Full GC?** Minor GC collects only the Young Generation (frequent, usually fast); Major/Full GC collects the Old Generation (and sometimes the whole heap), less frequent but typically much more expensive/disruptive.
- 3 **What is the current default garbage collector (Java 9+)?** G1 (Garbage First).
- 4 **What makes ZGC/Shenandoah different from G1?** They aim for extremely low, near-constant pause times (often sub-millisecond) even on very large heaps, via more extensive CONCURRENT (non-stop-the-world) collection phases, at some cost to raw throughput compared to G1.
- 5 **Can the Garbage Collector fix a memory leak?** No — GC only reclaims genuinely UNREACHABLE objects; a "leak" (objects unintentionally kept reachable via lingering references) is invisible to the GC by definition, since those objects are technically still reachable.
- 6 **Does `System.gc()` guarantee immediate collection?** No — it's purely an advisory hint; the JVM may ignore it entirely.
- 7 **What does G1's region-based approach do differently from older collectors?** It divides the heap into many small, equal-sized regions (not one contiguous generational block) and prioritizes collecting the regions containing the MOST garbage first, aiming to meet a configurable pause-time goal.

---

## TOPIC 87: JVM ARCHITECTURE (RECAP & FULL SUMMARY)

---

### 1. Recap

*(Fully covered across [\[\[JDK, JRE, JVM|Topic 2\]\]](#) (class loading, execution engine, overall architecture) and [\[\[Java Compilation Process|Topic 3\]\]](#) (bytecode, class file format) — this final topic consolidates the complete picture as the capstone of Core Java.)*

## 2. The Complete JVM Architecture — Final Consolidated Diagram

```
+-----+
| JVM ARCHITECTURE |
| |
| 1. CLASS LOADER SUBSYSTEM |
| Loading (Bootstrap -> Platform -> Application, delegation model) |
| -> Linking (Verify -> Prepare -> Resolve) |
| -> Initializing (static blocks/fields run here, ONCE per class) |
| |
| 2. RUNTIME DATA AREAS |
| SHARED: Method Area/Metaspace (class metadata), Heap (all objects, |
| generational: Young [Eden+S0+S1] -> Old) |
| PER-THREAD: JVM Stack (call frames), PC Register, Native Method |
| Stack |
| |
| 3. EXECUTION ENGINE |
| Interpreter (executes bytecode line-by-line) |
| JIT Compiler (compiles HOT bytecode to native machine code) |
| Garbage Collector (Serial/Parallel/G1[default]/ZGC/Shenandoah) |
| |
| 4. NATIVE METHOD INTERFACE (JNI) -> platform-native libraries |
+-----+
```

## 3. The Full Journey: Source Code to Output (End-to-End Capstone Summary)

```
1. YOU write: HelloWorld.java
2. javac COMPILES it: -> HelloWorld.class (bytecode, magic number 0xCAFEBAE, ...)
3. `java HelloWorld` LAUNCHES a JVM instance:
  a. Class Loader loads HelloWorld.class (+ any referenced classes, lazily)
  b. Bytecode Verifier checks it's safe
  c. Execution Engine INTERPRETS bytecode; hot methods get JIT-compiled to native machine code
  d. Objects created via `new` go on the HEAP; method calls push frames onto the current thread's STACK
  e. Garbage Collector reclaims unreachable Heap objects in the background
4. OS executes the native instructions -> OUTPUT appears
```

## 4. Frequently Asked Interview Questions

- 1 **Name the four major components of JVM architecture.** Class Loader Subsystem, Runtime Data Areas, Execution Engine, Native Method Interface (JNI).
- 2 **What are the three phases of class loading?** Loading, Linking (Verify, Prepare, Resolve), Initializing.
- 3 **What are the three components of the Execution Engine?** Interpreter, JIT Compiler, Garbage Collector.
- 4 **Walk through the complete pipeline from writing HelloWorld.java to seeing console output.** (See the full step-by-step summary above — compilation to bytecode, class loading/verification, interpretation/JIT execution, Heap/Stack memory usage, and GC reclamation.)

---

## FINAL SUMMARY — CORE JAVA COMPLETE

This concludes the entire **Core Java (Java 8 & Java 17)** section — 87 topics across:

- 1 **Java Fundamentals** (Topics 1-13): Introduction to Java, JDK/JRE/JVM, Compilation Process, Memory Model, Variables, Data Types, Operators, Type Casting, Keywords, Control Statements, Arrays, Methods, Command Line Arguments.
- 2 **Object-Oriented Programming** (Topics 14-27): Classes & Objects, Constructors, Constructor Chaining, Method Overloading, Method Overriding, Inheritance, Polymorphism, Abstraction, Encapsulation, Interfaces, Abstract Classes, Object Class, Packages, Access Modifiers.
- 3 **Advanced Core Java** (Topics 28-40): Wrapper Classes, String, StringBuilder, StringBuffer, Immutable Objects, Enums, Annotations, Generics, Reflection API, Inner Classes, Nested Classes, Records, Sealed Classes.
- 4 **Exception Handling** (Topics 41-44): Exception Hierarchy/Checked/Unchecked, Try-Catch-Finally, Throw/Throws, Custom Exceptions.
- 5 **Collections Framework** (Topics 45-65): List family, Queue family, Set family, Map family, Comparable, Comparator, Iterator, ListIterator.
- 6 **Java 8 Features** (Topics 66-73): Lambda Expressions, Functional Interfaces, Method References, Streams API, Optional, Date & Time API, Predicate/Function/Consumer/Supplier, Default/Static Interface Methods.
- 7 **Multithreading** (Topics 74-81): Thread/Runnable, Thread Lifecycle, Synchronization, Locks, Executor Framework, Callable/Future, CompletableFuture, Concurrent Collections.
- 8 **File Handling** (Topics 82-84): File API, NIO, Serialization/Deserialization.
- 9 **JVM Internals** (Topics 85-87): Heap/Stack/Metaspace recap, Garbage Collection, JVM Architecture recap.

**You now have a complete, interview-ready, production-ready reference covering Core Java from absolute fundamentals to Java 17's latest language features.**

---

*(END OF CORE JAVA MASTER NOTES — ready to proceed to Advanced Java whenever you say "Next Topic.")*